Inspired Bilingual

引言

> Introduction

1980年代中期，我是一名年轻的软件开发者，在惠普（HP）公司从事一个备受瞩目的产品项目。当时人工智能正值热潮，我很幸运能在业内最优秀的公司之一工作，并加入了一支非常强大的软件工程团队（该团队的几位成员后来在行业内各公司取得了巨大成功）。我们的任务十分艰巨：在一台低成本、通用的工作站上交付一款软件，而此前这款软件需要专用的软硬件组合，每位用户的成本超过 10 万美元——这是很少有人能负担得起的价格。

我们刻苦工作了一年多，牺牲了无数个夜晚和周末。在此期间，我们为惠普的专利组合增添了几项专利。我们按照惠普严格的质量标准开发软件。我们对产品进行了国际化处理，并为多种语言进行了本地化。我们培训了销售团队。我们向媒体预览了我们的技术，获得了极好的评价。我们准备好了。我们发布了。我们庆祝了发布。

只有一个问题：没人买它。

这款产品在市场上完全失败了。是的，它在技术上令人印象深刻，评论者也喜欢它，但它不是人们想要或需要的东西。

团队当然对这一结果感到沮丧。但很快我们开始问一些

重要的问题：谁来决定我们应该实际构建什么产品？他们如何决定？他们怎么知道我们要构建的东西会有用？

我们年轻的团队学到了一些非常深刻的东西——我相信许多团队都曾经历过艰难的发现：如果你的工程团队没有被赋予值得构建的东西，那么无论他们有多优秀都没用。

更一般地说，我们学到的是，仅仅把产品工程做得好是不够的。同样重要的是发现一款有价值、可用且可行的产品。

在试图追溯我们失败的根本原因时，我了解到关于构建什么的决定来自一位"产品经理"——一位通常隶属于营销部门、负责定义我们要构建的产品的人。但我也了解到，惠普并不特别擅长产品管理。后来我了解到大多数公司都不擅长这个，事实上大多数公司现在仍然不擅长。

我向自己承诺，除非我知道产品是用户和客户想要的，否则我绝不会再为一款产品如此努力工作。

在接下来的 20 年里，我非常幸运地参与了我们这个时代一些最成功的高科技产品——首先在惠普经历了个人电脑的兴起，然后在网景通讯/AOL（Netscape Communications/AOL）经历了互联网的兴起，我在那里担任平台与工具副总裁，最后在 eBay 经历了电子商务的兴起，我担任产品管理与设计高级副总裁。

并非所有的产品努力都像其他的那样成功，但我很高兴地告诉大家，

没有一个是失败的，其中几个被全球数百万人所喜爱和使用。

在我离开 eBay 后不久，我开始接到来自产品组织的电话，希望改进他们生产产品的方式。当我开始与这些公司合作时，我发现最优秀的公司生产产品的方式与大多数公司生产产品的方式之间存在巨大差异。我意识到最先进水平与普遍实践之间存在很大差距。

大多数公司仍在使用陈旧低效的方式来定义和创建产品。我还发现，无论是学术界，包括最好的商学院课程，还是行业组织，都无法提供多少帮助，它们似乎无望地固守在过去失败的模式中——就像我在惠普工作时所处的模式一样。

我有过一些很棒的经历，我特别感谢我有机会与业内一些最优秀的产品专家一起工作。这本书中最好的想法来自这些人。你会在致谢中找到他们中的许多人。我从他们所有人那里学到了东西，我对他们每个人都心存感激。

我选择这个职业是因为我想从事客户喜爱的产品；那些能够启发人心并提供真正价值的产品。我发现大多数产品领导者也希望创造能够启发人心并获得成功的产品。然而大多数产品并不令人启发，而生命太短暂，不应该浪费在糟糕的产品上。

我写这本书的希望是，它将有助于分享最成功的产品公司的最佳实践，

而结果将是更多真正令人启发的产品——客户喜爱的产品。

本书的目标读者

本书是专门为软件产品团队的成员撰写的——特别是互联网软件产品团队——包括消费级和企业级，他们负责定义要构建的产品。通常这些产品领导者被称为"产品经理"，但他们也可能是公司创始人、高管、首席工程师或设计师。

除了产品经理之外，本书还面向用户体验设计师、软件工程师和架构师、项目/程序经理、产品营销经理，以及产品组织不同部门的管理者。

根据我的经验，这里描述的信息适用于各种各样的产品开发团队：

你的公司可能是初创公司，或拥有许多产品的大型企业，或介于两者之间。你可能正在开发全新的 1.0 产品，或正在对现有产品进行渐进式改进。你的产品开发团队可能使用敏捷方法（如 Scrum），或传统的基于瀑布的方法。

你的产品可能是互联网服务、可交付软件、设备或平台。它可能面向消费者、小型企业或企业。例如，产品可能是

电子商务网站、幻想体育或游戏网站、消费电子产品、面向企业的托管服务，或支持特定类型互联网应用和服务的平台，如社交网络或视频分享。

我必须提出的一个警告是，本书不适用于从事非软件产品（如药品）的人员，也不适用于非产品软件工作，如定制软件项目。这并不是说我描述的方法和策略在其他环境中不起作用，而是我在产品软件世界中开发并实践了这些概念，因此它们在以外的地方可能效果不佳。

本书的结构

当我刚进入网景（Netscape）的高级管理岗位时，我发现我的日常工作分为三个不同的领域：人员（People）、流程（Process）和产品（Product）：

精心设计创造伟大产品

我不相信令人启发的产品是偶然发生的。在每一种情况下，在每一个成功的、令人启发的产品背后，我都发现存在一些真理。这里有十条这样的真理，我在每个产品工作中都努力牢记：

1. 产品经理的工作是发现有价值、可用且可行的产品。
2. 产品发现是产品经理、交互设计师和软件架构师之间的协作。
3. 工程开发很重要也很困难，但用户体验设计更重要，通常也更困难。
4. 工程师通常非常不擅长用户体验设计——工程师以实现模型的方式思考，但用户以概念模型的方式思考。
5. 用户体验设计既包括交互设计也包括视觉设计（对于基于硬件的设备，还包括工业设计）。
6. 功能（产品需求）和用户体验设计本质上是相互交织的。
7. 产品创意必须在实际目标用户身上尽早且经常地进行测试，才能产生有价值且可用的产品。
8. 我们需要一个高保真原型，这样我们才能快速、轻松且频繁地在真实用户身上使用真实的用户体验来测试我们的想法。
9. 产品经理的工作是识别满足目标的最小可行产品——有价值、可用且可行——最小化上市时间和用户复杂性。
10. 一旦这个最小成功产品被发现和验证，它就不是可以被拼凑起来并期望获得相同结果的东西。

我继续与太多固守陈旧、失败的产品创造方式的产品团队交谈。接受这些真理是本书的核心内容。

人员（People）指的是产品组织，以及团队成员在定义和开发产品时的角色和职责。

流程（Process）指的是用于反复发现和构建令人启发且成功的产品的流程、活动和最佳实践。

产品（Product）指的是这些令人启发的产品的定义特征。

这三个领域对于发现和创造令人启发的产品都至关重要。一切都始于人员，但流程是使这些人员能够持续生产令人启发且成功产品的关键。

我将本书按照这三个部分来组织。每个部分分为几个独立的主题。每个部分内的顺序通常不重要——你可以从一个主题跳到另一个主题。每个主题都是自成一体的。在书的最后，我总结了我认为最重要的 10 个实践和技巧。

本书讨论的许多主题揭示了一些世界顶级公司应用的最佳实践。其他主题基于对业内一些最优秀产品专家的访谈。还有一些基于我自己在我工作过和合作过的公司的经验。

请记住：只有当本书能帮助你交付更好的产品时，它才有用，所以每个主题的意图都是发人深省、相关且可操作的。

我希望这些信息能帮助你创造更成功的产品，我很想听到你的经验。请访问硅谷产品集团网站（www.svpg.com）在线联系我，让我知道你的想法。

祝你成功，祝你能发现客户喜爱的、令人启发的产品。

在线示例

我非常相信示例的力量，无论是好的还是坏的。但是，由于我们的行业变化如此之快，像这样的书面临的挑战之一是提供及时和相关的示例。另一个问题是，加入我最喜欢的示例会为这本书增加一百多页。

因此，我把许多示例放在了硅谷产品集团网站上（www.svpg.com/examples）。这样我就可以继续更新和添加示例，而无需更新书的内容。与网站上的所有材料一样，不需要付费或注册。

在线示例包括机会评估、产品原则、产品策略、产品路线图、产品规格、原型、用户画像，以及原型测试的问题和任务。

我为此道歉，这意味着你需要互联网连接才能看到许多示例，我意识到这打破了阅读书籍的连贯性，但我希望你会发现这种方法的好处大于不便。

你会在以下章节中找到对网站上示例的引用。

产品组织

每个产品都始于产品团队的人员。你如何定义角色，以及选择谁来组建团队，很可能将成为决定其成败的关键因素。

在本节中，我们将描述现代软件和互联网产品团队的关键角色与职责。

这是许多产品团队表现不佳的领域，他们固守过去的旧模式。对于许多组织而言，此处讨论的角色和职责与他们习惯的方式有着显著不同。

第 1 章：关键角色与职责

> Chapter 1: Key Roles And Responsibilities
> The Modern Software Product Organization

现代软件产品组织

> Throughout this book I'll be referencing the key roles on the product team, and in this first chapter I will define exactly what I mean by each. I realize that not every company uses these titles or assigns the responsibilities in exactly this way, but I believe the most successful companies do, and that each of these roles is essential to creating successful products. Remember that when I say “software product organization” I'm referring to not just shipped software for business or

> consumers, but also Internet or web services for businesses or consumers, and consumer electronics or other software-centric devices.

在整本书中，我将引用产品团队的关键角色，而在本章中，我将准确定义每个角色的含义。我意识到并非每家公司都使用这些头衔，或完全以这种方式分配职责，但我相信最成功的公司确实如此，而且每个角色对于创造成功的产品都至关重要。

请记住，当我说"软件产品组织"时，我指的不仅是面向企业或消费者的发售软件，还包括面向企业或消费者的互联网或网络服务，以及消费电子或其他以软件为中心的设备。

> Product Manager

产品经理（Product Manager）

> The product manager has two key responsibilities: assessing product opportunities, and defining the product to be built. Typically, new ideas can come from anywhere—company executives, discussions with customers, usability testing, your own product team, your sales or marketing staff, industry analysts, to name a few. But then someone needs to take a hard look at the idea and decide if it is something worth pursuing. The product manager is

产品经理有两个关键职责：评估产品机会，以及定义要构建的产品。通常，新创意可以来自任何地方——公司高管、与客户的讨论、可用性测试、你自己的产品团队、你的销售或营销人员、行业分析师等等。但随后需要有人仔细审视这个想法，并决定是否值得追求。产品经理

负责此项评估（许多公司称之为 MRD——市场需求文档（Market Requirements Document），但我将描述一种更轻量级的版本，称为机会评估（Opportunity Assessment））。

一旦你确定了一个良好的机会，且你的公司适合追求它，那么就需要有人去发现解决方案——即产品——实际上是什么，包括必要的特性和功能、用户体验以及发布标准。同样，这个人是产品经理，这项任务是他或她工作的核心。有些公司称这份规格说明书为 PRD（产品需求文档（Product Requirements Document）），另一些称之为产品规格说明书（Product Spec）或功能规格说明书（Functional Spec）。同样，我将提倡一种基于原型而非文档的轻量级方法，但关键在于它描述了要构建的产品的功能和行为，而非其实现方式。

> responsible for this assessment (many companies call this an MRD—Market Requirements Document—but I'll describe a lighter-weight version of this called an Opportunity Assessment).

用户体验设计师（User Experience Designer）

> Once you've decided that you have a good opportunity and your company is well-suited to pursue it, then someone needs to discover what the solution—the product—actually is, including the necessary features and functionality, the user experience, and the release criteria. Again, this someone is the product manager, and this task is the heart of his or her job. Some companies call this spec a Product Requirements Document (PRD), and others call it a Product Spec or Functional Spec. Again, I'll advocate a lighter-weight approach that's based on prototypes and not paper, but

> the key is that it describes the functionality and behavior of the product to be built, and not how it will be implemented.

用户体验设计组织中实际上有几个角色，稍后我将详细深入每个角色。这里的关键角色是交互设计师（interaction designer）（也称为信息架构师（information architect）、用户界面设计师（user interface designer）和用户体验架构师（user experience architect））。这些人负责深入理解目标用户（你试图在产品中满足的每个用户画像（persona）），并提出既可用又高效的任务、导航和流程。交互设计师与产品经理紧密合作，发现满足用户需求的特性和设计的融合。其目标是达到软件既可用（用户能弄清楚如何使用）又有价值（用户确实想使用）的程度。

> User Experience Designer

项目管理（Project Management）

> There are actually several roles within the user experience design organization and I'll dive into more detail later on each of these. The key role here is the interaction designer (also known as information architect, user interface designer, and user experience architect). These people are responsible for developing a deep understanding of the target users (each persona that you're trying to satisfy in your product), and coming up with the tasks, navigation, and flow that are both usable and productive. The interaction designer works closely with the product manager to discover the blend of requirements and design that meet the needs of the user. The idea is to get to the point where the software is both usable (users can figure out how to use it) and valuable (users actually want to use it).

产品定义完成后，产品开发团队将接手项目并开始构建产品。项目排期和跟踪功能是项目管理的核心。关于究竟由谁来处理排期和跟踪，有几种不同的模式。有时由专职的项目经理（project managers）管理，有时由工程经理管理（因为大部分资源通常来自他或她的团队），在某些情况下，产品经理也被要求兼任项目管理。这通常更多地取决于公司的文化和项目规模。大型项目尤其受益于专职且技能娴熟的项目经理。

> Project Management

工程（Engineering）

> Once the product has been defined, the product development team will take on the project and begin building the product. The project scheduling and tracking function is the core of project management. There are several different models regarding who exactly handles the scheduling and tracking. Sometimes it is managed by dedicated project managers, sometimes by the engineering manager (since most of the resources are usually from his or her team), and in some cases the product manager is asked to project manage as well. It often depends more on the culture of the company and the size of the project. Larger projects especially benefit from a dedicated and skilled project manager.

也称为产品开发（product development）或软件开发人员（software developers），这些是实际负责构建产品的人员。在某些公司，这被称为"IT"（信息技术（information technology）），但区分面向客户的软件和面向内部使用的软件（如人力资源应用）很重要。IT 通常是支持内部员工的团队，而工程组织则构建和维护面向外部客户的产品。

敏捷团队怎么办?

我合作的许多产品组织都在使用某种形式的敏捷方法（Agile Methods），特别是最流行的 Scrum。如果你尚未听说过这些，我将在"敏捷方法的成功实践"一章中更深入地介绍它们。

在大多数采用 Scrum 的软件组织中，产品经理担任产品负责人（Product Owner），项目经理通常担任 Scrum 主管（ScrumMaster）。前面定义的其他角色基本相同。

然而，对于敏捷团队的产品组织，有几个非常重要的考虑因素，例如整合用户体验设计和管理发布流程，我们将在整本书中讨论这些要点。

> Engineering

网站运营（Site Operations）

> Also known as product development or software developers these are the people responsible for actually building the product. In some companies this is called “IT” (information technology), but it's important to draw a distinction between the software created for customers and the software created for internal use such as an HR application. IT is typically the group that supports internal employees, and the engineering organization builds and maintains products for external customers.

对于互联网服务，产品通常在中央服务器上运行，通过网络访问。网站运营团队负责保持此服务的运行。虽然有些公司要求工程团队负责此项工作，但大多数公司发现这需要一套专门的技能，且重要性极高，不能作为次要职责。

> what About Agile Teams?
>
> Many of the product organizations I work with are using some form of Agile Methods, in particular, the most popular called Scrum. If you haven't heard of these, I cover them more in depth in the chapter Succeeding with Agile Methods.
>
> In most Scrum software organizations, the product manager serves as the Product Owner, and the project manager typically serves as the ScrumMaster. The other roles defined earlier are essentially the same.
>
> There are, however, several very important considerations for Agile teams when it comes to product organizations, such as incorporating user experience design, and managing the release process, and we will discuss these points throughout the book.
>
> Site Operations

产品营销（Product Marketing）

> For Internet services, the product is typically run on central servers and accessed over the Web. The site operations team is responsible for keeping this service running. While some companies ask the engineering team to cover this, most find that it demands a specialized set of skills and is far too important to be a secondary responsibility.

产品营销团队成员负责向外界宣传产品、管理面向外部的产品发布、为销售渠道提供营销和销售产品的工具，以及主导关键项目，如在线营销活动和大咖营销项目（influencer marketing programs）。通常，公司要求同一人同时负责产品管理（产品定义）和产品营销职责。这可能很困难，因为所需的技能截然不同，但尽管如此，这在许多公司中仍然存在。

附注：在微软（Microsoft），定义产品并推动项目进度的人被称为程序经理（program managers），这是一个不幸的称谓，因为该术语在行业中已被广泛用于描述多团队项目管理。然而，他们也不能使用产品经理（product manager）这个术语，因为他们已经用该术语来代表产品营销职能。虽然我希望他们使用不同的头衔（对两者都如此），但总体而言，我认为他们在定义产品的关键产品管理职责方面做得非常好。

产品。

各角色的合理配比是多少?

在任何软件产品组织中，你都会发现产品经理、设计师和工程师之间存在一些自然的比例关系。这是因为要让特定的工程团队忙于构建有价值且可用的软件，产品经理和设计师必须完成一定量的工作（因此需要一定数量的这些职位人员）来支持工程师。

当然，还有其他几个因素会影响角色的正确配比，例如正在生产的软件类型，以及员工的经验和技能，但以下的比例可以给你一个指导。

问：我们需要多少产品经理？ 答：通常，每 5-10 名工程师配备一名产品经理。

问：我们需要多少用户体验设计师？ 答：一名交互设计师通常可以支持两名产品经理，一名视觉设计师通常可以支持四名交互设计师。

问：我们应该有专职的项目经理吗？ 答：对于规模较大的项目，例如超过 5 名工程师的项目，是的。此外，如果你使用"火车模式"发布（即每 1-4 周发布一次，

如果某个功能未准备好，它就会被移到下一班可用的火车上），你肯定需要为每个发布分配专职的项目经理（每个发布通常包含来自多个项目的软件）。

> Product Marketing
>
> The product marketing team member is responsible for telling the world about the product, managing the external-facing product launch, providing tools for the sales channel to market and sell the product, and for leading key programs such as online marketing campaigns and influencer marketing programs. Often companies ask the same person to cover both the product management (product definition) and the product marketing responsibilities. This can be difficult since the skills required are very different, but nevertheless it occurs at many companies.

> Side Note: At Microsoft, the people that define the product and drive the project schedule are called program managers, which is an unfortunate title given that the term is already widely used in the industry to describe multiple-team project management. However, they could not use the term product manager either because they already use that to represent the product marketing function. While I wish they would use different titles (for both), in general I think they do a very good job on the critical product management role of defining
>
> products.
>
> What Are The Right Ratios of Roles?
>
> In any software product organization, you will find that there are some natural ratios between product managers, designers and engineers. This is because to keep a particular engineering team busy with valuable and usable software to build, there is a certain amount of work that the product managers and designers must do (and therefore a certain number of people in these positions) to support the engineers.
>
> There are of course several other factors that influence the right ratio of roles, such as the type of software being produced, and the experience and skill of the staff, but the ratios that follow should give you a guideline.
>
> Q: How many product managers do we need?
>
> A: Generally, one product manager for every 5-10 engineers.
>
> Q: How many user experience designers do we need?
>
> A: One interaction designer can generally support two product managers, and one visual designer can typically support four interaction designers.
>
> Q: Should we have dedicated project managers?
>
> A: For significant-sized projects, such as those with more than 5 engineers, yes. Further, if you use the “train model” of releases (where you make a release every one to four weeks
>
> consistently, and if a given feature is not ready it simply moves to the next available train), you'll definitely need dedicated project managers assigned to each release (which generally contains software from multiple projects).

第 2 章：产品管理与产品营销

> Chapter 2: Product Management
>
> VS. PRODUCT MARKETING

产品管理vs. 产品营销

> They Are Not The Same Thing

它们不是一回事

> Industry pundits claim that as much as nine out of ten product releases are failures in that they fail to meet their objectives. Even if your organization performs better than this, I strongly believe that most releases are ill-conceived. Countless release cycles are wasted on products that are either not valuable or not usable.

行业专家声称，多达九成的产品发布未能达到其目标，因此是失败的。即使你的组织表现比这好，我也强烈认为大多数发布都是构思不当的。无数的发布周期被浪费在那些要么没有价值、要么不可用的事情上。

> There are many reasons for bad products, and each topic in this book is intended to speak to some aspect of where these bad products come from, but I have long argued that the root cause of wasted releases can most often be traced to poor definition of the role of the product manager in a company, and inadequate training of the people chosen for this role.

糟糕的产品有很多原因，本书的每个主题都旨在探讨这些糟糕产品的某些方面，但我长期以来一直认为，浪费的发布最根本的原因通常可以追溯到公司对产品经理角色的定义不清，以及担任此角色的人员培训不足。

> This is a topic I've been thinking about for a long time—it is critically important because it gets to the core of what the job of the product manager truly needs to be.

这是一个我思考了很久的话题——它至关重要，因为它触及了产品经理工作真正需要的核心。

> As discussed earlier, the job of the product manager is to define—in detail—the product that the engineering team will build. In contrast, the role of product marketing is to tell the world

如前所述，产品经理的工作是详细定义工程团队将构建的产品。相比之下，产品营销的角色是向外界

> about this product. These are extremely different jobs.

介绍这个产品。这是截然不同的工作。

> To be clear right from the start, I argue that every product needs a single, accountable product manager, who is responsible for the product definition (the combination of product requirements and user experience design that describe the product to be built). Unfortunately, when I begin working with a company, all too often I encounter one of three different situations:

从一开始就明确一点，我认为每个产品都需要一个单一的产品经理，对产品定义（产品需求和用户体验设计的结合，描述要构建的产品）负责。不幸的是，当我开始与一家公司合作时，我常常会遇到以下三种情况之一：

> Marketing-Driven Product: There is a product marketing or product manager-titled person responsible for market requirements, and then the product goes straight to engineering—bypassing detailed product requirements and the many tough decisions that are encountered

> through the discovery process (and also very often bypassing user experience design, but that's the subject of another topic).

营销驱动的产品：有一位产品营销或名义上的产品经理负责市场需求，然后产品直接进入工程开发——绕过了详细的产品需求和发现过程中遇到的许多艰难决策（而且通常也绕过了用户体验设计，但那是另一个话题）。

> Two People, One Role: The product definition role is split between a product marketing person responsible for “high-level” business requirements and a product manager person responsible for the “low-level” product requirements.

两人一职：产品定义角色在负责"高层级"业务需求的产品营销人员和负责"低层级"产品需求的产品经理之间分割。

> One Person, Two Roles: A product marketing person is asked to cover both roles (and the company sometimes calls these people product managers and sometimes product marketing).

一人两职：产品营销人员被要求同时担任两个角色（公司有时称这些人为产品经理，有时称为产品营销）。

> Let's discuss each of these three situations and the problems they create:

让我们讨论这三种情况及其造成的问题：

> Marketing-Driven Product

营销驱动的产品

> This situation is pretty easy to spot. The rest of the product team views this person as the marketing resource who might be useful for creating data sheets, training the sales force, and coming up with the naming and pricing, but in terms of defining the product, this person is largely discounted and ignored. There are plenty of Dilbert cartoons portraying this person, and we've all known this type of product manager.

这种情况很容易发现。产品团队的其他成员将这个人视为营销资源，可能对创建数据表、培训销售团队、提出命名和定价有用，但在定义产品方面，这个人很大程度上被忽视和忽略。有很多呆伯特漫画描绘这类人，我们都认识这种类型的产品经理。

> While these people might be great at marketing, they are way over their heads when it comes to trying to discover and define in detail a valuable, usable and feasible product. In this situation, hopefully someone else on the product team—sometimes a lead engineer, sometimes an interaction designer, and sometimes a manager—steps in and performs the true product management function. If this person has the skills in addition to the bandwidth, the product may yet succeed. More often, however, the product is already in trouble right from the start.

虽然这些人可能在营销方面很出色，但在试图发现和详细定义有价值、可用且可行的产品时，他们就力不从心。在这种情况下，希望产品团队中的其他人——有时是首席工程师，有时是交互设计师，有时是经理——介入并履行真正的产品管理职能。如果这个人除了有空闲时间外还具备这些技能，产品可能还有成功的机会。但更多时候，产品从一开始就已经陷入麻烦了。

> My first exposure to product management was in this exact situation, and it initially kept me from wanting to have any association at all with this role. Fortunately, I met a guy that showed me what product management was really all about, and since that day I've worked to highlight the successful product managers and to redefine the role around these people.

我第一次接触产品管理就是这种情况，它最初让我根本不想与这个角色有任何关联。幸运的是，我遇到了一个人，他向我展示了产品管理真正是关于什么的，从那天起，我一直努力突出成功的产品经理，并围绕这些人重新定义这个角色。

> Two People, One Role

两人一职

> This situation is also easy to spot, as there is no single person responsible for the product. A product marketing person (sometimes in this model called the “business owner” or the “business product manager”) is responsible for the high-level product requirements, and a product manager (sometimes called a “technical product manager” or “product owner” in Agile teams) is responsible for the low-level product requirements.

这种情况也很容易发现，因为没有一个人对产品负责。产品营销人员（在这种模式中有时称为"业务负责人"或"业务产品经理"）负责高层级产品需求，产品经理（有时称为"技术产品经理"或敏捷团队中的"产品负责人"）负责低层级产品需求。

> The problem is that neither person truly owns the product and, more importantly, neither person feels nor behaves like they are ultimately responsible for the product. Further, this model is based on a flawed view of software that holds that you can define high-level requirements independent of detailed requirements and especially the user experience.

问题是两个人都没有真正拥有产品，更重要的是，两个人都没有感觉或表现得像是最终对产品负责的人。此外，这种模式基于一种有缺陷的软件观，认为你可以独立于详细需求和特别是用户体验来定义高层级需求。

> In companies with this model, product managers become little more than a spec-generation service. It is a frustrating job that tends to stifle innovation and rarely produces successful products.

在这种模式的公司中，产品经理不过是规格生成服务。这是一份令人沮丧的工作，往往会扼杀创新，很少能产出成功的产品。

> Many larger companies with multiple business units evolve into these roles and then wonder why they can no longer come up with inspiring products that their customers love.

许多拥有多个业务部门的大公司演变成这些角色，然后想知道为什么他们再也无法想出客户喜爱的、令人启发的产品。

> One Person, Two Roles

一人两职

> The problem with combining the product manager and product marketing roles into one position is that it is very hard to find someone who can do both types of jobs well. Each of these roles is critical for a product's success, and each requires special skills and talents.

将产品经理和产品营销角色合并为一个职位的问题是，很难找到能同时做好这两类工作的人。这两个角色对产品成功都至关重要，每个都需要特殊的技能和才能。

> Creating a product is much different than telling the world about that product. I have known some truly exceptional people who can excel in both roles, but these people are very rare. As an organizational model, it just doesn't scale. Further, for all but the simplest of products, the role of the product manager as defined here is an all-consuming, full-time job, requiring a dedicated person. If you ask the product marketing person to cover the product management role, even if the person has the skills and talents required for both, it is unlikely he or she will have the bandwidth to do both jobs well.

创建产品与向外界介绍产品是截然不同的。我认识一些真正优秀的人，他们可以在两个角色上都表现出色，但这样的人非常罕见。作为一种组织模式，它根本无法扩展。此外，对于除了最简单的产品之外的所有产品，这里定义的产品经理角色是一个全职的、耗时的工作，需要专人负责。如果你要求产品营销人员同时承担产品管理角色，即使这个人具备两个角色所需的技能和才能，他或她也不太可能有足够的精力把两个工作都做好。

> This is most frequently a problem at enterprise software companies where supporting the sales force is a big job in itself, and there is a strong tendency for the product managers simply to pass along (perceived) requirements—from the big customers, to the sales reps, to the product managers, and then to the engineers. And—no surprise—this almost never results in valuable and usable products.

这在企业软件公司中最常成为问题，因为在那里支持销售团队本身就是一项繁重的工作，而且产品经理有一种强烈的倾向，只是传递（被认为的）需求——从大客户到销售代表，再到产品经理，然后到工程师。——毫不奇怪，这几乎从来不会产生有价值且可用的产品。

> It is important to recognize that there are reasons for each of the organizational models described above, and I do understand that. But these companies are sacrificing far more than they realize. They are wasting entire product release cycles, and they are creating products that customers don't want, or must struggle with to use.

重要的是要认识到，上述每种组织模式都有其存在的理由，我理解这一点。但这些公司牺牲的远比他们意识到的多。他们浪费了整个产品发布周期，他们创造的产品是客户不想要的，或者必须费力才能使用。

> The Way Out

出路

> The way out of these situations is to clearly define the distinct roles of product manager and product marketing in your company. The product manager is responsible for defining—in detail—the product to be built, and validating that product with real customers and users.

摆脱这些情况的出路是在你的公司明确界定产品经理和产品营销的不同角色。产品经理负责详细定义要构建的产品，并用真实客户和用户验证该产品。

产品营销人员负责向外界介绍该产品，包括定位、信息和定价，管理产品发布，为销售渠道提供营销和销售产品的工具，以及主导关键项目，如在线营销和大咖营销项目。

> The product marketing person is responsible for telling the world about that product, including positioning, messaging and pricing, managing the product launch, providing tools for the sales channel to market and sell the product, and for leading key programs such as online marketing and influencer marketing programs.

请注意，虽然我个人的重点是产品管理，但这种关注不应被误解为我认为产品营销角色不重要。恰恰相反。我了解到它很重要，伟大的产品营销极其有价值。但它与我这里描述的产品经理角色关系不大。

> Please note that while my personal focus is on product management, that focus should not be misconstrued as a belief that the product marketing role is unimportant. Quite the opposite. I have learned that it is important, and that great product marketing is extremely valuable. But it has little to do with the product manager role that I have described here.

一般来说，产品经理和产品营销人员会经常沟通，并在特定话题上偶尔合作，但主要有两个互动。首先，产品营销人员将是产品经理拥有的产品需求的几个关键输入来源之一。其次，产品经理将是产品营销拥有的营销信息的几个关键输入来源之一。

> In general, the product manager and product marketing person will communicate often and collaborate occasionally on specific topics, but there are two main interactions. First, the product marketing person will be one of several key sources of input to product requirements owned by the product manager. Second, the product manager will be one of the several key sources of input to marketing messages owned by product marketing. Regardless of the title or organizational model, I promise you that behind every great product you will find an individual who is responsible for the definition of that product. Remember: It doesn't matter how great your engineering organization is if the product manager doesn't give the engineers something valuable, usable and feasible to build.

无论头衔或组织模式如何，我向你保证，在每一个伟大产品的背后，你都会找到一个负责该产品定义的个人。

记住：如果产品经理不给工程师有价值、可用且可行的东西来构建，你的工程组织有多优秀都没用。

第 3 章：产品管理 vs. 项目管理

> Chapter 3: Product Management Vs. Project MANAGEMENT

互联网也改变了这一点

在上一章中，我谈到了明确区分产品管理和产品营销角色的重要性。许多公司遭受另一个相关问题：当产品管理和项目管理的角色被合并时。

如此多的互联网公司仍然将产品管理定义为包括项目管理，原因是我们许多实践来自可交付软件世界。在可交付软件世界（例如微软的 Office 软件产品），让产品经理兼任项目管理角色是很常见的。然而，虽然这可能适用于可交付软件，但这种方法在互联网上就行不通了。

要了解原因，首先要了解一点互联网历史。当互联网服务出现时，大约在1996年左右，起初我们纠结于是否继续称自己为产品经理，因为像旅游网站这样的东西似乎比传统的可交付软件产品更偏向服务导向。但我们很快就克服了这一点。

我们最初尝试让产品经理继续兼任项目经理角色。像网景（Netscape）和雅虎（Yahoo!）这样的早期互联网公司尝试了这种方法，但他们遇到了一个问题：在可交付软件世界中，产品通常被打包成一个独立的单元，一个发布包接着另一个发布包，往往间隔数月甚至数年。所以产品通常与项目具有相同的粒度和频率，这使得产品经理兼任项目经理相对容易。但在网络服务世界中，这种模式就崩溃了。

大多数互联网服务公司发现，他们需要更频繁地发布更小的版本到更大的共同代码库。而且由于一个典型的项目需要比一个发布周期（通常从每周到每月不等）更多的工作，这很快变成了并行开发和软件火车模式（train model）的发布。大多数超越创业阶段的互联网公司都使用这种火车模式。

火车模式本身就是一个话题。对本书来说最重要的一点是一个发布火车需要积极且强大的项目管理，这不是针对特定项目，而是针对整个发布。一个火车通常包含来自许多项目和产品经理的功能，它有重大的协调要求，如发布管理、工程开发、网站运营、客户服务，以及产品管理。一些互联网公司把发布火车的项目经理称为火车的"列车长"（conductor）。

如果你使用火车模式，并且有专职的项目经理负责发布火车，你通常不需要产品经理也兼任项目管理。

回到历史课。随着雅虎（Yahoo!）、网景（Netscape）、美国在线（AOL）等公司的发布流程随着产品和网站的发展变得更加复杂，项目管理职责从产品管理角色中分离出来，所有这些公司都发展出了非常强大且专职的项目管理能力。许多较新的互联网公司，如 eBay 和谷歌（Google），如果没有

他们非常强大的横跨产品管理、工程开发和网站运营的项目管理团队，就无法发布如此数量和质量都优秀的软件。

长话短说，对于互联网服务公司来说，重要的是将这些角色分开。如果不这样做，你会在发布管理中疲于奔命，发布将持续被延迟，耗时也会比应有的长。

如果你正在创建可交付软件，我仍然认为将角色分开是有用的。这更多是由于产品管理的性质，它关乎发现有价值、可用且可行的产品；而项目管理则是关于执行交付该产品。

什么造就了伟大的项目经理?

看看任何成功的公司，你会发现一群脱颖而出的人，他们真正使公司与众不同。这可能是一个伟大产品和一个糟糕产品之间的区别。或者是获得公司需要触达客户的商业伙伴关系，还是默默无闻地迷失的区别。或者是成功发布产品，还是让它陷入永久延迟的区别。

eBay 按任何人的定义都是一家非常成功的公司，它在这些领域和其他领域都有一些非常强大的人才。

eBay 有一个非常不寻常的产品开发流程，但这个流程有三个关键特点：极其高效、要求极高，而且这个流程建立在一个极其强大的项目管理能力之上。

为 eBay 建立这种项目管理能力的人是 Lynn Reedy，我有幸合作过的最优秀的项目管理专家。在我加入 eBay 之前，我以为我在项目管理方面还不错，但她向我展示了真正的标准在哪里。

在一些公司（例如微软的大部分），产品经理也负责运行部分或全部项目管理。我相信培养强大的项目管理技能对产品经理来说是一个巨大的优势。至少，你的

产品将更快上市，而且——最终——这可能决定你的产品是否能够发布。然而，我也主张产品经理和项目经理应该是分开的角色。

我认为大多数人把项目管理等同于 Microsoft Project。但这忽略了项目管理的真正要点。以下是我认为像 Lynn 这样伟大项目经理的七项特征技能：

紧迫感。Lynn 一走进房间就能立刻传达出一种紧迫感。会议前的闲聊最多 60 秒，然后就是正题。这在一定程度上是由于她独特的糖和咖啡因饮食，但事实上，紧迫感和效率是 eBay 文化的核心，而 Lynn 是这种文化的最佳体现。

会议框架师。漫无目的、毫无成效的会议有很多原因，但最大的罪魁祸首之一是与会者并不总是清楚会议的确切目的是什么，要解决什么问题，以及具体的问题或障碍是什么。伟大的项目经理懂得如何清晰简洁地识别和框定问题，并主持富有成效的会议。

清晰思考。典型的商业问题通常包括多个根本原因，加上大量的政治因素、个人议程和个性。这导致了如果不加以解决，就会拖延开发进度的混乱。项目经理需要把各个问题分开，理清情绪和包袱，以暴露根本问题，让每个人都专注于寻求解决方案。

数据驱动。伟大的项目经理理解数据在准确告知他们当前位置和目标位置方面的关键作用。他们不断寻求改进产品开发过程和结果，他们知道这始于测量。拍脑袋做决定太容易了——尤其是在时间紧迫的情况下——所以项目经理必须坚持数据，确保决策是建立在对事实的了解之上。

果断。在大多数组织模式中，产品团队成员实际上并不向项目经理汇报，但他或她必须推动决策。这就是项目经理必须传达紧迫感、清晰框定问题、拥有理性和透明的推理，并根据数据做出决策的地方。项目经理还需要知道何时适合收集团队的数据和建议，何时将问题升级给高级管理层。

判断力。以上很大程度上依赖于良好的判断力——知道何时推动、何时升级、何时获取更多信息，以及何时私下找某人谈谈。这种特质更难教，但经验可以帮助。

态度。最后，总是有数百个非常正当的理由说明产品为什么还没准备好发布——资源不够、时间不够、资金不够等等。项目经理的工作是克服每一个障碍。归根结底，伟大的项目经理是伟大的问题解决者。伟大的项目经理不找借口，她让它实现。她不知疲倦、势不可挡。

我真心相信，如果没有 Lynn 带到公司和文化的项目管理纪律，eBay 不会成为今天这样的成功企业。

> The Internet Changed This Too
>
> In the previous chapter, I wrote about how important it is to clearly distinguish the roles of product management and product marketing. Many companies suffer from another related problem: when the roles of product management and project management are combined. The reason so many Internet companies still define product management as including project management is because many of our practices came from the shipped software world. In the shipped software world (such as the Office software products from Microsoft became famous for), it is common to have product managers cover the project management role. However, while it might work for shipped software, this approach just doesn't migrate well to the Internet.
>
> To understand why, first a little bit of Internet history. When Internet services came about, around 1996 or so, at first we struggled with whether to continue to call ourselves product managers, because things like a travel Web site seemed more service-oriented than a traditional shipped software product. But we quickly got over that.
>
> We initially tried to continue having the product manager cover the project manager role. Early internet companies like Netscape and Yahoo! tried this approach but they ran into a problem: in the shipped software world, the product was generally packaged as a self-contained unit, with one release package serially following another often months or even years later. So the product generally was in the same granularity and frequency as the project, making it relatively easy for the product manager to double as the project manager. But in the Web services world, this model breaks down.
>
> Most Internet service companies found that they needed to make more frequent, smaller releases to a larger common code base. And since a typical project required more work than a release interval (usually ranging from weekly to monthly), this quickly turns into parallel development and the software train model of releases. Most Internet companies beyond the startup phase use this train model.
>
> The train model is really a topic unto itself. The most important point for this book is that a train requires active and strong project management which is not tied to specific projects, but rather to the release as a whole. A train typically contains features from many projects and product managers, and it has significant coordination requirements such as release management,

engineering, site operations, customer service, and product management. Some Internet companies refer to the project manager of a release train as the train's conductor.

If you use the train model, and you have project managers dedicated to the release trains, you generally don't need product managers to cover project management too.

Back to the history lesson. As the release process at companies like Yahoo!, Netscape, AOL, and others became more sophisticated as the products and sites grew, the project management responsibilities were untangled from the product management role, and all of these companies developed very strong and dedicated project management competencies. Many newer Internet companies such as eBay and Google could not release the quantity and quality of software they do without their very strong project management team spanning product management, engineering and site operations.

Long story short, for Internet services companies, it is important that the roles be separate. You'll thrash in release management if you don't, and releases will consistently be delayed and take longer than they should.

If you are creating shipped software, I still think it's useful to separate the roles. This is more due to the nature of product management, which is all about discovering a product that is valuable, usable and feasible; versus project management, which is all about executing to deliver that product.

What Makes A Great Project Manager?

Look at any successful company and you'll find a set of people who stand out and are the ones that really make the difference from other companies. It may be the difference between a great product ora terrible one. Or the difference between getting the business partnerships the company needs to reach its customers or getting lost in obscurity. Or the difference between getting the product out or having it stuck in perpetual delays.

eBay is by anyone's definition a very successful company, and it has some extremely strong people in each of these areas and more.

eBay has a very unusual product development process, but three key characteristics of this process stand out: it is extremely productive, extremely demanding, and it is a process predicated on an extremely strong project management competency.

The person that established this project management competency for eBay was Lynn Reedy, the very best project management mind I've ever had the privilege of working with. Before I joined eBay I thought I was pretty good at project management, but she showed me where the bar really was.

In some companies (much of Microsoft for example), the product manager is also responsible for running some or all of project management. I believe that developing strong project management skills is a big advantage for product managers. At the very least, your

product will get to market faster, and—ultimately—it could make the difference between getting your product shipped at all. However, I also argue that the product manager and the project manager should be separate roles.

> I think most people equate project management with Microsoft Project. But this is missing the real point of project management. Here are the seven skills that I think characterize great project managers like Lynn:
>
> Sense of urgency. Just by walking into the room Lynn would instantly convey a sense of urgency. Pre-meeting banter was maybe 60 seconds, and then it was down to business. Partly this was due to her unique diet of sugar and caffeine, but in fact a sense of urgency—and efficiency—is at the heart of the eBay culture and was best personified by Lynn.
>
> Framers. There are so many reasons for aimless, unconstructive meetings, but one of the biggest culprits is that it's not always clear to the participants exactly what the purpose of the meeting is, what problem is to be solved, and what the specific issues or obstacles are. Great project managers understand how to clearly and concisely identify and frame problems and run constructive meetings.
>
> Clear thinking. The typical business issue generally includes multiple underlying causes with a healthy dose of politics, personal agendas and personalities thrown in. This results in a murky confusion that if left unaddressed, delays development progress. The project manager needs to isolate the separate issues, and untangle the emotion and baggage to expose the underlying problem and get everyone focused on pursuing the solution.
>
> Data driven. Great project managers understand the key role that data plays in informing them about precisely where they are and where they need to go. They are constantly looking to improve the product development process and the result, and they know this begins with measurement. It is all too easy to just shoot from the hip—especially in time-sensitive situations—so it's essential for the project manager to insist on the data to make sure the decisions are made with the facts behind them.
>
> Decisiveness. In most organizational models, the members of the product team don't actually report to the project manager, yet he or she must drive decisions. This is where the project manager must communicate the sense of urgency, clearly frame the problem, have rational and transparent reasoning, and make decisions based on the data. The project manager also needs to know when it is appropriate to collect data and recommendations from the team, and when to escalate issues to senior management.
>
> Judgment. Much of the above hinges on good judgment—knowing when to push, when to escalate, when to get more information, and when to take someone aside and have a little private chat. This trait is harder to teach, but experience can help.
>
> Attitude. Finally, there are always hundreds of very valid reasons why a product isn't ready to ship—not enough resources, not enough time, not enough money, etc. The job of the project manager is to get over each and every one of these obstacles. At their core, great project managers are great problem solvers. The great project manager doesn't make excuses, she makes it happen. She is tireless and unstoppable.
>
> Itruly believe that eBay would not be the success it is today without the project management discipline Lynn brought to the company and the culture.

第 4 章：产品管理 vs. 设计

> Chapter 4: Product Management Vs. Design
>
> Understanding User Experience Design

理解用户体验设计

> Many product people complain to me that their company doesn't staff or even understand user experience design, and they know their product suffers for it. Most say that the UI engineers just do whatever they can and that, by default becomes the design. Sometimes it's the product managers who wade into the design waters and do whatever they can. And in other cases, companies try to outsource some visual design at the end of the product development process to add in pretty veneer just before the product goes into QA.

许多产品人员向我抱怨，他们的公司没有配备甚至不理解用户体验设计，他们知道产品因此受到影响。大多数人说，UI 工程师只是尽其所能，然后这默认就成为了设计。有时是产品经理涉足设计领域尽其所能。在其他情况下，公司试图在产品开发过程的最后外包一些视觉设计，在产品进入 QA 之前添加漂亮的外表。

> Others tell me that their company values good user experience design, but they don't really understand the roles or how a good design comes about.

其他人告诉我，他们的公司重视良好的用户体验设计，但他们并不真正理解这些角色或好的设计是如何产生的。

> This is a very serious problem that not enough companies are aware of.

这是一个非常严重的问题，没有足够多的公司意识到。

> It seems to me that the design community hasn't been doing enough to address this lack of recognition for their importance to a product. While they do a good job communicating among themselves (and there are some outstanding talents in the design community, including Mark Hurst, Hugh Dubberly, and Alan Cooper, to name a few), in general I think these guys spend a lot of time preaching to the choir. The message about the value they

在我看来，设计界在解决对其重要性缺乏认识的问题上做得还不够。虽然他们在彼此之间做得很好（设计界有一些杰出的人才，包括 Mark Hurst、Hugh Dubberly 和 Alan Cooper，仅举几例），但总的来说，我认为这些人花了很多时间在唱诗班布道。关于他们对产品价值的

> deliver is most needed by the teams without designers. One way to do this is to work on educating the wider product team about the need and benefits of designers to a product, especially the product managers.

信息最需要传达给没有设计师的团队。一种方法是向更广泛的产品团队宣传设计师对产品的需求和好处，特别是产品经理。

> The reason I care so much about this problem is simple. A good product requires a good user experience. And a good user experience requires the close collaboration of product management and user experience design.

我这么关心这个问题的原因很简单。好的产品需要好的用户体验。而好的用户体验需要产品管理和用户体验设计的紧密协作。

> This is a big topic, so let's start by getting on the same page in terms of what design includes. In this chapter I spell out what I consider the design-related roles essential to creating a good user experience. Note that I'm emphasizing roles rather than people, as it's possible to find people that can competently handle more than one role. But one way or the other you need these roles if you want a good user experience:

这是一个大话题，所以让我们从设计包含什么达成一致开始。在本章中，我阐述了我认为创造良好用户体验所必需的设计相关角色。请注意，我强调的是角色而不是人，因为有可能找到能胜任多个角色的人。但无论如何，如果你想要好的用户体验，你需要这些角色：

> Interaction Design. These people are responsible for developing a deep understanding of the target users and coming up with the tasks, navigation, and flow that are both valuable and usable. Generally, the interaction designer maps product requirements to a design represented by wireframes, and passes them to the visual designer.

交互设计。这些人负责深入理解目标用户，并提出既可用又有价值的任务、导航和流程。通常，交互设计师将产品需求映射到用线框图表示的设计，然后传递给视觉设计师。

> Visual Design. These people put the flesh on the wireframe and create the actual pages and user interface look and feel, which includes everything from the precise layout, colors, and fonts, but more importantly, the visual design communicates and evokes emotion in the product (which is far more important than you may think).

视觉设计。这些人给线框图添加血肉，创建实际的页面和用户界面外观感觉，包括从精确的布局、颜色、字体的一切，但更重要的是，视觉设计在产品中传达和唤起情感（这远比你可能认为的更重要）。

> Rapid Prototyping. The prototypes work to capture the ideas of the product manager and

快速原型。原型致力于将产品经理和设计师的想法捕获到可以在真实用户身上测试并迭代

> designers into a prototype that can be tested on real users, and iterated upon.

的原型中。

> Usability Testing. This person specializes in research and analysis of the users, evaluating whether products or prototypes allow a given user to easily achieve objectives. It includes recruiting appropriate test subjects, administering the tests, evaluating the results, and recommending alternatives.

可用性测试。这个人专门研究和分析用户，评估产品或原型是否允许特定用户轻松实现目标。包括招募合适的测试对象、管理测试、评估结果和推荐替代方案。

> The four design roles above work closely with the product manager to discover the blend of requirements and design that meet the needs of the user. The idea is to get to the point where the software is both usable (users can figure out how to use it) and valuable (users actually want to use it).

上述四个设计角色与产品经理紧密合作，发现满足用户需求的特性和设计的融合。目标是达到软件既可用（用户能弄清楚如何使用）又有价值（用户确实想使用）的程度。

> You will also need to ensure the software you're designing is feasible, so you need to have a software architect reviewing the progress and prototypes. More on this later.

你还需要确保你设计的软件是可行的，所以你需要让软件架构师审查进度和原型。更多内容稍后介绍。

> For large products—especially at consumer Internet service companies—you really do need all four roles represented on your team. If you're an enterprise company and you'd like to differentiate your product from your competition, one of the easiest ways to do this is to create a good user experience. As a general rule, most enterprise products are very weak in this respect.

对于大型产品——特别是消费互联网服务公司——你真的需要在团队中有这四个角色。如果你是企业公司，你想让自己的产品与竞争对手区分开来，最简单的方法之一就是创造良好的用户体验。一般而言，大多数企业产品在这方面非常薄弱。

> For smaller products, you may be able to double-up some of the roles. For example, I recently worked with a consumer Internet service startup in the Web 2.0 space which had assembled a terrific team of three: a product manager, an interaction designer (who also covered usability testing), and a visual designer (who also covered prototyping). This team of

对于较小的产品，你可能可以兼任一些角色。例如，我最近与 Web 2.0 领域的一家消费互联网服务初创公司合作，他们组建了一个很棒的三人团队：产品经理、交互设计师（同时也负责可用性测试）和视觉设计师（同时也负责原型）。这个

> three worked extremely well together to quickly come up with numerous prototypes that

三人团队合作得非常好，快速想出了许多原型，然后在目标用户身上进行了测试。

> they then tested with target users.

另一个重要的注意事项：许多公司意识到他们需要在这个领域做些什么，但他们认为可以将用户体验工作外包给设计公司。在某种程度上你可以，但要注意某些功能比其他功能更适合外包。例如，我不建议外包交互设计师角色，原因有三：

> One other important note: Many companies realize they need to do something in this area,

1. 真正培养对用户和客户必要理解需要时间，需要经历几个项目。大多数设计合同没有足够的时间来做这个，即使有，当下一个版本到来时，这些知识也会丢失；

> but they think they can outsource this user experience work to a design firm. To some

1. 交互设计师需要全程深度参与项目，从开始到发布。开发和测试期间会出现数百个详细问题——让交互设计师在那里立即做出正确决策至关重要；

> degree you can, but beware that certain functions are more appropriate than others. For example, I don't recommend outsourcing the interaction designer role because of these three reasons:

1. 产品的用户体验对公司来说太核心了，不应该外包。如果可以选择，外包 QA 是更好的选择。

> 1.It takes time, over the course of several projects, to truly develop the necessary understanding of users and customers. Most design contracts don't have the time to do that, and even if they do, that knowledge is lost when the next release comes up;

你可以外包视觉设计，因为有很多工作室可以做你需要的事情，特别是如果你有一个强大的交互设计师在职。你也可以外包用户研究和/或可用性测试，虽然这通常很贵，而且我非常喜欢非正式测试（见"原型测试"一章）。产品经理和交互设计师通常可以合作来完成这个。

> 1. The interaction designer needs to be on hand and deeply involved all the way through the project, from the beginning to launch. Hundreds of detailed questions will come up during development and test—having an interaction designer there to make the right decisions immediately is critical;

对于快速原型师，你可以从工程团队借一个开发人员，只要

> 1. The user experience of the product is simply too core to the company to not have in-house. Given the option, it's a better choice to outsource QA.

你非常清楚地向那个人说明，这完全不同于生产级编码，他或她不应该试图构建任何可以在以后的真实产品中重用的原型。事实上，他们应该把原型中的所有代码都视为一次性代码。

> You can get away with outsourcing visual design, as there are a number of studios that can

这个关键话题有很多内容无法在这简短的章节中涵盖，但希望这个讨论奠定了基础。这些角色中哪些目前在你的产品团队中已涵盖，哪些缺失？

> do what you need, especially if you have a strong interaction designer on staff. You can also outsource user research and/or usability testing, although it's often expensive and I'm a big fan of informal testing (see the chapter Prototype Testing). The product manager and interaction designer can often team up to cover this.
>
> For the rapid prototyper, you can borrow a developer from your engineering team, as long as you make very clear to that person that this is completely different from production-level coding, and that he or she should not try to build a prototype where any of it can be reused later in the real product. In fact, they should consider all the code in a prototype as throw-away.
>
> There's a great deal more to say on this critical topic than can be covered in this brief chapter, but hopefully this discussion lays the foundation. Which of these roles are currently covered within your product team and which ones are missing?

第 5 章：产品管理 vs. 工程

> Chapter 5: Product Management Vs. Engineering
>
> Building The Right Product Versus Building The Product

构建正确的产品 vs. 正确地构建产品

如果一个伟大的产品是真实客户需求与刚刚成为可能性的解决方案的结合，那么很容易看出为什么产品经理和工程团队之间的关系如此关键。

产品经理负责定义解决方案，但工程团队最清楚什么是可能的，他们必须最终交付该解决方案。作为产品经理，你很快就会学到，如果你与工程有良好的关系，那么这份工作可以是伟大的。如果你没有，只能说你会经历一些非常漫长和沮丧的日子。

这种关系的一个关键是你们每个人都要明白你们是平级的——任何一方都不从属于另一方。作为产品经理，你负责定义正确的产品，你的工程对应方负责正确地构建产品。你们两个都需要。你需要让你的工程对应方做他认为必要的任何事情来构建高质量产品，他也需要给你空间来提出有价值且可用的产品。

双方可以互相提供巨大帮助。具体来说，工程师可以在你致力于发现成功产品方面提供很大帮助。记住，他们通常比任何人都更清楚什么是可能的。

以下是你如何利用工程师来提出更好产品的三种方法：

1. 让你的工程师接触用户和客户。他们不仅会从亲眼目睹用户挣扎中学到很多，而且会对问题及其严重性有更好的理解。这往往是更好想法和解决方案的灵感来源。你可以通过邀请工程师参加你的原型测试来轻松启动这一点。
2. 在技术发展过程中，争取工程师的帮助来探索什么正在成为可能。集思广益讨论可用或即将可用的不同技术，以及它们如何帮助解决手头的问题。

3. 从产品发现过程的最开始就让你的工程师（或至少一名首席工程师）或架构师参与，以便尽早评估不同想法的相对成本，并帮助识别更好的解决方案。产品经理最常犯的错误之一是想出一个伟大的产品定义，然后把它扔给工程部门。这只会推迟想要的与可能的之间的关键协商过程，直到没有足够的时间来做出良好和明智的决策。

同样，你实际上可以在很大程度上帮助你的工程师。以下是你可以帮助他们完成工作的三种方法：

1. 专注于最小化产品。稍后更多介绍，但作为产品经理，你的工作

不是定义最终产品，而是定义能够满足目标的最小可能产品。这一点本身将从根本上改善产品管理和工程之间的动态关系。

1. 尽一切努力减少工程开始开发产品后的变更。变更是指更改需求和产品定义。一些变更是不可避免的，工程师也明白有些事情超出了你的控制，但请记住，这不是尝试你最新最棒想法的时候。
2. 在实施过程中不可避免地会出现问题，包括遗漏或用例没有完全考虑清楚的情况。即使在最好的产品团队中，这也是正常的。然而，你作为产品经理在实施阶段的使命是尽快解决他们的问题，始终专注于最小化产品和减少变更。

考虑到所有这些，我总是鼓励最优秀的工程师来尝试产品管理。我提醒他们，如果团队没有被赋予值得构建的东西，工程再伟大也没用。我指出许多伟大的产品和公司都是由一个知道什么是可能的并解决构建什么这个更大问题的工程师创造的。这对他们的职业发展很有好处，也可能对产品（因此对客户和公司）很有好处。

如何与远程开发人员合作成功?

今天最常见的情况之一是产品经理在一个地方，而工程团队在另一个地方。我不仅仅是指外包到印度。远程开发团队可能来自收购或合并，或者你的组织足够大，开发人员位于你不在的设施中。

当开发人员不坐在你旁边时，所有正常的沟通和执行挑战都会被放大，往往达到许多团队对远程开发感到极度沮丧的程度，一些成员公开质疑所谓的成本节约的好处。

如果你发现自己有一个远程开发团队，有三件关键的事情可以大大提高你成功的机会：

1. 团队离你越远——语言、文化和时区的沟通挑战越大——确保你在产品规格上做得非常彻底的压力就越大。噩梦般的情况是产品经理不确定他想要什么（或不断改变主意），而远程工程团队在打转。就像被鞭子抽打一样——这是失败的配方。

在"重新定义产品规格"一章中，我写了高保真原型作为产品规格基础的重要性。我不会在这里重复所有的原因，但

suffice 说，如果你的团队是远程的，你绝对要确保使用高保真原型作为你的主要沟通机制——既用于沟通实际规格，也用于沟通变更。书面文档已经很难让人去读了，但如果它是用非母语写的，而且作者不在走廊尽头的隔间里可以澄清问题，那你就是在自找大麻烦。

1. 当开发团队是本地的时候，资源冲突（例如，两个不同的经理给出矛盾的指令）通常会被快速发现并解决。对于远程团队，你可以预期很多不愉快的意外，而且 literally 可能过去几个月后才发现脱节。这通常是因为远程开发人员被迫对想要什么以及如何解释各种指令做出假设（当然，假设并不总是正确的）。让某人在当地管理与远程团队的所有协调是至关重要的。这不意味着所有沟通都必须通过这个人，但工程团队对谁负责应该没有疑问。有时这是项目经理，有时是在当地（靠近产品经理）的工程总监或副总裁。
2. 当今大多数企业内部都有许多良好的沟通机制可用。除了电子邮件和即时消息外，视频会议技术已经有了很大改进，VoIP 大大降低了国际电话的成本。也就是说，仍然没有什么可以替代建立个人面对面关系。至少每季度一次，产品经理应该出去与工程团队共度一些高质量的面对面时间，与关键架构师和经理会面。这些面对面的访问将改善关系和沟通。另一个很好的沟通建设

技巧是让关键开发人员来与产品经理住一段时间。

有了一个才华横溢的远程团队，并按照我描述的方式管理关系，你实际上可以享受这种安排。特别是对于在印度的工程师，时差可以让每天早上你上班时都能看到进展，你可以把白天（和他们的夜晚）花在审查、测试和提供反馈上。结果可以是极快的周期时间。

请注意，你也可以将原型资源放在远程位置，但你需要在沟通上更努力，并在时间上更灵活，因为周期时间非常快（每天几次迭代）。

处理远程开发团队问题的另一个解决方案是将整个产品团队一起放在远程位置。我看到这一趋势刚刚开始，我认为它会增长。也就是说，你暂时不必担心这个。在这些远程地点开发工程和 QA 能力花了 10 年时间，可能还需要再过 10 年，这些地点才会有熟练的产品经理和设计师。

那外包呢?

现在几乎我交谈的每家公司都在某种程度上进行外包。然而结果却是好坏参半。我认为公司遇到问题有几个原因。问题通常源于产品开发流程的问题，或语言或文化问题，但更多时候我认为核心问题源于出于错误的原因使用外包。

如果你试图创造令人启发的产品，对于大多数专业职位，外包不应该是为了节省成本——它应该是为了为产品组建合适的人员。很多时候，你不得不超越你直接的地理区域寻找最优秀的人才。这可能意味着雇用住在另一个州或另一个国家的人。

关于硅谷的悲哀真相是，它已经变得如此昂贵，以至于许多你想雇用的人无法以你能负担得起的薪水在这里生活。通勤只能在一定程度上解决。最终人才供应会耗尽，你不得不去其他地方寻找合适的团队。

幸运的是，在一些地方有杰出的产品人才，如印度、东欧（特别是捷克共和国、匈牙利、波兰和斯洛伐克）、北欧（特别是荷兰、瑞典和德国）、以色列、中国、新加坡、澳大利亚和新西兰。我在这些地方的每一个都认识一些了不起的人。

我有幸领导的最好的团队之一有成员分布在瑞典、硅谷、波士顿和印度。我们正在构建需要支持超过 2000 万用户的基础设施，如果没有这些特定个人的才能，我们是否能成功还很不清楚。

我最喜欢的公司之一是 MySQL，这家公司多年来一直秉承这种理念。他们名义上位于硅谷和瑞典，但他们的产品团队甚至高管遍布全球。他们是一个真正的虚拟组织，他们能够受益于一些最优秀的数据库和系统软件人才。管理一个完全分布式的产品团队对他们来说并不容易，但我敢争辩，如果他们选择了地球上的一个单一地点，并试图成为一个典型的集中式公司，他们很可能无法取得他们所取得的成就。

就像制造业工作在 80 年代被迫离开硅谷一样，今天有几种其他类型的工作正在转移出去——特别是客户服务、QA，以及在较小程度上，工程。让架构师和 QA 经理位于公司总部，以及产品管理和设计，然后让团队的其他人一起或分散在全球各地，这正变得越来越普遍。

关键是要意识到，一切都是关于团队和组成它的个人的素质。许多经理不太理解这一点，当他们得知同一工作类别的员工之间可能存在 20 倍的生产力差异时，他们感到震惊。

哪一个会做得更好——由五个根据其 proven 技能选择的非常优秀的人组成的团队，还是

基于位置雇用或组建的 15 人团队？这个生产力因素很容易超过感知的财务节约。同样，它可以让住在昂贵城市斯德哥尔摩的顶尖工程师变成一笔划算的交易。

还有其他因素在起作用，但我坚信一切都始于正确的产品团队，如果你的产品团队决定你需要外包，我希望你是出于正确的原因——为你的产品团队找到合适的人，而不是仅仅因为你想省几块钱。

工程团队想要重写！？！

对于产品经理来说，没有什么比听到工程团队说"不要再加新功能了！我们需要停下来重写！我们的代码库一团糟，它跟不上用户数量，它是一纸牌屋，我们不能再维护它或保持它运行了！"更令人恐惧的了。

这种情况过去发生在许多公司，今天继续发生。它发生在 1999 年的 eBay，公司离崩溃比大多数人意识到的要近得多。它发生在几年前的 Friendster，为 MySpace 接管社交网络打开了大门。它发生在网景与微软的浏览器大战期间，每个人都知道谁赢了。事实是，大多数公司永远无法恢复。

当一家公司真的陷入这种情况时，公司通常会责怪工程。但根据我的经验，残酷的事实是，这通常是产品管理的错。原因是，当这种情况出现时，通常是因为几年来，产品经理一直在 pounding 工程组织，要求工程团队尽可能多地交付功能。结果是，在某个时刻，如果你忽视基础设施，所有软件都会达到无法再支持其需要的功能的程度。

在这次重写期间，你被迫停止客户所看到的前进进度。你可能认为重写只需要几个月（下面更多内容），但

invariably 它需要更长的时间，你被迫袖手旁观，看着你的客户离开你去选择你的竞争对手，而他们在此期间继续改进他们的产品。

如果你还没有达到这种情况，以下是你需要做的以确保你永远不会这样做——你需要将一定比例的工程能力分配给我们在 eBay 称之为"余量"（headroom）的东西。由于快速增长带来的许多问题与规模有关，余量背后的想法是避免撞到天花板。你通过为用户增长、交易增长和功能增长创造空间来做到这一点；本质上，保持产品的基础设施能够满足组织的需求。

与工程的交易是这样的：产品管理将团队 20% 的能力从顶部拿走，交给工程按照他们认为合适的方式使用。他们可能用它来重写、重新架构或重构代码库中有问题的部分，或者更换数据库管理系统，提高系统性能——无论他们认为什么是必要的，以避免不得不对团队说"我们需要停下来重写。"

如果你今天的状况真的很糟糕，你可能需要把这提高到 30% 甚至更多的资源。然而，当我发现团队认为他们可以远低于 20% 时，我会感到紧张。

如果你目前处于这种情况，事实是，你的公司可能无法生存。但如果你要有机会渡过难关，以下是你需要做的：

第 1 步：为工程识别的必要变更制定一个现实的时间表和时间线。在正常开发项目中，经验丰富的工程团队通常会给出相当准确的估算。这一规则的例外情况是重写的情况——这里的估算往往过于乐观——主要是因为很少有团队对真正的重写有任何实际经验。你必须在这种情况下做出明智的决定，所以你必须逐行检查时间表，确保日期是现实的。

第 2 步：如果 humanly 有可能将重写分解成块，与用户可见的产品开发一起在网站上增量完成，你绝对应该这样做。即使重写现在可能需要两年而不是九个月，如果你能想办法继续在用户可见的功能上取得进展——即使只有 25% 到 50% 的能力——这对产品保持在市场中的相关性非常重要，特别是在快节奏的互联网领域。

第 3 步：由于你交付用户可见功能的能力非常有限，你需要选择正确的功能，并确保你正确定义它们。

在 eBay 的濒死经历之后，团队确保他们不会再次让公司处于危险之中。这意味着立即开始另一次重写，这次是在问题出现之前 well in advance。事实上，由于他们的快速增长，eBay 最终第三次重写，这次是将整个网站翻译成不同的编程语言和架构。他们在几年内完成了这个庞大的、数百万行的重写，同时设法交付了创纪录的新功能——最重要的是——没有影响用户群。这是在飞行中重建引擎的最令人印象深刻的例子

在飞行中重建引擎是我所知道的最令人印象深刻的例子。

处理这种情况的最佳策略是不要到达这一点。你需要支付你的税费，并记住至少 dedicating 20% 给余量。如果你还没有和你的工程对应方讨论过这个问题，今天就做。

> Right
>
> If a great product is the result of combining a real customer need with a solution that's just now becoming possible, then it's easy to see why the relationship between the product manager and the engineering team is so critical.

The product manager is responsible for defining the solution, but the engineering team knows best what's possible, and they must ultimately deliver that solution. As a product manager, you'll quickly learn that if you have a good relationship with engineering, then the job can be a great one. If you don't, let's just say that you're in for some very long and frustrating days.

One key to this relationship is for each of you to understand that you are peers—neither position is subordinate to the other. As product manager, you are responsible for defining the right product, and your engineering counterpart is responsible for building the product right. You need both. You need to let your engineering counterpart do what he believes necessary to build a quality product, and he needs to give you the room to come up with a

valuable and usable product.

Both sides can be a huge help to each other. Specifically, the engineers can be a big help to you as you work to discover a winning product. Remember that they generally know what's possible better than anyone else.

Here are three ways you can use your engineers to come up with a better product:

1. Get your engineers in front of users and customers. Not only will they learn a great deal from seeing users struggle first hand, but they will get a better appreciation for the issues and their severity. This is often the inspiration for much better ideas and solutions. You can jumpstart this easily by inviting an engineer along to your prototype testing.
2. Enlist the help of your engineers in exploring what's becoming possible as technology develops. Brainstorm the different technologies that are available or coming available, and how they might help solve the problems at hand.
3. Involve your engineers (or at least a lead engineer) or architect from the very beginning of the product discovery process to get very early assessments of relative costs of the different ideas, and to help identify better solutions. One of the most common mistakes that product managers make is to come up with a great product definition, and then throw it over the wall to engineering. That just postpones the critical negotiation process of what's wanted vs. what's possible until there's not enough time to be able to make good and informed decisions.

Similarly, you can actually be a big help to your engineers. Here are three ways you can help them do their job:

1. Keep the focus on minimal product. More on this later, but your job as product manager

is not to define the ultimate product, it's to define the smallest possible product that will meet your goals. This point alone will fundamentally improve the dynamic between product management and engineering.

1. Do everything you can to minimize churn once engineering begins to develop the product. Churn is changing requirements and product definition. Some churn will be unavoidable, and engineers understand that some things are beyond your control, but remember that this is not the time for trying out your latest-and-greatest ideas.
2. There will inevitably be questions that arise during implementation, including use cases that were missed or weren't completely thought through. This is normal, even in the best

of product teams. However, your mission as the product manager during the implementation phase is to jump on their questions and get answers as fast as humanly possible, always keeping the focus on minimal product and minimizing churn.

With all this in mind, I always try to encourage the best engineers to come try their hand at product management. I remind them that it doesn't matter how great the engineering is if the team is not given something worthwhile to build. And I point out the many great products and companies that have been created by an engineer who knew what was possible and tackled the bigger question of what to build. It's great for their career development, and can be great for the product (and therefore the customers and the company).

How Do We Succeed With Remote Developers?

One of the most common situations today is where the product manager is in one location, and the engineering team is somewhere else. I don't only mean outsourcing to India, either. The remote development team might emerge from an acquisition or merger, or possibly your organization is large enough where the developers are located in a facility you are not. When the developers are not sitting right next to you, then all of the normal challenges of communication and execution are magnified, often to the point where many teams get extremely frustrated with remote development, and some members openly challenge the benefits of the purported cost savings.

If you find yourself with a remote development team, there are three key things you can do to dramatically improve the odds of your success:

1. The farther away the team is from you—and the more the communication challenges of language, culture and time zones—the more pressure there is to ensure that you have done a very thorough job on the product spec. The nightmare project is where the product manager isn't sure what he wants built (or keeps changing his mind) and the remote engineering team is thrashing. It's like being on the wrong end of a whip—and a recipe for failure.

In the chapter Reinventing the Product Spec, I write about the importance of a high-fidelity prototype as the basis of the product spec. I won't repeat all the reasons for that here, but

suffice it to say that if your team is remote, you absolutely want to make sure you use the high-fidelity prototype as your main communication mechanism—both for communicating the actual spec and for communicating changes. Written documents are hard enough to get people to read, but if it's written in a language that isn't native, and if the author isn't in the cube down the hall to clarify questions, you're asking for big trouble.

1. When a development team is local, resource conflicts (for example, two different managers give conflicting instructions) are typically caught and resolved quickly. With remote teams, you can expect lots of unpleasant surprises, and literally months can pass before the disconnects are identified. This is usually because the remote developers are forced to make assumptions about who wanted what and how to interpret the various instructions (and, of course, the assumptions are not always correct). It is critical to have someone local manage all coordination with the remote team. This doesn't mean that all communication must be funneled through this person, but there should be no question to whom the engineering team is accountable. Sometimes this is a project manager and

sometimes this is a director or VP of engineering who is based locally (near the product manager).

2. There are many good communication mechanisms available within most businesses today. In addition to e-mail and instant messaging, video conferencing technology is much improved, and VoIP has brought the costs way down for international calls. That said, there still is no substitute for establishing personal face-to-face relationships. At least once a quarter, the product manager should get out and spend some quality face time with the engineering team, meeting with the key architects and managers. These face-to-face visits will improve relationships and communication. Another great communication-building

technique is to have an exchange program where key developers come stay with the product manager for a while.

With a talented remote team, and managing the relationship as I've described, you can actually come to enjoy this arrangement. Especially with engineers based in India, the time difference can make it such that every morning when you come in to work, you see progress waiting for you, and you can spend your day (and their night) reviewing, testing, and providing feedback. The result can be extremely fast cycle times.

Note that you can also have your prototyping resources in a remote location, but you'll have to work a little harder on the communication and be more flexible on your hours because of the very quick cycle times (several iterations a day).

Yet another solution to the problems of dealing with a remote development team involves locating the entire product team together in the remote location. I see this trend just starting, and I think it will grow. That said, you don't have to worry about this just yet. It has taken 10 years to develop the engineering and QA capabilities in these remote locations, and itll likely take another 10 before these locations have skilled product managers and designers as well.

what About Outsourcing?

Just about every company I talk to now is outsourcing to one degree or another. Yet the results are decidedly mixed. I think there are several reasons for the problems that companies are having. Often the problems stem from issues with the product development process, or from language or cultural issues, but more often than not I think the core issue stems from using outsourcing for the wrong reasons.

If you're trying to create inspiring products, for most professional positions, outsourcing should not be about cost savings—it should be about assembling the right people for the product. Very often, you'll have to look beyond your immediate geographic vicinity for the best people. This might mean hiring someone that's based in another state, or in another country.

The sad truth about Silicon Valley is that it has become so expensive that many of the people you would like to hire can't afford to live here at the salary you can afford to pay them. Commuting only works to a degree. Eventually the talent supply runs out and you have to look elsewhere for the right team.

Fortunately, there are some terrific sources of outstanding product talent in places such as India, Eastern Europe (especially the Czech Republic, Hungary, Poland, and Slovakia), Northern Europe

(especially the Netherlands, Sweden, and Germany), Israel, China, Singapore, Australia, and New Zealand. I know some amazing people in every one of these

locations. One of the best teams I ever had the privilege to lead had members spread across Sweden, Silicon Valley, Boston, and India. We were building infrastructure that needed to support more than 20 million users and it's not at all clear we could have succeeded without the talents of the specific individuals involved.

One of my favorite companies is MySQL, which is a company that has embodied this philosophy for years. They are nominally based in Silicon Valley and Sweden, but their product team and even their executives are scattered all over the globe. They are a true virtual organization, and they are able to benefit from some of the very best database and system software talent to be found anywhere. It is not easy for them to manage a completely distributed product team, but I'd argue that very likely they would not have been able to accomplish what they have if they had picked a single location on the globe, and tried to be a typical centralized company.

Just as manufacturing jobs were forced out of Silicon Valley in the ‘80s, several other types of jobs are moving out today—especially customer service, QA, and to a somewhat lesser extent, engineering. It's becoming common to have the architects and QA managers located at the company's headquarters, along with product management and design, and then to have the rest of the team at locations either together or dispersed around the globe.

The key is to realize that it's all about the team and the caliber of the individuals it consists of. Many managers don't quite get this, and they are stunned to learn that there can be as much as a 20X difference in productivity among their staff in the same job class.

Which will do better—the team that has five very strong people chosen for their proven

skills, or the team of 15 that was hired or assembled based on their location? This productivity factor can easily dwarf perceived financial savings. Similarly, it can turn a top engineer living in the very expensive city of Stockholm into a bargain.

There are additional factors that come into play, but it's my firm belief that everything begins with the right product team, and if your product team decides you need to outsource, I hope you do it for the right reasons—to get the right people for your product team and not just because you think you'll save a few bucks.

Engineering Wants To Rewrite!?!

Few words are more dreaded by product managers than being told by engineering: “No more new features! We need to stop and rewrite! Our code base is a mess, it can't keep up with the number of users, it's a house of cards, we can no longer maintain it or keep it running!”

This situation has happened to many companies in the past, and continues to happen today. It happened to eBay in 1999, and the company came far closer to collapsing than most people ever realized. It happened to Friendster a few years ago, opening the door for MySpace to take over social networking. It happened to Netscape during the browser wars with Microsoft, and everyone knows who won. The truth is that most companies never recover.

When a company does get into this situation, the company typically blames engineering. But in my experience, the harsh truth is that it's usually the fault of product management. The reason is that when this comes up, it's usually because for several years, product managers have been pounding

the engineering organization to deliver as many features as the engineering team possibly can produce. The result is that at some point, if you neglect the infrastructure, all software will reach the point where it can no longer support the functionality it needs to.

During this rewrite, you're forced to stop forward progress for what the customers see. You might think that the rewrite will take only a few months (more about that below), but

invariably it takes far longer, and you are forced to stand by and watch your customers leave you for your competitors, who in the meantime, are continuing to improve their product.

If you haven't yet reached this situation, here's what you need to do to make sure you never do—you need to allocate a percentage of your engineering capacity to what at eBay we called "headroom." Since many of the issues you run into with rapid growth have to do with scale, the idea behind headroom is to avoid slamming into ceilings. You do this by creating room for growth in the user base, growth in transactions, and growth in functionality; essentially, keep the product's infrastructure able to meet the organization's needs.

The deal with engineering goes like this: Product management takes 20% of the team's capacity right off the top and gives this to engineering to spend as they see fit. They might use it to rewrite, re-architect, or re-factor problematic parts of the code base, or to swap out database management systems, improve system performance—whatever they believe is necessary to avoid ever having to come to the team and say, "we need to stop and rewrite."

If you're in really bad shape today, you might need to make this 30% or even more of the resources. However, I get nervous when

I find teams that think they can get away with much less than 20%.

If you are currently in this situation, the truth is that your company may not survive. But if you are to have a chance of pulling through, here's what you'll need to do:

Step 1: Do a realistic schedule and timeline for making the necessary changes that

engineering identifies. Most of the time in a normal development project, an experienced engineering team will come up with fairly accurate estimates. The exception to this rule is this case of rewrites—here the estimates are often wildly optimistic—largely because few teams have any real experience with true rewrites. You must make informed decisions in this situation, so you'll have to go through every line item on the schedule to make sure that the dates are realistic.

Step 2: If there's any way humanly possible to break up the rewrite into chunks to be done incrementally with user-visible product development continuing on the site, you should absolutely do so. Even though the rewrite might now stretch over two years instead of nine months, if you can find a way to continue to make forward progress on user-visible functionality—even if it's only with 25% to 50% of the capacity—this is incredibly important for the product to stay relevant in the marketplace, particularly in the fast-paced Internet space.

Step 3: Since you'll only have very limited ability to deliver user-visible functionality, you will need to pick the right features, and make sure you define them right.

After eBay's near-death experience, the team made sure they wouldn't put the company at tisk again. This meant immediately beginning another rewrite, this time well in advance of issues. In fact, due to their very rapid growth, eBay ended up rewriting a third time, this time translating the entire site into a different programming language and architecture. And they did this massive,

> multi-million-line rewrite over several years while at the same time managing to deliver record amounts of new functionality and—most importantly—without impacting the user base. It's the most impressive example of rebuilding the engine in
>
> mid-flight that I know of.
>
> The best strategy for dealing with this situation is to not get to this point. You need to pay your taxes and remember to dedicate at least 20% to headroom. If you haven't had this discussion with your engineering counterpart, do so today.

第 6 章：招聘产品经理

> Chapter 6: Recruiting Product Managers
>
> Finding Great Product Managers

寻找伟大的产品经理

> Probably the single most common question I get from CEOs is this: Where can I find great product managers?

可能我从 CEO 那里得到的唯一最常见的问题是：在哪里可以找到伟大的产品经理?

> I tell them that often the great product managers they're looking for are already in their organization, hiding under a different title-maybe a software engineer, user experience designer, or a Systems Engineer (SE), just waiting to be discovered. But whether you recruit product managers from inside or outside, the easiest way to spot them is to have a clear understanding of the characteristics to look for. So in this chapter I'll enumerate the specific traits and skills you're looking for.

我告诉他们，他们正在寻找的伟大产品经理往往已经在他们的组织中，隐藏在不同的头衔下——可能是软件工程师、用户体验设计师或系统工程师（SE），只是等待被发现。但无论你是从内部还是外部招聘产品经理，发现他们的最简单方法是对要寻找的特征有清晰的理解。所以在本章中，我将列举你要寻找的具体特质和技能。

> Personal Traits and Attitude
>
> Most skills can be learned, however, there are some traits that are very difficult to teach, and as such they should form the foundation of any search for a product manager.
>
> Product Passion

个人特质和态度

> There are some people out there who just love products—they live, eat, and breathe them. Great product managers have a love and respect for good products, no matter where they come from, and they live to create them.

大多数技能都可以学习，然而，有一些特质很难教，因此它们应该构成任何产品经理搜索的基础。

> This passion for product is an essential ingredient as it will often be called upon to provide the motivation to get through the many very difficult challenges—and long hours—of defining a great product. Further, the product manager will need to inspire the rest of the product team, and the passion for a product is contagious.
>
> It is fairly easy to determine whether or not you are talking to such a person by simply asking them what some of their favorite products are and why. It is hard to feign passion—the insincerity comes through loud and clear. Ask for examples from different domains. Ask what they would improve on their favorite product if they were the product manager. Ask about bad products too.
>
> Customer Empathy

产品热情

> The ideal product manager does not necessarily have to come from your target market (there are pros and cons to this), but they absolutely need to be able to empathize with your target market. This trait is often difficult to find in high-technology companies trying to produce mass-market products. We tend to want to think of our users as we think of ourselves and our friends. However, the target market very likely has quite different values, priorities, perceptions, tolerances, experiences, and technical understandings.

有些人就是热爱产品——他们生活、吃饭、呼吸产品。伟大的产品经理对好产品有着热爱和尊重，无论它们来自哪里，他们活着就是为了创造它们。

> Ask the candidates about the target market and how they believe they might be different from themselves. Try and detect how the candidate feels about the target market and, most importantly, does the candidate respect and empathize with that target market? Or does he view his job as “enlightening” the target market?

这种对产品的热情是一种必不可少的成分，因为它经常被用来提供动力，以度过定义伟大产品的许多非常困难的挑战——和长时间的工作。此外，产品经理需要激励产品团队的其他成员，而对产品的热情是会传染的。

> This is doubly important for international products, or those products targeted at specific countries or cultures. There are many similarities and, more importantly, many differences between cultures. Many of the differences are incidental and not important to defining products. However, some of the differences are essential. Does the candidate you are talking to have enough understanding of the target market to know which is which?

通过简单地问他们一些他们最喜欢的产品是什么以及为什么，很容易确定你是否正在和这样的人交谈。假装热情很难——不真诚会很明显地显露出来。要求来自不同领域的例子。问如果他们是产品经理，他们会改进他们最喜欢的产品的什么地方。也问问糟糕的产品。

> Intelligence

客户同理心

> There is really no substitute for innate intelligence. Product management is about insights and judgment, both of which require a sharp mind. Hard work is also necessary, but for this job, it is not sufficient.

理想的产品经理不一定必须来自你的目标市场（这有利有弊），但他们绝对需要能够同情你的目标市场。这种特质在试图生产大众市场产品的高科技公司中往往很难找到。我们倾向于希望像我们看待自己和我们的朋友一样看待我们的用户。然而，目标市场很可能有相当不同的价值观、优先级、观念、容忍度、经验和技术理解。

> Hiring very smart people is harder than it sounds. Much depends on the strength and security of the hiring manager. You've probably heard the old adage about A's hiring A's, and B's hiring C's. It's true. Hiring smart people speaks to the company culture which is another important topic in its own right, but suffice it to say here that if your goal is a truly great product, it is simply not going to happen if you can't find a truly bright product manager.

问候选人关于目标市场以及他们认为自己可能与目标市场有何不同。试着察觉候选人对目标市场的感受，最重要的是，候选人是否尊重并同情那个目标市场？还是他认为他的工作是"启发"目标市场？

> Assuming you are anxious to find the brightest, most insightful person possible, one technique is to drill your candidates on their ability to problem solve. Microsoft is famous for their very intensive and effective interviewing for intelligence based on problem solving. The technique is to use one or more experts in some topic to drill the candidate on a problem. The interviewer is not looking so much at whether or not the candidate simply knows the right answer (knowledge rather than intelligence), but rather, how well they deal with not knowing the answer. How does the candidate work out problems? What is the thought process? When the candidate comes up with a solution, the interviewer changes the question somewhat and asks what the candidate would do then. This is done continuously until the candidate is eventually forced to deal with a scenario he or she doesn't know the answer to. Then the candidate is asked to verbalize how he or she would go about solving that problem. With practice, this can be a very effective technique in assessing a candidate's problem solving capability.

对于国际产品，或针对特定国家或文化的产品，这一点尤为重要。文化之间有许多相似之处，更重要的是，有许多差异。许多差异是偶然的，对定义产品不重要。然而，有些差异是至关重要的。你正在交谈的候选人是否对目标市场有足够的了解，知道哪些是哪些？

> Another approach is to ask two or three people in your organization who are well known for their intellectual prowess, to interview this person, and help you determine the candidate's problem-solving ability.
>
> Work Ethic

智力

> Not every role in the product team requires the same level of commitment and effort. However, the product manager role is not for someone who is afraid of hard work. It comes along with the responsibility; the product manager is the person ultimately held accountable

天生的智力真的没有替代品。产品管理关乎洞察力和判断力，两者都需要敏锐的头脑。努力工作也是必要的，但对于这份工作，光努力是不够的。

> for the success of the product, and this burden weighs heavily on the successful product manager.

雇用非常聪明的人比听起来更难。很大程度上取决于招聘经理的实力和安全感。你可能听过一句古老的格言：A 级人才雇用 A 级人才，B 级人才雇用 C 级人才。这是真的。雇用聪明人体现了公司文化，这本身就是另一个重要话题，但 suffice 说，如果你的目标是真正伟大的产品，如果你找不到真正聪明的产品经理，它就不会发生。

> Even when skills such as time management and the techniques of product management are mastered, the successful product manager is still consumed with the product. Can you have a family and a non-work life and be a successful product manager? I believe you can. At least once you have some experience. But there are many people that want to be able to work 40 hours a week and—most importantly—leave their work problems at the office when they go home at the end of the day. This unfortunately is not the life of a successful product manager.

假设你渴望找到最聪明、最有洞察力的人，一种技巧是深入考察候选人解决问题的能力。微软以基于问题解决的对智力的非常密集和有效的面试而闻名。这种技巧是使用一个或多个某个主题的专家来深入考察候选人的问题。面试官不太关注候选人是否简单地知道正确答案（知识而非智力），而是更关注他们如何处理不知道答案的情况。候选人如何解决问题？思考过程是什么？当候选人想出一个解决方案时，面试官稍微改变一下问题，问候选人然后会怎么做。这持续进行，直到候选人最终被迫处理一个他或她不知道答案的情况。然后候选人被要求口头描述他或她会如何解决这个问题。通过练习，这可以成为评估候选人问题解决能力的非常有效的技巧。

> I believe in being very frank with new candidate product managers about the level of effort required for successful product management. It is not about requiring the product manager to work certain hours. If you have to actually ask or tell the product manager to come in to work during a critical point or otherwise point out to them that their presence is needed in the office, you have the wrong person for the job.

另一种方法是让你组织中两三个以智力著称的人来面试这个人，帮助你确定候选人的问题解决能力。

> Keep in mind that the level of effort and commitment is not uniform throughout the lifecycle of the project. There are certain phases that are much more intense than others. What won't change for the successful product manager is the degree to which they care and worry about their product and the lengths they are willing to go to ensure its success.
>
> Integrity

工作态度

> Of all the members of the product team, the product manager most needs to reflect the values of the company and the product. In most organizational structures, the product manager does not directly manage the people on the project team, and as such, he can't simply direct the people to do his bidding. Rather, he must work by influencing those on his team. This persuasion is done by mutual trust and respect—both of which depend on the integrity of your product manager.

产品团队中的每个角色不需要相同程度的承诺和努力。然而，产品经理这个角色不适合害怕努力工作的人。这是伴随着责任而来的；产品经理是最终对产品成功负责的人，这个负担沉重地压在成功的产品经理身上。

> Trust and respect is built over time by the successful product manager demonstrating the traits and skills of a strong product team leader. If the product manager is not perceived to have integrity, or honesty, or fairness when dealing with his teammates, then the product manager will not have the degree of collaboration and team effectiveness that he needs to get the job done well.

即使掌握了时间管理和产品管理技巧等技能，成功的产品经理仍然被产品所消耗。你能有家庭和非工作生活并成为成功的产品经理吗？我相信你可以。至少一旦你有了一些经验。但有很多人希望能够每周工作 40 小时——最重要的是——在一天结束回家时把工作问题留在办公室。不幸的是，这不是成功产品经理的生活。

> The product manager may not be an expert in every role of the product team, but he should have a deep understanding and respect for what each team member is responsible for, and he should be willing to trust those people to do their job.

我相信对新的产品经理候选人非常坦率地说明成功产品管理所需的投入程度。这不是要求产品经理工作特定的小时数。如果你不得不实际要求或告诉产品经理在关键时刻来上班，或者向他们指出他们需要在办公室，那你就找错了人。

> As the main interface between the product team and both the executive team and the sales organization, the product manager is often put in difficult situations, such as being asked to deliver products earlier, or with special features for large customers. The product team will watch closely how the product manager handles these situations.

请记住，投入程度和承诺在项目生命周期中并不是均匀的。有些阶段比其他阶段紧张得多。对于成功的产品经理来说，不会改变的是他们关心产品的程度，以及他们愿意为确保其成功而付出的努力。

> As with intelligence, assessing someone's integrity can be difficult—especially outside candidates who are unknown to your organization. For candidates with previous experience
>
> as product managers, you can ask them about how they dealt with the stresses in past products. Press for details of particular situations; what made the situation hard and how was it dealt with?
>
> Confidence

诚信

> Many people think of confidence as a result of experience. However, while experience may help build confidence, many very experienced product managers simply do not project confidence. At the same time, you can sometimes find brand-new college graduates simply bursting with confidence (although this is generally the confidence that comes from not yet knowing what they're in for).

在产品团队的所有成员中，产品经理最需要反映公司和产品的价值观。在大多数组织结构中，产品经理不直接管理项目团队的人员，因此，他不能简单地指示人们按他的吩咐做。相反，他必须通过影响他团队中的人来工作。这种说服是通过相互信任和尊重来完成的——两者都取决于产品经理的诚信。

信任和尊重是由成功的产品经理通过展示强大产品团队领导者的特质和技能而建立起来的。如果产品经理被认为没有诚信，或在处理队友时没有诚实或公平，那么产品经理就不会获得他需要的协作程度和团队效率来完成工作。

> Confidence is an important trait because the entire product team, executive team, and sales organization is looking to the product manager to convince them that what they are investing their time and money and careers in will be successful, and that the vision is a good one. In communicating persuasively, confidence is a critical ingredient, and people are more likely to follow a leader who has it, rather than one who does not.

产品经理可能不是产品团队每个角色的专家，但他应该对每个团队成员的职责有深入的理解和尊重，他应该愿意相信这些人能做好他们的工作。

作为产品团队与执行团队和销售组织之间的主要接口，产品经理经常被置于困难的情况中，比如被要求提前交付产品，或为大型客户提供特殊功能。产品团队会密切关注产品经理如何处理这些情况。

与智力一样，评估某人的诚信可能很困难——尤其是对你的组织来说未知的外部候选人。对于有产品经理经验的候选人，

你可以问他们过去如何处理产品中的压力。追问特定情况的细节；是什么让情况变得困难，是如何处理的?

> Attitude

自信

> The successful product manager sees himself as the CEO of the product. He takes full responsibility for the product, and does not make excuses. The successful product manager knows he is ultimately responsible for the success of the product. More importantly, he knows there are many very valid reasons for the product not to ship, or fail in the market

许多人认为自信是经验的结果。然而，虽然经验可能有助于建立自信，但许多非常有经验的产品经理根本没有表现出自信。同时，你有时可以发现刚毕业的大学生充满自信（虽然这通常是因为还不知道他们要面对什么而带来的自信）。

> when it does—the product is too difficult to build, it will take too long to get to market, it will cost too much, it will be too complicated, etc. But he knows it is his job to see that each and every one of these obstacles is overcome.

自信是一种重要的特质，因为整个产品团队、执行团队和销售组织都在指望产品经理来说服他们，他们投入时间、金钱和事业的东西将会成功，愿景是好的。在有说服力的沟通中，自信是一种关键成分，人们更有可能跟随有自信的领导者，而不是没有自信的领导者。

> This does not mean that he micromanages the product team, or that he tries to do it all himself, but rather that he is quick to take the blame if something goes wrong, and equally quick to give credit to the rest of the team when it goes well. The successful product manager knows that it is through the rest of the team that his product vision will become a reality, but that it is his product vision they are building.
>
> Skills

态度

> In order to succeed at the job of product management, there are several skills that are important. If the person has the right personal traits, I believe all these skills can be learned.

成功的产品经理将自己视为产品的 CEO。他对产品负全责，不找借口。成功的产品经理知道他最终对产品的成功负责。更重要的是，他知道产品不发布或在市场上失败有很多非常正当的理由——

产品太难构建，上市时间太长，成本太高，太复杂等等。但他知道他的工作是确保克服每一个这些障碍。

这并不意味着他微观管理产品团队，或试图自己做所有事情，而是当出现问题时，他很快承担责任，当进展顺利时，他同样快地把功劳归于团队其他成员。成功的产品经理知道，通过团队其他成员，他的产品愿景将成为现实，但正是他的产品愿景，他们正在构建。

> Applying Technology

技能

> One reason so many successful product managers come from the engineering ranks is that a big part of defining a successful product is in understanding new technology and seeing how it might be applied to help solve a relevant problem.

为了在产品管理工作上取得成功，有几项技能很重要。如果这个人有正确的个人特质，我相信所有这些技能都可以学习。

> While you don't need to be able to invent or implement the new technology yourself in order to be a strong product manager, you do need to be comfortable enough with the technology that you can understand it and see its potential applications.
>
> There are many ways to develop this skill. Taking classes, reading books and articles, and talking with engineers and architects can help you learn. Ask the senior engineers on your product team

> their recommendations for ways to learn more about the technological possibilities. Brainstorming sessions with the engineering team are another way to learn how new technologies might be applied.
>
> Focus

应用技术

> As the saying goes, “The main thing is to keep the main thing the main thing.” There are so many distractions out there, especially for the product manager trying to create a product that customers will love. The ability to keep the focus on the key problem to be solved at any given moment, and not succumb to creeping featurism, or the loud voices of a few key people or customers, requires tremendous discipline—both company discipline and personal discipline.

这么多成功的产品经理来自工程队伍的原因之一是，定义成功产品的很大一部分在于理解新技术并看到它如何被应用来帮助解决相关问题。

虽然你不需要自己能够发明或实现新技术才能成为强大的产品经理，但你确实需要对技术足够熟悉，能够理解它并看到它的潜在应用。

> The truth is that nearly every product has features that are not really all that important—if the features were never there, it would not significantly impact the sales or customer satisfaction. Much more often, if the features were not there, the product would be better for itas more users could comprehend and appreciate the resulting simpler product. Focus will help you reduce the number of cluttering features, reduce the time it takes you to build the product, and therefore the time it takes you to get to market and your costs of getting it there.

有很多方法可以培养这种技能。上课、阅读书籍和文章、与工程师和架构师交谈都可以帮助你学习。询问产品团队中的高级工程师，他们推荐哪些方式来更多地了解技术可能性。与工程团队的头脑风暴会议是学习新技术如何被应用的另一种方式。

> Time Management

专注

> In today's e-mail, instant message, and mobile phone-based world where distractions abound, it is so very easy to come in to work early in the morning, work frantically all day—even skipping food—and then head home late into the night, not having actually accomplished anything important for your product. This is because you have spent the day chasing fires and working on “urgent” items.

俗话说，"主要的事情是保持主要事情为主要事情。"外面有太多的干扰，尤其是对于试图创造客户会喜爱的产品的产品经理。在任何时候都将注意力集中在要解决的关键问题上，不屈服于功能蔓延，或少数关键人物或客户的大声疾呼，需要巨大的纪律——既是公司纪律也是个人纪律。

> It is absolutely essential to get very skilled at quickly distinguishing that which is important from that which is urgent, and learn to prioritize and plan your time appropriately. If you can't manage

> to get the time to focus on those tasks which are truly important to your product, your product will fail.

事实是，几乎每个产品都有一些并非真正那么重要的功能——如果这些功能从未存在过，也不会显著影响销售或客户满意度。更多的时候，如果这些功能不存在，产品会更好，因为更多用户能够理解并欣赏由此产生的更简单的产品。专注将帮助你减少杂乱功能的数量，减少构建产品所需的时间，因此减少你上市的时间和成本。

> I have known too many product managers who burn themselves out with 70-hour workweeks, but the worst part is when I tell them that they're not actually doing their job. The natural response is that they just don't have any more time and can't work any harder. I then go into my lecture on time management and working smarter. Rather than feel like they need to attend every meeting or be constantly available on e-mail, they need to spend their time on the activities that will actually make a difference. So much of what these people spend time doing is avoidable.
>
> Communication Skills

时间管理

> While communication skills can, for the most part, be learned, it can take years to become an effective speaker or writer, and these skills will be required from the start. As discussed above, the product manager influences others by persuasion rather than authority—making his case by communicating either through writing, speaking or both.

在当今电子邮件、即时消息和手机无处不在、干扰横行的世界里，很容易早上很早就来上班，整天疯狂工作——甚至不吃东西——然后深夜才回家，但实际上并没有为产品完成任何重要的事情。这是因为你花了一整天在救火和处理"紧急"事项上。

> Speaking skills can be partially assessed during the interview itself, but written skills should be assessed specifically. I like to suggest that product manager candidates bring in examples of written material such as nonproprietary white papers or strategic documents.

必须非常熟练地快速区分什么是重要的，什么是紧急的，并学会适当地优先排序和规划时间。如果你不能抽出时间专注于那些真正对你的产品重要的任务，你的产品就会失败。

> While good communication skills are absolutely essential, it is important to emphasize that speaking with an accent, or minor grammatical issues with a non-native language, do not constitute poor communication skills. The person must speak clearly enough to be easily understood, and write powerfully enough to persuade, but perfect pronunciation or grammar are not required.

我认识太多每周工作 70 小时把自己累垮的产品经理，但最糟糕的是当我告诉他们他们实际上没有做他们的工作时。自然的反应是他们没有更多时间了，不能再努力工作了。然后我就给他们讲时间管理和更聪明工作的道理。他们不需要感觉必须参加每个会议或时刻在电子邮件上可用，而需要把时间花在那些真正能产生影响的活动上。这些人花这么多时间做的很多事情其实是可以避免的。

> Product managers spend a great deal of time writing—composing e-mails, specs, white papers, strategy papers, data sheets, competitive product reviews, and more. The successful product manager only takes the time to write these documents if he believes people are going to read them. And since they are going to be read, they need to do their job well, which is typically to describe, educate and/or persuade.
>
> Being able to write clear and concise prose is a skill that product managers use every day. The successful product manager realizes that the readers are constantly evaluating him based on his writings. Especially with senior management, sometimes these writings are all
>
> they have to go on.
>
> The other major form of communication that product managers frequently need to employ is giving a presentation. Presenting in front of a group—particularly a large group—is hard for many people. Presenting effectively is even harder. Yet this is an important skill for a product manager since many of the most important events in the life of a product require the product manager to stand up in front of a group of company executives, major customers or the company sales force and—in the short time he or she has—explain what the product is about and why it is important.
>
> We have all sat through terrible presentations—with slide after endless slide, the speaker simply reading the bullets, people straining to read the too-small print, meaningless graphics, and being unclear what the key messages are and why you should care. Not only are these presentations ineffective at conveying the purpose to the team, they are also a waste of time.
>
> In contrast, the successful product manager has a minimal number of slides, he is engaging, clearly knowledgeable and passionate about his product, he speaks clearly and to the point, his slides provide relevant supporting data for what he is saying, and he has unambiguously stated his main points and what he needs from the audience after the presentation. His presentation finishes on time (or even early), he entertains questions and, if he can't provide a clear, useful answer immediately, he follows up diligently and promptly with the questioner and, if appropriate, the entire audience. The book Presenting to Win: The Art of Telling Your Story by Jerry Weissman is a good guide to improving your presentation skills.
>
> Business Skills

沟通技巧

> Finally, business skills are also important for the product manager. As the main interface with the rest of the company, the product manager will need to work with company finance staff, marketing people, sales, and executive management—using the language and concepts that these people deal with.

虽然沟通技巧在很大程度上可以学习，但可能需要数年时间才能成为有效的演讲者或作家，而这些技能从一开始就需要。如上所述，产品经理通过说服而非权威来影响他人——通过写作、演讲或两者兼而有之来进行沟通。

> I sometimes talk of product managers needing to be bilingual. Not in Chinese and English. They need to be able to converse equally well with engineers about technology as they do with executives and marketers about cost structures, margins, market share, positioning, and brand.

演讲技巧可以在面试过程中部分评估，但书面技能应该专门评估。我建议产品经理候选人带一些书面材料的例子，如非专有的白皮书或战略文件。

> This is one reason why so many product managers are recruited out of business school. The product organization knows that they need someone who can talk the language of the business side, so they hire an MBA. I have known some great product managers that have come through the MBA path, but business skills are but one part of the mix required for a successful product manger, and they can certainly be learned in places other than business schools. It is at least as common that an engineer moves into product management and acquires the business skills required by reading books, taking courses, and getting coaching and assistance from mentors in the finance and marketing organizations.

虽然良好的沟通技巧绝对必要，但重要的是要强调，带有口音，或非母语的轻微语法问题，并不构成糟糕的沟通技巧。这个人必须说得足够清楚以便容易理解，写得足够有说服力，但完美的发音或语法不是必需的。

> So where do you find these people?

产品经理花大量时间写作——撰写电子邮件、规格说明书、白皮书、战略文件、数据表、竞争产品评审等等。成功的产品经理只有在相信人们会阅读这些文件时才会花时间写。既然它们会被阅读，它们就需要做好工作，这通常是描述、教育和/或说服。

> After reading this list of traits and skills, you may be thinking that such people are extremely rare. They are rare—about as rare as good products. But few hires you make will be as critical as your product managers, so it is worthwhile to interview for these characteristics and set the bar high.

能够写出清晰简洁的散文是产品经理每天使用的技能。成功的产品经理意识到读者会根据他的写作不断评估他。特别是对于高级管理层，有时这些写作是

> There are different schools of thought on recruiting product managers. Many companies think that all you need is someone from the marketing organization or someone with an MBA. In the old-school definition of product manager, this may have been true, but today this is a recipe for failure.

他们唯一需要参考的东西。

> Many companies prefer MBAs from top business schools who have a technical undergraduate degree combined with applicable industry experience. This can work well if you keep in mind that a consistent problem with MBA programs—even from the top-tier schools—is that they almost never teach product management, so it is dangerous to assume that the recent MBA grad has any idea how to be a product manager.

产品经理经常需要使用的另一种主要沟通形式是演讲。在团体面前演讲——特别是大型团体——对许多人来说很难。有效演讲更难。然而，这对产品经理来说是一项重要的技能，因为产品生命中许多最重要的事件需要产品经理站在公司高管、主要客户或公司销售团队面前——在他或她拥有的短时间内——解释产品是什么以及为什么它很重要。

> My favorite source for product managers is to look for people with the characteristics described above and then use training, an informal mentoring program, and/or a formal employee development program to develop these people into strong product managers. Such people might be found virtually anywhere in the company. I've seen outstanding product managers come out of engineering, user experience design, customer service, professional services, product marketing, sales, and the user community. Often these people will approach management asking how they can get more involved in the product. It can also be useful for senior management to approach top performers from across the company

我们都经历过糟糕的演示——幻灯片一张接一张没完没了，演讲者只是在读要点，人们费力地阅读太小的字体，毫无意义的图形，不清楚关键信息是什么以及为什么你应该关心。这些演示不仅无法向团队传达目的，而且也是浪费时间。

> about the possibility of product management, as this can be essential experience for those on an executive track.

相比之下，成功的产品经理有最少数量的幻灯片，他有吸引力，对产品知识渊博且充满热情，他说话清晰且切中要害，他的幻灯片为他所说的提供相关的支持数据，他明确陈述了他的主要观点以及演讲后他需要观众做什么。他的演讲按时（甚至提前）结束，他接受提问，如果他不能立即提供清晰有用的答案，他会勤奋及时地跟进提问者，如果合适的话，跟进整个听众。Jerry Weissman 的书《Presenting to Win: The Art of Telling Your Story》是提高演讲技巧的好指南。

> pow Important Is Domain Experience?
>
> Recently a friend called to ask my opinion of a product management leader job candidate—I'll call him David—with whom I've worked in the past. The hiring manager is an exec at a large consumer Internet services company. He really liked David, but his question to me was: “He's clearly an expert at enterprise software, but could he succeed at our type of business?”
>
> Thad to laugh, and I went on to explain that just over four years ago I got a similar call about David. His future manager asked me, “Clearly the guy was great at infrastructure software, but could he handle enterprise software?”
>
> In truth, David is not an infrastructure guy, or an enterprise guy, or a consumer services guy. His education wasn't even in technology—he studied finance. He is, however, one very smart guy, and one of the best product managers I know. One of the things that David does extremely well is that he can tackle new domains and new technologies very quickly, and this has allowed him to excel with several very different types of products in different domains and industries.
>
> I'm often asked about the need for applicable domain or industry expertise for product managers. Many product managers are, in fact, hired exclusively for their domain experience, and I do think there are a few products where domain expertise is truly essential.

If I ever need a defibrillator one day, I hope there was a product manager on the team that designed it that knew something about cardiac care. But in my experience, this is by far the exception rather than the rule.

Tl go even further. It can be dangerous for a product manager to have too much domain expertise. I say that because people that have spent a long time building their mastery of one domain often fall into another common product management trap: they believe they can speak for the target customer, and that they are more like their target customer than they really are. The product manager needs to constantly revisit assumptions about the domain and the customers. It's not impossible for people with deep domain expertise to do this, but they have to work harder at it to remain open-minded to new developments and options. This is not to say that you don't need domain expertise in order to do a good job with your product—in fact I think understanding your product domain is absolutely essential, and I don't mean superficial knowledge either. But I believe that strong product managers can come up to speed on most new product domains very quickly if they approach the education process aggressively. I've learned that it generally takes me one to three months to come up to speed on a domain I haven't worked on before, to the point where I feel confident charting a product strategy. Some people can probably learn faster, and others might take a little longer.

I also believe that there are some different skills required for leading enterprise products, versus infrastructure, versus consumer services versus consumer electronics. For example, if your product is sold to a relatively small number of large enterprises (as opposed to

hundreds of thousands or millions of consumers), then some different techniques are used to understand requirements and try out product ideas. It is also important to understand the different types of sales and distribution channels because these impact the product as well. If hardware is involved, you need to understand the process and time-line differences. For large-scale consumer services, the issues of scale and community management can destroy you if you don't know how to manage them.

Overall, I consider about 80% of the skills and talents of a product manager to be applicable across the different types of products.

This is also not a statement against the value of experience in general, but I have found that the most valuable experience is not what you learn about some product domain or technology (that is probably obsolete now anyway), but rather what you've learned about the process of creating great products, leading a product team, and managing growth. It's also what you've learned about yourself and how to improve the next time.

Very related to domain expertise is the topic of technology expertise. This used to come up more frequently than it does now, but I still hear it. I saw a job description the other day for an enterprise application company looking for a product manager with direct experience creating Linux-based products. While it's true that there are some important differences between operating systems, if the product manager can't very quickly assess the important points as it impacts his or her product, then this person has much greater issues than just lacking Linux expertise.

With high-tech products, it's all about how quickly you can learn new technologies and,

more importantly, envision how you can apply the new technology to the problems you are trying to solve.

Technologies change so fast that product managers must be skilled at quickly learning new technologies and solving problems in new domains. When I interview prospective product managers, I'm looking not to see exactly what they already know, but for each of their products, what did they need to learn, how long did it take them, and how did they apply that knowledge?

Does Age Matter?

If you have been wondering what's going on with all these startups with 20-something-year-old founders and product leaders, you're not alone. There are some great companies that have been started by some very young people, several of whom dropped out of college to pursue their ideas. While experience can be extremely valuable, there is a serious problem with people discounting product leaders because of their youth.

In truth, there are outstanding product leaders across the age spectrum. But how is it that someone can be just 25 years old and already an exceptional product leader? First, remember that the Internet has only been used by the widespread public since 1995 or so, so anyone who today is 25 or older probably has the same amount of experience online as the rest of us do. And people who were in their teens during the rise of the Internet grew up taking for granted technologies that many are still trying to figure out.

Further, while experience can play an important role and naturally develops over time, other traits such as innate intelligence and product passion are not a function of age.

I personally had to get used to the idea of working for someone in their early 20s when I worked for Marc Andreessen, the young co-founder of Netscape. But I quickly forgot about how old he was once I started seeing how quickly he absorbed new technologies and assimilated the literally hundreds of customer visits he was doing at the time. Anyone just listening to him would assume he was at least in his 40s, based on his command of the

business.

But what this is really all about is finding great product people—regardless of their age, gender or race. I'm not here to talk about the moral issues involved in the different forms of discrimination, but I do want to talk about the business issues. I believe there are still stereotypes and biases that get in the way of companies creating the best product teams and products possible.

One reason I love Silicon Valley so much is because the people here are so diverse. But even. in the most progressive of companies, I think there are often hiring biases based on our mental image of a great product leader. For example, we know that communication skills are essential for a strong product manager, so we sometimes look for someone with native English-language skills, even though others who may be much stronger candidates have more than passable language skills.

I point this out not to chastise anyone but just to raise awareness that some truly outstanding product leaders may be overlooked by unintentionally restricting our view of what makes a great product manager and where great product ideas come from. So the next time that 22-year-old college hire comes to you with a product idea, you may want to listen. Her idea might be the next Facebook.

商业技能

最后，商业技能对产品经理也很重要。作为与公司其他部门的主要接口，产品经理需要与公司财务人员、营销人员、销售人员和执行管理层合作——使用这些人处理的语言和概念。

我有时说产品经理需要双语。不是中文和英文。他们需要能够像与工程师谈论技术一样，与高管和营销人员谈论成本结构、利润、市场份额、定位和品牌。

这就是为什么这么多产品经理从商学院招聘的原因之一。产品组织知道他们需要能够谈论商业方面语言的人，所以他们雇用 MBA。我认识一些通过 MBA 路径成为伟大产品经理的人，但商业技能只是成功产品经理所需素质的一部分，它们当然可以在商学院以外的地方学习。至少同样常见的是工程师进入产品管理，通过阅读书籍、上课，以及从财务和营销组织的导师那里获得指导和帮助来获得所需的商业技能。

那么在哪里可以找到这些人呢?

读完这份特质和技能清单后，你可能在想这样的人极其罕见。他们确实罕见——就像好产品一样罕见。但你做出的招聘中很少有关键程度能超过产品经理，所以值得面试这些特征并设定高标准。

关于招聘产品经理有不同的思路。许多公司认为你只需要来自营销组织的人或有 MBA 的人。在旧式的产品经理定义中，这可能是真的，但今天这是失败的配方。

许多公司更喜欢来自顶尖商学院的 MBA，他们有技术本科学位并结合适用的行业经验。如果你记住即使是顶尖商学院的 MBA 项目也几乎从不教授产品管理，所以假设刚毕业的有 MBA 学位的人知道如何做产品经理是危险的，这可能会很好地发挥作用。

我最喜欢的产品经理来源是寻找具有上述特征的人，然后使用培训、非正式导师计划和/或正式的员工发展计划将这些人培养成强大的产品经理。这样的人几乎可以在公司的任何地方找到。我见过杰出的产品经理来自工程、用户体验设计、客户服务、专业服务、产品营销、销售和用户社区。通常这些人会主动接近管理层，询问他们如何更多地参与产品。对于高级管理层来说，接近公司各部门的顶尖人才

关于产品管理的可能性也很有用，因为这对那些走高管路线的人来说可能是至关重要的经验。

领域经验有多重要?

最近一位朋友打电话来询问我对一个产品经理领导职位候选人的意见——我叫他 David——我以前和他一起工作过。招聘经理是一家大型消费互联网服务公司的高管。他真的很喜欢 David，但他的问题是："他显然是企业软件方面的专家，但他能在我们这种类型的业务中取得成功吗？"

我不得不笑，然后继续解释，就在四年前，我接到了类似的关于 David 的电话。他未来的经理问我，"这家伙显然在基础架构软件方面很棒，但他能处理企业软件吗？"

事实上，David 不是基础架构专家，也不是企业软件专家，也不是消费服务专家。他的教育甚至不是技术方面的——他学的是金融。然而，他是一个非常聪明的人，是我认识的最好的产品经理之一。David 做得非常好的一件事是他能够快速处理新领域和新技术，这使他能够在不同的领域和行业表现出色。

我经常被问及产品经理是否需要适用的领域或行业专业知识。事实上，许多产品经理正是因为他们的领域经验而被聘用的，我确实认为有一些产品真正需要领域专业知识。

如果有一天我需要除颤器，我希望设计它的团队中有一位了解心脏护理的产品经理。但根据我的经验，这绝对是例外而非常规。

我要更进一步。产品经理拥有太多领域经验可能很危险。我这样说是因为花很长时间建立了一个领域精通的人往往会落入另一个常见的产品管理陷阱：他们相信自己可以代表目标客户，而且比他们实际更像他们的目标客户。产品经理需要不断重新审视关于领域和客户的假设。具有深厚领域专业知识的人这样做并非不可能，但他们必须更加努力地保持对新发展和选择的开放心态。

这并不是说你在做好产品工作时不需要领域专业知识——事实上，我认为理解你的产品领域是绝对必要的，我指的不是肤浅的知识。但我相信，如果强大的产品经理积极地对待教育过程，他们可以很快掌握大多数新产品领域。我了解到，我通常需要一到三个月的时间来掌握我以前没有工作过的领域，达到我有信心制定产品策略的程度。有些人可能学得更快，其他人可能需要稍微长一点的时间。

我也认为，领导企业产品、基础设施、消费服务与消费电子产品需要一些不同的技能。例如，如果你的产品卖给相对较少的大型企业（而不是

数十万或数百万消费者），那么使用一些不同的技术来理解需求和尝试产品创意。了解不同类型的销售和分销渠道也很重要，因为这些也会影响产品。如果涉及硬件，你需要了解流程和时间线的差异。对于大规模消费服务，如果你不知道如何管理，规模和社区管理的问题可能会毁了你。

总的来说，我认为产品经理约 80% 的技能和才能适用于不同类型的产品。

这也不是对一般经验价值的否定，但我发现最有价值的经验不是你对某个产品领域或技术学到的（反正现在可能已经过时了），而是你对创造伟大产品的过程、领导产品团队和管理增长学到的。这也是你对自己学到的以及如何在下次改进的了解。

与领域专业知识密切相关的是技术专业知识的话题。这比以前更常出现，但我仍然听到。前几天我看到一个企业应用公司的职位描述，寻找有直接创建基于 Linux 产品经验的产品经理。虽然操作系统之间确实存在一些重要差异，但如果产品经理不能很快评估它如何影响他或她的产品的要点，那么这个人就有比缺乏 Linux 专业知识更大的问题。

对于高科技产品，关键在于你能多快学习新技术，而且

更重要的是，设想如何将这些新技术应用到你试图解决的问题上。

技术变化如此之快，产品经理必须善于快速学习新技术并在新领域解决问题。当我面试 prospective 产品经理时，我不是要看他们确切知道什么，而是对于他们的每个产品，他们需要学习什么，花了多长时间，以及他们如何应用这些知识?

年龄重要吗?

如果你一直在想这些由 20 多岁的创始人和产品领导者创办的初创公司是怎么回事，你不是一个人。有一些伟大的公司是由一些非常年轻的人创办的，其中一些人从大学辍学去追求他们的想法。虽然经验可能非常宝贵，但人们因为年轻而看轻产品领导者是一个严重的问题。

事实上，在各个年龄段都有杰出的产品领导者。但是一个只有 25 岁的人怎么可能已经成为一个卓越的产品领导者呢？首先，记住互联网大约从 1995 年才开始被公众广泛使用，所以任何今天 25 岁或更大的人可能拥有和我们其他人一样多的在线经验。而且在互联网兴起时期正值十几岁的人，把许多人仍在努力理解的技术视为理所当然。

此外，虽然经验可以发挥重要作用并自然随着时间发展，但其他特质如天生的智力和产品热情与年龄无关。

当我为年轻的网景联合创始人马克·安德森（Marc Andreessen）工作时，我个人必须习惯为 20 出头的人工作。但一旦我开始看到他如何快速吸收新技术并消化他当时正在进行的数百次客户访问，我很快就忘了他多大了。任何只听他说话的人都会假设他至少 40 多岁，基于他对业务的掌控。

但这实际上是要找到伟大的产品人才——无论他们的年龄、性别或种族。我不是来讨论不同形式歧视涉及的道德问题，但我想谈谈商业问题。我相信仍然存在阻碍公司创建最佳产品团队和产品的刻板印象和偏见。

我如此热爱硅谷的原因之一是这里的人如此多样化。但即使在最进步的公司里，我认为在招聘时往往基于我们对伟大产品领导者的心理形象存在偏见。例如，我们知道沟通技巧对强大的产品经理至关重要，所以我们有时会寻找有母语英语技能的人，即使其他可能强得多的候选人拥有绰绰有余的语言技能。

我指出这一点不是为了责备任何人，而只是为了提高认识，一些真正杰出的产品领导者可能因为我们无意中限制了我们对什么造就伟大产品经理以及伟大产品想法来自哪里的看法而被忽视。所以下次那个 22 岁的大学毕业生带着产品想法来找你时，你可能想听听。她的想法可能是下一个 Facebook。

第 7 章：管理产品经理

> Chapter 7: Managing Product Managers
> Building The Team That Will Build Your Company

建立将建立你公司的团队

> I have long advocated setting the bar high for your product managers. These people are absolutely essential to the success of your products, and hence your business. Yet, when I speak with managers of product management, they often explain that they have inherited an organization where many of the people with “product manager” titles were really product marketing people. These so-called product managers had all the problems I described in earlier chapters, and they were struggling with how to correct the situation.

我一直主张为你的产品经理设定高标准。这些人对你的产品成功绝对至关重要，因此对你的业务也至关重要。然而，当我与产品管理经理交谈时，他们经常解释说他们继承了一个组织，其中许多有"产品

经理"头衔的人实际上是产品营销人员。这些所谓的产品经理有我在前面章节中描述的所有问题，他们正在努力如何纠正这种情况。

> So here I'd like to discuss the role and responsibilities of those that manage product managers.

所以在这里我想讨论那些管理产品经理的人的角色和职责。

> Typically this individual is given the title Director or VP of Product Management. This job is among the most important positions in any high-tech company. Few positions will have more impact on the future success of the company than the head of product management. A successful product can literally redefine the course of the business, while a failed product can sink it. As a result, this job is usually characterized by massive success or massive failure—and little in between.

通常这个人被赋予产品管理总监或副总裁的头衔。这个职位是任何高科技公司中最重要的职位之一。很少有职位比产品管理负责人对公司未来的成功有更大的影响。一个成功的产品可以 literally 重新定义业务的进程，而一个失败的产品可能会使公司沉没。因此，这份工作通常以巨大的成功或巨大的失败为特征——中间状态很少。

> These people have two essential responsibilities. First, they must build a strong team of product managers. Second, they are responsible for the company's overall product strategy and the various products in the company's portfolio. I'll discuss each of these responsibilities in turn.

这些人有两个基本职责。首先，他们必须建立一个强大的产品经理团队。其次，他们负责公司的整体产品战略和公司产品组合中的各种产品。我将依次讨论这些职责中的每一个。

> Building the Product Management Team
>
> It is the primary job of every manager to build and develop the capabilities of his or her team, but this is especially crucial for the role of Director of Product Management because of the high-impact nature of the product management position. An inadequate product manager is almost certain to fail, resulting in wasted product cycles, frustrated users, and lost customers. For many positions in the company, the truth is that you can often get away with subpar employees because there are others who will pick up the slack. However, given that most products just have a single product manager for this position, this is rarely the case. Your only real hope if the product manager isn't fully capable is that someone else on the product team—perhaps a lead engineer—steps up to do what's necessary.
>
> If you find yourself managing a team of product managers where some are not up to the task, then you have to correct the situation immediately. Some people will just never be successful product managers—they just aren't a fit for the position's demands and no amount of training or coaching is going to change that. However, I've found that for many product managers, you can in fact significantly improve their performance. I don't want this to sound self-serving since it is no secret that I spend a good deal of my time helping
>
> companies develop the skills of their product managers, but one way or another the product management leader needs to ensure that every team member is up to speed.

I believe that every new product manager needs roughly three months of hard learning before you can entrust them with the responsibility of guiding a product. During this time, the new product manager needs to immerse herself with target users and customers, get educated on the relevant technologies, and study the market and competitive landscape. Throughout the process, the manager should be facilitating and overseeing the learning curve. Note that this already assumes the product manager understands the actual skills and responsibilities of a product manager. These three months apply even to experienced product managers. They still need to learn about your unique customers and domain.

When you hire a new product manager to the team, establish a program so that he or she can get the needed exposure to users and technologies. For those product managers that you already have and are already in the midst of managing their products, if they're behind on understanding the users, then you will want to make sure that they start a program like this in parallel with their other responsibilities. But make sure they clearly understand their need to get up to speed.

If you determine that the product manager is unable or unwilling to do what is necessary to execute their duties at any level, it is your job as the manager to find someone that is. To anyone that has ever had to remove someone, you know this is no fun. But this is one reason for your higher pay, and you owe it to the rest of your team, company, and customers to correct the situation. Do everything you can to help the person find a job they can succeed at, but keep your focus laser sharp on getting the right people in place who can do the job

that is necessary.

Once you are convinced that the members of your team are capable of success and properly equipped to succeed, then you will need to step back and let these talented people do their job. If you micromanage your product managers, they will not step up and take ownership the way you need them to. If you can't trust your product managers, you need to find product managers you can trust. This doesn't mean you shouldn't ask questions and be constantly available to help—you absolutely should. But if you empower strong people and let them do their jobs, I promise you that you'll be amazed at what they can do.

Note that you've first got to make sure you have strong people before you empower them. If you empower people who aren't capable, you are abdicating your responsibility as manager. And if you micromanage people who are not capable, you are essentially doing their job for them.

Every good manager knows that the best way to look good is for the members of his or her team to look good. As such, always hire people that you believe are smarter than yourself, and then do everything you can to help them succeed.

Defining the Company's Product Strategy

Nobody is more responsible or more accountable for the suite of products a company offers than the head of the product management organization. This person decides what products to pursue, and then closely reviews the strategy and progress for each product.

The head of product management must have a deep and current understanding of the company's business strategy so that she can ensure the product strategy directly supports the business strategy. This individual also takes the leadership role in defining the product vision and working with the product managers on the team to deliver on this vision. The product principles (see the

chapter Product Principles) are typically established by the full product management team, but the head of product management leads this effort and ensures that the products adhere to these principles as much as possible.

Even with the best team of product managers, each doing an outstanding job, you will still have cross-product conflicts as each product manager works to optimize his own product. The head of product management must work to identify and resolve these cross-product issues.

Similarly, the head of product management is responsible for the portfolio roadmap—taking an overall look at the many product releases that are planned, and considering the business needs and customer impact.

Finally, the head of product management will need to manage executive relationships within the organization. It is essential that all of the key players in the company—particularly the CEO—have a good and trusting relationship with the head of product management. The entire company depends on this person, and he must be open and transparent with his decisions and reasons, accessible and approachable by all. She must be receptive to new ideas from any source, but also respected enough that she can push back when appropriate on conflicting or vacillating priorities.

As you can see, this is an extremely demanding job, but great product companies have great people in this role, and this is no coincidence.

How Do We Meaure Product Managers?

Very often I'm asked how product managers should be measured. I have long believed that the only true measure of the product manager is the success of his or her product. While I still believe that, it's not a very satisfying answer, as it's not clear what is the best way to measure a product's success. Is it revenue? Profit?

The number of users? Page views? All of these indicators can be useful, but they don't really provide the big picture needed for a true measurement of a product manager's success. Recently, a new business metric has been gaining traction across a number of different industries: Net Promoter Score (NPS). It's a very simple metric, and you can read all about it at www.netpromoter.com.

Here's how it works: You ask your customers how likely they would be to recommend your product to others, on a scale of 0-10. Those who rate 9 or 10 are considered "promoters" (they're out there telling their friends how much they love your product, and are actively evangelizing for you); those who rate 7-8 are lukewarm or neutral; and those who rate 0-6, the "detractors," that is, they not only won't recommend your product, they may even actively warn their friends or the public against your product. If you take the percentage of promoters and subtract the percentage of detractors, you get the NPS, which essentially tells you if you have more people cheering for you or against you.

Quite a few companies already track this metric, and those that rate very highly won't surprise you—companies like Apple, Amazon, Google and eBay.

I like this metric a lot because it puts the focus on the product and the overall customer experience. If, as we like to say in this business, it's all about creating happy customers, then this is a measure of exactly that. Yes, in theory you could have 100% very happy customers, but you end up going out of business because you're losing money on every customer. But for a single metric, I

believe this keeps the focus of the company on creating happy customers, which is what fuels growth. And in terms of profitability, the most cost-effective sales or marketing program is your own customers doing the sales and marketing work for you. A great product with happy customers lowers these other (often very significant) costs, which contributes to profitability.

Another benefit of this metric is that it leads to the concept of good revenue and bad revenue. For example, let's say the company is approached by a big potential sponsorship or advertising partner that covets your user base and believes they can effectively sell their product on your site. This could be a good thing or a bad thing, depending on howit's done. If it's done poorly, the short-term revenue might be nice, but if it hurts your customer experience, this will be reflected in the NPS, and your business growth will be slowed. On the other hand, if it's done well—for example, by working collaboratively with the advertising partner—it could be either neutral to the customer experience or even contribute to the experience. This will help your business grow more quickly.

This is why specials (doing explicitly-defined work for a specific customer typically in exchange for their agreement to purchase your product) are so dangerous. They represent bad revenue, and hurt your company's ability to deliver products that create happy users. You can compare your NPS across companies and even across industries, which is interesting, but NPS is most useful as a way to measure your own progress towards improving your products and services. So if you don't already measure your NPS, consider doing so as soon as possible. Then you can start watching how the changes you make to your products impact the score. Make sure you're always moving in the right direction, and consider the impact on the NPS of everything you do.

Where Should Product Management Live?

Many companies struggle with where product management should live in the organization. The choices for most companies are most often engineering or marketing. If you have the right personalities, it can work in either place, but I'm actually not a fan of it residing in either.

The most common situation I find in the companies I work with is that product management lives within the marketing organization. The problem with this organizational design is that it is based on the misconception that you get products from talking to your customers, and that it is marketing's job to talk to customers. I won't repeat all the fallacies of this line of thinking here, but suffice it to say that there are several key reasons why you won't find successful products just by asking your customers.

Also problematic is the fact that what usually happens is that the product marketing role and the product management role get combined within the marketing organization. These roles—and the skills required to execute them well—are so different that what usually ends up happening is that one or the other (or both) gets poorly executed.

The next most common situation I encounter is that product management is put in the engineering organization. While this has the benefit of putting the people who invent and design the product next to the people who actually build the product, this can also be problematic. Why? Because engineering organizations are typically designed to focus on

building a product right, rather than building the right product. It's easy for the product management team to be consumed in the details and pressures of producing detailed specs rather than looking at the market opportunity and discovering a winning product strategy and roadmap.

> It takes a different mindset and different skills to come up with the right product to build. Moreover, engineering is essentially an execution-based activity, and it can be hard to perform discovery activities in an organization optimized for execution.
>
> So if not the marketing organization, and if not the engineering organization, then where?
>
> I am a big believer in raising the level of the product organization to be organizationally on par with engineering and marketing. Ideally, the product organization includes the design team, because the interaction between product management and user experience design absolutely needs to be as close as possible. Increasingly, you'll see an organization with the name of Product or Product Management or Product Management and Design—often with a VP of Product or even a Chief Product Officer running it.
>
> There are several benefits to this organizational design, but the biggest reason is that the head of product needs to have a seat at the table on the executive team. Companies are all about products, and marketing and engineering are each critical components with their own considerations and challenges, and it's easy for the product to get lost in these issues. Additionally, this organizational structure makes it clear that the product is not being driven by the technology, and not being driven solely by the sales or marketing needs either.
>
> One special case exists in many larger companies. Often large companies have a centralized engineering function and decentralized business units. This lets the company focus on
>
> multiple business lines, while potentially enjoying efficiencies in common engineering services. In such organizations, the product management and design function might be located in the centralized engineering/product development organization, or it might be in its own organization, or it might be a part of the business units themselves. Often in such an organization, the business unit managers must play a major product management role, creating problems if the product management team isn't part of the business unit. In these situations, I usually prefer integrating product management and design into the business units.
>
> While I've explained my reasons for the ideal locations, it can be very hard to implement organizational structural change, and your company may not be willing or able to go this route. This does not necessarily mean that you're destined for problems. It still boils down to the people involved and the skills they bring. If you can develop your product team's skills and demonstrate their value across the company, any of these organizational structures can succeed.
>
> Orxamples

建立产品管理团队

> You can see example product strategies, product roadmaps, and portfolio roadmaps at www-svpg.com/examples.

每个经理的主要工作是建立和发展他或她团队的能力，但对于产品管理总监这个角色来说，这尤其关键，因为产品管理职位具有高度影响力。一个不合格的产品经理几乎肯定会失败，导致浪费产品周期、沮丧的用户和失去客户。对于公司中的许多职位，事实是你可以经常容忍不合格的员工，因为其他人会弥补。然而，鉴于大多数产品只有一个产品经理担任这个职位，这种情况很少。如果你的产品

经理不能完全胜任，你唯一的真正希望是产品团队中的其他人——也许是首席工程师——站出来做必要的事情。

如果你发现自己在管理一个产品经理团队，其中一些人不能胜任任务，那么你必须立即纠正这种情况。有些人永远不会成为成功的产品经理——他们只是不适合这个职位的要求，再多的培训或指导也无法改变这一点。然而，我发现对于许多产品经理，你实际上可以显著提高他们的表现。我不想这听起来像自我推销，因为我花大量时间帮助公司发展产品经理的技能已不是秘密，但无论如何，产品管理负责人需要确保每个团队成员都达到标准。

我相信每个新产品经理在被委以指导产品的责任之前，大约需要三个月的艰苦学习。在此期间，新产品经理需要沉浸在目标用户和客户中，了解相关技术，并研究市场和竞争格局。在整个过程中，经理应该促进和监督学习曲线。注意，这已经假设产品经理理解产品经理的实际技能和职责。这三个月甚至适用于有经验的产品经理。他们仍然需要了解你独特的客户和领域。

当你雇用新的产品经理到团队时，建立一个项目，让他或她能够获得对用户和技术所需的接触。对于那些你已经拥有并正在管理他们的产品的产品经理，如果他们在理解用户方面落后了，那么你要确保他们在其他职责的同时开始这样的项目。但要确保他们清楚地理解他们需要达到标准的需要。

如果你确定产品经理不能或不愿意做任何必要的事情来执行他们的职责，那么作为经理，你的工作就是找到能这样做的人。对于任何曾经不得不开除某人的人来说，你知道这并不好玩。但这正是你薪酬更高的原因之一，而且你欠你的团队、公司和客户的，要纠正这种情况。尽你所能帮助这个人找到他们能成功的工作，但要把注意力 laser sharp 地放在让合适的人来做必要的工作上。

一旦你相信你的团队成员有能力成功并配备了成功的条件，那么你需要退后一步，让这些有才华的人做他们的工作。如果你微观管理你的产品经理，他们不会以你需要的方式站出来承担责任。如果你不能信任你的产品经理，你需要找到你可以信任的产品经理。这并不意味着你不应该提问并时刻提供帮助——你绝对应该。但如果你授权给强大的人并让他们做他们的工作，我保证你会对他们能做的事情感到惊讶。

注意，你必须首先确保你有强大的人，然后才能授权给他们。如果你授权给没有能力的人，你就是在放弃你作为经理的责任。如果你微观管理没有能力的人，你实际上就是在替他们做他们的工作。

每个好的经理都知道，让自己看起来好的最好方法是他或她的团队成员看起来好。因此，始终雇用你认为比自己更聪明的人，然后尽你所能帮助他们成功。

定义公司的产品战略

没有人比产品管理组织的负责人对公司的产品套件负有更多责任或更多问责。这个人决定追求什么产品，然后密切审查每个产品的战略和进展。

产品管理负责人必须对公司的业务战略有深入和 current 的理解，这样她才能确保产品战略直接支持业务战略。这个人还在定义产品愿景以及与团队合作实现这一愿景方面发挥领导作用。产品原则（见"产品原则"一章）通常由完整的产品管理团队建立，但产品管理负责人领导这项工作，并确保产品尽可能遵守这些原则。

即使有最好的产品经理团队，每个人都在做着出色的工作，当每个产品经理努力优化自己的产品时，你仍然会有跨产品冲突。产品管理负责人必须努力识别和解决这些跨产品问题。

同样，产品管理负责人负责产品组合路线图——总体查看计划中的许多产品发布，并考虑业务需求和客户影响。

最后，产品管理负责人需要管理组织内的执行关系。公司所有关键人物——特别是 CEO——与产品管理负责人有良好的信任关系至关重要。整个公司都依赖这个人，他必须对他的决定和理由开放和透明，所有人都可以接近和接触。她必须接受来自任何来源的新想法，但也要受到足够的尊重，以便在适当的时候对有冲突或摇摆不定的优先级进行反驳。

正如你所见，这是一个要求极高的工作，但伟大的产品公司有伟大的人担任这个角色，这不是巧合。

如何衡量产品经理?

我经常被问到应该如何衡量产品经理。我一直认为衡量产品经理的唯一真正标准是他或她的产品的成功。虽然我仍然相信这一点，但这不是一个令人满意的答案，因为不清楚衡量产品成功的最佳方式是什么。是收入？利润?

用户数量？页面浏览量？所有这些指标可能都有用，但它们并没有真正提供衡量产品经理成功所需的全局视野。

最近，一个新的商业指标在许多不同行业获得了关注：净推荐值（Net Promoter Score，NPS）。这是一个非常简单的指标，你可以在 www.netpromoter.com 上阅读所有相关内容。

它的工作方式是这样的：你问你的客户他们有多大可能向他人推荐你的产品，评分从 0-10。那些评 9 或 10 的人被认为是"推荐者"（他们在外面告诉朋友他们有多喜欢你的产品，并积极地为你布道）；那些评 7-8 的人是温和的或中立的；而那些评 0-6 的人，"贬损者"，即他们不仅不会推荐你的产品，甚至可能积极警告他们的朋友或公众反对你的产品。如果你用推荐者的百分比减去贬损者的百分比，你就得到了 NPS，它 essentially 告诉你是有更多人为你还是反对你加油。

相当多的公司已经在跟踪这个指标，那些评分非常高的公司不会让你惊讶——像苹果、亚马逊、谷歌和 eBay 这样的公司。

我非常喜欢这个指标，因为它把注意力集中在产品和整体客户体验上。如果，正如我们在这个行业喜欢说的，一切都是关于创造快乐的客户，那么这正是对此的衡量。是的，理论上你可能有 100% 非常快乐的客户，但最终因为每个客户都在亏钱而倒闭。但对于单个指标，我相信这让公司的注意力集中在创造快乐的客户上，这是增长的燃料。就盈利能力而言，最具成本效益的销售或营销项目是你自己的客户为你做销售和营销工作。一个拥有快乐客户的伟大产品降低了这些其他（通常非常显著的）成本，这有助于盈利能力。

这个指标的另一个好处是，它引出了好收入和坏收入的概念。例如，假设公司被一个觊觎你的用户基础并相信他们能有效地在你的网站上销售他们产品的大潜在大赞助商或广告合作伙伴接近。这可能是好事也可能是坏事，取决于如何做。

如果做得不好，短期收入可能不错，但如果它损害了你的客户体验，这将反映在 NPS 中，你的业务增长将会放缓。另一方面，如果做得好——例如，通过与广告合作伙伴合作——它可能对客户体验是中性的，甚至有助于体验。这将帮助你的业务更快地增长。

这就是为什么特殊需求（为特定客户做明确定义的工作，通常以换取他们同意购买你的产品）如此危险。它们代表坏收入，并损害公司交付创造快乐用户产品的能力。

你可以跨公司甚至跨行业比较你的 NPS，这很有趣，但 NPS 最有用的地方是衡量你自己在改进产品和服务方面的进展。所以如果你还没有衡量你的 NPS，考虑尽快这样做。然后你可以开始观察你对产品所做的改变如何影响分数。确保你始终朝着正确的方向前进，并考虑你所做的一切对 NPS 的影响。

产品管理应该属于哪里?

许多公司都在纠结产品管理应该在组织中处于什么位置。对于大多数公司来说，选择通常是工程或营销。如果你有合适的个性，它在任何地方都可以工作，但我实际上不喜欢它属于任何一个。

我在合作的公司中发现最常见的情况是产品管理属于营销组织。这种组织设计的问题在于它基于一种误解，即通过与客户交谈就能获得产品，而且与客户交谈是营销的工作。我不会在这里重复这种思维方式的所有谬误，但 suffice 说，有几个关键原因说明为什么你不会仅仅通过询问客户就找到成功的产品。

同样有问题的是，通常情况下产品营销角色和产品管理角色在营销组织中合并。这些角色——以及执行它们所需的技能——差异如此之大，以至于通常最终发生的是一个或另一个（或两者）执行得很差。

我遇到的第二常见的情况是产品管理被放在工程组织中。虽然这有好处，让发明和设计产品的人与真正构建产品的人在一起，但这也可能有问题。为什么？因为工程组织通常被设计为专注于正确地构建产品，而不是构建正确的产品。产品管理团队很容易被消耗在产生详细规格的细节和压力中，而不是着眼于市场机会和发现成功的产品战略和路线图。想出正确的产品来构建需要不同的心态和不同的技能。此外，工程本质上是一种基于执行的活动，在优化执行的组织中很难进行发现活动。

所以如果不是营销组织，如果不是工程组织，那么在哪里?

我强烈主张提高产品组织的级别，使其在组织上与工程和营销平级。理想情况下，产品组织包括设计团队，因为产品管理和用户体验设计之间的互动绝对需要尽可能紧密。越来越多的情况下，你会看到一个名为产品或产品管理或产品管理和设计的组织——通常由产品副总裁甚至首席产品官管理。

这种组织设计有几个好处，但最大的原因是产品负责人需要在执行团队中有一席之地。公司都是关于产品的，营销和工程都是各自有各自考虑和挑战的关键组成部分，产品很容易在这些问题的讨论中被淹没。此外，这种组织结构明确了产品不是由技术驱动的，也不是仅由销售或营销需求驱动的。

在许多大公司中存在一个特殊情况。通常大公司有一个集中的工程职能和分散的业务单元。这让公司能够专注于多条业务线，同时可能享受共同工程服务的效率。在这样的组织中，产品管理和设计职能可能位于集中的工程/产品开发组织，或者它可能在自己的组织，或者它可能是业务单元本身的一部分。通常在这样的组织中，业务单元经理必须发挥重要的产品管理作用，如果产品管理团队不是业务单元的一部分，就会产生问题。在这些情况下，我通常更喜欢将产品管理和设计整合到业务单元中。

虽然我解释了理想位置的原因，但实施组织结构变革可能非常困难，你的公司可能不愿意或无法走这条路。这并不一定意味着你注定会有问题。归根结底还是取决于涉及的人员和他们带来的技能。如果你能发展你的产品团队的技能并展示他们在整个公司的价值，任何这些组织结构都可以成功。

示例

你可以在 www.svpg.com/examples 上看到示例产品战略、产品路线图和产品组合路线图。

第 8 章：巴顿给产品经理的建议

> Chapter 8: Patton'S Advice For
> PRODUCT MANAGERS

巴顿给产品经理的建议

> Managing By Objective

目标管理

> “Never tell people how to do things. Tell them what to do, and they will surprise you with their ingenuity.”

"永远不要告诉人们如何做事。告诉他们该做什么，他们会用他们的聪明才智让你惊讶。"

> —General George S. Patton, Jr. General Patton was quite a character. From what I know of him, it sounds like he would have been quite a product manager. He's the source of countless quotes and advice, but I want to consider the above quote and how I think it applies to product managers in two very important ways. First, on the receiving end, customers and users will very often try to tell you as product manager how your product should work, rather than what your product should do. We all have experienced this, as it's simple human nature for us to try to envision solutions to problems. But when a product manager focuses on what problem to solve, it's pretty amazing how many possibilities open up as to how to best solve the problem. Customers and users really aren't in a position to come up with a good solution themselves.
>
> They just don't know what's possible, and it's also extremely hard to envision the solution in advance.
>
> Also relevant is the other side of this point, where the product manager tells the user experience designers and engineers how to design and build the product, rather than telling them what the product needs to do. This is an especially common problem with user experience design. This problem is exacerbated by the fact that companies typically have too few design resources, and sometimes the product manager is in fact the only person available to do the interaction design.
>
> Unfortunately, the skills for interaction design are very different from product management, and it's the rare product manager who's good at both. Another problem I see that complicates this is that, in many companies, the user experience design resources are part of a service organization

> where they're pulled in “as needed” to help with design after the spec is complete. The problem here is that this severely limits the contribution of a good designer. Designers are most valuable very early in the process, when the product manager is working to understand the target market and come up with a solution.
>
> A product manager who (a) has a good appreciation for the role of user experience design, (b) is in a company that has the user experience design function staffed and available, (c) gives the user experience designer the latitude to come up with solutions that meet the needs, has a big advantage in coming up with winning products.
>
> Strong user experience designers, especially interaction designers, are hard to find. There simply aren't enough good academic programs turning out these critically important people.
>
> But when you do find one, be sure to fully utilize her. Make the designer a key part of your product team, and include her in your customer visits, personas, product brainstorming, and in deciding your product strategy and roadmap. Let her explore alternative designs and listen closely to her input in terms of user behavior and preferences.
>
> All of the above also applies with engineers. The engineering team doesn't appreciate the product manager spelling out the details of the implementation any more than the product manager wants the customer to dictate the specifics of the requirements. In my experience, this is somewhat less frequent because the line between product managers and engineers is fairly clear, but when I review specs I find that it still happens a great deal.
>
> So the more latitude you can give your engineers and user experience designers in coming up with the solutions to the problems you are trying to solve, the more likely they will come up with something that customers will love.

——小乔治·S·巴顿将军

巴顿将军是个相当有个性的人。据我所知，他会是一个相当出色的产品经理。他是无数名言和建议的来源，但我想考虑上述名言，并思考它以两种非常重要的方式适用于产品经理。

首先，在接收端，客户和用户经常会试图告诉你作为产品经理你的产品应该如何工作，而不是你的产品应该做什么。我们都经历过这种情况，因为对我们试图想象问题的解决方案，这是简单的人性。但当产品经理专注于解决什么问题时，令人惊讶的是有多少可能性会打开，关于如何最好地解决问题。

客户和用户真的无法自己提出一个好的解决方案。

他们就是不知道什么是可能的，而且提前设想解决方案也非常困难。

同样相关的是这一点的另一面，即产品经理告诉用户体验设计师和工程师如何设计和构建产品，而不是告诉他们产品需要做什么。这在用户体验设计中是一个特别常见的问题。这个问题因公司通常设计资源太少而加剧，有时产品经理确实是唯一可以来做交互设计的人。

不幸的是，交互设计的技能与产品管理截然不同，能同时擅长两者的产品经理很少见。我看到的另一个使情况复杂的问题是，在许多公司中，用户体验设计资源是服务组织的一部分，他们在规格完成后被"按需"拉入帮助设计。这里的问题是这严重限制了好设计师的贡献。设计师在产品经理努力理解目标市场并提出解决方案的过程早期最有价值。

一个（a）对用户体验设计的角色有很好的理解，（b）在一个有用户体验设计职能并配备人员且可用的公司，（c）给用户体验设计师自由度来提出满足需求的解决方案的产品经理，在提出成功产品方面有巨大优势。

强大的用户体验设计师，特别是交互设计师，很难找到。好的学术项目培养出这些至关重要的人还不够多。

但当你找到一个时，一定要充分利用她。让设计师成为你产品团队的关键部分，并让她参与你的客户访问、用户画像、产品头脑风暴，以及决定你的产品战略和路线图。让她探索替代设计，并仔细听取她关于用户行为和偏好的意见。

以上所有也适用于工程师。工程团队不欣赏产品经理详细说明实现细节，就像产品经理不希望客户规定需求的具体内容一样。根据我的经验，这种情况发生得较少，因为产品经理和工程师之间的界限相当清楚，但当我审查规格时，我发现这种情况仍然经常发生。

所以你能在多大程度上给你的工程师和用户体验设计师自由度来提出你试图解决的问题的解决方案，他们就越有可能想出客户会喜欢的东西。

第 9 章：副产品经理

> Chapter 9: Deputy Product Managers
>
> The Smartest Guy In The Room

房间里最聪明的人

> As product people, we're essentially in the idea business. It's our job to come up with great ideas and then make them a reality. While this takes skill and practice, the main ingredient is something that I don't know how to teach. We depend on smart people for the smart ideas. Sometimes these ideas come from within ourselves, but if we depend only on ourselves for the smart ideas, we're severely limiting our potential universe of smart ideas.

作为产品人员，我们本质上处于创意业务中。我们的工作是想出伟大的想法，然后使它们成为现实。虽然这需要技能和练习，但主要成分是某种我不知道如何教的东西。我们依赖聪明人的聪明想法。有时这些想法来自我们自己，但如果我们只依赖自己获取聪明想法，我们就严重限制了潜在聪明想法的宇宙。

> Probably the single most important lesson I've learned in the product business is to start by seeking out the smartest people in the company. I've found that every organization has at least a few very smart people, and these people may hold the key to unlocking your company's potential—if you can find them. They're not always where you'd guess, and sometimes they're being hidden from you. I never cease to be amazed at how petty office politics, ego, xenophobia and insecurity can get in the way of something so potentially beneficial for a company.

我可能在产品业务中学到的最重要的一课是，从寻找公司里最聪明的人开始。我发现每个组织至少有几个非常聪明的人，如果你能找到他们，这些人可能掌握着解锁公司潜力的关键。他们并不总是在你猜测的地方，有时他们被对你隐藏着。我从不停止惊讶于 petty 办公室政治、自我、仇外心理和不安全感如何能阻碍对公司如此潜在有益的事情。

> When you do find these people, you can use them any number of ways. I like to consider these people “deputy product managers” and sometimes I even give them public recognition as such. Often I'll recruit these people to come join the product team.

当你找到这些人时，你可以以各种方式使用他们。我喜欢把这些人视为"副产品经理"，有时我甚至给他们公开认可。我经常招募这些人来加入产品团队。

> To illustrate the many different corners of your company that may be hiding these people, here are some favorite examples from my own career. Every one of these examples is based ona real person, but I have changed their names.

为了说明可能隐藏这些人的公司不同角落，以下是我职业生涯中的一些最喜欢的例子。每一个例子都基于一个真实的人，但我改变了他们的名字。

> Sam: It took me longer to find Sam than it should have because his manager was actively bad-mouthing him. However, it quickly became clear that it was the manager who was clueless, because what was really going on was that the manager was insecure and intimidated by Sam's mind. So not only had Sam not been recognized and utilized, he had actually been demoted! Today, that manager is history and Sam is one of the best product leaders I know.

Sam：找到 Sam 花了我比应该花的时间更长，因为他的经理在积极地说他坏话。然而，很快就很清楚是那个经理一无所知，因为真正发生的是经理不安全，被 Sam 的头脑吓到了。所以 Sam 不仅没有被认可和利用，他实际上被降级了！今天，那个经理已经不存在了，Sam 是我认识的最好的产品领导者之一。

> Chris: I met Chris when I was out assisting a customer visit with HP, and our salespeople were making little sense when briefing us on the local considerations. Finally, an SE (systems engineer—they provide technical assistance to the sales staff) stepped in and did an outstanding job articulating what the situation was. I could see the respect that the customer had for the SE, and afterwards I invited him to grab a beer. It was soon very clear to me that I was sitting with an extremely talented guy. I asked him why he was hiding in the Midwest as an SE, and he explained he had family in the area, he had never thought of living elsewhere, and that he had taken the best job he could find. I immediately began to use Chris as a sounding board and source of product ideas and, while it took a while, I finally got him to relocate. Today he's a general manager at a Fortune 500 company. While engineers often have great insight into the available technologies, people from the field often have great insight into customer needs, which for Chris, combined to give him extraordinary insight into user problems plus the possible solutions.

Chris：我在协助 HP 的客户访问时遇到了 Chris，我们的销售人员在向我们介绍当地情况时说的没什么道理。最后，一位 SE（系统工程师——他们为销售人员提供技术援助）站出来，出色地阐述了情况是什么。我能看到客户对 SE 的尊重，之后我邀请他喝一杯。很快我就很清楚，我正坐在一个非常有才华的人面前。我问他为什么躲在中西部做 SE，他解释说他有家人在那里，他从未想过住在别的地方，而且他找了能找到的最好的工作。我立即开始把 Chris 用作咨询对象和产品想法的来源，虽然花了一段时间，但我终于让他搬了家。今天他是财富 500 强公司的总经理。虽然工程师通常对可用技术有很好的洞察力，但来自现场的人往往对客户需求有很好的洞察力，对于 Chris 来说，两者结合使他对用户问题加上可能的解决方案有非凡的洞察力。

> Alex: As is so often the case, I found Alex deep in the ranks of an engineering team. He was shy and introverted, and not especially ambitious. But the guy was incredibly smart. He not only knew technologies extremely well, but he had a natural product sense, understood the broader technology trends, and he was a constant champion of the user experience. He's a great engineer, and people assumed that that represented his full potential. However, Alex had an equally talented product mind, and he was one of those rare people who are great at just about everything. He never made the move to product, but he did become one of the thought leaders in the company and was consulted on virtually every major product decision. And the company and the products we built were much better for it.

Alex：和往常一样，我在工程团队的深处找到了 Alex。他害羞且内向，不是特别雄心勃勃。但这家伙非常聪明。他不仅对技术非常了解，而且有天然的产品感，理解更广泛的技术趋势，而且他是用户体验的坚定拥护者。他是一位伟大的工程师，人们认为这代表了他的全部潜力。然而，Alex 同样有才华的产品头脑，他是那些几乎在所有事情上都很出色的人之一。他从未转到产品部门，但他确实成为了公司的思想领袖之一，几乎所有重大产品决策都会咨询他。公司和我们的产品因此变得更好。

> Matt: I wish it was not the case, but there are many forms of discrimination in business, even in high tech. But one form of discrimination that I had thought would have been long gone by now is discrimination due to youth. Matt is probably the most brilliant person I've ever worked with. He graduated college while still in his teens, and he never slowed down. But when I met Matt, he was dramatically underutilized because his manager couldn't imagine giving someone so young that much responsibility. Big mistake. Matt jumped ship and went on to co-found a startup that has improved the lives of millions.

Matt：我希望不是这种情况，但商业中存在许多形式的歧视，即使在高科技领域也是如此。但我原以为到现在应该已经消失的一种歧视形式是因年轻而产生的歧视。Matt 可能是我合作过的最聪明的人。他在十几岁时就大学毕业了，而且从未放慢脚步。但当我遇到 Matt 时，他被严重低估利用，因为他的经理无法想象给这么年轻的人那么多责任。大错误。Matt 跳槽并继续共同创立了一家改善了数百万人生活的初创公司。

> Mira: This talented individual had it twice as tough—she was female and Indian. In this loud, heavily male, technology-driven industry, women are easily overlooked. And culturally, Indians are often quiet and reluctant to challenge authority—that of their managers or their colleagues. But Mira was easily the smartest “guy” in the room, and it didn't take long to draw her out of her shell

> and for her to establish herself as the product leader she was meant to be. I've seen this with Chinese nationals as well. Don't let cultural

Mira：这位才华横溢的人有两倍困难——她是女性，而且是印度人。在这个嘈杂、男性主导、技术驱动的行业中，女性很容易被忽视。而且，从文化上讲，印度人通常很安静，不愿意挑战权威——无论是他们的经理还是同事。但 Mira 很容易是房间里最聪明的"家伙"，没过多久就把她从壳里拉出来，让她建立自己注定要成为的产品领导者的地位。我在中国公民身上也看到了这一点。不要让文化规范或口音让你失望——这些可能是你正在寻找的产品思维。

> norms or an accent through you off—these may be the product minds you're searching for. I've learned that sometimes the greatest product minds are right there in front of you. You may be at a company that's enjoyed some success, and the product mind that got you there is now CEO or chairman of the board, and seemingly unreachable to today's product team. If the founder is good, he's probably trying not to step in and micro-manage things himself, but that doesn't mean that he's not willing to help. If you're lucky enough to have great product people as founders, you should initiate a channel with them and invite their feedback and suggestions on your product plans. They're often all too happy to do so, and you should absolutely find a way to utilize that resource if it is available to you.

我了解到有时最伟大的产品思维就在你面前。你可能在一家已经取得一些成功的公司，让你到达那里的产品思维现在是 CEO 或董事会主席，对今天的产品团队来说似乎无法触及。如果创始人很好，他可能正试图不介入和微观管理事情 himself，但这并不意味着他不愿意帮忙。如果你有幸有伟大的产品人作为创始人，你应该与他们建立沟通渠道，并邀请他们对你的产品计划提供反馈和建议。他们通常非常乐意这样做，你绝对应该找到一种方法来利用这种资源（如果它对你可用）。

> The bottom line is that these minds can be hidden anywhere—engineering, sales, customer service, professional services, the exec team, or your board of directors. It's your job to find them. Now—how do you do that?

底线是这些思维可能隐藏在任何地方——工程、销售、客户服务、专业服务、执行团队或你的董事会。你的工作就是找到他们。现在——你怎么做?

> - Ask! Ask at all levels of the company who people think the really great minds are, and you may be surprised by their answers.

询问！询问公司各级人员他们认为真正伟大的思维是谁，你可能会对他们的答案感到惊讶。

> - MBWA. From the HP Way—Management By Wandering Around. Managers need to get out of their office or cubes and spend time with the people from across the company, not in meetings, but informally. It's easy and it works.

MBWA。来自惠普之道——走动式管理。管理者需要走出他们的办公室或隔间，花时间与全公司的人在一起，不是在会议上，而是非正式地。这很简单，而且有效。

> · Listen (really listen) to the dialog in meetings and conversations.

倾听（真正倾听）会议和对话中的对话。

> · Keep your door open—make sure everyone knows that they are welcome to drop in with product suggestions.

保持你的门敞开——确保每个人都知道他们可以随时带着产品建议来找你。

> · Share. If you are willing to share with others the issues that you're struggling with, you'll find that word will get around and people may stop by with suggestions.

分享。如果你愿意与他人分享你正在努力解决的问题，你会发现消息会传开，人们可能会带着建议来找你。

> · Hang Out. All too often the product people hang out with other product managers, and

闲逛。产品人员往往和其他产品经理一起闲逛，

> execs with other execs. But if you make an effort to spend time with people at all levels of

高管和其他高管在一起。但如果你努力花时间与公司各级的人在一起，你会对总体情况有更好的了解，特别是谁是隐藏的宝石。

> your company, you'll get a much better idea of what's going on in general, and who the

最后，我认为大多数产品经理没有关注以上几点的主要原因是自我。他们认为他们必须是那个总是提出大想法的人，如果其他人提出了，他们的目的会是什么？虽然有时我会想出一个我喜欢的想法，但更多时候我是从他人那里获得灵感或核心想法本身。你仍然需要能够识别那个伟大的想法，并使那个想法成为现实。只要记住，对你的公司来说真正重要的唯一事情是交付伟大的产品。任何帮助你做到这一点的事情都是好的。

> hidden gems are in particular. Finally, the main reason why I think most product managers are not focused on the above is ego. They think that they are the ones that must always come up with the big ideas, and that if someone else did, what would their purpose be? While sometimes I come up with an idea I like, more often than not I get my inspiration or the core idea itself from others. You still need to be able to recognize that great idea, and make that idea a reality. Just remember that the only thing that really matters for your company is shipping great products. Anything that helps you do that is all good.

第 10 章：向上管理

> Chapter 10: Managing Up
>
> Top 10 List

十大技巧

> One of the most common questions I get from product managers, most often at large companies, is how to manage their managers. They are frustrated with their managers—it's not that they don't like them, but they feel like the sands keep shifting. Their managers give them different and conflicting direction each week, and it's always two steps forward and one step back. Especially in big companies, there are so many influencers and stakeholders that getting a company to move in a single direction long enough to get a product out can be a true challenge.

我从产品经理那里得到的最常见的问题之一，最常在大型公司，是如何管理他们的经理。他们对他们的经理感到沮丧——不是他们不喜欢他们，而是他们感觉沙子在移动。他们的经理每周给他们不同和冲突的方向，总是前进两步后退一步。特别是在大公司，有如此多的影响者和利益相关者，让公司朝着一个方向移动足够长的时间以推出产品可能是一个真正的挑战。

> There are many reasons for the constant change in direction. It's not just you and your manager who need to be on the same page, your manager's manager—and others on up the chain—all bring their own initiatives to the table. And there are outside influences as well, such as competitive pressures, changing technologies, mergers and acquisitions, business development deals, budget and staffing constraints, and more. Each of these can—and usually do—have a direct or indirect impact on your product plans. That's part of the cost of working at a large firm. The benefit, however, is that if you can find a way to leverage the resources of your large company, you can have a dramatic impact on the marketplace on a

方向不断改变有很多原因。不只是你和你的经理需要在同一页上，你的经理的经理——以及链条上的其他人——都把他们的倡议带到桌面上。还有外部影响，如竞争压力、技术变化、并购、业务发展交易、预算和人员限制等等。每一个都可能——而且通常会——对你的产品计划产生直接或间接的影响。这是在大公司工作的成本的一部分。然而，好处是，如果你能想办法利用你大公司的资源，你可以对市场产生在小公司难以匹敌的规模上的巨大影响。

> scale that's hard to match at a small firm.

即使在小型公司和初创公司，这些挑战也存在。但我认为在大公司尤其困难。我在大大小小的产品组织工作过，在每个级别，我收集了一些帮助我这个问题的技巧。但我要先说，挑战是巨大的，它们不会消失；你能做的最好的事情是缓解这些问题，这里有一些方法可以做到这一点。

> Even in small companies and startups, these challenges exist. But I think that in large companies it is especially difficult. I have worked in product organizations large and small, at every level, and I've collected a list of techniques that have helped me with this problem. But I will say up front that

> the challenges are substantial and they won't go away; the best you can do is to mitigate the issues, and here are some ways to do that.

所以这里是我向上管理的十大技巧：

> So here's my list of ten techniques for managing up:

1. 衡量和计划变更。

> 1. Measure and plan for churn.

变更是我用来代表各种演练、返工和计划改变所导致的成本的术语，这些正是你首先感到沮丧的原因。虽然你不应该期望变更降到零，但你可以不断努力减少它。这从提高对变更的认识开始，而这从衡量开始。有很多方法可以做到这一点，但以某种形式，尝试跟踪你的周/月/季度中有多少花在了向前进展上。现在你对变更水平有了更多认识，计划一定程度的变更会有很大帮助。当你安排项目时，要知道会有一定比例的时间用于响应这些变化和相应调整，而且你的一些努力最终会搁置在架子上。这将有助于管理你的压力水平，使你的时间表更准确，并帮助你识别可以尝试改进的问题。

> Churn is the term I use to represent the cost of the various drills, rework, and changes in plans that cause the frustration you feel in the first place. While you shouldn't expect churn to go down to zero, you can constantly strive to reduce it. This starts by increasing awareness of churn, and that begins with measuring it. There are lots of ways to do this, but in one form. or another, try to track how much of your week/month/quarter is spent on forward progress. Now that you're more aware of the level of churn, it helps a great deal to plan for some amount of churn. When you're scheduling your projects, know that there will be a percentage of your time devoted to responding to these changes and adjusting accordingly, and that some amount of your efforts will end up sitting on the shelf. It will help manage your stress level, make your schedules more accurate, and help you identify issues you can try to improve.

1. 沟通风格和频率。

> 1. Communication style and frequency.

就像产品经理并不都一样，经理也不都一样。有些经理喜欢随时了解每个小细节。其他人不想你打扰他们，除非有升级或严重问题需要你的经理的帮助。有些人喜欢用书面形式提供详细的支持材料更新，其他人喜欢在走廊里快速聊几句。你需要确定你的经理喜欢的风格，并尽力满足这种需求，即使这不是你自己喜欢的风格。

> Just as product managers are not all the same, managers are not all the same either. Some managers prefer to be kept apprised of every little detail on a continual basis. Others don't want you to bother them unless there's an escalation or serious issue that needs your manager's help. Some prefer updates in writing with detailed supporting material, and others prefer a quick chat in

> the hall. You need to determine the style that your manager prefers and do your best to meet that need, even if it's not your own preferred style.

1. 会前工作。

> 1. Pre-meeting work.

大多数产品公司有很多会议——在我看来太多了。然而，你的组织中的影响者和利益相关者越多，你就会被要求有越多的检查和评审会议来让每个人都在轨道上并了解情况。有很多运行良好会议的技巧，但这里的主要点是在正式会议之前实际上进行真正的会议。这意味着在实际会议之前单独去找关键影响者和利益相关者，给他们预览你的要点，倾听他们的问题，并确保在小组会议发生时他们已经在船上。如果你做得好，小组会议应该很快而且没有意外。然而，正式会议仍有重要目的，就是让桌上的每个人都看到其他人在船上。

> Most product companies have lots of meetings—too many in my view. However, the more influencers and stakeholders there are in your organization, the more you'll be asked to have checkpoint and review meetings to keep everyone on track and informed. There are many techniques for running good meetings, but the main point here is to actually conduct the real meetings before your official meeting. This means going individually to the key influencers and stakeholders prior to the actual meeting and giving them a preview of your points, listening to their issues, and ensuring that they are already on board by the time the group meeting happens. If you do this well, the group meeting should be quick with no surprises. The formal meeting still has an important purpose however, which is for everyone at the table to see that everyone else is on board.

1. 建议，而不是问题。

> 1. Recommendations, not issues.

大多数经理更喜欢看到你关于如何解决遇到问题的建议，而不仅仅是问题的陈述。理想情况下，取决于问题的大小，这意味着对几种替代方案的分析以及你的建议和理由。

> Most managers prefer to see your recommendations on how to solve problems you encounter rather than just a statement of the problem. Ideally, depending on the size of the problem, this means an analysis of several alternatives along with your recommendation and rationale.

1. 利用你的经理。

> 1. Use your manager.

经理通常是一个非常有用的工具，但大多数员工利用不足。例如，假设你正在努力解决一个问题，你有分析和建议，但一些关键影响者不愿意抽出时间让你按照上述会前工作中描述的那样预先简报他

们。你的经理通常可以获得你无法获得的访问权限。所以给你的经理提供工具和请求，让她为你举行这个私人会议。你的经理会想要做好准备，但通常很乐意以这种方式帮助你。

> Managers can often be a very useful tool that most employees underutilize. As an example, suppose there's a problem you're working to solve, and you have an analysis and recommendation, but some of the key influencers are not anxious to make the time available you need to pre-brief them as described in the pre-meeting work above. Your manager can often get the access you can't. So provide your manager with the tools and the request that she hold this private session for you. Your manager will want to be prepared, but is often happy to help in this way.

1. 做好功课。

> 1. Do your homework.

产品经理犯的最大错误之一就是没有做好功课。经理通常很聪明，能快速识别思维和计划中的漏洞——那是他们的工作。你为此准备的最好方式就是做好功课。你需要彻底理解问题并做好准备。

> One of the biggest mistakes product managers make is in not doing their homework. Managers are usually smart and can quickly identify holes in thinking and in plans—that's their job. The best way for you to prepare for this is by doing your homework. You need to understand the issues thoroughly and be prepared.

1. 简短的电子邮件。

> 1. Short e-mails.

另一个常见错误是产品经理喜欢给他们的经理写长而详细的电子邮件，但当这些邮件没有被阅读或回复时会感到沮丧。你需要意识到你的经理可能每天要收到数百封电子邮件，正在寻找简短、切中要害的通信。你发送的人越高级，你就越希望邮件简短。提供额外的材料，但不要让经理读超过几行。

> Another common mistake is that product managers like to write long, detailed e-mails to their managers, but then get frustrated when they're not read or responded to. You need to realize that your manager is probably getting hundreds of e-mails a day, and is looking for short, to-the-point communications. The more senior the person you're sending to, the shorter you'll want the e-mail to be. Offer additional material, but don't make the manager read more than a few lines.

1. 使用数据和事实，而不是观点。

> 1. Use data and facts, not opinions.

与经理打交道时——尤其是高级经理——必须记住你的工作是提供数据和事实。网景前 CEO Jim Barksdale 在面对困难决策时有一句很棒的话。他说，"如果我们要基于观点做决策，我们将使用我的观点。"如果你做了功课，收集并展示了数据，你的建议应该基于事实而不是观点就很清楚了。

> When dealing with managers—especially senior managers—it's essential to remember that your job is to provide the data and facts. Jim Barksdale, the former CEO of Netscape, had a great line when he was confronted with difficult decisions. He said, “If we're going to make this decision based on opinions, we're going to use my opinion.” If you do your homework, and have collected and laid out the data, your recommendation should be clear based on the facts and not opinion.

1. 布道。

> g. Evangelize.

产品经理工作的一个重要部分是在整个组织中为产品布道。但很少有产品经理像我他们认为的那样认真对待这一点。如果你有效地布道，一切都会变得更容易——尤其是与公司其他团队合作。如果他们知道你在做什么，并对你的产品将为公司做什么感到兴奋，他们会更愿意找到帮助的方法。

> A big part of a product manager's job is to evangelize the product across the organization. But few product managers seem to take this as seriously as I think they should. If you evangelize effectively, everything will become easier—especially working with other groups in the company. If they know what you're doing and are excited about what your product will do for the company, they'll be much more likely to find ways to help.

1. 低维护员工。

> 1. Low-maintenance employees.

几乎每个经理都认为但很少有人会承认的一个秘密是，他们真正在员工中寻找的是低维护的人。高维护员工消耗经理不成比例的时间和注意力，虽然确保他的团队高效是经理的工作，但一天只有那么多时间，这种类型的手把手通常不是你的经理急于花费他一天时间做的事情。不要试图把你的经理当作导师——从你的直接管理链之外找另一位导师。并 thoughtful 地使用你经理的时间。我可以向你保证，你的经理会感激的。

> One of the secrets that nearly every manager thinks—but few will admit—is that what they ‘re really looking for in an employee is someone who is low maintenance. High-maintenance employees consume a disproportionate amount of the manager's time and attention, and while it's your manager's job to ensure that his team is productive, there is only so much time in the day, and this type of hand-holding is not usually what your manager is anxious to spend his day doing. Don't try to use your manager as a mentor—find another mentor from outside of your direct management chain. And be thoughtful of how you use of your manager's time. I can promise you that your manager will appreciate it.

许多产品经理感到沮丧，特别是在大公司。如果这是你的情况，试试这些技巧。你不会消除这些问题，但希望你能看到真正的改善。

> Many product managers get frustrated, especially in large companies. If this is your situation, give these techniques a try. You won't eliminate the issues, but hopefully you'll see a real improvement.

> a PROCESS
> ONE OF US WILL H UNLESS IT GETS

活动和最佳实践

当今顶级软件和互联网公司使用的流程和技术，在许多情况下与大多数公司使用的非常不同。

本节描述用于反复发现和构建令人启发和成功产品的流程、活动和最佳实践。

> HAVE TO READ THIS | |2 DESTROYED IN
> GIGANTIC PRODUCT | |F A FREAK
> REQUIREMENTS i ACCIDENT.
> DOCUMENT. a ie
> fof i
> (U2 sANT
> Noe tS Y
> 2] (IT'S LIKE WATCHING
> | C THOMAS EDISON
> 3] (work. i) 8 (TUNE some
> 3 £4 OILY RAGS
> 8] (Pe cute. I
> Cue A
> DILBERT: © Scott Adams/Dist. by United Feature Syndicate, Inc. Activities and Best Practices
> The processes and techniques used by today's top software and Internet companies are in many cases very different than those used by most companies.
> This section describes the processes, activities, and best practices used to repeatedly discover and build inspiring and successful products.

第 11 章：评估产品机会

> Chapter 11: Assessing Product Opportunities
> Defining The Problem To Be Solved

定义要解决的问题

> Opportunities for new products exist all around us, in every market—even mature markets. This is because what is possible is always changing. New technologies are constantly emerging, competitors come and go, and new people with new talents and new ideas join your company.

新产品机会存在于我们周围，在每个市场——即使是成熟市场。这是因为可能发生的事情总是在变化。新技术不断涌现，竞争对手来来去去，有才华和新想法的新人加入你的公司。

> The product manager must be able to quickly evaluate opportunities to decide which are promising and which are not; what looks appealing, which should be pursued, which are better left for others, and which ideas are not yet ready for productization.

产品经理必须能够快速评估机会，决定哪些是有前途的，哪些不是；什么看起来有吸引力，哪些应该追求，哪些最好留给别人，以及哪些想法还没有准备好产品化。

> In many companies, the decree comes down from above, in something akin to, “We really need to do this product.” In other companies, the marketing organization determines what products are needed. And in yet others, the ideas come from engineering.

在许多公司，命令从上面下达，类似于"我们真的需要做这个产品。"在其他公司，营销组织决定需要什么产品。在另一些公司，想法来自工程部门。

> Unfortunately, too often the process of deciding whether or not to build a product is left to intuition (or worse, a large customer will offer to fund a “special,” and this becomes the basis for a product effort).

不幸的是，决定是否构建产品的过程往往留给直觉（或者更糟，大客户会提供资金做一个"特殊需求"，这成为产品努力的基础）。

> Typically someone on the business side or in marketing will create some form of a Market Requirements Document (MRD) that is intended to describe the problem to be solved and usually includes a business justification. The purpose of the MRD is to describe the opportunity, not the solution—at least, that's the theory. In practice, many companies don't really do MRDs, or if they do, they're essentially attempts at product specs that are misnamed as MRDs. Even if a true MRD is done, they suffer many of the same problems as PRDs—that is, they take too long to write, they aren't read, and they often don't answer the key questions they need to address.

通常业务方面或营销部门的人会创建某种形式的市场需求文档（MRD），旨在描述要解决的问题，通常包括业务理由。MRD 的目的是描述机会，而不是解决方案——至少，这是理论。在实践中，许多公司并不真正做 MRD，或者如果他们做，它们本质上是试图做产品规格但被错误命名为 MRD。即使做了真正的 MRD，它们也遭受与 PRD 相同的许多问题——即，它们需要太长时间来写，它们没有被阅读，而且它们经常不回答需要解决的关键问题。

> The result is that many product managers ignore the MRD altogether. But there's a problem with not doing anything and just jumping right into the product: it is generally a good idea to look before you leap. The challenge is to do this in a quick, lightweight, yet effective manner. I consider the Product Opportunity Assessment an extremely important responsibility of the product manager. The purpose of a good product opportunity assessment is to either (a) prevent the company from wasting time and money on poor opportunities by ultimately proving the idea should be shelved for now, or (b) for those opportunities that are good ones, focus the team and understand what will be required to succeed and how to define that success.

结果是许多产品经理完全忽视 MRD。但什么都不做就直接跳入产品是有问题的：一般来说，在跳跃之前先看看是个好主意。挑战是要以快速、轻量级但有效的方式做到这一点。

我认为产品机会评估是产品经理极其重要的责任。好的产品机会评估的目的是要么（a）通过最终证明这个想法现在应该搁置，防止公司在糟糕的机会上浪费时间和金钱，要么（b）对于那些好的机会，聚焦团队并理解成功需要什么以及如何定义成功。

> Fortunately, it's really not that hard to do a useful opportunity assessment. I ask product managers to answer ten fundamental questions:

幸运的是，做有用的机会评估并不难。我要求产品经理回答十个基本问题：

> 1. Exactly what problem will this solve? (value proposition)

1. 这究竟要解决什么问题？（价值主张）

> 1. For whom do we solve that problem? (target market)

1. 我们为谁解决这个问题？（目标市场）

> 1. How big is the opportunity? (market size) 4. How will we measure success? (metrics/revenue strategy) 5. What alternatives are out there now? (competitive landscape) 6. Why are we best suited to pursue this? (our differentiator) 7. Why now? (market window) 8. How will we get this product to market? (go-to-market strategy) 9. What factors are critical to success? (solution requirements) 10. Given the above, what's the recommendation? (go or no-go) Note that none of these questions speaks to the actual solution. This is both intentional and critically important. The opportunity assessment should just discuss the problem to be solved, not the particular solution you may have in mind. The majority of your time going forward will be focused on the solution, but this is the time to think clearly and concisely about the problem you are trying to solve. All too often what happens is that a product manager combines the problem to be solved with the solution and, when they run into difficulties with the particular solution they are pursuing, they abandon the opportunity. It's a classic example of throwing the baby out with the bathwater. The hardest question to answer is usually the first in the opportunity assessment, the value proposition, which surprises many people because it sounds like the easiest. But ask most product managers what problem their product is intended to solve, and you usually get a rambling list of features and capabilities, rather than the a crisp, clear and compelling statement of the exact problem that's being solved.

1. 机会有多大？（市场规模） 4. 我们如何衡量成功？（指标/收入策略） 5. 现在有哪些替代方案？（竞争格局） 6. 为什么我们最适合追求这个？（我们的差异化） 7. 为什么是现在？（市场窗口） 8. 我们将如何让这个产品上市？（上市策略） 9. 成功的关键因素是什么？（解决方案要求） 10. 鉴于以上，建议是什么？（做还是不做）

请注意，这些问题中没有一个谈到实际的解决方案。这既是故意的也是极其重要的。机会评估应该只讨论要解决的问题，而不是你可能想到的特定解决方案。你未来大部分的时间将专注于解决方案，但这是清晰简洁地思考你试图解决的问题的时候。

经常发生的情况是，产品经理把要解决的问题和解决方案结合起来，当他们遇到正在追求的特定解决方案的困难时，他们就放弃机会。这是典型的把婴儿和洗澡水一起倒掉的情况。

最难回答的问题通常是机会评估中的第一个，价值主张，这让很多人惊讶，因为它听起来最容易。但问问大多数产品经理他们的产品打算解决什么问题，你通常会得到一连串的功能和能力，而不是关于正在解决的确切问题的简洁、清晰和引人注目的陈述。

> Another difficult problem can be assessing the size of the opportunity. You can get thoughts on this from industry analysts, trade associations, your finance staff, and from your own bottom-up calculations. Just remember to be conservative and realize that not every opportunity needs to be a billion-dollar market.

另一个困难的问题可能是评估机会的大小。你可以从行业分析师、行业协会、你的财务人员和自己的自下而上的计算中得到关于这方面的想法。只要记住要保守，并意识到不是每个机会都需要是十亿美元的市场。

> The go-to-market strategy is especially important as it describes how the product will be sold, which can have very significant impact on product requirements.

上市策略尤其重要，因为它描述了产品将如何销售，这对产品需求可能有非常重大的影响。

> The success factors, or solution requirements, refer to any special needs or requirements that were identified during the investigation. Again, we're not describing the product here, but rather making clear any dependencies or constraints. For example, if this is a product that will be sold through system integrators, then these types of partners have requirements around extensibility of the products they deliver. Similarly, there may be branding or partnership requirements.

成功因素或解决方案要求指的是调查期间确定的任何特殊需求或要求。同样，我们在这里不是在描述产品，而是在明确任何依赖或约束。例如，如果这是一个将通过系统集成商销售的产品，那么这些类型的合作伙伴对他们交付的产品有可扩展性要求。同样，可能有品牌或合作要求。

> A product organization is all about pursuing good opportunities and providing great product solutions. Opportunities for new products are everywhere, and it is important that the product manager be able to effectively evaluate new opportunities and identify those that have the most potential for their company. It is just as important that bad product ideas get identified and rejected at this stage, before significant time and money is lost chasing them. Choosing the right set of products to pursue is among the most important decisions a company will make.

产品组织就是关于追求好的机会并提供伟大的产品解决方案。新产品的机会无处不在，产品经理能够有效地评估新机会并识别那些对其公司最有潜力的机会是很重要的。在这个阶段识别和拒绝坏的产品

想法同样重要，以免在追逐它们时浪费大量时间和金钱。选择追求正确的产品组合是公司要做的最重要决定之一。

> It is important that the results of the product opportunity assessment be presented and

重要的是产品机会评估的结果要呈现并与高级管理层讨论，

> discussed with senior management, and that the company make a clear decision on whether or not to pursue a product to meet this opportunity. If you do decide to proceed, you will be much better informed about what you are getting yourself into, and what it will take to succeed.

公司要明确决定是否追求产品以满足这个机会。如果你确实决定继续，你会更好地了解你将要做什么，以及成功需要什么。

> So what do you do if the CEO tells you that, like it or not, this is what we're doing, so just get to work on the product? First, realize that there are sometimes strategic reasons for doing a particular product, so you might need to pursue it even when it's unlikely to succeed in the marketplace. That said, doing a lightweight and quick product opportunity assessment is still valuable because you will become much better informed about what the product involves. It is possible that what you learn will change your CEO's opinion, but even if not at least you will have a clear understanding of your objective.

那么如果 CEO 告诉你，不管喜欢与否，这就是我们要做的，所以就开始做产品吧？首先，意识到有时做特定产品有战略原因，所以即使它在市场上不太可能成功，你也可能需要追求它。也就是说，做轻量级快速的产品机会评估仍然有价值，因为你会对产品涉及的内容有更好的了解。你学到的东西可能会改变你 CEO 的观点，但即使没有，至少你也会清楚你的目标。

> Build New Or Fix Old?

做新的还是修复旧的?

> I'm often asked what the right balance is between new product development and improving existing products. I suppose it's natural for companies to want to have some sort of percentage guideline, but I try to get companies to think about these investments a little differently. To me, all projects, whether a new 1.0 product, or an enhancement to an existing product, are product investments and instead of worrying about whether you're investing enough in new product lines versus existing product lines, I would rather have the team worry about investing in the best opportunities.

我经常被问到新产品开发和改进现有产品之间的正确平衡是什么。我想公司自然希望有某种百分比指导方针，但我试图让公司稍微不同地思考这些投资。对我来说，所有项目，无论是新的 1.0 产品，还是现有产品的增强，都是产品投资，与其担心你在新产品线与现有产品线上投资是否足够，我宁愿让团队担心投资最好的机会。

> Expanding your product offering so that you can offer additional products to existing customers or acquire new customers can be a great thing. Improving your existing products so that they generate more revenue from your existing customers plus make it easier to get new customers can also be a great thing.

扩展你的产品供应，以便向现有客户提供额外产品或获取新客户，可能是一件好事。改进现有产品，以便从现有客户那里产生更多收入，并更容易获得新客户，也可能是一件好事。

> The real key is that each of these projects is a product opportunity and, as such, it's the responsibility of the product team to assess the benefits and the costs. Then it's the responsibility of the management team (see the chapter The Product Council) to ensure the company is pursuing the best opportunities available. This might be completely or mostly new product opportunities, particularly for new companies, or it might be mostly product improvement opportunities for a more established business. It's not a bad thing to be opportunistic when it comes to choosing your product investments.

真正的关键是，这些项目中的每一个都是产品机会，因此，评估收益和成本是产品团队的责任。然后管理团队的责任（见"产品委员会"一章）是确保公司正在追求可用的最佳机会。对于新公司来说，这可能完全或主要是新产品机会，或者对于更成熟的业务来说，可能主要是产品改进机会。在选择产品投资时，机会主义并不是一件坏事。

> Many times, the best product opportunities are sitting right under the company's nose. In particular, often the biggest bang for the buck comes from improving existing products that are not performing at the level they should and could be. For example, you might find that for every 100 people who explicitly begin the subscription process for your product, only 9 make it through to successful completion. You know that if you can improve that number to 18, you've essentially doubled the revenue for that product. That's a great opportunity if you can find a good solution.

很多时候，最好的产品机会就在公司眼皮底下。特别是，通常最大的回报来自改进表现不佳的现有产品。例如，你可能会发现每 100 个明确开始订阅你的产品的人中，只有 9 个成功完成。你知道如果你能把这个数字提高到 18，你基本上就把那个产品的收入翻了一番。如果你能找到好的解决方案，那是一个很好的机会。

> And the ironic thing about this type of opportunity is that it is often the most straightforward to solve. With just a bit of prototyping and user testing, you can quickly identify the issues and come up with better solutions that are often not difficult to implement.

关于这种机会的讽刺之处在于，它往往是最容易解决的。只需一点点原型和用户测试，你就可以快速识别问题并提出更好的解决方案，这些方案通常实施起来并不困难。

> Here's another example: you may find that you're employing hundreds of customer service staff to help your users as they struggle to configure and use your product. Improving the product's usability can significantly reduce the need for customer service staff—and that's just the cost

> savings. The even bigger win may be the improvement in customer satisfaction and your corresponding NPS score.

这是另一个例子：你可能会发现你雇用了数百名客户服务人员来帮助你的用户，因为他们努力配置和使用你的产品。改进产品的可用性可以显著减少对客户服务人员的需求——这只是成本节约。更大的胜利可能是客户满意度的提高和相应的 NPS 分数。

> I often come into a company and look like a hero when I point out these “opportunities” and the big returns they can generate. But I think what's really going on is that there is a tendency in software companies to assume that the product is already about as good as it can be, and continued investment won't make much of a difference. Companies tend to believe that their products are inherently complex, or that a 9% conversion rate isn't bad, or that they just need to spend more on customer acquisition marketing or advertising, or that

我经常进入一家公司，当我指出这些"机会"和它们能产生的大回报时，我看起来像个英雄。但我认为真正发生的是软件公司有一种倾向，认为产品已经尽可能好了，继续投资不会有多大区别。公司倾向于认为他们的产品本质上很复杂，或者 9% 的转化率还不错，或者他们只需要在客户获取营销或广告上花更多钱，或者

> investing in customer service is just a necessary cost of doing business.

投资客户服务只是做生意的必要成本。

> However, what's actually going on is that the product is weak, and the company is just trying to make the best of what they have.

然而，实际发生的是产品很弱，公司只是在尽力利用他们拥有的东西。

> On one level, this is just another symptom of companies under-investing in design and user experience. But more generally, the truth is that many products are poorly done and, rather than improve a product to the point where it can generate real revenue and success, many organizations view it as easier to create a new product instead. But unless they change the way they produce that new product, they're likely going to end up with yet another under-performing product in need of improvement.

在一个层面上，这只是公司在设计和用户体验上投资不足的另一个症状。但更普遍的是，事实是许多产品做得很差，与其改进产品到能够产生真正收入和成功的程度，许多组织认为创造新产品更容易。但除非他们改变生产新产品的方式，否则他们很可能会以另一个需要改进的表现不佳的产品告终。

> @where's the Money?

钱在哪里?

> Do you understand the economics of your product? Do you know your exact revenue model? Do you know the total costs of your product? Do you know how much you pay for each new customer?

> Do you know their lifetime value to the company? Do you know the return your product has generated for the company?

你了解产品的经济学吗？你知道你的确切收入模式吗？你知道产品的总成本吗？你知道为每个新客户支付多少钱吗？你知道他们对公司的终身价值吗？你知道产品为公司产生的回报吗？

> In my experience, most product managers—especially product managers with a technical background—have only a very shallow understanding of how their product (or their company) makes money. This is especially the case for those of us that came to product management from engineering.

根据我的经验，大多数产品经理——尤其是有技术背景的产品经理——对他们的产品（或公司）如何赚钱只有非常肤浅的理解。对于我们这些从工程转到产品管理的人来说尤其如此。

> Tlearned a long time ago that I could benefit a great deal from making a friend in the finance department. In every company I've ever worked at, I have asked the CFO for someone that could help me answer these questions. It always amazed me how willing these people were to help, and just how much information they had available for those who asked.

我很久以前就学到，与财务部门的人交朋友可以让我受益匪浅。在我工作过的每家公司，我都要求 CFO 找个人可以帮助我回答这些问题。他们总是让我惊讶，这些人多么愿意帮忙，而且他们的信息对询问的人有多么丰富。

> I've found that my friends in finance can help with three big things.

我发现财务部门的朋友可以在三件大事上提供帮助。

> Understanding Your Product

理解你的产品

> Sit down with your friend and ask for help determining and evaluating the financial aspects

坐下来向你的朋友寻求帮助，确定和评估财务方面

> of your product, starting with the questions I posed above. If you have partnerships, read the contracts. If you license technology, look at the agreements. Ask your friend to help you assess your product. Is it carrying its own weight? Is it a good investment for the company? What trends does your friend see? Is the product heading in the right direction?

你的产品，从我在上面提出的问题开始。如果你有合作伙伴，阅读合同。如果你授权技术，看看协议。请你的朋友帮助你评估你的产品。它是否自负盈亏？它对公司来说是一项好的投资吗？你的朋友看到什么趋势？产品是否朝着正确的方向前进？

> Understanding Your Customers

理解你的客户

> While we typically have good tools for understanding how users behave on a Web site (via Web analytics), the finance department often has a wealth of additional information on the actual customers, accumulated from transaction histories, payment information, customer data, and management reporting. You both need to be sensitive about what information you can view and how you use it, but in terms of understanding your customers it can be extremely useful.

虽然我们通常有很好的工具来理解用户在网站上的行为（通过网站分析），但财务部门通常有大量关于实际客户的额外信息，从交易历史、支付信息、客户数据和管理报告中积累而来。你们都需要对可以查看什么信息以及如何使用它保持敏感，但就理解你的客户而言，它可能非常有用。

> More than once I've uncovered information about my products from the financial staff that truly surprised me, and exposed fantastic opportunities. One time in particular I remember asking my friend in finance why nobody knew about one such opportunity, and he replied “Because nobody asked.” Realize the life of a finance person is largely thankless, and driven by extreme deadlines (“Our quarter closes on Thursday and we're announcing earnings on Monday!”). I also have a theory that people in finance are often fairly quiet, and not the type to come to your desk advocating product opportunities. Usually, you've got to go to them.

我不止一次从财务人员那里发现了关于我的产品的真正让我惊讶的信息，并揭示了极好的机会。有一次我特别清楚地记得问我在财务部门的朋友为什么没有人知道这样一个机会，他回答说"因为没人问。"意识到财务人员的生活很大程度上是不被感谢的，而且由极端的截止日期驱动（"我们的季度在周四结束，我们在周一公布收益！"）。我还有一个理论，财务人员通常相当安静，不是那种会来你办公桌前倡导产品机会的人。通常，你得去找他们。

> Preparing The Business Case

准备商业案例

> You've got a great idea, but you're not sure about the business case—now what? Your friend in finance can help. You'll need to provide most of the inputs, but your friend will know how to put together the case. If it's a good case, you'll also find that having someone in finance who has studied the economics of the potential product can be a big help for you when discussing with senior management.

你有一个伟大的想法，但你不确定商业案例——现在怎么办？你在财务部门的朋友可以帮助。你需要提供大部分输入，但你的朋友会知道如何整理案例。如果这是一个好的案例，你还会发现让财务部门研究潜在产品经济学的人在与你与高级管理层讨论时对你有很大帮助。

> So go make a friend in finance. You need the information they have and their help in interpreting the information and putting it to good use. I think you'll find that these people want to help their company, and appreciate the opportunity to do so.

所以去和财务部门的人交朋友吧。你需要他们拥有的信息以及他们帮助解释信息并善加利用的帮助。我想你会发现这些人想要帮助他们的公司，并感谢有机会这样做。

> ° Examples

示例

> You can see example opportunity assessments at www.svpg.com/examples.

你可以在 www.svpg.com/examples 上看到示例机会评估。

第 12 章：产品发现

> Chapter 12: Product Discovery
> Defining The Right Product

定义正确的产品

> Software projects can be thought of as having two distinct stages: figuring out what to build (build the right product), and building it (building the product right). The first stage is dominated by product discovery, and the second stage is all about execution.

软件项目可以被认为有两个不同的阶段：弄清楚要构建什么（构建正确的产品），以及构建它（正确地构建产品）。第一阶段以产品发现为主导，第二阶段完全是关于执行。

> When in product discovery, you welcome and explore new ideas, talk with scores of users and customers, learn how you can apply new technologies, flesh out your product concepts and test them out, and spend a lot of time thinking about the overall product direction, both immediate and longer term. It is all about discovering that mix of form and function that results in a winning product.

当处于产品发现阶段时，你欢迎并探索新想法，与数十个用户和客户交谈，学习如何应用新技术，充实你的产品概念并测试它们，并花很多时间思考整体产品方向，包括近期和长期。这都是关于发现那种形式和功能相结合的混合体，从而产生成功的产品。

> However, once you've spec'd out this product, and your engineering team begins the process of building it, a very profound and important shift needs to take place for the product team. Now the game is all about execution—getting the product built, tested, and delivered to market. In this stage, you spend your time keeping everyone focused, chasing down the countless issues that inevitably arise, and getting these issues resolved immediately. Acquisitions, competitors, organizational and management changes—these are all distractions, and your job is to keep the team on track so this product can get out there when

然而，一旦你规格化了这个产品，你的工程团队开始构建它的过程，产品团队需要发生一个非常深刻和重要的转变。现在游戏完全是关于执行——把产品构建、测试并交付到市场。在这个阶段，你花时间让每个人保持专注，追逐无数不可避免出现的问题，并立即解决这些问题。收购、竞争对手、组织

和管理变化——这些都是干扰，你的工作是让团队保持在轨道上，以便这个产品能在需要的时候推出。

> it needs to be.

在无数产品团队中，这种心态转变实际上并没有发生——或者至少直到很晚才发生，往往晚到进入 QA。相反，产品经理继续探索新想法，公司高管继续将产品规格视为可塑的。结果是委婉地称为"变更"，产品规格继续以显著的方式变化，影响工程和团队其他成员。结果，发布日期被推迟，或功能被削减，或质量受到损害。或者以上所有。

> In countless product teams, this shift in mindset doesn't actually happen—or at least it doesn't happen until much later, often as late as entering QA. Instead, product managers continue to explore new ideas, and company execs continue to view the product spec as malleable. What results is euphemistically referred to as “churn,” where the product spec continues to change in significant ways, impacting engineering and the rest of the product team. As a result, the release dates get pushed out, or features get cut, or the quality gets compromised. Or all the above.

如果你有幸有一个伟大的项目经理，那么你可能有帮助在执行期间保持一切在轨道上。但即使你有，作为产品经理，你需要意识到这种必要的心态转变；否则，产品经理很容易成为产品无法上市的原因。

> If you're lucky enough to have a great project manager, then you probably have help keeping everything on track during execution. But even if you do, as a product manager you'll need to be cognizant of this necessary change in mindset; otherwise, it is all too easy for the product manager to be the source of the product's inability to get to market.

然而，我认为重要的是要认识到我们都有自己的独特偏好和技能。如果你天生是一个发现型的人——更喜欢发明过程的自由和创造力——那么在执行期间你将不得不格外努力地抑制这些冲动。另一方面，如果你更自然地是喜欢把事情推出门的项目经理类型，那么你需要努力培养你的战略思维和发现技能——记住最重要的是创造客户喜爱的产品。

> However, I think it's important to recognize that we all have our own unique preferences and skills. If you're naturally a discovery kind of person—preferring the freedom and creativity of the invention process—then you'll have to work extra hard to contain those urges during execution. On the other hand, if you're more naturally the project manager type who loves getting things out the door, then you'll need to work on your strategic thinking and discovery skills—remembering that what matters most is creating a product that your customers love.

我发现一个非常有用的技巧是始终保持两个版本的产品并行进行。换句话说，一旦你开始工程 1.0 版本并切换到该项目的执行模式，

> One technique I have found very useful is to always keep two versions of a product going in parallel. In other words, as soon as you start the engineering for release 1.0 and switch into

你就开始并行进行 2.0 版本的发现。始终保持创新引擎运转——一旦某个版本进入工程阶段，就把你的创造力重新定向到下一个版本。

> execution mode for that project, then you start up the discovery for release

一个警告：你确实需要小心，这种方法不会分散当前项目的执行工作。

> 2.0 in parallel. Always keep that innovation engine working—once a given release goes to engineering, redirect your creative urges to the next release.

总的来说，我发现有这样的出口是一件好事。下次公司高管带着一个大的新需求来找你时，与其影响你正在炉子里的产品，你已经在发现阶段有了下一个版本，你可以在那里开始探索新需求的工作。

> One note of warning: You do need to be careful that this approach doesn't detract from the execution work for the current project.

我不想让这一切听起来过于简单，但我确实相信有纪律地管理是可以做到的。你必须发展你的发现技能（以确保你想出成功的产品）以及你的执行技能（以确保这些伟大的想法真正到达你的客户）。

> Overall, I've found that having this outlet is a good thing. The next time a company exec drops by with a big, new requirement, rather than impacting the product you have in the oven, you already have the next release in the discovery stage and you can start the work of exploring the new requirement there.

你能安排发现吗?

> I don't mean to make this all sound overly simple, but I do believe that with discipline it can be managed. It's essential that you develop both your discovery skills (to ensure you're coming up with winning products) as well as your execution skills (to ensure that these great ideas actually make it to your customers).

你以前经历过这种情况吗？你的公司对一个产品想法非常兴奋，作为产品经理，你被要求定义它。你被告知工程师将在四周内完成他们当前的项目，所以这意味着你可以花所有你需要的时间——只要你在四周内准备好。

> 8 Can You Schedule Discovery?

没问题，你说（毕竟，有时你只给你几天，所以四周听起来很棒）。你将从机会评估开始，了解要解决的问题，然后你会花时间采访真实用户，并确定一套初步需求。到第二周开始时，你应该能够与交互设计师一起制作原型，在第三周，你将用原型进行用户测试，在第四周，你将充实用例的细节并与工程部门一起审查原型和规格。

> Have you experienced this situation before? Your company gets all excited about a product idea, and as product manager you are asked to define it. You are told that the engineers will be finished with their current project in four weeks, so that means you can take all the time you need—so long as you are ready in four weeks.

这些都是很好的实践。但发生的事情通常不那么好。在你最初的用户讨论中，你发现用户不像你的管理层那样对这个想法感到兴奋，和/或你努力想出一个用户能弄明白的原型，和/或用户在尝试时对原型中的想法不感到兴奋。

> No problem, you say (after all, sometimes you're only given days, so four weeks sounds great). You'll start with an opportunity assessment to understand the problem to be solved, then you'll spend quality time interviewing real users, and identify a preliminary set of requirements. By the start of the second week, you should be able to work with an interaction designer on a prototype, in the third week, you'll do user testing with the prototype and, in the fourth week, you'll flesh out the details of the use cases and review the prototype and spec with engineering.

但时间到了，工程师准备好了，所以你给他们你有的东西。

> These are all great practices. But what happens isn't usually so great. During your initial user discussions, you find that users aren't as excited about the idea as your management is, and/or you struggle to come up with a prototype that users can figure out, and/or the users aren't excited about the ideas in the prototype when they try it.

结果？在接下来的三到六个月里，工程继续构建你在原型中看到的同样

> But time is up, the engineers are ready, so you give them what you have.

不可用且不能让人兴奋的产品，你发布它，然后你的管理层当然对结果感到失望。

> The result? During the next three to six months, engineering proceeds to build that same

问题不是软件的可靠性，所以工程团队没有责任——毕竟，他们只是按照你要求他们做的来构建。那么是谁的错呢？每个人都知道是你的错——你是产品经理。

> unusable and unexciting product that you saw in your prototype, you ship it, and then your management is of course disappointed with the results.

与用户交谈、创建原型、与用户测试，如果你不能根据你学到的东西调整你的方向，就没有帮助。

> The problem isn't the reliability of the software, so the engineering team isn't to blame—after all, they just built what you asked them to. So whose fault is it? Everyone knows it's your fault—you're the product manager.

这种需求和设计作为产品开发过程中顺序、可预测和安排好的阶段的概念在我们的行业中如此根深蒂固，以至于它往往是产品组织最难打破的习惯之一。但我们都需要克服这一点。产品组织需要接受这样一个事实，即产品发明过程从根本上来说是一个创造性过程。它更像艺术而不是科学。

> It doesn't help to talk to users, create prototypes, and test with users, if you don't adjust your course based on what you learn.

这就是为什么我更愿意把这一阶段称为"产品发现"而不是"需求和设计"。我认为这个命名强调了两个非常重要的点：

> This notion of requirements and design as a sequential, predictable and scheduled phase in a product development process is so ingrained in our industry that it's often one of the most difficult habits for product organizations to break. But we all need to get past this. Product organizations need to come to terms with the fact that the product invention process is fundamentally a creative process. It is more art than science.

首先，你需要发现是否真的有用户想要这个产品。换句话说，你需要识别你的市场并用你的客户验证机会。

> This is why I prefer to think of this phase as “product discovery” more than “requirements and design.” I think this nomenclature emphasizes two all-important points:

其次，你需要发现一个对这个问题的产品解决方案，它是有价值的、可用的和可行的。换句话说，你需要设计你的解决方案并用你的

> First, you need to discover whether there are real users out there who want this product. In other words, you need to identify your market and validate the opportunity with your customers.

客户和你的工程团队验证它。

> Second, you need to discover a product solution to this problem that is valuable, usable, and feasible. In other words, you need to design your solution and validate it with your

有时产品发现阶段是 straightforward 的。其他时候它极其困难。根据我的经验，发现和验证市场机会并不难，但发现解决方案往往非常具有挑战性。即使有优秀的设计师和优秀的工程师的帮助，有些问题真的很难（至少许多值得追求的且尚未解决的问题）。

> customers and your engineering team.

制药行业提供了一个极端的例子。

> Sometimes the product discovery phase is straightforward. Other times it is extremely difficult. In my experience, it's not so hard to discover and validate the market opportunity, but it's often very challenging to discover the solution. Even with the help of great designers and great engineers, some problems are just truly hard (at least many of the ones worth pursuing and that haven't been solved already).

市场发现过程通常并不困难——不乏值得解决的好的医疗问题（如挽救你的生命、延长你的生命或提高你的生活质量）。困难的部分当然是发现产品解决方案。制药公司进入这个发现阶段时充分意识到他们不能保证会想出任何东西，或者如果会的话，需要多长时间。作为一个行业，他们已经接受了这种不确定性（这种风险定价在他们的产品中）。

> The pharmaceutical drug industry provides an extreme example.

然而，对于软件，即使我们都知道它非常难——而且我们知道大多数软件发布未能达到其目标——我们仍然坚持像计划建造房子一样安排发现阶段。

> The market discovery process is usually not very difficult—there are no shortage of good medical problems worth solving (like saving your life, extending your life, or improving the quality of your life). The hard part of course is discovering a product solution. Drug companies go into this discovery phase fully aware that there's no guarantee they'll come up with anything or, if they do, how long it may take. As an industry, they have come to terms with this element of uncertainty (and this risk is priced into their products).

管理层尤其难以接受产品发现这个概念。我认为这有两个根本原因：

> Yet with software, even though we all knowit is very hard—and we know that the majority of software releases fail to meet their objectives—we still insist on scheduling the discovery phase like we're planning the construction of a house.

首先，发现过程不像我们希望的那样可预测。管理层担心你可能

> Management especially struggles with this notion of product discovery. I think there are two underlying reasons for this:

花几个月追逐一个解决方案，最后却一无所获。至少如果他们继续构建，他们可以说他们发布了东西。这就是为什么许多经理对 Scrum 等敏捷方法感到不舒服的同样原因。他们想知道需要多少个 30 天的冲刺才能完成。

> First, the discovery process isn't as predictable as we wish it was. Management fears you may

其次，几乎每个软件产品组织中最受约束和最昂贵的资源是工程师，而工程团队可能无所事事只玩桌上足球的想法让管理层抓狂。

> spend months chasing a solution and end up with nothing to show for it. At least if they go ahead and build, they can say that they shipped something. It's the same reason why many managers are uncomfortable with Agile methods like Scrum. They want to know how many 30-day sprints it will take before they're done.

讽刺的是，正是这种推理直接导致工程资源的浪费。

> Second, the most highly constrained and expensive resource in just about every software product organization is the engineers, and the thought that an engineering team might be sitting around with nothing to do but play Foosball just drives management nuts.

意识到几乎每家公司都在执行我在这里描述的发现过程，只是他们不是用一个原型师几周，而是用完整的工程团队完整的发布周期来构建软件，然后进行 QA 并部署到生产系统。他们正在使用工程组织来构建一个非常、非常昂贵的原型，并使用他们的实时客户作为不知情的测试对象。这也是为什么公司通常需要三到五个发布，历时一到两年，才能得到他们可以赚钱的东西。

> Ironically, it is precisely this reasoning that leads directly to wasted engineering resources. Realize that almost every company executes the discovery process I've described here, only instead of using one prototyper for a few weeks, they use the full engineering team for full release cycles to build the software that is then QA'd and deployed into production systems. They are using the engineering organization to build a very, very expensive prototype, and they use their live customers as unwitting test subjects. This is also why it typically takes companies three or more releases over one to two years to get something they can make money from.

这也是为什么这么多初创公司失败的一个重要原因。大多数初创公司根本没有资金在进入市场之前坚持两年。所以他们雇用工程师，尽力而为，看看会发生什么。准备，开火，瞄准。

> This is also a big reason why so many startups fail. Most startups simply don't have the funding to go two years before they gain traction in the marketplace. So they hire engineers, take their best shot, and see what happens. Ready, fire, aim.

初创公司尤其必须把精力集中在这种更快的产品发现过程上。一旦他们发现了有效的解决方案——一个有价值、可用且可行的解决方案——

> Startups especially must focus their energies on this much faster product discovery process. And once they discover the solution that works—one that is valuable, usable and

可行——然后就是关于执行。在那之前，他们不必马上雇用太多工程师——他们已有的工程师可以而且应该积极参与产品发现过程。而且，在某种程度上，工程师可以在这个发现进行的同时准备基础设施。

> feasible—then it's all about execution. Until that point, they don't have to hire too many engineers right away—the engineers they already have can and should actively participate in the product discovery process. And, to a degree, the engineers can prepare the infrastructure while this discovery is going on.

你可以帮助你的管理层理解产品发现过程的本质。如果你始终强调你有义务确保工程构建的东西必须是有价值的和可用的，并争取他们的努力来发现成功的解决方案，你就会开始把他们的注意力转移到这一点上——产品开发过程中最关键的阶段。

> You can help your management understand the nature of the product discovery process. If you consistently emphasize your obligation to ensure that what engineering builds must be valuable and usable, as well as enlist their efforts to discover a successful solution, you'll start to move their focus to this—the most critical stage of the product development process.

第 13 章：产品原则

> Chapter 13: Product Principles

> Deciding What's Important

决定什么是重要的

> Another tool that can be a big aid in determining the right tradeoffs and priorities is a good set of product principles. The product principles are a public declaration of your beliefs and intentions. I like them because if they're done well, they can serve as a useful complement to a product strategy and significantly speed the product discovery process.

另一个可以帮助确定正确权衡和优先级的工具是一套好的产品原则。产品原则是你信念和意图的公开声明。我喜欢它们，因为如果做得好，它们可以作为产品策略的有用补充，并显著加快产品发现过程。

> When I start working with a product team, once I understand the business strategy, often one of the first activities we do together is to define a set of product principles.

当我开始与一个产品团队合作时，一旦我理解了业务策略，我们经常一起做的第一项活动之一就是定义一套产品原则。

> Coming up with product principles means deciding what is important to you—and what is incidental—and deciding what is strategic and fundamental, and what is simply tactical and temporary.

制定产品原则意味着决定什么对你重要——什么是偶然的——并决定什么是战略性和根本性的，什么只是战术性和暂时性的。

> There are other benefits to developing product principles. The process serves as a way for me to understand the DNA of a company, and what the founders hope to achieve. Most importantly, it serves as a framework for evaluating the many alternatives in front of every product and company.

制定产品原则还有其他好处。这个过程可以作为我理解公司 DNA 和创始人希望实现什么的方式。最重要的是，它作为评估每个产品和公司面临的许多替代方案的框架。

> Product principles are not a list of features and, in fact, are not tied to any one product incarnation. In this sense, they are most aligned with a product strategy for an entire product line, or with a company mission statement for a startup. A good set of principles serves as the basis or foundation for inspiring product features.

产品原则不是功能列表，事实上，不与任何一个产品化身绑定。在这个意义上，它们最符合整个产品线的产品策略，或初创公司的公司使命宣言。一套好的原则作为启发产品功能的基础或基础。

> An example of a product principle for a movie site may be that the team believes that the user community's opinions on movies are more valuable than those of professional reviewers. Later, if a studio wants to place reviews on your site, you can then decide if this is consistent with your principles or not.

电影网站的一个产品原则例子可能是，团队相信用户社区对电影的意见比专业影评人的更有价值。后来，如果一家电影公司想在你的网站上放置评论，你就可以决定这是否符合你的原则。

> Whether you choose to go public with your product principles depends on your purpose. Often the principles are simply a tool for the product team, much like a product strategy document. But, in other cases, the principles serve as a clear statement of what you believe, intended for your users, customers, partners, suppliers, investors, and employees.

你是否选择公开你的产品原则取决于你的目的。通常原则只是产品团队的工具，就像产品策略文档一样。但在其他情况下，原则作为你信念的明确声明，旨在传达给你的用户、客户、合作伙伴、供应商、投资者和员工。

> Another benefit I have found is that more than any other document, a set of product principles can bring together the product team—especially product management, user experience design, engineering, and marketing—and get the team on the same page.

我发现的另一个好处是，比任何其他文件都更重要的是，一套产品原则可以把产品团队——特别是产品管理、用户体验设计、工程和营销——团结在一起，让团队在同一页上。

> While there's value in identifying your guiding product principles, you also need to prioritize them. Countless products are trying to be easy to use and also safe and secure. But what matters is the priority. Is ease of use paramount? Or is safety and security the primary concern?

虽然确定你的指导产品原则有价值，但你也需要对它们进行优先排序。无数产品试图做到易用且安全。但重要的是优先级。易用性是至高无上的吗？还是安全和保障是主要关注点？

> Finally, many teams make a couple of mistakes when they first try creating a set of product principles. The first is that they state principles that are so generic that they aren't really useful (“must be reliable”). The second is that they confuse their product principles with design principles. For example, a common design principle is to provide a well-lit path (so the user always knows where to go next). That's not a product principle.

最后，许多团队在第一次尝试创建一套产品原则时会犯几个错误。第一个是 stated 的原则太笼统，以至于没有真正的用处（"必须可靠"）。第二个是混淆了产品原则和设计原则。例如，一个常见的设计原则是提供一条光线充足的路径（所以用户总是知道下一步去哪里）。这不是产品原则。

> So, if you don't yet have a clear statement of the beliefs and principles that guide your product team, consider getting the team together for a couple hours to discuss, identify, and prioritize what you think is important.

所以，如果你还没有关于指导你产品团队的信念和原则的明确声明，考虑让团队聚在一起几个小时来讨论、识别和优先考虑你认为重要的东西。

> © Resolving Conflicts

解决冲突

> Quite a few product managers tell me that they're constantly struggling with endless meetings without structure or decisions, second guessing of earlier decisions, vetoes, politics and what I call “drive-bys” (when a manager drops in every so often, shoots down your progress, and then is gone again without providing the feedback or guidance that could help you address her concerns).

相当多的产品经理告诉我，他们不断在没有结构或决定的无穷会议、对早期决定的反复猜测、否决、政治和我所谓的"路过"（当经理时不时路过，击落你的进展，然后又消失了，没有提供可以帮助你解决她的关切的反馈或指导）中挣扎。

虽然这种情况可能发生在公司必须做出的几乎任何决定上，但我认为这在产品决定上尤其常见。在我看来，有几个原因。首先，每个人对公司产品的意见——这可能是首先吸引他们加入公司的原因。其次，每个人对产品都有强烈的感觉，因为在某种程度上——我们都意识到公司需要钱才能生存，钱来自客户，客户因产品而来。第三，你的许多同事认为自己比实际更像你的目标客户（或认为他们比实际更了解目标客户）。

> While this type of situation can occur with virtually any decision a company must make, I think this is an especially common problem with product decisions. As I see it, there are several reasons for this. First, everyone has an opinion about the company's products—they're probably what attracted them to join the company in the first place. Second, everyone feels strongly about the product since—at some level—we all realize that companies need money to survive, money comes from customers, and customers come for the products. Third, many of your colleagues view themselves as much more like your target customer than they really are (or think they understand the target customer much more than they really do).

再加上这样一个事实——在大多数组织中——产品团队实际上不向产品经理汇报，因此几乎没有组织权力来强制；产品经理必须说服团队而不是命令它。结果就是使产品管理如此困难且有时极其令人沮丧的原因。

> Combine this with the fact that—in most organizations—the product team doesn't actually report into the product manager and so has little organizational power to coerce; the product manager must persuade the team and not dictate to it. The result is what makes product management so difficult and at times extremely frustrating.

在一些团队中，产品决策过程可能变得如此有争议和僵局，以至于必须将决定升级给高级经理才能继续前进。如果发生这种情况（有时是不可避免的），我认为这是一个非常糟糕的结果。你想要辩论和争论，但你希望最后每个人都在船上。在大多数情况下，会产生更好的产品。高级经理总是可以为你做决定，但除了产生的怨恨之外，产品是最大的输家。

> In some teams, the product decision process can become so contentious and deadlocked that the decision must be escalated to a senior manager in order to move forward. If this happens (and sometimes it is unavoidable), I consider this to be a very bad result. You want the debate and the arguments, but you want everyone on board at the end. In most cases, a much better product will result. A senior manager can always make the call for you but, besides the resentment this creates, the product is the biggest loser. I suppose it's little surprise that so many struggle to find an

> effective process for making product decisions. I will not pretend that there's a way to make the product discovery process painless—there isn't. Constructive debate and argument is an essential ingredient to coming up with a great product. While I know those arguments are necessary, it doesn't mean I always enjoy them. That said, as product manager you can make a very significant impact on this process—minimizing churn and maximizing creativity and quality of the result by doing the following: For virtually all product decisions, the key is to properly frame the decision to be made, and to first get everyone on the same page in terms of:

我想这并不奇怪，这么多人都在努力寻找有效的产品决策过程。

我不会假装有一种方法可以让产品发现过程无痛——没有。建设性的辩论和争论是想出伟大产品的必要成分。虽然我知道这些争论是必要的，但这并不意味着我总是享受它们。

也就是说，作为产品经理，你可以在这个过程中产生非常重大的影响——通过做以下事情来最小化变更并最大化结果的创造力和质量：

对于几乎所有的产品决策，关键是正确框定要做出的决定，并首先让每个人都在同一页上：

> - What problem exactly are you trying to solve?

你究竟要解决什么问题?

> - Who exactly are you trying to solve this problem for—which persona?

你究竟要为谁解决这个问题——哪个用户画像?

> - What are the goals you are trying to satisfy with this product?

你试图用这个产品满足什么目标?

> - What is the relative priority of each goal?

每个目标的相对优先级是什么?

> In my experience, most of the time when there's strong disagreement within the product team, it's not really over the facts of the situation—it's instead because each person has a different interpretation or weighting of the goals and the priorities.

根据我的经验，大多数时候当产品团队内部有强烈分歧时，实际上并不是关于情况的事实——而是因为每个人对目标和优先级有不同的解释或权重。

> For example, you should be arguing about what's most important to your target user: ease of use, speed, functionality, cost, security, privacy—this is the right argument. Once you've agreed on what the goals are and who exactly you want to satisfy—and, just as important, the relative priority—then you all have a common basis for evaluating and assessing the options. It is extremely important to take prioritization seriously—you should get the team to agree on the specific ordering, most to

> least important. Don't just wave your hands and group the goals into something like “critical” and “very important.” Be sure your team can identify what is most critical, then second most critical, and so on.

例如，你应该争论什么对你的目标用户最重要：易用性、速度、功能性、成本、安全性、隐私——这是正确的争论。一旦你同意目标是什么以及你究竟想满足谁——而且，同样重要的是，相对优先级——那么你们就有了评估和评估选项的共同基础。

> When I am called in on controversial product decisions, all too often the group has skipped this step, and is deep in the weeds of each option—everyone passionately arguing his or her case but without a common basis for evaluation. Everyone assumes the objectives and the priority. Even if you have done a great job developing these objectives, you should remind the team prior to the decision process. Put it up on the white board so that the team can see the exact framework for evaluating the options and making the decision.

认真对待优先级排序极其重要——你应该让团队同意具体的排序，从最重要到最不重要。不要只是挥手把目标分成"关键"和"非常重要"之类。确保你的团队能够识别什么是最关键的，然后是第二关键的，依此类推。

> Moreover, I think it is very important for product managers to be completely transparent in their decision making process and reasoning. You don't want the team thinking that you're just following your intuition. Every member of the team should be able to see the goals and

当我被请来处理有争议的产品决策时，往往团队跳过了这一步，深陷于每个选项的杂草中——每个人都激情地争论他或她的情况，但没有评估的共同基础。每个人都假设目标和优先级。即使你在制定这些目标方面做得很好，你也应该在决策过程之前提醒团队。把它写在白板上，这样团队就可以看到评估选项和做出决定的确切框架。

> objectives you are using, their priority, and how you assess each option. The decision—and the reasoning behind how you got there—should be clear to all.

此外，我认为产品经理在决策过程和推理方面完全透明非常重要。你不希望团队认为你只是在跟随你的直觉。团队的每个成员都应该能够看到你在使用的目标和目的，它们的优先级，以及你如何评估每个选项。决定——以及你是如何到达那里的推理——应该对所有人清晰。

> So the next time you find your product team battling it out and getting into an unproductive and demoralizing state, bring the team back from the edge, and revisit the goals and priorities. Make sure you're all on the same page before returning to evaluating the different options.

所以下次你发现你的产品团队在争斗并进入非生产性和令人沮丧的状态时，把团队从边缘带回来，重新审视目标和优先级。在返回评估不同选项之前，确保你们都在同一页上。

> ° Examples

示例

> You can see example product principles at: www.svpg.com/examples.

你可以在 www.svpg.com/examples 上看到示例产品原则。

第 14 章：产品委员会

> Chapter 14: The Product Council
>
> Timely And Definitive Product Decisions
>
> Even in small companies, getting decisions made is often time consuming and frustrating. Every product company needs a mechanism to get the key stakeholders and decision makers together to make timely and informed product decisions.

我在这里描述的内容曾向 eBay 前首席运营官梅纳德·韦伯（Maynard Webb）介绍过。自那以后，我与多家公司合作，细化和精简了具体的职责。

> My favorite way to ensure this is to establish a product council.
>
> In general, I really am not a fan of committees or even most meetings. But I have found product councils to be very valuable, and their use can speed up the overall product development process considerably. This is because key product decision makers are all available with the express purpose of making those decisions the product organization needs to get products to market.
>
> The challenge is to do this in a way that provides the visibility and oversight that management needs and is responsible for, without the micromanaging (and disempowerment) that comes with company executives trying to design products.
>
> Many companies have some variations of this, but I credit the origin of the concept as
>
> described here to Maynard Webb, the former COO of eBay. I've worked with several companies since then to refine and streamline the specific responsibilities.
>
> Purpose

目的

> The purpose of the product council is to set the strategic product direction, allocate product resources and investments, and provide a level of oversight of the company's product efforts. This group is not trying to set the company's business strategy, but rather—given the business strategy—come up with a product strategy that will meet the needs of the business. The decisions this group makes will directly impact the success of the business.

产品委员会的目的是设定战略产品方向、分配产品资源和投资，并对公司的产品工作提供一定程度的监督。这个小组不是试图设定公司的业务战略，而是——在给定业务战略的情况下——制定一个能够满足业务需求的产品战略。这个小组做出的决定将直接影响业务的成功。

> Membership

成员组成

> The Product Council is typically comprised of the cross-functional set of managers responsible for product development. Every company has its own considerations, but an example might be:

产品委员会通常由负责产品开发的跨职能管理人员组成。每个公司都有自己的考虑因素，但一个示例可能是：

> · CEO, COO or Division GM

« CEO、COO 或事业部总经理

> · VP/Director of Product Management
> · VP/Director of User Experience Design
> · VP/Director of Marketing
> · VP/Director of Engineering
> · VP/Director of Site Operations
> · VP/Director of Customer Service
>
> As with any such group, the effectiveness of the meetings will largely be a function of the meeting skills of the leader. The leader needs to be good at staying on task, framing issues, and driving to decisions. Usually, the leader of the product council is either the head of product or—for smaller companies—the head of the company.
>
> Make sure you have representation from the key areas, but try to keep the group at 10 or less. If more people want to be members (they will) make sure they each know who is there to represent their views. For example, the VP Sales might use the VP Marketing, and the head of QA might use the VP Engineering.
>
> Specific Responsibilities

¢ 产品管理副总裁/总监

> This is not a group that designs or builds products. This group should oversee the flow of products through the product development process, and make the key decisions required. For each product effort, there are four major milestones for product council review and decision making:
>
> Milestone 1: Review proposed product strategies and product roadmaps, and initiate opportunity assessments for specific product releases. That is, select the product opportunities to be investigated.
>
> Milestone 2: Review opportunity assessments and recommendations, and issue go/no-go decisions to begin discovering a solution.
>
> Milestone 3: Review product prototypes, user testing results and detailed cost estimates, and issue go/no-go decision to begin engineering.
>
> Milestone 4: Review final product, QA status, launch plans, and community impact assessments, and issue go/no-go decision to launch.

> Additional Notes

« 用户体验设计副总裁/总监

> - For small organizations, one product council typically covers all products. For large companies, the product council typically aligns with the business unit.
>
> «There is no need to review minor updates or fixes in this process—there should be an expedited process for the minor changes needed to run the business (which are typically more content related).
>
> - These are not product design sessions—if there are issues with the product then the team should work on them and return to the product council when they have been addressed.
> - While you will often have a very preliminary sense of estimated cost at Milestone 2, no one should take that estimate very seriously as the solution has not yet been outlined—anything beyond “small, medium or large” effort is not fair to engineering. However, the estimate of time and cost at Milestone 3 should be detailed, and something the full product team is prepared to commit to.
> - You will need to decide if this group will also address issues of policy (such as product end-of-life policy, privacy policies, etc.) Often, this group does discuss such issues, but these topics can become long unstructured discussions if not managed. Make sure policy discussions don't delay product oversight responsibilities.
> - The frequency of meetings depends on the number of product efforts going on. You may only need to meet for one hour a month, or you may need to meet two hours a week.
>
> «It can be useful for the product council to review the business performance of the products that have launched. The product council may request a presentation on the business results of the product launch, typically 3-6 months post-launch. This sort of accountability will help the council better understand which investments and decisions they made were good ones, and why.
>
> - Ideally, the product manager should present his product to the product council. His manager should help him prepare for this presentation, to ensure that he has done his homework and the recommendations are sound. The smart product manager will have individually briefed the members of the product council prior to his presentation to learn of any issues and resolve them, so he's not caught by surprise.
>
> Ifyou find your product organization taking too long to make decisions, consider instituting a product council. Hopefully you'll find that this one meeting eliminates countless others and makes the decision process informed, transparent and timely.
>
> when Do We Estimate Project Costs?
>
> Even though we've been estimating project costs since the very beginning of software development, it's remarkable to me how much confusion remains. I believe the root cause of this confusion is that management needs cost information very early in the process, yet engineering doesn't have the information it needs for a reasonably accurate estimate until much later in the process (because there's virtually no good information yet on the solution required). The result is either premature estimates that prove wildly inaccurate, or surprises because people had different

> assumptions all along and—when the accurate estimate eventually does come in—management experiences a severe case of sticker shock.
>
> Here's the process that I advocate—while it is intertwined with the product development process I support, it can also be applied in most situations.
>
> Recall that I strongly encourage all product efforts to start with a 10-question opportunity assessment (see the chapter Opportunity Assessments). This assessment is what the management team uses to decide whether there is a problem worth trying to solve. There's no solution yet, just a problem and an opportunity. But, for most teams, there's a clear need at this stage for a very preliminary estimate of project scope. Of course, since there's no solution spelled out at this stage, it's going to be very much a
>
> SWAG (a “scientific wild-assed guess”) which is why I recommend that the estimates at this stage be limited to “Small,” “Medium,” or “Large.” It's usually fairly clear at this granularity
>
> what the cost will be, although there will still be occasional surprises.
>
> If the opportunity looks good relative to the estimated cost, management will likely allow the project to proceed to defining the solution. It is at this stage that the product solution is spelled out in detail, ideally with a high-fidelity prototype that you validate against real target users with prototype testing.
>
> Throughout the process of coming up with the solution, the product manager and interaction designer should include a member of engineering to evaluate the various options and estimate costs for the different alternatives. This information is then considered by the product manager and designer and the product is adjusted as needed. But at the end of the spec process, there should be a very detailed and high-confidence estimate based on a detailed description of the product proposed to be built.
>
> At this stage, the management team has a detailed product spec, and a corresponding high-confidence cost estimate, and they should be able to make a final decision on whether to proceed to build this product or not. It may be that the solution turned out to be more complex and expensive than they thought, or they might not like the actual solution, but if they do proceed the entire product team knows the cost and the product they'll get for that investment.
>
> To summarize, I'm suggesting a preliminary estimate at opportunity assessment time, followed by a detailed estimate that accompanies the final product spec.

¢ 营销副总裁/总监

¢ 工程副总裁/总监

¢ 网站运营副总裁/总监

¢ 客户服务副总裁/总监

与任何此类小组一样，会议的有效性将很大程度上取决于会议主持人的会议技巧。主持人需要擅长保持任务进度、界定问题和推动做出决策。通常，产品委员会的负责人要么是产品负责人——对于较小的公司——要么是公司负责人。

确保你有来自关键领域的代表，但尽量将小组人数控制在 10 人或更少。如果有更多人想成为成员（他们会的），确保他们每个人都知道谁是代表他们观点的人。例如，销售副总裁可能通过营销副总裁来表达，QA 负责人可能通过工程副总裁来表达。

具体职责

这不是一个设计或构建产品的小组。这个小组应该监督产品在产品开发流程中的流动，并做出所需的关键决策。

对于每个产品工作，产品委员会审查和决策有四个主要里程碑：

里程碑 1：审查提议的产品战略和产品路线图，并启动对特定产品发布的机会评估。即，选择要调查的产品机会。

里程碑 2：审查机会评估和建议，并发出开始发现解决方案的继续/不继续决策。

里程碑 3：审查产品原型、用户测试结果和详细的成本估算，并发出开始工程的继续/不继续决策。

里程碑 4：审查最终产品、QA 状态、发布计划和社区影响评估，并发出发布的继续/不继续决策。

附加说明

¢ 对于小型组织，一个产品委员会通常涵盖所有产品。对于大型公司，产品委员会通常与业务单元保持一致。

- 在此过程中无需审查小更新或修复——对于运行业务所需的小变更（通常更多是与内容相关的），应该有一个快速处理流程。
- 这些不是产品设计会议——如果产品存在问题，团队应该处理这些问题，并在问题解决后返回产品委员会。

« 虽然在里程碑 2 时你通常会对估算成本有一个非常初步的了解，但没有人应该过于认真对待这个估算，因为解决方案尚未概述——超出"小型、中型或大型"工作量的任何估算对工程都是不公平的。然而，在里程碑 3 时对时间和成本的估算应该是详细的，并且是整个产品团队准备承诺的。

« 你需要决定这个小组是否还将处理政策问题（如产品生命周期终止政策、隐私政策等）。通常，这个小组确实会讨论这些问题，但如果管理不当，这些话题可能会变成漫长而无组织的讨论。确保政策讨论不会延误产品监督职责。

« 会议频率取决于正在进行的产品工作数量。你可能每个月只需要开会一小时，或者你可能需要每周开会两小时。

«产品委员会审查已发布产品的业务表现可能是有用的。产品委员会可能会要求就产品发布的业务结果进行演示，通常是在发布后 3-6 个月。这种问责制将帮助委员会更好地理解他们做出的哪些投资和决策是好的，以及原因。

«理想情况下，产品经理应该向产品委员会展示他的产品。他的经理应该帮助他准备这次演示，以确保他做了功课，建议是有根据的。聪明的产品经理会在演示前单独向产品委员会成员进行简要介绍，以了解任何问题并解决它们，这样他就不会感到意外。

如果你发现你的产品组织做决定花费太长时间，考虑建立产品委员会。希望你能发现这一个会议消除了无数其他会议，并使决策过程知情、透明和及时。

@ 我们何时估算项目成本?

尽管我们从软件开发的最初就开始估算项目成本，但令我惊讶的是仍然存在如此多的困惑。我相信这种困惑的根本原因是管理层在流程的早期就需要成本信息，而工程部门直到流程的后期才有进行合理准确估算所需的信息（因为关于所需解决方案的可靠信息几乎还没有）。结果是要么是过早的估算被证明完全不准确，要么是意外，因为人们一直有不同假设——当准确的估算最终出现时——管理层会经历严重的"标价震惊"。

以下是我倡导的流程——虽然它与我支持的产品开发流程交织在一起，但它也可以应用于大多数情况。

回想一下，我强烈鼓励所有产品工作从 10 个问题的机会评估开始（参见"机会评估"一章）。这个评估是管理团队用来决定是否有一个值得尝试解决的问题的。目前还没有解决方案，只有问题和机会。但是，对于大多数团队来说，在这个阶段显然需要对项目范围有一个非常初步的估算。当然，由于在这个阶段还没有详细的解决方案，这将很大程度上是一个

SWAG（"科学性的胡乱猜测"），这就是为什么我建议在此阶段的估算仅限于"小型"、"中型"或"大型"。在这个粒度上通常相当清楚成本会是多少，尽管偶尔仍会有意外。

如果机会相对于估算成本看起来不错，管理层可能会允许项目 proceed 到定义解决方案。正是在这个阶段，产品解决方案被详细说明，最好是用一个高保真原型，你可以通过与真实目标用户进行原型测试来验证。

在提出解决方案的整个过程中，产品经理和交互设计师应该包括一名工程成员来评估各种选项并估算不同替代方案的成本。然后产品经理和设计师会考虑这些信息，并酌情调整产品。但在规格流程结束时，应该有一个基于详细描述拟建产品的详细且高可信度的估算。

在这个阶段，管理团队有详细的产品规格和相应的高可信度成本估算，他们应该能够就 proceed 构建该产品与否做出最终决定。可能是解决方案比他们想象的更复杂和昂贵，或者他们可能不喜欢实际的解决方案，但如果他们 proceed，整个产品团队都知道成本以及他们将为此投资获得的产品。

总结一下，我建议在机会评估时进行初步估算，然后是伴随最终产品规格的详细估算。

第 15 章：特许用户计划

> Chapter 15: Charter User Programs
>
> Your Product Development Partners

你的产品开发合作伙伴

> Most marketing people will tell you that nothing is more important or compelling when launching a product than to have a solid set of reference customers (or reference applications for a platform product). Yet it continually amazes me how many products launch with none.

大多数营销人员会告诉你，在发布产品时，没有什么比拥有一组稳固的参考客户（或平台产品的参考应用）更重要或更有说服力的了。然而， continually 令我惊讶的是有多少产品在没有参考客户的情况下就发布了。

> If at launch there are a half-dozen marquee names publicly stating their use and satisfaction with a product, then the job of the sales and marketing folks is dramatically easier, as the greatest risk the potential customer faces is dramatically reduced. On the other hand, if good reference customers are missing, all the creative marketing and clever sales tactics in the world will only take you so far.

如果在发布时有六个知名企业公开声明他们使用并对产品满意，那么销售和营销人员的工作就会大大简化，因为潜在客户面临的最大风险大大降低了。另一方面，如果缺少好的参考客户，世界上所有的创意营销和聪明的销售技巧也只能带你走那么远。

> If there are no references, this is a huge red flag, and it usually means the product is either bad, or not yet ready for prime time. And you don't want to be the first to try to use it.

如果没有参考客户，这是一个巨大的危险信号，通常意味着产品要么不好，要么还没有准备好投入主流使用。你不想成为第一个尝试使用它的人。

> If there's only one or a few reference sites, I worry that what's been built is essentially a special, or custom solution, and that it won't be generally useful.

如果只有一两个参考站点，我会担心所构建的基本上是一个特殊或定制的解决方案，它不会具有普遍适用性。

> Note that this applies whether we're talking a platform technology, a business application, or even a consumer Internet service. Potential customers need to know that this product really works for people like them.

请注意，这适用于我们讨论平台技术、业务应用，甚至消费者互联网服务的情况。潜在客户需要知道这个产品真的对像他们这样的人有效。

> Now let's move for a moment from our focus on the launch to the very beginning of the project.

现在让我们暂时把注意力从发布转移到项目的最开始。

> As product manager, you know your job is to gain a deep understanding of your target customer, the problems to be solved, and whether you can come up with a product that meets these needs. You know you need to work closely and directly with customers to develop a product that will meet the needs of hundreds of customers (and thousands—or even millions—of users), but you also know there aren't enough hours in the day to work directly with this many customers.

作为产品经理，你知道你的工作是深入了解你的目标客户、要解决的问题，以及你是否能想出一个满足这些需求的产品。你知道你需要与客户紧密直接合作，开发一个能满足数百个客户（以及数千甚至数百万用户）需求的产品，但你也知道一天中没有足够的时间与这么多客户直接合作。

> My favorite technique for addressing both of these problems—getting deep insight into my target customers, and having great reference customers at launch—is to use a charter user program (also known as a Customer Advisory Board, Customer Council, or Voice of the Customer). This is not a new technique (I did my first at HP about 20 years ago), and many companies do this. But I'm surprised how many do not.

我最喜欢的解决这两个问题的方法——深入了解我的目标客户，并在发布时拥有出色的参考客户——是使用特许用户计划（也称为客户咨询委员会、客户委员会或客户之声）。这不是一种新技术（我大约 20 年前在惠普做了我的第一个），许多公司都在这样做。但我惊讶于有多少公司没有这样做。

> The program is fairly straightforward. Your goal is to end with at least six happy, live, referenceable customers from your target market. That means you'll probably need to start with 8-10. So your job is to recruit these customers right at the start of your project. You're looking for customers in your target market who would make great references. They may be

这个计划相当简单。你的目标是最终至少获得六个来自目标市场的满意、活跃、可作为参考的客户。这意味着你可能需要从 8-10 个开始。所以你的工作是在项目开始时立即招募这些客户。你正在寻找会成为出色参考的目标市场的客户。他们可能来自你现有的客户群，或是潜在客户，或通常是两者的混合。关键是他们相信这是一个需要解决的真正问题，并且他们需要尽快解决它。

> from your existing customer base, or prospects, or often a blend of both. The key is that they believe this is a real problem to solve and they need it solved as quickly as possible.

这就是交易： 加入的客户/用户的好处：

> Here's the deal:

- 他们获得早期且重要的产品投入——他们认识到这个产品试图解决的问题，他们感受到痛苦，并渴望确保找到一个好的解决方案

> The benefits to the customers/users that join:

« 他们获得产品的早期访问权——同样，他们感受到痛苦，所以他们能越早获得缓解越好

> · They get early and significant product input—they recognize the problem that this product is trying to solve, they feel the pain, and are anxious to ensure they find a good solution

· 通常，成本会显著降低，如果有的话

> · They get early access to the product—again, they feel the pain, so the sooner they can get relief the better

对你的好处：

> · Typically, there is a significantly reduced cost, if any

« 你有一组用户和客户可用于持续的问题和对话

> The benefits to you:

«你可以访问客户办公室和该公司的用户（如果是平台产品，则可以访问该公司的开发人员）

> · You have a set of users and customers available for ongoing questions and dialog

« 客户/用户同意定期到你的办公室参加小组会议

> +You have access to the customer's offices and the users at that company (or the company's developers if it's a platform product)

« 客户同意及时部署测试版本并提供及时反馈（你通常会和他们在一起）

> · The customers/users agree to come to your offices periodically for group sessions

«如果他们对交付的产品满意，客户同意作为公开参考客户

> · The customer agrees to deploy test versions promptly and provide timely feedback (you'll typically be there with them)

几个关键点：

> «If they are happy with the delivered product, the customer agrees to serve as a public reference customer

«　重要的是客户不要提前支付参与这个计划的费用。那会使这是一种非常不同类型的关系。你想要一个开发产品的合作伙伴——你不想只为他们构建一个定制解决方案，你也不是一个项目商店。你可以在他们爱上产品后收取他们的钱。

> A few critical points:

«如果你像大多数公司一样，你会被想要参与的客户所淹没。这确实是一笔很好的交易，客户知道这一点。如果你有销售组织，他们会试图把这当作讨价还价的筹码，结果是你会被迫纳入比你能处理的更多的客户。这有时需要技巧，但重要的是特许用户计划的成员是合适的。（有时公司会创建一个早期发布计划，供那些想要提前获得软件但不适合特许用户计划的客户使用。这没问题。只是确保你不要把超过大约 10 个客户纳入特许用户计划，因为你无法管理他们并按照你需要的方式与那么多客户密切合作。）

> - It's important that the customer not pay in advance to participate in this program. That would make this a very different type of relationship. You want a partner in developing the product—you do not want to build a custom solution just for them, and you're not a project shop. You can take their money after you deliver them a product they love.

如果你发现你真的很难招募特许用户和客户，那么你很可能在追求一个不那么重要的问题，你可能很难销售这个产品。这是确保你将时间花在值得做的事情上的第一批现实检查之一。如果客户对这个问题不感兴趣，你可能需要重新考虑你的计划。

> «If you're like most companies, you will be overwhelmed with customers that want to participate. It really is a great deal, and customers know this. If you have a sales organization, they'll try to use this as a bargaining chip, and the result is that you'll be leaned on to include many more customers than you can handle. This will take finesse at times, but it's important that the members of the charter user program be the right set. (Sometimes companies create an early release program that is available for those customers that want the software early, but aren't right for the charter user program. This is fine. Just make sure you don't accept more than about 10 customers into the charter user program as you won't be able to manage them and work as closely as you need to with that many.)

«你需要确保你的特许用户和客户真正来自你的目标市场。很容易最终只得到早期采用者，他们宽容得多，很容易导致一个只对早期采用者感兴趣的产品（参见"情感采用曲线"一章）。

> - If you find that you are having real trouble recruiting charter users and customers, then it's very likely you are chasing a problem that isn't that important, and you will probably have a very hard time selling this product. This is one of the very first reality checks to make sure you are spending your time on something worthwhile. If customers aren't interested in this problem, you may want to rethink your plans.

«　你需要向计划的每个成员解释，你正在尝试想出一个通用的产品——你可以成功销售给大量客户的东西。你不是试图构建一个仅适用于特定公司的定制解决方案，而且他们在任何情况下都不会真的想要那样（因为如果你不能用那个产品建立一个真正的业务，你会倒闭，他们会留下没有支持的、死路一条的

> «You need to make sure your charter users and customers are truly from your target market. It's easy to end up with early adopters, who are much more tolerant and can easily lead to a product of interest only to early adopters (See the chapter Emotional Adoption Curve).

软件）。然而，你深深地致力于想出一个对他们非常有效的产品。

> · You will need to explain to each member of the program that you are trying to come up with a general product—something you can successfully sell to a large number of customers. You're not trying to build a custom solution that works only for that particular company, and they wouldn't really want that in any case (because if you can't build a real business with that product, you'll go under and they'll be left with unsupported, dead-end

«　你需要把这些特许用户和客户视为开发合作伙伴。你们在一起。你需要把他们当作同事。你们互相帮助。你会发现你们建立的关系可以持续几十年。

> software). You are, however, deeply committed to coming up with a product that works very well for them.

«你将在整个项目生命周期中与这些人互动——你会向他们展示原型并与他们的用户一起测试，你会问数百个详细的问题，你会在他们的环境中测试发布候选版本。

> · You need to think of these charter users and customers as development partners. You're in this together. You need to treat them as colleagues. You are helping each other. You'll find that the relationships you create can last for decades.

«　确保你在正式发布之前向这些人发布软件，并确保在公开发布之前他们已经在使用并感到满意。当你发布时，他们会准备好支持你。

> · You will be interacting with these people throughout the project lifecycle—you'll be showing them prototypes and testing with their users, you'll be asking hundreds of detailed questions, and you'll be testing release candidates in their environment.

¢　你很可能会与产品营销密切合作，为准备特许客户成为公开参考，而且他们通常也会帮助寻找这些合作伙伴。

> · Make sure you release the software to these people before the general release, and make sure they are live and happy before the public release. When you launch, they'll be ready to stand up for you.

¢　如果你的产品是平台产品（其他人将在你的产品之上编写和交付应用程序），那么这个计划尤其关键。主要区别是你希望以六个参考应用程序而不是六个客户来结束这个计划。你需要与你的应用程序

合作伙伴密切合作，以确保他们在你的平台上构建的应用程序也能成功满足其用户的需求（做到这一点的一个好方法是鼓励他们拥有特许用户）。

> - You'll likely be working very closely with product marketing on preparing the charter customer to be a public reference, and they'll often help with finding these partners as well.

请注意，虽然我的许多示例是针对企业软件或平台产品意义上的客户，但这些技术也适用于消费者互联网服务和消费者设备的最终用户。对于消费者服务，你会希望将集合扩大到

> - If your product is a platform product (others will write and deliver applications on top of your product), then this program is especially critical. The main difference is that you want to focus on ending the program with six reference applications rather than six customers. And you'll need to work closely with your application partners to ensure that the applications they build on your platform are also successful with their users (and a great way to do that is to encourage them to have charter users).

10-15 人左右，但关键是要真正了解这些人以及他们使用产品的环境——家里或办公室。在设计消费者网站时，很容易直到流程的后期（测试版甚至发布）才对真正的目标用户有足够的了解。这是非常危险的，像这样的计划有助于产品经理专注于为真正的用户提供真正的价值。就营销而言，当消费者决定购买或使用产品时，他们可能不会像企业购买者那样看待参考客户，但消费者会受到媒体和用户评论网站的影响，当媒体撰写关于你的产品的报道时，他们首先要寻找的是真正的用户。

> Note that while many of my examples are for customers in an enterprise software or a platform product sense, these techniques also work with end-users for consumer Internet services and consumer devices. For consumer services, you will want to expand the set to

这确实是一种简单而强大的技术，可以确保你正在构建客户想要的产品，并且你可以向潜在客户证明，如果他们选择你，他们很可能会成功和满意。

> 10-15 or so, but the key is to really get to know these people and the environment in which they will be using your product—home or office. It is all too easy when designing a site for consumers to not have enough exposure to true target users until very late in the process (beta or even launch). This is very dangerous, and a program like this helps keep the product manager focused on providing real value to real users. In terms of marketing, when consumers decide to buy or use a product, they may not look at reference customers the same way a business purchaser would, but consumers are influenced by the press and user review sites, and when the press writes a story about your product, the first thing they'll look for is real users.

9° 不与客户交谈?

> This really is an easy and powerful technique to ensure that you're building a product that customers want, and that you can prove to prospective customers that they're likely to be successful and happy if they go with you.

每隔一段时间，我会遇到一位产品经理告诉我，她不被允许与用户或客户交谈。有时是因为销售代表希望控制所有此类访问，或者可能是因为营销应该是与客户的接口，或者产品是通过渠道销售的。偶尔，会有公司政策限制直接客户访问，因为担心不适当的声明或承诺。无论是什么原因，如果你在一家被告知不能与你的用户交谈的公司工作，我的建议是先努力改变这个政策。如果这行不通，那就擦亮你的简历，找一个你可以实践你的手艺并有机会创造成功产品的地方。

> © Don't Talk To Customers?

我真的不知道如果你不对这些用户有深入的了解，你怎么能建立用户会喜欢的产品，而如果没有大量的直接交流——包括面对面互动——你就无法获得这种了解。

> Every so often I meet a product manager who tells me she is not allowed to talk to users or customers. Sometimes it's because the sales reps want to control all such access, or maybe it's because marketing is supposed to be the interface with the customer, or perhaps the product is sold through a channel. Occasionally, there is a corporate policy restricting direct customer access because of worries about inappropriate statements or commitments. Whatever the reason, if you work at a company where you're told you can't talk to your users, my advice is to first try hard to change this policy. If that doesn't work, dust off your resume and find a place where you can practice your craft and have a shot at creating successful products.

通常，特别是在较大的公司中，有许多不同的过滤器试图"帮助"你作为产品经理了解市场。你会发现营销团队委托进行调查和焦点小组，生成关于用户（认为）他们想要什么的报告，销售组织会有像销售工程师（销售代表的技术助理）这样的人被指定来汇总客户输入并将其传递给产品经理。或者你会发现客户服务经理负责每月关于热门问题的报告。

> Treally don't know how you can build products users will love without a deep understanding of those users, and you won't get that without lots of direct communication—including face-to-face interactions.

所有这些输入都很好，但它绝不能替代产品经理获得他完成工作所需的与用户的直接互动。

> Often, especially in larger companies, there are many different filters set up to try to “help” you as product manager understand the market. You'll find marketing groups that commission surveys and focus groups that produce reports on what users (think) they want, and sales organizations that will have someone like a sales engineer (technical assistant to sales reps) designated to aggregate customer input and pass it along to the product manager. Or you'll find customer service managers who are responsible for monthly reports on the top issues.

正是通过了解足够多的这些人——并深入挖掘每个人的潜在需求——我们才获得发现伟大产品所需的洞察力。这些洞察力不会来自表面层面的对话，如"我需要定制这个页面，并获得一个按资源数量划分时间的报告"，或者"72% 的用户说他们想要更高分辨率的视频"。

> All that input is fine, but it is in no way a substitute for the product manager getting the direct interaction with users he needs to do his job.

一定要利用你的组织。许多营销组织拥有用户研究能力，可以在促进和分析用户互动方面提供巨大帮助。只要确保你在那里与研究人员一起工作。带一个营销人员去也很好，这样她可以开始思考信息和定位。而且我喜欢带上首席工程师，这样他可以开始思考如何解决这些潜在问题。

> To be very clear, I believe that the product manager should attend every user interview, every site visit, every usability test, and every charter user program meeting that pertains directly to his product

但不要放弃你理解用户的责任。

> It is from getting to know enough of these people—and digging with each of them into their underlying needs—that we get the insights necessary to discover great products. The insights won't come from the surface level dialog of “I need to customize this page, and get a report with the time spent divided by the number of resources,” or, “72% of our users said they want higher-resolution videos.”

最后一点：当你会见用户时，你会开始自然地发现有些人在符合你的目标档案或他们能提供的洞察水平方面对你更有用。对于这些人，建立持续的关系。获取他们的电话号码和电子邮件地址，并放在你的办公室里 handy。

> By all means, leverage your organization. Many marketing organizations have user research capabilities that can be a huge help in facilitating and analyzing user interactions. Just make sure you're there working with the researcher. It's also fine to bring along a marketing person so she can start thinking about messaging and positioning. And I'm a fan of bringing along the lead engineer so he can start thinking about how these underlying issues might be solved.

他们可能是特许用户计划的绝佳候选人，或者至少你可以用他们进行额外的用户测试。

> But don't abdicate your responsibility to understand the user.
>
> One final point: As you meet users, you'll start to naturally find that some are much more useful to you than others in terms of fitting your target profile, or the level of insight they
>
> can provide. For these people, establish an ongoing relationship. Get their phone number and e-mail address and keep them handy in your office.
>
> They might be great candidates for the charter user program, or at the least you can use them for additional user testing.

第 16 章：市场研究

> Chapter 16: Market Research
>
> Understanding The Capabilities And Limitations

理解能力和局限性

> If your company is like most, there's some amount of natural tension between marketing and product. One often controversial topic is the appropriate role in product discovery of market research tools and techniques such as focus groups, customer surveys, site analytics, site visits, usability testing/field testing and competitive analysis.

如果你的公司像大多数公司一样，营销和产品之间存在一定程度的自然紧张关系。一个经常有争议的话题是焦点小组、客户调查、网站分析、网站访问、可用性测试/现场测试和竞争分析等市场研究工具和技术在产品发现中的适当角色。

> Unfortunately, I think this is an area of significant confusion, fueled in part by the various camps—those from a marketing background that may see the benefits of these tools, and those from product that see the limitations. The result is that some product teams miss out because they don't take advantage of the information these tools and techniques can offer. Yet other teams go astray because they depend on these techniques to answer questions the tools are incapable of answering.

不幸的是，我认为这是一个存在重大困惑的领域，部分原因是各个阵营——来自营销背景的人可能看到这些工具的好处，而来自产品的人则看到局限性。结果是，一些产品团队错过了，因为他们没有利用这些工具和技术可以提供的信息。然而，其他团队则误入歧途，因为他们依赖这些技术来回答工具无法回答的问题。

> This is a big topic, but I'd like to discuss the major market research tools and consider how they can help you—and where they can't.

这是一个很大的话题，但我想讨论主要的市场研究工具，并考虑它们如何帮助你——以及它们在哪里不能。

> The tools for market research have improved dramatically in the past decade. Many of the concerns of the past—which I'll discuss shortly—are addressed by new technologies for easily

过去十年，市场研究工具已经显著改进。过去的许多担忧——我很快就会讨论——通过新技术得到解决，这些技术可以轻松接触大量用户和客户。这些技术还可以帮助你分析用户的活动和行为——他们是谁以及他们如何使用你的产品。也就是说，市场研究工具仍然存在一些非常基本的、固有的局限性，所以理解这一点也很重要。

> reaching out to large numbers of users and customers. These technologies can also help you analyze your user's activity and behavior—who they are and what they do with your product. That said, there are still some very fundamental, inherent limitations to market research tools, so it's important to understand that too.
>
> Capabilities

能力

> Let's begin with a summary of the main tools and techniques:

让我们从主要工具和技术的总结开始：

客户调查。网络使这种方法变得简单而强大。结合联合分析等技术（帮助用户按偏好排序），客户调查既简单又便宜，是任何产品都必须做的。然而，有两件重要的事情需要注意。首先，提出调查问题本身是一门艺术——并不像听起来那么容易。仔细思考问题背景和语境，否则你会发现公司里的人会质疑结果。他们会争论"垃圾进，垃圾出"，如果问题表述不清楚或有偏见，这很可能是真的。其次，在公司中设定期望，这些数据只是答案的一个输入——它不是答案本身。你可能让每个用户都回来对你说"我想要 X"，但对你的公司来说，给他们"Y"可能仍然更有意义。

> Customer surveys. The Web has made this approach easy and powerful. Combined with techniques such as conjoint analysis (to help users rank order their preferences), customer surveys are so easy and so inexpensive, that they're a must-do for any product. However, there are two important things to note. First, there is an art to coming up with the survey questions themselves—it's not as easy as it sounds. Think hard about the questions and context, otherwise you'll find that people in your company will discount the results. They'll argue “garbage in, garbage out,” which may very well be true if the questions are unclear or biased in their phrasing. Second, set expectations in your company that this data is but one input to the answer—it isn't the answer itself. You may very well have every user come back and say “I want X” and it still may make more sense for your company to instead give them

网站分析。如果你的产品是一个网站，有很棒的工具可以帮助你了解用户如何使用它。你需要做一些工作来确保你的网站被适当地埋点，但这是值得的。尽早部署网站分析并持续观察和学习——并调整。如果你的产品不是网站，你通常可以为你的产品埋点，使其记录关于产品使用方式的有价值信息并将数据发送给你。你可能必须向客户明确你发送的是汇总数据，没有个人身份信息，但获取这些数据是值得的。

> Site analytics. If your product is a Web site, there are terrific tools out there for understanding how your users are using it. You'll have to do a little work to make sure your site is instrumented appropriately, but it's well worth it. Get the site analytics in place early

数据挖掘。你会从许多来源收集数据，例如我上面提到的网站分析、计费信息和用户账户信息，以及你自己产品的数据。今天有比以往更好的工具来分析和挖掘这些数据。想知道使用你某些服务组合的用户的性别细分？或者特定角色的活动水平层级和分布？你通常可以用新一代数据分析工具轻松快速地回答这些问题和数千个其他问题。

> and continually watch and learn—and adjust. If your product is not a Web site, you can usually instrument your product so that it records valuable information about how the product is used and sends the data to you. You may have to be clear to your customers that you're sending aggregated data and nothing personally identifiable, but it's worth getting it. Data mining. You'll collect data from many sources, such as the site analytics I've mentioned above, billing and user account information, and your own product's data. Today there are better tools than ever for analyzing and harvesting that data. Want to know the gender breakdown of people that use some combination of your services? Or the activity level tiers and distribution of a specific persona? You can usually answer these and thousands of other questions easily and quickly with the new breed of data analysis tools. Site visits. There is no real substitute for visiting with your users in their native

> habitat—home, office, mall—wherever they will use your product. It can be expensive and time-consuming yet, whenever I do a site visit, I realize something I wouldn't have known any other way. Site visits are extremely valuable, but for cost and time considerations you'll want to pick them carefully.

现场访问。没有什么能真正替代在用户原生环境中拜访他们——家里、办公室、商场——无论他们将在哪里使用你的产品。这可能既昂贵又耗时，然而，每当我进行现场访问时，我都会意识到我以其他方式不会知道的事情。现场访问非常有价值，但考虑到成本和时间因素，你会想要仔细选择。

用户画像。我喜欢用户画像，特别是用于产品定义和设计。市场研究人员也使用用户画像，虽然用途不同。必须意识到没有单一的"用户"，你的工作是深入理解市场上的主要用户类型——那些是你现在的客户，以及那些你希望将来拥有的客户。参见"产品管理的用户画像"一章。

> Personas. I like personas, especially for product definition and design. Market researchers use personas too, although not for the same purposes. It's essential to realize that there is no single “user” and your job is to deeply understand the major types of users out there—those who are your current customers, and those you want to have in the future. See the chapter Personas for Product Management.

可用性测试。我是可用性测试的忠实粉丝，并提倡尽早和经常使用（参见"原型测试"一章）。你也可以将此工具用于现有产品，以更好地了解用户的真实想法。本质上，这是一种通过他们的眼睛看的方式，同时他们使用你的产品——你可以衡量热情或沮丧，并观察行动（而不仅仅是言语）。有远程执行此操作的工具，以及记录和分析人们确切行为的工具，但这些都只是锦上添花。

> Usability testing. I am a huge fan of usability testing, and advocate its use early and often

竞争分析。产品团队经常把竞争对手视为无知的，但根据我的经验，每个产品至少有一些做得好的地方，你的工作就是找到这些东西。从他们的成功和错误中学习。

> (see the chapter Prototype Testing). You can also use this tool with existing products to better understand what users really think. Essentially, it's a way to see through their eyes while they use your product—you can gauge enthusiasm or frustration, and watch actions (and not just words). There are tools for doing this remotely, and for recording and analyzing what exactly people do, but this is all just icing on the cake. Competitive analysis. Too frequently product teams write off competitors as clueless, but in my experience every product has at least some things that the product does well, and it's your job to find these things. Learn from their successes and their mistakes. With these tools and techniques you can get some very real help answering the following important product questions:

有了这些工具和技术，你可以在回答以下重要的产品问题时获得一些非常真实的帮助：

> - Do you understand who your users really are?

« 你真的了解你的用户是谁吗?

> · Howare users using your product?

« 用户如何使用你的产品?

> · Can users figure out how to use your product? Where do they stumble?

¢ 用户能弄清楚如何使用你的产品吗？他们在哪里遇到困难?

> · Why do users use your product?

« 用户为什么使用你的产品?

> · What do users like about your product?

« 用户喜欢你的产品的什么?

> · What do users want added to or changed in your product?

« 用户希望在你的产品中添加或改变什么?

> Limitations

局限性

> Notice that while these questions are critically important, they do not directly address the fundamental question for most product people: What product to build? This information certainly is an input to the product creation process, but you're in trouble if you try to steer

请注意，虽然这些问题至关重要，但它们并没有直接解决大多数产品人员的核心问题：构建什么产品？这些信息当然是产品创造过程的输入，但如果你试图用市场研究来指导你的产品，你就有麻烦了。

> your product with market research. The product discovery process is about answering these questions:

产品发现过程是关于回答这些问题的： « 我可以应用哪些技术以更好的方式解决这个问题？ « 用户体验应该是什么?

> · What technologies can I apply to solve this problem in a better way?

尽管市场研究工具和技术很有用，但我知道没有一个成功的产品是通过市场研究创造出来的。不是谷歌，不是 eBay，不是 iPod 或 iPhone，不是 FaceBook 或 MySpace。没有一个。

> - What should the user experience be? As useful as market research tools and techniques are, I know of no winning product that was created by market research. Not Google, not eBay, not the iPod or iPhone, not FaceBook or MySpace. None. Winning products come from the deep understanding of the user's needs combined with an equally deep understanding of what's just now possible. I wish we could simply ask customers what they want, but if you do that you'll end up with incremental and evolutionary improvements to what they already have (at best) or—more likely—a random collection of band-aid features, and not the new and dramatically better solution that you're looking for. If you've already launched your product, and if you have a set of active customers, you can learn a great deal from talking to them about what parts they like—and what parts they don't—and get their views on incremental features. The key is to understand the limitations of each, and that this data is about refining an existing product rather than conceiving a new So by all means use market research tools to help refine your product and make it as good as

成功的产品来自对用户需求的深刻理解，加上对现在刚刚可能实现什么的同等深入的理解。

> it can possibly be. Just don't expect the techniques to produce the idea for the next Facebook, Flickr, or YouTube.

我希望我们可以简单地问客户他们想要什么，但如果你这样做，你最终只会得到对现有产品的渐进式和进化式改进（充其量）——或者——更有可能——一堆随机的权宜功能，而不是你正在寻找的新的、显著更好的解决方案。

> What About Focus Groups?

如果你已经发布了你的产品，并且如果你有一组活跃的客户，你可以通过与他们交谈了解他们喜欢哪些部分——不喜欢哪些部分——并获得他们对渐进式功能的看法，从而学到很多东西。关键是理解每个的局限性，以及这些数据是关于完善现有产品而不是构思新产品。

> One tool I did not mention above is focus groups. That's because I have very mixed feelings

所以一定要使用市场研究工具来帮助你完善你的产品并使其尽可能

> about focus groups. I like anything that puts the product manager in front of his target users

好，但不要把它们误认为是如何提出下一个大创意的替代品。

> and customers, and it is possible for focus groups to do that. If handled well, there are

产品发现仍然是关于深入的用户理解和技术能力的结合，以提出新的和更好的解决方案。市场研究可以指导和完善，但不要指望它创新。

> benefits to be had.

它可能做到的好。只是不要指望这些技术能产生下一个 Facebook、Flickr 或 YouTube 的创意。

> That said, while you can learn about your target customers, I can promise you that you won't

© 焦点小组怎么样?

> discover great products from focus groups. How come?

我在上面没有提到的一个工具是焦点小组。那是因为我对焦点小组有非常复杂的感觉。我喜欢任何能让产品经理面对其目标用户和客户的事情，焦点小组有可能做到这一点。如果处理得当，是有好处的。

> There are two very fundamental reasons why not:

也就是说，虽然你可以了解你的目标客户，但我可以保证你不会从焦点小组中发现伟大的产品。为什么呢?

> 1.Customers don't know what's possible. Most have no idea about the enabling technologies involved.

有两个非常根本的原因： 1.客户不知道什么是可能的。大多数人对所涉及的使能技术一无所知。 2. 客户不知道他们想要什么。如果不实际看到解决方案，很难设想你想要的解决方案。

> 1. Customers don't know what they want. It's very hard to envision the solution you want without actually seeing it.

焦点小组还有其他缺点： 首先，当用户聚在一起时会发生一种动态。

> There are other drawbacks to focus groups:

他们彼此影响如此之大，以至于你失去了每个人的纯粹输入，而得到了一个由最善于表达或最直言不讳的与会者影响的扭曲代表。

> First, there is a dynamic that happens when users get together.

其次，关于客户不知道他们想要什么的观点，除非客户能够实际看到和与产品互动，否则很难获得关于产品想法的有用数据。大多数情况下，焦点小组是在那个阶段之前进行的。

> They influence each other so much that you lose the pure input of each, and instead get a

第三，与调查一样，进行焦点小组是一门艺术，找到一个真正知道如何有效进行焦点小组的人，以及知道何时应该和不应该使用它们。再加上需要足够理解你的产品领域以引发你需要的深度对话的要求，找到合适的人选可能很困难。

> skewed representation influenced by the most articulate or vocal attendees.

因此，焦点小组经常被用于政治目的，而不是为了获得你正在寻找的对用户的深入理解。

> Second, related to the point about customers not knowing what they want, it's very hard to get useful data about a product idea unless customers can actually see and interact with the product. Most often, focus groups are conducted prior to the stage where that is possible. Third, as with surveys, there is an art to conducting focus groups, and finding someone who actually knows how to conduct them effectively, and when they should and should not be used. Add a requirement to understand your product domain enough to elicit the depth of conversation you need, and it can be tough finding the right person for the job.

如果你觉得必须进行焦点小组，确保产品经理亲自参加每一个。将物流和主持委托给外部公司是可以的，但不要委托数据的解释和分析。

> As a result, too often focus groups are used for political purposes rather than to get that deep understanding of users that you're looking for.
>
> If you feel you must conduct focus groups, make sure the product manager attends every one in person. It is okay to delegate the logistics and the facilitation to an outside firm, but do not delegate the interpretation and analysis of the data.

第 17 章：产品管理的用户画像

> Chapter 17: Personas For Product Management
>
> Understanding The Target User

理解目标用户

> Product management is all about choices—making decisions about what opportunities are worth chasing, which problems are worth solving, what features will provide the most value, what the best time-to-market trade-offs are, and which customers are most important. While you'll never make all the right choices, you have to make most of them right for your product to succeed.

产品管理就是关于选择——决定哪些机会值得追逐，哪些问题值得解决，哪些功能将提供最大价值，什么是最佳的上市时间权衡，以及哪些客户最重要。虽然你永远不会做出所有正确的选择，但你必须做出大部分正确的选择，你的产品才能成功。

> One of my favorite tools for helping to make the hard decisions is a persona (aka user profile)—a technique for capturing the important learnings from interviewing users and customers, and identifying and understanding the different types of people who will be using your product. The persona is an archetype description of an imaginary but very plausible user that personifies these traits—especially their behaviors, attitudes, and goals.

我最喜欢的帮助做出艰难决策的工具之一是用户画像（又名用户档案）——一种从采访用户和客户中捕获重要学习的技巧，识别和理解将使用你的产品的不同类型的人。用户画像是对一个虚构但非常合理的用户的原型描述，这些特质被具体化——尤其是他们的行为、态度和目标。

> The tool was first described in 1998 in one of my all-time favorite books, The Inmates Are Running the Asylum, by Alan Cooper. If you haven't read this book you should—it"s a classic for product managers, designers, and engineers.

这个工具最早于 1998 年在 Alan Cooper 的一本我一直以来最喜欢的书《精神病院里的囚犯》中描述。如果你还没有读过这本书，你应该读——它是产品经理、设计师和工程师的经典之作。

> There's a good chance that your designers already use personas in the design process. The

很有可能你的设计师已经在设计过程中使用用户画像了。设计社区似乎已经采用了这种技术，因为我遇到的大多数设计团队都使用这个工具。每个团队都对什么构成一个好的用户画像有自己的看法，有些比其他更正式，但在我看来，他们都是好的。

> design community seems to have adopted this technique as most of the design teams I meet use this tool. Each has their own spin on what makes a great persona, and some are much more formal about them than others, but in my view they're all good.

甚至有可能你的营销人员在创建他们的信息和广告项目时使用用户画像。这些用户画像的用途是相似的——而且都很有用——但它们并不完全相同，因为它们用于有些不同的目的。营销人员试图确定接触目标客户和吸引潜在情感的最佳方式。设计师最关心的是用户的目标和在线行为。

> And there's even a chance your marketing people use personas as they create their messaging and advertising programs. These uses of personas are similar—and they're both useful—but they're not quite the same thing as they are used for somewhat different purposes. The marketing folks are trying to determine the best ways to reach the target customers and appeal to the underlying emotions. The designers care most about the user's goals and online behaviors.

作为产品经理，这一切对你都非常有用。

> As product manager, this is all extremely useful to you.

不幸的是，虽然这是一个真正强大的工具，但它往往在产品定义/发现过程中的后期才被采用，比它应该的晚。通常是设计师推动这一点，而他们往往被带入流程的时间比他们应该的晚。

> Unfortunately, while this is a truly powerful tool, it is often not employed until later in the product definition /discovery process than it should be. Often it is the designers who drive this, and they are all too often brought into the process later than they should be.

为了获得用户画像的真正潜力，产品经理需要深入参与他们的创建和优先级排序，特别是用户访谈和研究，以识别他们。用户画像的创建应该是产品经理和交互设计师之间的协作——如果你足够幸运拥有他们——还有你的用户研究团队。但无论你做什么，都不要把这个任务委托出去。出于同样的原

因，产品经理需要亲自参加其产品的每一次可用性测试，他或她需要在每一次用户访谈中。产品经理需要来自与尽可能多的用户和客户交谈的对目标用户的深入理解。

> To get the true potential of personas, the product manager needs to be deeply involved in their creation and prioritization, and especially the user interviews and research that goes into identifying them. The creation of personas should be a collaboration between the product manager and interaction designer and—if you are lucky enough to have them—your user research team. But whatever you do, don't delegate this task. For the same reason that the product manager needs to be present at every usability test of his product, he or she needs to be at every user interview. The product manager needs that deep understanding of the target user that comes from talking with as many users and customers as possible.

我总是鼓励产品经理积极参与用户画像的创建——并确保它们在流程中尽早完成。

> Ialways encourage product managers to actively participate in the creation of personas—and make sure they are done as early in the process as possible.

设计社区已经广泛撰写了关于用户画像的文章，所以我不会在这里试图重复所有细节，只是指出一些与产品经理相关的问题。详细描述请参阅 Alan Cooper 的《About Face 3》一书。

> The design community has written extensively about personas so I won't try to duplicate all the details here, other than to point out some issues as they relate to product managers. For a detailed description, see the book About Face 3 by Alan Cooper.

使用用户画像作为产品管理工具有许多好处：

> There are numerous benefits of using personas as a tool for product management:

¢ 用户画像帮助你确定什么是重要的。如果你已决定让"玛丽"成为此次发布的目标，那么如果这个功能对"玛丽"至关重要，那就放进去，如果它是为"山姆"的，那就不要。正如你所看到的，与决定发布是为谁的一样重要的是，决定它不是为谁。产品试图取悦每个人最终却谁都不满意是一个极其常见的错误。这个过程有助于防止这种情况。

> · Personas help you prioritize what's important. If you have decided to make “Mary” the target for this release, then if this feature is critical for “Mary” then put it in, if it's for “Sam” then it's out. As you can see, just as important as deciding who a release is for, is

> deciding who it is not for. It is an extremely common mistake for a product to try to please everyone and end up pleasing no one. This process helps prevent that.

- 产品团队最常犯的错误之一是把自己和客户混淆。我真正喜欢用户画像的一点是，它们有助于揭示这个太普遍的问题。

> - One of the most common mistakes product teams make is confusing themselves with their customers. One thing I really like about personas is that they help shine a light on this all too prevalent problem.

«许多产品有许多类型的用户——不同类型的最终用户、经理、管理员等，很容易认为你应该为这些人中的每一个都放一些功能，然后，你可能会得到一个混乱的烂摊子。这在某种程度上是一个设计问题，但用户画像通常帮助你确定这些不同用户的重要性，并意识到你在哪里需要单独的用户体验。

> - Many products have many types of users—different types of end-users, managers, administrators, etc., and it's easy to think that you should just put some features in for each of these people, and again, you can end up with a muddled mess. This is partly a design problem, but personas often help you prioritize the importance of these different users, and also realize where you need a separate user experience.

«　用户画像是一个非常有用的工具，用于向你的整个产品团队描述产品是为谁设计的，他们将如何使用它，以及他们为什么会关心。

> - Personas are a very useful tool for describing to your entire product team who the product is for, how they will use it, and why they will care.

«　更一般地说，很像产品原则，用户画像有团结团队围绕共同愿景的好处。在产品发布过程中需要处理的细节有成千上万。产品经理（或设计师）不可能每一个都做出决定。如果每个经理、设计师、作家、开发人员和测试人员都把产品原则和用户画像记在心里，那么在面对一个开放问题时，他们更有可能做出正确的决定。

> - More generally, much like the product principles, personas have the benefit of rallying

这些都是相当好的好处。但也有一些陷阱需要注意：

> the team around a common vision. There are literally thousands of details that will have to be addressed in the course of a product release. The product manager (or designer) can't possibly make every one. If every manager, designer, writer, developer, and tester has taken the product principles and personas to heart, then when faced with an open question, they are more likely to make the right call.

«　一些团队创建用户画像，但他们没有采取下一步——对哪个用户画像应该是优先级做出艰难的选择。说你的产品适合每个人是不行的——你只是在欺骗自己。虽然这对大多数产品经理来说极其困

难，但我努力让产品经理专注于每个发布的一个主要用户画像。这并不是说该发布对其他用户不会有价值和可用，但你在每个发布上的重点应该是为仅一种类型的目标用户做好工作。

> These are some pretty great benefits. But there are also some pitfalls to watch out for:

- 有时团队基于他们对用户的假设和刻板印象创建用户画像，而实际上并不花时间与真实用户交谈并验证这些理论化的人是否真的存在。我已经多次感到惊讶——事实上如此多次，以至于我学会了将我的最初印象仅仅视为一种理论，并在与真实用户交谈之前推迟真正的判断。创建用户画像的过程绝不能替代与真实目标用户的面对面交谈，以及在真实用户上测试你的设计。

> - Some teams create personas but they don't take the next step—to make the hard choices about which persona should be the priority. It's not ok to say your product is for everyone—you're only deluding yourself. While this is extremely difficult for most product managers, I try hard to get the product manager to focus each release on a single primary persona. It's not that the release won't be valuable and usable by others, but your focus on each release should be to do a great job for just one type of target user.

« 一个经常出现的问题是——当你为用户招募原型测试时——你只测试来自主要用户画像的用户吗？当然你需要确保你的产品对目标人群很棒，然而，你也会想测试一些来自这个用户画像之外的人，因为你不会有只有主要用户画像使用你的产品的奢侈。所以对于原型测试，你会想招募一系列可能的用户。

> - Sometimes teams create personas based on their assumptions and stereotypes of their users, and they don't actually take the time to talk to real users and verify if these theoretical people really exist. I have been surprised many times—so many times in fact that I have learned to consider my initial impressions as just a theory, and hold off on real judgments until after talking with real users. In no way is the process of creating a persona a substitute for talking face-to-face with real target users, and testing your designs on real users.
> - One question that often comes up is—as you recruit users for your prototype testing—do you only test with users from your primary persona? Certainly you need to make sure your product is great for the people it is intended for, however, you'll want to test with some people from outside this persona as well, as you won't have the luxury of having
>
> only primary personas using your product. So for prototype testing, you'll want to recruit a range of possible users.
>
> ° Examples

° 示例

> You can see example personas at www.svpg.com examples.

你可以在 www.svpg.com/examples 上看到示例用户画像。

第 18 章：重塑产品规格

> Chapter 18: Reinventing The Product Spec
>
> R.LP. PRD

安息吧 PRD

> I think the product spec is long overdue for a renovation. Some would argue that Agile methods accomplish this by doing away with the spec altogether. While there are other issues with that, in many respects I think they are on the right track.

我认为产品规格早就该翻新了。有些人会认为敏捷方法通过完全取消规格来完成这一点。虽然这有其他问题，但在许多方面我认为他们走对了方向。

但在我们操之过急之前，让我们先讨论一下当今基于纸张的规格的问题。

当然，产品规格的范围很大，从我们称呼它们开始（产品需求/PRD、市场需求/MRD、业务需求/BRD、功能规格/FS 等）。涵盖的主题可能有很大差异（这些不同的文件本来不是用于相同目的的，但随着时间的推移，它们已经合并和变形，失去了许多原来的区别），详细程度，当然还有规格本身的质量。甚至形式也有很大变化——许多人使用 MS Word 文档，但有些人使用电子表格，有些人将规格发布在 Wiki 网站上，其他人使用商业需求管理工具之一。

如果你同意我的观点，即产品经理的核心职责是确保向工程团队交付一个描述成功产品的产品规格，那么我们必须承认典型规格流程的弱点，并仔细审视产品是如何被定义的。

以下是我认为一个好的、有用的产品规格的要求：

- 规格必须描述完整的用户体验——不仅仅是产品需求，还有用户交互和视觉设计。到现在，希望每个人都认识到需求与用户体验设计是如何紧密交织在一起的。

«规格必须准确代表软件的行为——我们需要承认，文字和漂亮的图片在描述这种行为方面的能力太有限了。

«规格有几个关键的消费者——工程、QA、客户服务、营销、网站运营、销售和高管。因此，规格需要以所有这些群体都能获得他们需要的东西的方式来传达产品的行为。

¢ 规格会改变——一旦工程开始，变化的速度应该会显著减慢，但会出现决策和问题，规格应该改变以反映最新的决策。

«在创建规格的过程中有许多工件，例如优先级需求列表、线框图和模型，但需要有一个单一的规格主

表示，以最大程度地减少混乱、歧义和版本问题。

在我看来，只有一种形式的规格能够满足这些要求，那就是高保真原型。

"高保真"一词指的是这应该是对提议的用户体验的真实表示。除了最简单的用户界面之外，我不喜欢所谓的纸质原型。有了今天可用的工具，现在对于大多数产品来说，创建一个高保真原型是如此快速、

简单和便宜，以至于没有理由不这样做。这仍然是一个原型，所以伪造（模拟）后端处理和数据是可以的，只要用户体验是可信的。

在过去几年中，我的想法已经从只是原型化用户体验的几个关键组件，演变为现在我主张原型化几乎所有内容——所有页面/屏幕和所有主要用例。仍然会有一些错误条件和边角情况不值得原型化，但拥有一个完整产品团队可以与之互动以理解要构建的产品的高保真表示的好处是如此之大，以至于它们超过了增量成本。

确实，你仍然需要补充原型，因为提议产品行为的某些方面不容易在原型中表示，例如业务逻辑（如税表和运费）、发布需求（如可靠性、性能、可扩展性）或平台交付需求（如安装需求或要支持的浏览器版本列表）。作为补充有用的还有用例，描述产品中最重要的流程。

仍然有一个问题是如何最好地表示这些补充材料。我真正想要的是注释原型，但在该技术 readily available 之前，我更喜欢使用 Wiki 或其他形式的内部网站。最大的原因是产品团队中的每个人都知道在任何时间在哪里可以找到最新的答案，而不是有各种随机版本的文档四处漂浮。发布问题和答案也很容易，在规格更新时设置自动电子邮件通知，并跟踪决策历史。

> But before we get ahead of ourselves, let's start by discussing the problem with today's paper-based specs.

但产品规格的大部分应该是高保真原型，代表用户体验的功能需求、信息架构、交互设计和视觉设计。

除了满足上述要求之外，我认为最重要的好处是——与纸质文档不同——高保真原型可以被测试。你可以把它放在实际目标用户面前，确保他们能弄清楚如何使用你的产品（可用性），并确定他们是否愿意使用你的产品（价值）。在你的原型通过这两项测试之前，你实际上没有一个值得交给工程的规格。在 QA 或 Beta 阶段进行这种形式的测试在流程中太晚了。

我可以保证，如果你尝试一下——创建一个提议功能和用户体验的高保真原型——你的产品团队绝对会喜欢它。工程师是直接的赢家，因为他们终于得到了一个有效且明确地描述他们需要构建的产品的规格，当他们困惑于某事应该如何表现时，他们可以随时参考它。QA 的工作也同样变得

更容易，因为他们现在知道在测试实际产品时应该发生什么。营销、销售和客户服务会很高兴能够在周期中更早地学习产品。你也会发现你的高管也会喜欢它，因为他们可以向投资者、董事会成员和合作伙伴描述你正在做的事情（并演示原型），比任何 PowerPoint 演示文稿都更有效。

> There is of course tremendous range among product specs, starting with what we call them

但等等，还有更多。

对大多数团队来说最大的惊喜是，以这种方式创建规格通常会显著缩短上市时间。是的。我意识到这可能听起来违反直觉，但要理解为什么总上市时间更快，你必须更深入地看看软件项目中几乎总是发生的事情。因为典型的规格如此糟糕（不完整、有歧义，尤其是未经测试），而且很少有难题和关键细节被实际处理和解决，所以在工程阶段，团队被迫解决这些问题。这导致了巨大的变动（规格不断

变化导致工程中的延迟和沮丧），或者工程师只能尽力做出假设，发布的产品一团糟，需要一个或多个更新版本才能实际向客户提供有用的东西。无论哪种情况，上市时间都比应该的要长。

所以我真的希望在你的下一个产品上你能尝试一下。与其花费数周时间在一个很少有人会阅读且无法测试的 50 页 Word 文档上，不如与你的设计师合作创建一个你提议的产品的原型。然后向目标用户以及你的产品团队展示该原型。你最终会迭代几次（现在比工程花费数月构建一个糟糕的产品要好！），但当你得到正确的

方案时，使用该原型作为你交付给工程的规格的基础——看看会发生什么。

> (Product Requirements/PRDs, Market Requirements/MRDs, Business

° 示例

> Requirements/BRDs, Functional Specs/FS, and more). The topics covered can vary greatly (these different documents were not intended to serve the same purposes, but over time they have merged and morphed and lost many of their original distinctions), the level of detail, and of course the quality of the spec itself. Even the form varies greatly—many use MS Word documents, but some use spreadsheets, some post the spec on a Wiki site, and others use one of the commercial requirements management tools.

你可以在 www.svpg.com/examples 上看到示例高保真原型和产品规格。

> Ihave seen a few truly good product specs, but most specs take too long to write, they are
>
> seldom read, and they don't provide the necessary detail, don't address the difficult questions, nor contain the critical information they need to. And—most important—it is all too easy for the mere existence of the spec to serve as a false indicator to management and the product team that everything is proceeding just fine™
>
> If you agree with me that the central responsibility of the product manager is to make sure that you deliver to the engineering team a product spec that describes a product that will be successful, then we have to acknowledge the weaknesses in the typical spec process and take a hard look at how products are defined.
>
> Here are what I consider the requirements for a good and useful product spec:
>
> · The spec must describe the full user experience—not just the product requirements but also the user interaction and visual design. By now, hopefully everyone recognizes how closely intertwined the requirements are with the user experience design.
>
> +The spec must accurately represent the behavior of the software—and we need to acknowledge that words and pretty pictures are just too limited in their ability to describe this behavior.
>
> · There are several critical consumers of the spec—engineering, QA, customer service, marketing, site operations, sales, and executives. As such, the spec needs to communicate the behavior of the product in a way that all these groups get what they need.

- The spec will change—the rate of change should slow down dramatically once engineering gets started, but there will be decisions and issues that arise, and the spec should change to reflect the very latest decisions.
- There are a number of artifacts in the creation of a spec, such as lists of prioritized requirements, wireframes, and mock-ups, but there needs to be a single master

representation of the spec to minimize confusion, ambiguity and versionitis. In my mind, there's only one form of spec that can deliver on these requirements, and that is the high-fidelity prototype. The term "high-fidelity" refers to the fact that this should be a realistic representation of the proposed user experience. Except for the most trivial of user interfaces, I am not a fan of so-called paper prototypes. With the tools available today, it is now so quick, easy, and inexpensive to create a high-fidelity prototype for most products that there is no reason not to do so. This is still a prototype, so it's fine to fake (simulate) the backend processing and data, so long as the user experience is plausible. Over the past few years, my own thinking has evolved here from just prototyping a few critical components of the user experience, to now I advocate prototyping virtually everything—all pages/screens, and all the major use cases. There will still be some error conditions and corner cases that don't pay to prototype, but the benefits of having a high-fidelity representation of the product that the full product team can interact with to understand the product to be built are so great that they dwarf the incremental costs. It is true that you will still need to supplement the prototype, as there are aspects of the proposed product's behavior that are not easily represented in a prototype, such as business logic (e.g. tax tables and shipping charges), the release requirements (e.g. reliability, performance, scalability) or platform delivery requirements (such as_ installation requirements, or the list of browser versions to be supported). Also useful as a supplement are use cases, describing the most important flows through the product.

There is still the question of how to best represent this supplementary material. What I really want is to annotate the prototype, but until that technology is readily available, I prefer using a Wiki or other form of Intranet site. The biggest reason is that everyone on the product team knows where to find the latest answers at any time, rather than having various random versions of documents floating around. It is also easy to post questions and answers, set up automatic e-mail notifications whenever the spec is updated, and track the history of decisions.

But the majority of the product spec should be the high-fidelity prototype, representing the functional requirements, the information architecture, the interaction design, and the visual design of the user experience.

In addition to meeting the requirements described above, the most important benefit in my view is that—unlike a paper document—a high-fidelity prototype can be tested. You can put it in front of actual target users and ensure that they can figure out how to use your product (usability), and also determine if they care to use your product (value). You don't actually have a spec worth handing over to engineering until your prototype passes these two tests. Doing this form of testing while you are in QA or Beta is far too late in the process.

Ican promise that if you give this a try—creating a high-fidelity prototype of the proposed functionality and user experience—your product team will absolutely love it. Engineers are the

> immediate winners as they finally get a spec that effectively and unambiguously describes the product they need to build, and that they can refer to at any time when they're confused about how something is supposed to behave. The job of QA is similarly made
>
> easier as they now know what should happen when they test the actual product. Marketing, sales, and customer support will love being able to learn the product much earlier in the cycle. You'll also find that your execs will love it too as they can describe what you're doing (and demo the prototype) to investors, board members, and partners much more effectively than any PowerPoint deck can do.
>
> But wait, there's more.
>
> The biggest surprise for most teams is that creating a spec this way will typically significantly reduce time to market. Yep. I realize this may sound counter-intuitive, but to understand why the total time to market is faster, you have to look a little deeper at what almost always happens in a software project. Because the typical spec is so poor (incomplete, ambiguous, and especially untested), and so few of the hard questions and critical details are actually addressed and resolved, it is during the engineering phase that the team is forced to tackle these issues. This results in either tremendous churn (the specs keep changing resulting in delays and frustration in engineering), or the engineers just make assumptions as best they can, and the product that ships is a mess and one or more update releases are required before you actually get something useful to your customers. In either case, the time to market is longer than it should be.
>
> So I truly hope on your next product you'll try this out. Rather than spending weeks working on a 50-page Word document that few will read and is impossible to test, work with your designer to create a prototype of the product you are proposing. Then show that prototype to target users, as well as your product team. You'll end up iterating several times (better now than after engineering spends months building a bad product!), but when you get the recipe
>
> right, use that prototype as the basis for the spec you deliver to engineering—and see what happens.
>
> Orxamples
>
> You can see example high-fidelity prototypes, and product specs at www.svpg.com/examples.

第 19 章：设计与实现

> Chapter 19: Design Vs. Implementation
>
> Define The User Experience Before Proceeding To Build

在继续构建之前定义用户体验

> There are many things in the software development process that can and should be done in parallel. For example, I have long argued that requirements (functionality) and design (user experience design) are intertwined and should be done together. I don't like the old waterfall model of a product manager doing “requirements” and handing that off to interaction designers

> that do “design.” Most teams understand now that this is an obsolete view of product development.

在软件开发过程中有许多事情可以而且应该并行完成。例如，我早就主张需求（功能）和设计（用户体验设计）是交织在一起的，应该一起完成。我不喜欢产品经理做"需求"并将其交给交互设计师做"设计"的旧瀑布模型。现在大多数团队都明白这是产品开发的一种过时观点。

我也相信，学习了并行进行实现和测试价值的软件工程团队已经取得了巨大进步。工程师编写软件然后全部交给 QA 人员测试的旧模型实际上需要更长时间，而且结果不太可靠。XP 等敏捷方法展示了协调进行实现和测试的价值。

> I also believe that great strides have been made by software engineering teams that have learned the value of doing implementation and testing in parallel. The old model of the engineer writing software and then handing it all off to a QA person to test actually takes longer and the result is less reliable. Agile methods like XP demonstrate the value of doing implementation and testing in concert.

也就是说，有一件事许多团队试图并行完成——但不应该——是用户体验设计和实现。

> That said, one thing that many teams try to do in parallel—but should not—is user experience design and implementation.

这有几个原因：

> There are several reasons for this:

首先，软件团队中有一个重要的动态需要认识到：一旦实现开始，随着你处理用户体验设计想法，做出可能必要的基本变更就变得越来越困难。部分原因是技术性的——工程团队必须基于对需求和设计的假设做出一些早期的架构决策，以便取得进展。这些早期决策很重要，有重大影响。部分原因是心理上的——一旦团队转变为实施模式，就会有一种心态占据主导，倒退会令人沮丧。部分原因是实际的——时间在流逝，返工和变动只会增加团队面临的压力。所以即使像 Agile 这样的方法倡导拥抱变化，你很快会发现有些变化比其他变化更受欢迎。

> First, there is a dynamic in software teams that is important to recognize: once implementation begins, it becomes increasingly difficult to make the fundamental changes that will likely be necessary as you work through your user experience design ideas. Partly this is technical—engineering teams must make some early architectural decisions based on assumptions about the requirements and designs in order to make progress. These early decisions are important and have big consequences. Partly, this is psychological—there is a mindset that takes hold once the team shifts into implementation mode, and it's demotivating to go backwards. Partly, this is practical—the clock is ticking, and rework and churn just compounds the pressure the team is under. So even though methods like Agile advocate embracing change, you quickly find that some changes are much more welcome than others.

其次，用户体验设计涉及可用性和价值的非常困难的问题，为了想出一个既可用又有价值的产品，你需要尽早并经常尝试想法。一个常见的回应是"我们会在测试版期间获得反馈"，或者对于敏捷团队，"我们会在冲刺结束时测试这个想法"。不幸的是，等待测试想法的时间太长了。一个好的用户体验设计师希望在几天内尝试几十种想法和方法，想到即使等待两到四周的冲刺也会令人沮丧，因为频率慢了一个数量级。

> Second, user experience design deals with very difficult questions of both usability and value and, in order to come up with a product that is both usable and valuable, you will need to try ideas out—early and often. One common response is “We'll get feedback during beta,” or with Agile teams, “We'll test the idea out at the end of the sprint.” Unfortunately, this is far too long to wait to test out an idea. A good user experience designer will want to try out dozens of ideas and approaches in a matter of days, and the thought of waiting even for a two- to four-week sprint would be debilitating as the frequency is an order of magnitude too slow.

第三，与上述相关，我认为要尝试一个想法，你需要一个高保真原型。有些人会认为测试版发布可以被视为原型，或者冲刺的结果可以被视为原型。但除了等待太久才能获得该软件进行测试之外，重要的是要意识到原型软件与生产软件有很大不同。原型软件需要真正是一次性的。它需要是可以

> Third, and related to the above, I argue that to try out an idea you need a high-fidelity prototype. Some will argue that the beta release can be viewed as the prototype, or that the result of the sprint can be viewed as the prototype. But even beyond waiting too long for that

在几个小时内大幅改变的东西。生产软件所需要的东西就像拖着船锚一样拖累原型。你还会发现不同类型的人喜欢编写原型软件与生产软件。

> software to be available for test, it's important to realize that prototype software is far different than production software. Prototype software needs to be truly disposable. It needs to be something that can be changed substantially in a few hours. What is necessary for production software is like dragging around a boat anchor for a prototype. You'll also find that different types of people like to write prototype versus production software.

第四，虽然将发布分成几个迭代来实现通常非常有意义（这降低了风险，提高了质量，并简化了集成），但用户体验通常不是可以分块设计的东西。你必须整体看待用户体验——它在每次发布时都必须对用户有意义。虽然很容易"存根化"尚未可用的软件，但对用户体验这样做就不那么容易了。

> Fourth, while it often makes excellent sense to break up a release into several iterations to implement (this reduces risk, improves quality, and eases integration) a user experience is often not something that can be designed in pieces. You have to look at the user experience holistically—it has to make sense to the user at each release. While it's easy to “stub out” software that's not yet available, it's not so easy to do the same for the user experience. Finally, user experience designers don't necessarily require a lot of time (just as with software engineering, it depends on the methodology they are using, the particular product requirements, and the skills and experience of the specific people), but they do require some time. Even if it's only a week or two.

最后，用户体验设计师不一定需要很多时间（就像软件工程一样，这取决于他们使用的方法、特定的产品需求以及特定人员的技能和经验），但他们确实需要一些时间。即使只是一两周。

> If you try to start implementation at the same time as design, here's what you will almost certainly see: the designers will be stressed trying to accomplish weeks worth of work in just days, the engineers get anxious as they wait for the designers to give them something, soon the designers will reluctantly make some preliminary guesses to allow the engineers to get going and then hurry to try to get something decent put together before the engineers get too far down the path. However, when they finally do have something, it'll be too late and the engineers will say “we can get to it in the next round” but of course the next round has its own priorities. The designers do not feel good about what is built and shipped, and the

如果你试图在设计与实现同时开始，以下是你几乎肯定会看到的：设计师会感到压力，试图在短短几天内完成数周的工作，工程师在等待设计师给他们东西时会感到焦虑，很快设计师会不情愿地做出一些初步猜测，让工程师开始工作，然后急忙试图在工程师走得太远之前把一些像样的东西放在一起。然而，当他们最终有了东西时，已经太晚了，工程师会说"我们可以在下一轮处理它"，但当然下一轮有自己的优先级。设计师对构建和发布的东西感觉不好，

> customers don't like the result either.

客户也不喜欢结果。

> In the worst-case situation, the designers come to the conclusion that they need to go work for a company that prioritizes the user experience.

在最坏的情况下，设计师得出结论，他们需要去一家优先考虑用户体验的公司工作。

> Fortunately, this really isn't a hard problem to solve. The key is that the user experience design needs to happen before the implementation begins. This is one situation where sequential is important. The requirements and design happen together, and then implementation and test can happen together.

幸运的是，这真的不是一个难题。关键是用户体验设计需要在实现开始之前发生。这是顺序很重要的一个情况。需求和设计一起发生，然后实现和测试可以一起发生。

> For Agile teams, the product manager and user experience designers use the “sprint zero” concept to stay a step or two ahead of the engineers. You are still working to design in as small of increments as possible. It requires a somewhat richer definition of what's in the backlog, but the team will be happier and the product will be better for it.

对于敏捷团队，产品经理和用户体验设计师使用"冲刺零"概念，比工程师领先一步或两步。你仍然致力于尽可能小的增量进行设计。它需要对 backlog 中的内容有更丰富的定义，但团队会更快乐，产品也会因此更好。

> The exception to the rule is when the engineers have a lot of backend infrastructure work to do. In this situation, the engineering team can be working on this while the user experience is being defined. There will be some interdependencies, but they can be managed. If your user experience designers are about to revolt, have your engineers work on the infrastructure for a release cycle or two, as this gives the designers time to work on creating a backlog of good design.

规则的例外是当工程师有大量后端基础设施工作要做时。在这种情况下，工程团队可以在定义用户体验的同时进行这项工作。会有一些相互依赖关系，但它们可以被管理。如果你的用户体验设计师即将反抗，让你的工程师在一个或两个发布周期内从事基础设施工作，因为这给了设计师时间来创建一个良好的设计 backlog。

> Note that although I'm advocating that requirements and design are both done before implementation begins, you will still need at least someone from engineering to review the design work from the start, as it's critical for them to assess feasibility and costs along the

请注意，虽然我主张需求和设计都在实现开始之前完成，但你仍然需要至少一名工程人员从一开始就审查设计工作，因为让他们评估可行性和成本是至关重要的

> way. This is necessary to inform the design process. Remember that the objective is to ensure that you discover a product definition that is valuable, and usable.

方式。这是告知设计过程所必需的。记住，目标是确保你发现一个有价值的、可用的产品定义。

> There is a remarkable amount of confusion out there today in terms of incorporating good design, especially as many teams experiment with Agile methods. I think this is unfortunate as with only a few caveats and adjustments, the Agile methods can be a huge step forward for teams that previously used conventional waterfall methods. I talk about the root causes of this confusion and how you can get the best of both worlds in the chapter Succeeding with Agile Methods.

今天在整合良好设计方面存在大量的困惑，特别是随着许多团队尝试敏捷方法。我认为这是不幸的，因为只要有一些警告和调整，敏捷方法对于以前使用传统瀑布方法的团队来说可能是巨大的进步。我在"成功运用敏捷方法"一章中讨论了这种困惑的根本原因，以及你如何获得两个世界的最佳效果。

第 20 章：最小化产品

> Chapter 20: Minimal Product
>
> Cutting Features Versus Slipping Dates

削减功能与推迟日期

> Have you seen this movie before? The one where the product manager comes up with this great product spec that is packed with features—all clearly marked as Pi/Must Have, P2/High Want, or

> P3/Nice to Have. Then he hands the spec off to engineering, and they estimate the costs of the various features, lay the features out against their staff availability, and come up with a schedule that's typically months longer than the product manager needs. So then the negotiating game starts—arguing estimates, cutting features, minimizing QA and beta times, trying to hire some extra contract staff, etc.—all while the clock is ticking. I'm sure you know the story—even if you haven't seen the movie, you can guess the ending. The product that eventually ships is far from a coherent whole; and no one is happy with it—not the product manager, not the engineers, and definitely not the end users.

你以前看过这部电影吗？产品经理想出了一个很棒的产品规格，里面装满了功能——所有功能都清楚地标记为 P1/必须有、P2/高度想要或 P3/有就好。然后他把规格交给工程部门，他们估算各种功能的成本，根据员工可用性安排功能，并制定一个通常比产品经理需要的多几个月的进度表。然后谈判游戏就开始了——争论估算、削减功能、最小化 QA 和测试时间、试图雇佣一些额外的合同员工等等——所有这些都在时间滴答作响的同时进行。

我相信你知道这个故事——即使你没有看过这部电影，你也能猜到结局。最终发布的产品远非一个连贯的整体；没有人对它满意——不是产品经理，不是工程师，绝对不是最终用户。

> Many teams think this is just how the game is played. But this is really just the natural consequence of a flawed process.

许多团队认为这就是游戏的玩法。但这实际上只是一个有缺陷流程的自然后果。

> Instead, I argue for a very different model:

相反，我主张一个非常不同的模式：

> First, the job of the product manager, working with his designer, is to come up with a high-fidelity prototype with the minimal functionality necessary to meet the business objectives, yet with a user experience that users can figure out how to use—and actually want to use. The reason it's so important that the team come up with the minimal functionality is to minimize implementation time and user complexity.

首先，产品经理的工作是与他的设计师合作，想出一个高保真原型，其中包含满足业务目标所需的最少功能，但具有用户可以弄清楚如何使用——并且实际想要使用——的用户体验。团队提出最少功能如此重要的原因是为了最小化实现时间和用户复杂性。

> Second, starting at the very beginning of this design process, someone from the engineering team (typically an architect or lead engineer) needs to participate in reviewing the product ideas expressed in the prototype, so she can help the product manager and designer understand the relative and absolute costs of the various product ideas. She can point out any dangerous directions the product might be heading in, or she can go investigate any areas she's unsure about. But by the time the prototype is ready, this engineer must have provided detailed estimates of the surviving features. So the many trade-offs of what is in and what's cut have already been made—and made

> collaboratively—and at this point the engineering team must have a detailed estimate that they can commit to.

其次，从设计过程的最开始，工程团队的某个人（通常是架构师或首席工程师）就需要参与审查原型中表达的产品想法，以便她可以帮助产品经理和设计师理解各种产品想法的相对和绝对成本。她可以指出产品可能走向的任何危险方向，或者她可以去调查任何她不确定的领域。但到原型准备好时，这位工程师必须已经提供了幸存功能的详细估算。因此，关于什么在内、什么被削减的许多权衡已经做出——并且是协作做出的——在这一点上，工程团队必须有一个他们可以承诺的详细估算。

> Third, it's essential that this prototype be validated (tested) with real target users. Before committing the resources of the full product team, the product manager and designer must be confident they have come up with something that will succeed. It's not enough to just believe the product definition is good, you have to test to make sure. Just as you wouldn't allow an engineer to ship code just because he or she believed it was good, you must test that code to make sure.

第三，必须用真实目标用户验证（测试）这个原型至关重要。在承诺完整产品团队的资源之前，产品经理和设计师必须确信他们已经想出了会成功的东西。仅仅相信产品定义是好的是不够的，你必须测试以确保。就像你不会允许工程师仅仅因为他或她认为代码好就发布代码一样，你必须测试以确保。

> This is why once you've come up with this minimal product—and have tested it with target

这就是为什么一旦你想出了这个最小产品——并已经用目标用户测试到你有证据表明它会奏效的程度——你就不能随后只是削减一些更多的

> users to the point that you have evidence it will work—you can't later just cut out some more features and assume it will still work with users. If you could, then you didn't really identify the minimal product earlier.

功能并假设它对用户仍然有效。如果可以的话，那么你之前并没有真正识别出最小产品。

> You will still have some cases where you have the same tough decision—a common situation is when one or more features takes engineering longer to build than they anticipated—but in this model, the normal response is a schedule slip rather than a feature cut. Remember, you've already done the cutting. The good news is that the estimates in this process are better than normal because—with a high-fidelity prototype on which to base an estimate, rather than a paper document—engineering had more time to evaluate the functionality, they feel more ownership in their estimates, and there is also less product to build. So, when slips do occur, they're not as severe or frequent as we are used to.

你仍然会有一些情况下你有同样艰难的决定——一个常见的情况是当一个或多个功能需要工程团队比预期更长的时间来构建时——但在这种模式下，正常的反应是进度推迟而不是功能削减。记住，你已经做了削减。好消息是，这个过程中的估算比正常情况更好，因为——有了高保真原型作为估算的基础，而不是纸质文档——工程有更多时间来评估功能，他们对自己的估算更有主人翁意识，而且需要构建的产品也更少。所以，当确实发生推迟时，它们不像我们习惯的那么严重或频繁。

> Similarly, for essentially the same reasons, once the engineering is underway, the product manager can't just keep tossing in additional requirements. By far the most common reason product managers request changes to the spec is a consequence of not really thinking through the requirements in the first place. The high-fidelity prototype will force most of these issues to the surface much earlier in the process.

同样，出于基本相同的原因，一旦工程 underway，产品经理不能只是不断抛出额外的需求。到目前为止，产品经理要求更改规格的最常见原因是根本没有真正深思熟虑需求。高保真原型将迫使大多数这些问题在流程中更早地浮出水面。

> Some people think that Agile methods such as Scrum address these issues, but in a different way. While I would love it if most teams switched to Agile methods tomorrow, as they really can make a positive difference, you'll find they don't really address these issues, and they create a couple of their own as well. More about that in the chapter Succeeding with Agile Methods.

有些人认为 Scrum 等敏捷方法以不同的方式解决这些问题。虽然我希望大多数团队明天就转向敏捷方法，因为它们确实可以产生积极的影响，但你会发现它们并没有真正解决这些问题，而且它们还创造了几个自己的问题。更多内容在"成功运用敏捷方法"一章中。

> So by all means prioritize as you're thinking about the requirements and what's most important, but by the time you come up with your final spec, make sure your product is already the minimal possible. Then yank all those P1/P2/P3 annotations from the spec, and make it clear to the team that it describes an entire product. And if you remove a leg, then as an old boss of mine would say, that dog won't hunt.

所以一定要在思考需求和什么最重要时确定优先级，但到你想出最终规格时，确保你的产品已经是最小化的了。然后从规格中删除所有这些 P1/P2/P3 注释，并向团队明确说明它描述的是一个完整的产品。如果你砍掉一条腿，那么用我的一位老老板的话说，那只狗就不会打猎了。

第 21 章：产品验证

> Chapter 21: Product Validation
> Evidence of Valuable, Usable And Feasible

有价值、可用和可行的证据

> The past few chapters have had references to what I call product validation. This refers to verifying that the product spec is describing a product that you have evidence will be successful, but doing so without actually building out and deploying the product.

前几章提到了我所说的产品验证。这是指验证产品规格是否描述了一个你有证据表明会成功的产品，但在实际构建和部署产品之前这样做。

> This used to be a very expensive and difficult thing to do, and was generally only done for products that were very expensive to tool and manufacture, such as automobiles. However, for just about every type of product today, the costs to produce effective prototypes or simulations has come down so far that I am amazed I continue to encounter product teams that don't do this.

过去这是一件非常昂贵和困难的事情，通常只对工具化和制造成本非常高的产品进行，例如汽车。然而，对于今天几乎每种类型的产品，制作有效原型或模拟的成本已经下降到我惊讶地继续遇到不这样做的产品团队。

> One of the biggest and most common mistakes product teams make is to have far more confidence in their product specifications than they should. They move forward, thinking they'll adjust the product—if necessary—once they get beta feedback. But of course beta is far past the time for major changes, and it is little wonder so many initial product releases are so far off the mark.

产品团队犯的最大和最常见的错误之一是对其产品规格的信心远远超过他们应有的程度。他们继续前进，认为如果需要的话，他们会在获得测试版反馈后调整产品。但当然，测试版早已过了进行重大变更的时间，难怪这么多初始产品发布与目标相去甚远。

> As product manager, it is your responsibility to ensure that this doesn't happen to your

作为产品经理，你有责任确保这种情况不会发生在你的

> product. The key is to prove to yourself and to the rest of the product team that the spec you give them describes a winning product. You can do this, and it costs far less than you probably think.

产品上。关键是向你自己和产品团队的其他成员证明，你提供给他们的规格描述了一个成功的产品。你可以做到这一点，而且成本远低于你可能想象的。

> There are three important types of validation that you need to perform before you hand over a final product specification to the engineering team:

在向工程团队提交最终产品规格之前，你需要进行三种重要的验证：

> Feasibility Testing

可行性测试

> One immediate question is whether or not the product is going to be buildable, with the technology time and funds currently available. Your engineers and architects should be very involved in investigating technologies and exploring possible approaches. Some paths will be dead ends, but hopefully others will prove viable.

一个直接的问题是产品是否可以用目前可用的技术、时间和资金来构建。你的工程师和架构师应该非常深入地参与调查技术和探索可能的方法。有些路径会是死胡同，但希望其他路径会被证明是可行的。

> What is most important is that, if there are obstacles the engineering team considers insurmountable in this product's timeframe, it is important to know this now rather than much later in the process—after the time and money has been lost.

最重要的是，如果工程团队认为在这个产品的时间范围内有不可逾越的障碍，重要的是现在就知道，而不是在流程中更晚的时候——在时间和金钱已经损失之后。

> Some products have more technical risks than others, but if yours has significant risks regarding feasibility, make sure you address them early.

有些产品的技术风险比其他产品多，但如果你的产品存在重大的可行性风险，确保你尽早解决它们。

> Usability Testing

可用性测试

> Your interaction designers will be working very closely with you to come up with ways of

你的交互设计师将与你密切合作，想出在产品中呈现功能的方法，以便用户能够弄清楚如何使用它。

> presenting the functionality in the product so that users can figure out how to use it. Usability testing will often uncover missing product requirements and, also—if done well—identify product requirements that are not as necessary as originally thought. You should plan on multiple iterations before you come up with a successful user experience. The purpose of the prototype is to have something to test on real people, and usability testing is the art and science of getting useful feedback on your product from your target customers. Certainly, the product manager and designers will use the prototype and learn a great deal from it, but there is no substitute for putting the prototype in front of real people from the target customer base.

可用性测试经常会发现缺失的产品需求，而且——如果做得好——还能识别出不像最初认为的那样必要的产品需求。在你想出一个成功的用户体验之前，你应该计划多次迭代。

原型的目的是有东西可以在真实人身上测试，而可用性测试是从目标客户那里获得对你的产品有用反馈的艺术和科学。当然，产品经理和设计师会使用原型并从中学习到很多东西，但没有什么能替代把原型放在目标客户中的真实人面前。

> Note that for usability testing purposes, it is perfectly fine if complicated back-end processing is simulated—the key is to evaluate the user experience.

请注意，对于可用性测试目的，如果复杂的后端处理是模拟的，这完全没问题——关键是评估用户体验。

> Value Testing

价值测试

> Finally, it is not enough to know that your product is feasible to build and will be usable. What really matters is whether or not your product is something users will find valuable and want to buy—that is, how much do users and customers like and value what you're doing? This testing can typically be combined with the usability testing process, and the prototypes used are the same. But in usability testing, you're seeing if users can figure out how to do the necessary tasks, while in value testing you're seeing if they actually care about those tasks

最后，仅仅知道你的产品可行且可用是不够的。真正重要的是用户是否会觉得你的产品有价值并想要购买——即，用户和客户对你正在做的事情有多喜欢和多重视?

这种测试通常可以与可用性测试过程结合起来，使用的原型也是一样的。但在可用性测试中，你看的是用户是否能弄清楚如何做必要的任务，而在价值测试中，你看的是他们是否真正关心这些任务

以及你把它们解决得有多好。

> and how well you've solved them.

对于少数小型产品工作，简单地在纸上理清你的想法可能就足够了。但对于大多数产品——具有复杂的用户交互或技术的新用途——原型对于评估产品是否能满足其目标绝对是至关重要的。

> For a few small product efforts, simply working your ideas out on paper may be sufficient. But for most products—with complex user interactions or new uses of technology—prototypes are absolutely critical in order to assess whether or not the product will meet its objectives.

最常见的情况是，原型只是快速组装的可点击页面，代表最终的网站或软件服务。但对于其他类型的产品，原型可能是物理设备或设备和软件的组合。关键是它需要足够真实，以便你可以在实际目标客户身上测试原型，他们可以给你有用的反馈。

> Most often the prototype is simply quickly assembled and clickable pages representing the eventual web site or software service. But for other types of products the prototype may be a physical device or a combination of device and software. The key is that it needs to be realistic enough that you can test the prototype on actual target customers and they can give you useful feedback.

直到最近，关于"高保真"原型（我所描述的）与"低保真"原型（本质上是纸质绘图）的相对优点的争论一直存在。今天，我认为这种争论毫无意义，因为高保真原型的成本已经降得如此之低，而且反馈的质量要高得多。

> Until recently, there was debate over the relative merits of “highfidelity” prototypes (what I'm describing), versus “low-fidelity” prototypes (essentially paper drawings). Today, I consider this debate meaningless because the cost of high-fidelity prototypes has dropped so low, and the quality of the feedback is so much higher.

过去，这些原型方法有两个主要障碍。缺乏良好的原型工具意味着构建原型可能需要很长时间。另一个问题是无知的管理层不理解原型和真实产品之间的区别。在这里，团队会面临压力，要求使用原型作为最终产品的基础，这在实施质量上产生了可预见的结果。

> In the past, there were two major obstacles to these prototyping approaches. The lack of good prototyping tools meant that it could take a long time to construct the prototype. Another problem was in unenlightened management not understanding the difference between a prototype and the real product. Here, teams would be pressured to use the prototype as the basis for the final product, with predictable results in the quality of the implementation.

今天，有出色的原型工具可以让工程师或设计师快速创建功能非常强大的原型（通常在几小时或几天内），可以有效地模拟未来的产品——达到必要的程度——并形成真实用户测试的基础。

> Today, there are outstanding prototyping tools that can let engineers or designers rapidly create very functional prototypes (often in hours or days) that can effectively emulate the future product—to the degree necessary—and form the basis of realistic user testing. Moreover, most managers today understand that building a simulation and building the actual product are very different animals—akin to building a scale model of a house versus building the actual home.

此外，今天大多数经理都明白，构建模拟和构建实际产品是非常不同的东西——类似于建造房屋的比例模型与建造实际住宅。

> These are not the only ways to validate your product—especially for Internet services, there are other techniques that are also easy and effective. But I can't emphasize enough how important and valuable it is to validate your ideas before you actually go and build the product. There are always surprises, and it is far better to discover them early rather than when the product is in beta or released. Further, once the real engineering begins, a special type of inertia sets in—it becomes very difficult to make significant changes and the costs of these changes rise dramatically.

这些不是验证你的产品的唯一方法——特别是对于互联网服务，还有其他同样简单有效的方法。但我再怎么强调在真正去构建产品之前验证你的想法有多么重要和有价值都不为过。总会有惊喜，早早发现它们远比在产品处于测试版或发布时发现要好得多。此外，一旦真正的工程开始，一种特殊的惯性就会开始出现——进行重大变更变得非常困难，而且这些变更的成本会急剧上升。

> In the chapter Prototype Testing, I explain in detail the techniques for the usability and value testing.

在"原型测试"一章中，我详细解释了可用性和价值测试的技术。

> Orxamples

° 示例

> You can see example high-fidelity prototypes, and a list of tools for creating them, at www-svpg.com/examples.

你可以在 www.svpg.com/examples 上看到示例高保真原型，以及创建它们的工具列表。

第 22 章：原型测试

> Chapter 22: Prototype Testing
>
> Putting Your Ideas In Front Of Real Users

把你的想法放在真实用户面前

> By this point you should know that I view the high-fidelity prototype as the primary means of describing the product to be built—a prototype is significantly more useful to the product team than the typical paper-based specification. However, that's really the secondary benefit. The primary reason to create a high-fidelity prototype is to help you gain a much deeper understanding of your product, and—ultimately—so that you can test your ideas with real users before you have your engineering teams take months to go build something that you have no real evidence will serve its purpose.

至此，你应该知道我将高保真原型视为描述要构建的产品的主要手段——原型对产品团队来说比典型的基于纸张的规格要有用得多。然而，这实际上是次要的好处。创建高保真原型的主要原因是帮助你更深入地了解你的产品，并且——最终——以便你可以在让工程团队花费数月时间构建某些东西之前，用真实用户测试你的想法，而你并没有真正的证据表明它会达到其目的。

> In this chapter I describe how to do this prototype testing. I'll warn you up front that this chapter is relatively long, but I will also say that testing your ideas with real users is probably the single most important activity in your job as product manager.

在本章中，我描述了如何进行这种原型测试。我会提前警告你，本章相对较长，但我也想说，用真实用户测试你的想法可能是你作为产品经理工作中最重要的单一活动。

> If your company is large enough to have its own usability testing team, by all means secure as much of their time for your project as you can. Even if you can't get much of their time, these people are terrific resources and, if you can make a friend in user research or usability testing, it'll be a huge help to you.

如果你的公司足够大，有自己的可用性测试团队，一定要尽可能为项目争取他们的时间。即使你不能获得他们太多时间，这些人也是非常棒的资源，如果你能在用户研究或可用性测试中交个朋友，对你会有巨大帮助。

> If your organization has funds earmarked for outside services, you may be able to use one of many excellent firms to conduct testing for you. But at US$10,000-$20,000 per round of testing (typically around 10 users) that most of these firms charge, chances are that you won't be able to afford as much testing as your product will need.

如果你的组织有专门用于外部服务的资金，你可以使用许多优秀的公司之一来为你进行测试。但按照这些公司大多数收取的每轮测试 10,000-20,000 美元（通常约 10 个用户）计算，你可能无法负担你的产品所需的尽可能多的测试。

> If you're like most companies, you have few resources available, and even less money. But you can't let that stop you. It is absolutely essential that you test your ideas out with real users. As I said above, it is arguably the single most important part of your job.

如果你像大多数公司一样，你很少有资源可用，资金更少。但你不能让这阻止你。用真实用户测试你的想法是绝对必要的。正如我上面所说，这可能是你工作中最重要的部分。

> So I'll show you how to do this testing yourself. Don't get me wrong, you won't be as proficient as a trained usability engineer, and it'll take you a few sessions to get the hang of it. But in most cases you'l find that you can still identify the serious issues with your product, which is what's important.

所以我会向你展示如何自己进行这种测试。不要误会我的意思，你不会像受过培训的可用性工程师那样熟练，而且需要几次会议才能掌握窍门。但在大多数情况下，你会发现你仍然可以识别出产品的严重问题，这才是重要的。

> One thing to note is that, while usability testing (seeing if people can figure out how to actually use your product) is critical, you also need to test the value or usefulness of your product (do people actually want to use it?), and we'll discuss both forms of testing here.

需要注意的一件事是，虽然可用性测试（看人们是否能弄清楚如何实际使用你的产品）至关重要，但你也需要测试产品的价值或有用性（人们是否真的想使用它？），我们将在这里讨论这两种形式的测试。

> Finding Test Subjects

寻找测试对象

> Before you do the prototype testing, you'll need to round up some test subjects. If you're using a lab they'll recruit and schedule the users for you, which is a big help, but if you're on your own, you've got several options:

在进行原型测试之前，你需要召集一些测试对象。如果你使用实验室，他们会为你招募和安排用户，这是一个很大的帮助，但如果你自己进行，你有几种选择：

> «If you've established a charter customer program as I described in Charter User Programs, you should have quite a few users readily available. If you haven't yet established your program, then you should.

*如果你已经按照我在"特许用户计划"中描述的那样建立了特许客户计划，你应该有相当多的用户随时可用。如果你还没有建立你的计划，那么你应该建立。

> «If you're doing a product for business, then trade shows are a great source of target customers.

«如果你正在为商业做产品，那么贸易展是目标客户的绝佳来源。

> «It's increasingly common to advertise for test subjects on Craigslist. If you do this, try to keep your participant description a notch more general than specific, and then when you call interested test subjects to explore their participation you can screen for the right match.

« 在 Craigslist 上为测试对象做广告越来越普遍。如果你这样做，尽量让你的参与者描述比一般更笼统一些，然后当你打电话给有兴趣的测试对象以探讨他们的参与时，你可以筛选出合适的匹配。

> · For consumer products you can use your “friends and family” network, but try to avoid people too close to you, and those in the tech industry, unless that's specifically your target. Be sure to use subjects from outside this network, too.

¢ 对于消费产品，你可以使用你的"朋友和家人"网络，但尽量避开离你太近的人，以及科技行业的人，除非那确实是你的目标。也一定要使用来自这个网络之外的测试对象。

> · If you have a list of user e-mail addresses, you can do a selection from there. Often your marketing team can help you narrow down the list.

« 如果你有用户电子邮件地址列表，你可以从那里进行选择。通常你的营销团队可以帮助你缩小列表范围。

> · You can solicit volunteers on your Web site—lots of major sites do this now. Remember: you'll still call and screen the people to make sure you don't get a sample skewed with early adopter types.

« 你可以在你的网站上征集志愿者——现在很多主要网站都这样做。记住：你仍然会打电话并筛选人员，以确保你不会得到一个被早期采用者类型扭曲的样本。

> *One technique I especially like for medium to larger companies is to set up regular prototype test sessions—say every other Friday—where you arrange for 10-20 or so users to come into your offices for a couple hours each. Your product managers sign up for the time slots, so a given user might test a couple prototypes each. I like this because one person can do the logistics of invites and screening, and product teams can count on a ready set of test users on a steady basis.

· 我特别喜欢中大型公司的一种技术是建立定期的原型测试会议——比如每隔一个星期五——你安排 10-20 个左右的用户到你的办公室，每个用户几个小时。你的产品经理注册时间段，所以一个给定的用户可能会测试几个原型。我喜欢这个，因为一个人可以做邀请和筛选的物流工作，产品团队可以在稳定的基础上依赖一套现成的测试用户。

> · You can always take your show on the road and go to where your users congregate. If

« 你总是可以带着你的表演上路，去你的用户聚集的地方。如果你正在做一个电子商务产品，你可能想去购物中心。如果你正在做一个体育产品，去体育酒吧。如果你的产品解决了一个真正的需求，让

人们给你一个小时的时间不会有困难。只要带一些感谢礼物，尽量不要看起来像是试图改变他们的宗教信仰。

> you're doing an e-commerce product, you may want to go to a shopping mall. If you're doing a sports product, go to a sports bar. If your product addresses a real need, you won't have trouble getting people to give you an hour of their time. Just bring some thank you gifts, and try not to look like you're trying to convert their religion.

«如果你要求用户到你的地点——特别是对于商业用途——你可能需要补偿他们的时间。如果你正在做一个消费者服务产品，一个真诚的感谢以及一顶印有公司标志的帽子通常就足够了，因为人们真诚地希望帮助创造产品——特别是对于他们喜欢的公司。然而，如果你确实补偿测试对象，考虑提供相当于你网站上 50 美元信用的东西。

> «If you're asking users to come to your location—especially for business use—you will likely need to compensate the people for their time. If you're doing a consumer service product, a big sincere thank-you along with a hat with your company logo on it will often suffice, as people genuinely want to help in the creation of products—especially for companies they like. However, if you do compensate test subjects, consider providing something along the lines of US$50 of credit on your site.

¢ 要知道，当你安排人们来测试时，缺席率非常高——这只是生活的事实。虽然这个数字可能高达 30%，但你可以通过在前一天给你的测试对象打个人电话将其降到 5-10% 的范围。即使留下语音邮件也会有帮助，但要注意，发送电子邮件的效果并不一样好。

> · Realize that there's a very high no-show rate when you schedule people to come in for testing—it's just a fact of life. While this number can rise to as high as 30%, you can drop it to the 5-10% range by giving your subjects a personal phone call the day before. Even leaving a voicemail message will help, but note that sending an email message does not work equally well.
>
> Preparing the Test You'll need to define the usability tasks you'll want to test, and the interview questions concerning value:
>
> · Define in advance the set of tasks you want to test. Usually these tasks are fairly obvious. If you're building an e-mail client, for example, your users will need to do things such as compose a message, read new mail, and file away messages. There will also be more obscure tasks, but concentrate on the primary tasks—the ones that users will do most of
>
> the time. If you have time, you can get to less common tasks but it's essential the key tasks are tested well.
>
> · You have a one-time-only opportunity with each user you test—the opportunity to learn how they think about this problem today, without your product. If you're testing a new online restaurant rating service, rather than start them out at your prototype's home page, you might instead want to start them out with an empty browser and see what they do. What review sites do they use today? Do they use Google or Yahoo's search to find the

> specific restaurant, or do they go somewhere like OpenTable or Zagat? Do they search by neighborhood, by cuisine type, or price range? This type of incredibly valuable information is missed if you jump right into your prototype, which will necessarily have quite a few assumptions built in. Once your test subjects have the opportunity to play with your prototype for a while, they can tell you what they like better, but they will no longer be thinking about the problem the way a first-time visitor would.
>
> · You'll then want to get them to your prototype, but there's one more thing before you jump into your tasks. See if they can tell from the home page or landing page of your prototype what it is that you actually do, and especially what might be valuable or appealing to them. Again, once they jump into tasks, they'll lose that first-time visitor context, so don't waste the opportunity. You'll find that landing pages are incredibly important to bridging the gap between expectations and what the site actually does.
>
> · After you've seen if your user can figure out how to do the tasks you're testing, now is the right time to have a conversation with him or her. Think of it as a one-person focus group. Does the person use a different product or site for the same purpose today, or is this something they do manually or offline? How much better is this than what they use today? And don't forget to ask my favorite question, Net Promoter Score (NPS): How
>
> likely would you be to recommend this product to your friends? Now that the user has interacted with your prototype, they understand the topic and you can have an extremely useful dialogue with them about this problem. Most importantly, you're trying to gauge how much this person values the product.
>
> «It's useful if you structure your questions on a scale, such as 0-10, or with numeric answers in general. This is so that you can track the averages as they improve. One technique I like for gauging value is to ask how much the user would be willing to pay for it, even if you have no intention of actually charging for use this way. It's a way to assess value and—especially—to track how the average value goes up or down over time as you change the prototype.
>
> · Note that you don't have to wait until you have a complete prototype in order to begin testing. You can start with the main tasks, and it's okay if you have dead ends in the rest of the prototype. If the user wanders over to one of those dead ends, just ask “And what would you expect to happen if you did that?” This is a great question whether you have that path laid out or not. If you do have it laid out, you can see if they match. And if you don't, you'll get important info about what you'll need to do.
>
> The Test Environment

准备测试

> Here's how to prepare your test environment:

你需要定义你想要测试的可用性任务，以及关于价值的面试问题： ¢ 提前定义你想要测试的任务集。通常这些任务相当明显。例如，如果你正在构建一个电子邮件客户端，你的用户需要做一些事情，如

撰写消息、阅读新邮件和归档邮件。也会有更晦涩的任务，但专注于主要任务——用户大部分时间会做的任务。如果你有时间，你可以处理不太常见的任务，但关键任务被充分测试是至关重要的。

> · Formal testing labs will typically have setups with two-way mirrors or closed-circuit video monitors, as well as cameras that capture both the screen and a frontal view of the user. Just know that while that's great if you have it, you do not need these things to have an extremely useful and valuable test. I can't count how many prototypes I've tested at a tiny

· 你对每个测试的每个用户都有一次性的机会——了解他们今天如何思考这个问题的机会，没有你的产品。如果你正在测试一个新的在线餐厅评级服务，而不是让他们从你的原型主页开始，你可能想让他们从一个空浏览器开始，看看他们做什么。他们今天使用什么评论网站？他们使用谷歌或雅虎的搜索来找到特定的餐厅，还是去像 OpenTable 或 Zagat 这样的地方？他们是按社区、菜系类型还是价格范围搜索？如果你直接进入你的原型，就会错过这种非常有价值的信息，而你的原型必然会有相当多的假设。一旦你的测试对象有机会玩你的原型一段时间，他们可以告诉你他们更喜欢什么，但他们不再会像首次访问者那样思考这个问题。

> table at Starbucks—just big enough for a laptop—with three chairs around the table. In fact, in some ways this is preferable to the testing lab because the user feels a lot less like a lab rat and may be more candid and open in his or her responses.

· 然后你会想让他们看你的原型，但在你跳入任务之前还有一件事。看看他们是否能从你的原型的主页或登陆页看出你实际上做什么，特别是什么可能对他们有价值或有吸引力。同样，一旦他们跳入任务，他们就会失去首次访问者的背景，所以不要浪费这个机会。你会发现登陆页对于弥合期望与网站实际做什么之间的差距非常重要。

> · The other environment that works quite well is your customer's office. It may be time consuming to go there and get set up, but even 30 minutes in their office will often tell you a lot. And because they are “master of their domain,” they're frequently very open and talkative. Also, all the cues are there in the office to remind them of how they might actually use the product in their daily routine. You can also learn a lot from observing what the office looks like. How big is their monitor? How fast is their computer and network connectivity? How do they communicate with their colleagues on their work tasks?

« 在你看到用户是否能弄清楚如何做你正在测试的任务之后，现在是与他或她进行对话的合适时机。把它想象成一个单人的焦点小组。这个人今天是否使用不同的产品或网站来达到同样的目的，或者这是他们手动或离线做的事情？这比他们今天使用的要好多少？不要忘记问我最喜欢的问题，净推荐值（NPS）：你

> · There are tools for doing this type of testing remotely, but while you can see where their mouse is, and what the user is clicking on, it's not the same as looking at the person's eyes

> and body language. So, again, while more testing is generally better, this is not a substitute for face-to-face testing.

有多可能向你的朋友推荐这个产品？现在用户已经与你的原型互动过，他们理解这个话题，你可以与他们就这个问题进行非常有用的对话。最重要的是，你试图衡量这个人对产品的重视程度。

> - As product manager, you need to make sure you are at every single test—do not delegate this task. Real value comes from experiencing as many users as possible—first hand—interacting with and responding to your ideas. Even if you use an outside firm to arrange and administer the tests, you need to be there with them during the testing. No one knows your product as well as you do, and you will have insights from watching the slightest hesitation or confused look, or the nuance of a question that reveals that your test subjects don't understand the conceptual model or particular feature. What gets summarized for you by a proctor will probably miss several key insights.

«如果你以量表的形式构建你的问题，比如 0-10，或一般使用数字答案，这是很有用的。这样你就可以跟踪平均值的变化。我喜欢的一种衡量价值的技术是问用户愿意为此支付多少钱，即使你并不打算实际以这种方式收费。这是一种评估价值的方法——特别是——跟踪平均值如何随着你改变原型而上升或下降。

> - Some people believe that the product manager (and the interaction designer) are too close

¢　注意，你不必等到有一个完整的原型才能开始测试。你可以从主要任务开始，原型的其余部分有死胡同也没关系。如果用户走到其中一个死胡同，只要问"如果你那样做，你期望会发生什么？"这是一个很好的问题，无论你是否有那个路径。如果你有，你可以看看他们是否匹配。如果你没有，你会得到关于你需要做什么的重要信息。

> to the product to do this type of testing objectively—that they may either get their feelings hurt or only hear what they want to hear. My view is that good product managers and interaction designers get past this very quickly. They know they will get the product wrong initially—that almost nobody gets it right the first time—and they know that learning from these tests is the fastest path to an inspiring product. So to me the benefits far outweigh the risks.
>
> - Ideally, you should have one person administer the tests and another person taking notes. It's very useful to have someone to debrief with afterwards to make sure you both saw the same things and came to the same conclusions. That said, if it's just you and your laptop—and you've got a ready and willing target user in front of you—do it. It's all good.
> - Ifyou as product manager have a user researcher or usability engineer along with you, let him or her administer the test while you take notes. Otherwise, you'll probably be the one that administers. It's great to invite others on the product team to be your note taker. Most often this person will probably be the interaction designer, but the visual designer, managers, and especially engineers are all useful and they'll get a lot out of the experience.

> Testing Your Prototype

测试环境

> Now that you've got your prototype ready, you've lined up your test subjects, and you've prepared the tasks and questions, here are a set of tips and techniques for administering the actual testing:

以下是如何准备你的测试环境：

> · Greet the person warmly and offer a coffee or bottle of water, but the sooner you get to the prototype the better. Tell your subject you'll chat with them after they test the

« 正式的测试实验室通常会有带双向镜或闭路电视监视器的设置，以及捕捉屏幕和用户正面视图的摄像头。只要知道，虽然如果你有这些，那很好，但你不需要这些东西就能有一个非常有用和有价值的测试。我数不清我在星巴克的一张小桌子上——只够放一台笔记本电脑——周围有三把椅子测试了多少原型。事实上，在某些方面这比测试实验室更可取，因为用户感觉不那么像一只实验室老鼠，可能会更坦诚和开放地回应。

> prototype, but you want to get their untainted impressions first. The more you chat beforehand, the more clues you are giving away and the less of a true first impression your test subject can provide. If more than five minutes goes by without the user starting in on the prototype you are talking too much.

¢ 另一个相当有效的环境是你客户的办公室。去那里并设置可能很耗时，但即使在他们办公室里待 30 分钟通常也会告诉你很多东西。而且因为他们在自己的"地盘上"，他们通常非常开放和健谈。此外，办公室里所有的线索都在那里提醒他们如何在日常生活中实际使用该产品。你也可以通过观察办公室的样子学到很多东西。他们的显示器有多大？他们的电脑和网络连接有多快？他们如何与同事就工作任务进行沟通？

> · After your greeting, be sure to tell him or her that (1) this is just a prototype—it's a very early product idea—and it's not real, (2) they won't be hurting your feelings by giving you their honest opinion—good or bad, and (3) you're testing the prototype—you're not testing him or her. Your test subject can't pass or fail—only the prototype can pass or fail.

¢ 有工具可以远程进行这种测试，虽然你可以看到他们的鼠标在哪里，以及用户在点击什么，但这与看人的眼睛和肢体语言不一样。所以，再次强调，虽然更多的测试通常更好，但这不能替代面对面的测试。

> · When testing, you'll want to do everything you can to keep your users in “use mode” and out of “critique mode.” What matters is whether users can easily do the tasks they need to do, and whether they value the product. It really doesn't matter if the user thinks something on the page is ugly or should be moved or changed. Sometimes misguided testers will ask users questions like “What three things on the page would you change?” To me, unless that user happens to be an interaction designer, I'm not really interested in

> the answer to that question, or others like it. If users knew what they really wanted, software would be a lot easier to create. So watch what they do more than what they say.

« 作为产品经理，你需要确保你参加每一次测试——不要把这个任务委托出去。真正的价值来自于尽可能多地亲身体验用户——第一手——与你的想法互动和回应。即使你使用外部公司来安排和管理测试，你也需要在测试期间与他们在一起。没有人像你一样了解你的产品，你会从观看最轻微的犹豫或困惑的表情，或问题的细微差别中获得洞察力，这些细节揭示了你的测试对象不理解概念模型或特定功能。主持人为你总结的东西可能会遗漏几个关键见解。

> - During the testing, the main skill you have to learn is to keep quiet. Normally, when we see someone struggle, most of us have an urge to help the person out. You need to suppress that urge. You have to turn into a horrible conversationalist and get comfortable with silence.

« 有些人认为产品经理（和交互设计师）离产品太近，无法进行这种客观的测试——他们可能会受伤，或者只听到他们想听的。我的观点是，好的产品经理和交互设计师很快就能克服这一点。他们知道他们最初会把产品搞错——几乎没有人第一次就把它做对——他们知道从这些测试中学习是通往灵感产品的最快途径。所以对我来说，好处远远大于风险。

> - There are three important cases you're looking for: (1) the user got through the task with no problem at all and no help, (2) the user struggled and moaned a bit but eventually got through it, or (3) the user got so frustrated he gave up. Sometimes people will give up pretty quick, so you may need to encourage them to keep trying a bit longer. But if the

«理想情况下，你应该有一个人主持测试，另一个人做笔记。事后有一个人可以汇报，确保你们都看到了同样的事情并得出了相同的结论，这是非常有用的。也就是说，如果只是你和你的笔记本电脑——而且你有一个准备就绪、愿意参与的目标用户在你面前——那就去做吧。一切都很好。

> user gets to the point that you believe he would truly leave the site and go to a competitor, then that's when you note he truly gave up.

« 如果你作为产品经理有用户研究员或可用性工程师与你在一起，让他或她主持测试，而你做笔记。否则，你可能就是那个主持的人。邀请产品团队的其他成员做你的记录员是很棒的。最常见的人可能是交互设计师，但视觉设计师、经理，特别是工程师都很有用，他们会从这次经历中学到很多东西。

> - In general, you'll want to avoid giving any help or “leading the witness” in any way. If you see the user scrolling the page up and down and clearly looking for something, it's okay to ask the user what specifically he's looking for, as that information is very valuable to you. Some people ask users to keep a running narration of what they're thinking, but I find this tends to put people in critique mode, as it's not a natural behavior.
> - Act like a parrot. This helps in many ways. First, it helps avoid leading. If your test subject is quiet and you can't stand it any longer, tell them what they're doing: “I see that you're looking at the list on the right.” This will prompt them to tell you what they're trying to

do, looking for, whatever. If your subject asks a question, rather than giving a leading answer you can play back the question to them, “Will clicking on this make a new entry?” The subject will usually take it from there because they'll want to answer your question: “Yeah, I think it will.” Parroting also helps avoid leading value judgments. If you have the urge to say “Great!,” instead say “You created a new entry.” Finally, parroting key points also helps your note taker because he or she will have more time to write down important points.

- Fundamentally, you're trying to get an understanding of how your target users think about this problem, and to identify places in your prototype where the model the software presents is inconsistent or incompatible with how the user is thinking about the problem. That's what it means to be counterintuitive. Fortunately, when you spot this it is usually not hard to fix, and can be a big win for your product.
- You will find that you can tell a great deal from body language and tone. It is painfully obvious when test subjects don't like your ideas, and it is also clear when they genuinely

do. If they like what they see, they'll almost always ask for an email telling them when the product is out. And if they really like it, they'll try to get access from you before it's released to the general public. When I attend the prototype testing with my clients in Germany, even though I don't speak German, it is obvious what the issues are—which ideas work well and which ones don't. Updating the Prototype The point of this prototype testing is to identify what you need to fix in the prototype to make it more valuable and usable. So, as quickly as possible, you'll want to correct the problems.

- Some people believe you have to freeze the prototype, the tasks, and the questions for a complete round of testing (generally 6-8 users) before drawing any conclusions. I don't think that's true. I have found that you can significantly accelerate the process of getting to a good product by responding more quickly to feedback.You don't have to be hit on the head by eight users in a row to know you need to fix something big. So go ahead and fix it when you feel you've identified a problem, even if it's after only two or three users. The harder question is deciding when you're done. Generally, if you can get through six consecutive users who understand and appreciate the value of the product—and can get through the key tasks—you're in good shape and you've done your job.
- You might determine that you just aren't able to get people interested in this problem, or figure out a way to make the product clear or usable enough that your target users realize its value. In this case, you may decide to stop right there and put the idea on the shelf. Some product managers consider this a big failure. I view it as saving the company the

wasted cost of building and shipping a losing product, not to mention the opportunity cost of what your engineering team could be building instead. This whole process might sound complicated or difficult, but the remarkable thing is just how easy and effective it actually is. The best way to prove this to yourself is to take your laptop with your product or prototype on it to someone that hasn't seen it yet and just give it atry. I want to give credit here to two important sources (and great resources for you). The first is the book, Don't Make Me Think by Steve Krug. It's mostly a book on interaction design, but in the back third he makes a compelling case for this sort of informal testing, and he provides many useful tips. I have long recommended this book to both product

> managers and designers, and I hope you'll buy a copy and read it carefully. Second, my favorite product testing firm is Creative Good (www.creativegood.com). These guys specialize in a form of testing they call Listening Labs, which is a powerful form of undirected testing that can find huge issues with your product—functionality and design—resulting in dramatic improvements to the business results. While most testing firms test the products on a task basis, these guys are good at looking at the big picture, which is where many of the biggest improvements are to be found. Several of the techniques described above are adapted from their testing methodology.

测试你的原型

现在你的原型已经准备好了，你已经安排好了测试对象，你已经准备好了任务和问题，以下是一套管理实际测试的技巧和技术：　«　热情地问候对方，提供一杯咖啡或一瓶水，但你越早开始原型测试越好。告诉你的测试对象你会在他们测试原型后与他们聊天，但你想先获得他们未被污染的印象。你事先聊得越多，你透露的线索就越多，你的测试对象能提供的真实第一印象就越少。如果用户开始原型测试前超过五分钟，你说得太多了。

«　问候之后，一定要告诉他或她（1）这只是一个原型——这是一个非常早期的产品想法——它不是真的，（2）他们不会因为给你诚实的意见——好或坏——而伤害你的感情，（3）你在测试原型——你不是在测试他或她。你的测试对象不能通过或失败——只有原型才能通过或失败。

«　测试时，你会想尽一切办法让你的用户保持在"使用模式"，而不是"评论模式"。重要的是用户是否能轻松完成他们需要做的任务，以及他们是否重视该产品。用户是否认为页面上的某些东西很丑或应该被移动或改变并不重要。有时误导性的测试人员会问用户这样的问题"你会改变页面上的哪三件事？"对我来说，除非那个用户恰好是交互设计师，否则我对这个问题的答案或其他类似问题并不真的感兴趣。如果用户知道他们真正想要什么，软件会更容易创建。所以看他们做什么比听他们说什么更重要。

¢　在测试过程中，你必须学习的主要技能是保持安静。通常，当我们看到有人挣扎时，我们大多数人都有帮助这个人的冲动。你需要抑制这种冲动。你必须变成一个糟糕的健谈者，并习惯沉默。

«　有三种重要的情况你在寻找：（1）用户完全没有任何问题地完成了任务，（2）用户挣扎并抱怨了一会儿，但最终完成了，或者（3）用户太沮丧了以至于放弃了。有时人们会很快放弃，所以你可能需要鼓励他们再尝试一会儿。但如果

用户到了你认为他真的会离开网站并去竞争对手那里的地步，那么这就是你记下他真正放弃的时候。

- 一般来说，你会想避免提供任何帮助或"引导证人"。如果你看到用户上下滚动页面，显然在寻找什么，询问用户具体在找什么是可以的，因为这些信息对你非常有价值。有些人要求用户持续叙述他们在想什么，但我发现这往往会让人们进入评论模式，因为这不是一种自然的行为。

«　　表现得像一只鹦鹉。这在很多方面都有帮助。首先，这有助于避免引导。如果你的测试对象很安静，而你再也受不了了，告诉他们他们在做什么："我看到你正在看右边的列表。"这会促使他们告诉你他们想做什么，在找什么，无论什么。如果你的对象问一个问题，与其给出引导性的答案，你可以把问题回问他们，"点击这个会创建一个新条目吗？"测试对象通常会接下去，因为他们会想回答你的问

题："是的，我想会的。"鹦鹉学舌也有助于避免引导性的价值判断。如果你有说"太棒了！"的冲动，相反说"你创建了一个新条目。"最后，重复关键点也有助于你的记录员，因为他或她会有更多时间写下重要的点。

¢ 从根本上说，你试图了解你的目标用户如何思考这个问题，并识别原型中软件呈现的模型与用户思考问题的方式不一致或不兼容的地方。这就是反直觉的意思。幸运的是，当你发现这一点时，通常不难修复，而且可以成为你产品的一大胜利。

¢ 你会发现你可以从肢体语言和语调中看出很多东西。当测试对象不喜欢你的想法时，这是非常明显的，当他们真正喜欢时也是如此。

如果他们喜欢他们看到的，他们几乎总是会要求发一封电子邮件告诉他们产品什么时候发布。如果他们真的喜欢，他们会试图在产品向公众发布之前从你那里获得访问权限。当我在德国与我的客户一起参加原型测试时，虽然我不会说德语，但问题很明显——哪些想法有效，哪些无效。

更新原型

这种原型测试的目的是识别你需要在原型中修复什么，以使其更有价值和可用。所以，尽快纠正这些问题。

· 有些人认为，在得出任何结论之前，你必须在一轮完整的测试（通常是 6-8 个用户）中冻结原型、任务和问题。我不认为这是真的。我发现，通过更快地响应反馈，你可以显著加速获得好产品的过程。你不必被八个用户连续击中头部才知道你需要修复一些大问题。所以，当你觉得已经识别出一个问题时，就去修复它，即使只是在两三个用户之后。更难的问题是决定什么时候完成。一般来说，如果你能连续通过六个理解并欣赏产品价值——并能完成关键任务——的用户，你的状态就很好，你已经完成了你的工作。

« 你可能会确定你无法让人们对这个问题感兴趣，或者无法找到一种方法使产品足够清晰或可用，让你的目标用户意识到它的价值。在这种情况下，你可能会决定立即停止，把这个想法搁置。一些产品经理认为这是一个大失败。我把它视为为公司节省了构建和发布失败产品的浪费成本，更不用说你的工程团队本可以构建的其他东西的机会成本。

这整个过程可能听起来复杂或困难，但 remarkable 的是它实际上是多么简单和有效。向自己证明这一点的最好方法是带着你的笔记本电脑，上面有你的产品或原型，去找一个还没有看过它的人，试试看。

在这里，我想感谢两个重要的来源（以及对你的重要资源）。

第一个是 Steve Krug 的书《别让我思考》。它主要是一本关于交互设计的书，但在后三分之一中，他为这种非正式测试提出了一个令人信服的案例，并提供了许多有用的技巧。我早就向产品经理和设计师推荐了这本书，我希望你会买一本并仔细阅读。

第二，我最喜欢的产品测试公司是 Creative Good（www.creativegood.com）。这些人专门从事他们称之为"倾听实验室"的测试形式，这是一种强大的无指导测试形式，可以发现你产品的巨大问题——功能和设计——从而导致业务结果的显著改善。虽然大多数测试公司按任务基础测试产品，但这些家伙擅长看大局，这是许多最大改进所在的地方。上述描述的一些技术改编自他们的测试方法。

第 23 章：改进现有产品

> Chapter 23: Improving Existing Products
>
> It's Not About Adding Features

不是关于添加功能

> Many product managers get thrown off course when it comes to continuing development on an existing product. Most have a detailed product roadmap with all their great ideas of what they should add, or the roadmap is loaded with requirements that come from specific customers— “If you want me to buy this you'll need to add these six features...”

许多产品经理在继续开发现有产品时偏离了轨道。大多数人有一个详细的产品路线图，里面有他们所有关于应该添加什么的伟大想法，或者路线图上满载着来自特定客户的需求——"如果你想让我买这个，你需要添加这六个功能……"

> Most product organizations are essentially feature factories (with some bug fixing thrown in). For them it is all about adding features.

大多数产品组织本质上是功能工厂（还有一些错误修复）。对他们来说，一切都是关于添加功能。

> But all too often these added features just end up making the situation worse and not better. I try to get product teams to look at product improvement differently.

但太经常了，这些添加的功能最终只会让情况变得更糟而不是更好。我试图让产品团队以不同的方式看待产品改进。

> As with new product development, everything starts with having a very clear understanding of what you're trying to achieve. For example, let's say you're the product manager responsible for new insurance coverage applications at an online insurance site.

与新产品开发一样，一切都始于对你试图实现的目标有非常清晰的理解。例如，假设你是一个在线保险网站负责新保险覆盖申请的产品经理。

> There are all kinds of great metrics for you to track. How many users visit the start of the application process? How many drop off at each page? How many refuse to give personal

有各种各样的好指标供你跟踪。有多少用户访问申请流程的开始？有多少人在每一页放弃？有多少人拒绝提供个人信息？有多少人确认他们的电子邮件地址？有多少人完成流程？

> information? How many confirm their e-mail addresses? How many complete the process? Let's say that today only 7% of the people who begin the registration process actually complete it. If you can drive that number up to 15%, look what you've just done for your product and your company.

假设今天只有 7% 开始注册流程的人实际完成了它。如果你能把这个数字提高到 15%，看看你对你的产品和公司做了什么。

> To accomplish this goal, you might find that you need some new features, and the justification for the new feature is the degree to which it can improve these metrics.

为了实现这个目标，你可能会发现你需要一些新功能，新功能的理由是它能在多大程度上改善这些指标。

> It is likely there are elements of the user experience that can be improved. Or maybe you don't need as much personal information from the user at this stage as you think you do. Or maybe you're not being clear enough as to why you require this information. Or perhaps users are not sure they can trust you with this data.

用户体验中可能有可以改进的元素。或者也许你在这个阶段不需要像你认为的那么多来自用户的个人信息。或者也许你不够清楚为什么你需要这些信息。或者用户不确定他们可以信任你使用这些数据。

> Your job as product manager is to live and breath these metrics. Every day you should ask yourself what you can do to improve them. And you work closely with an interaction designer, user researcher, and lead engineer to consider possibilities.

作为产品经理，你的工作是生活和呼吸这些指标。每天你都应该问自己你能做什么来改善它们。你与交互设计师、用户研究员和首席工程师密切合作，考虑各种可能性。

> Your primary tools for gaining insight into what's going on are to study the site analytics, and to test your product on real users. You'll also gather data from NPS tracking, customer service, the sales force, and from win/loss analysis.

你获得洞察力的主要工具是研究网站分析，并在真实用户上测试你的产品。你还会从 NPS 跟踪、客户服务、销售团队以及赢/输分析中收集数据。

> This type of product improvement is often extremely rewarding, especially for Internet services because we have so much great data—near real-time data—giving us feedback on our ideas. It's amazing how quickly and how dramatically you can help your business as you

这种类型的产品改进通常非常有价值，特别是对于互联网服务，因为我们有这么多好的数据——近实时数据——给我们关于我们想法的反馈。当你根据从数据中学到的东西采取行动时，你能以多快、多大的程度帮助你的业务，这是令人惊叹的。

> act on what you have learned from the data.

记住，这不是关于某个特定客户认为添加什么是重要的，也不是调查或焦点小组的结果。重要的是什么实际上推动了你的指标的指针。

> Remember that it's not about what a particular customer thinks is important to add, or the result of a survey, or a focus group. What matters is what actually moves the needle on the metrics you are driving towards.

所以，当你试图改进现有产品时，不要陷入收集主观反馈、获取客户需求和追逐功能的陷阱。相反，专注于通过研究实时使用并将数字推向正确的方向来不懈地追求你的指标。

> So when you're trying to improve an existing product, don't fall into the trap of gathering subjective feedback, eliciting customer requirements and chasing features. Instead, focus on relentlessly pursuing your metrics by studying live use and working the numbers in the right direction.
> ° Examples

° 示例

> You can see example prototype testing questions at www.svpg.com/examples.

你可以在 www.svpg.com/examples 上看到示例原型测试问题。

第 24 章：温和部署

> Chapter 24: Gentle Deployment
> Help Prevent User Abuse

帮助防止用户滥用

> User abuse is when you unnecessarily and (hopefully) unintentionally mistreat your users by releasing changes to the user community that they don't appreciate. I know it's hard to believe that not every one of your users is waiting excitedly for all of your changes, but it's true. There are several reasons why your users may feel this way: +They may not have received any notice of the changes, so they were caught by surprise—and they weren't in the mood for a surprise. + They may not have time at the moment to learn the changes and you don't have a way for them to continue to use the old version until they do. + The new change may not actually work. + The new change may be incompatible with early versions (such as for accessing data). + The new change may work but it is perceived as gratuitous. + They may already be fatigued from all of the changes you have recently made. + They may have their own layer of process or behavior built around your previous version, and now that is broken and they have to update it.

用户滥用是指你不必要地（希望是）无意地虐待你的用户，向用户社区发布他们不欣赏的变更。我知道很难相信你的每一个用户都在兴奋地等待你的所有变更，但这是真的。你的用户可能有这种感觉有几个原因： «他们可能没有收到任何变更通知，所以他们被抓住了惊喜——而且他们当时没有心情接受惊喜。 * 他们现在可能没有时间学习变更，而你没有办法让他们继续使用旧版本，直到他们有时间学习。 « 新的变更可能实际上不起作用。 * 新的变更可能与早期版本不兼容（例如用于访问数据）。 *

新的变更可能有效，但被认为是不必要的。 * 他们可能已经因为你最近做的所有变更而感到疲劳。 ¢ 他们可能在你之前的版本上建立了自己的流程或行为层，而现在这被打破了，他们必须更新它。

> So what causes user abuse?

那么是什么导致了用户滥用?

> Mainly change. As a general rule, users don't like change. Sure they want the software to be great, and they clamor for new functionality, but most people aren't excited about taking the time to learn a new way to do something they can already do.

主要是变更。一般来说，用户不喜欢变更。当然，他们希望软件很棒，他们 clamor 新功能，但大多数人并不兴奋于花时间学习一种新方式来做他们已经能做的事情。

> Of course, that's a problem, as most of us are in the business of change. We have product teams working relentlessly to add value and deliver new capabilities to our users and customers. Needs change, technologies change, markets change—and our software must change along with them.

当然，这是个问题，因为我们大多数人都在变更的行业中。我们有产品团队不懈努力地为用户和客户提供价值并交付新功能。需求变化，技术变化，市场变化——我们的软件必须随之变化。

> The solution to user abuse isn't to prevent change, it's to be smart about deploying change. One of the fun things about working on a 1.0 product is that you get to start fresh with your community of users. It's true that your user base is still influenced by other products and services that they've been exposed to, but overall you don't have to worry much about things like backwards compatibility or retraining your users. However, for most of us, we're in the business of creating updates or new versions of existing products or services.

解决用户滥用的方法不是阻止变更，而是智能地部署变更。

> In the past, software companies could much more easily get away with sending out largely incompatible and disruptive updates. While users would gripe, they weren't as able to influence other potential users around the world, or take their business elsewhere. Today, with the pervasiveness of the Internet—and the free flow of user product reviews, both good

从事 1.0 产品的一个有趣之处是，你可以从你的用户社区重新开始。确实，你的用户群仍然受到他们接触过的其他产品和服务的影响，但总的来说，你不必担心太多向后兼容性或重新培训你的用户。然而，对于我们大多数人来说，我们在创建现有产品或服务的更新或新版本的业务中。

> and bad—if you turn out a bad release of your product, and you don't correct the problems quickly, you'd better brace yourself for some serious community backlash.

在过去，软件公司可以更容易地发布大致不兼容和破坏性的更新。虽然用户会抱怨，但他们不太能够影响世界各地的其他潜在用户，或者把业务带到其他地方。今天，随着互联网的普及——以及用户产品评论的自由流动，无论是好是坏——如果你发布了一个糟糕的产品版本，而且你没有迅速纠正问题，你最好为一些严重的社区反弹做好准备。

> For large-scale consumer Internet services, this is an even more serious concern. These sites need to consider community impact in everything they say and do, beginning with software updates. I call the process of deploying updates intelligently and carefully to a large community of users “gentle deployment.”

对于大规模消费者互联网服务，这是一个更严重的问题。这些网站需要考虑社区影响在他们所说和所做的每一件事中，从软件更新开始。我将向大量用户社区智能和谨慎地部署更新的过程称为"温和部署"。

> In the spirit of minimizing the disruption caused by change, there are several techniques that can be useful in deploying changes gently.

本着最小化变更造成的破坏的精神，有几种技术可以用来温和地部署变更。

> First, do everything you can to communicate the changes in advance, through vehicles such as newsletters, onsite education, and tutorials. But remember that many people will have neither the time nor the inclination to read what you write, so this technique can only take you so far.

首先，尽你所能提前沟通变更，通过通讯、现场教育和教程等渠道。但记住，许多人既没有时间也没有意愿阅读你写的东西，所以这种技术只能带你走这么远。

> Second, if there is any question about the new version of your product working properly—either due to reliability issues, or scale, or performance—then you need to redouble your QA efforts to ensure that you won't have to rollback your updates, which compounds community angst significantly.

其次，如果你的产品新版本是否正常工作有任何疑问——无论是由于可靠性问题、规模还是性能——那么你需要加倍努力进行 QA，以确保你不必回滚你的更新，这会显著加剧社区焦虑。

> Third, if the change is significant, it may be important to contain the risk by pursuing some form of gentle deployment such as parallel, or incremental deployment.

第三，如果变更是重大的，通过追求某种形式的温和部署（如并行或增量部署）来控制风险可能很重要。

> You can do this by deploying a parallel version of your product and inviting people to opt-in

你可以通过部署产品的并行版本并邀请人们选择加入来做到这一点

> and try out the new version when they have the time. Allow those who try to use the new version to make it their default if they like it. Once you are confident that the new version is working well—and the majority of your community has converted to using it—you can make it the default and allow customers to “opt-out” and continue to use the old version for a period of time if they don't have the time to learn the changes immediately. Give these people sufficient notice of when support for the old version will be discontinued. For a significant client or service with a large

> community, expect this process to take months. Also expect some heat from your engineering and site operations organizations because it is not easy to support parallel versions.

并在他们有时间的时候试用新版本。允许那些尝试使用新版本的人如果喜欢就把它设为默认版本。一旦你确信新版本运行良好——而且你的社区大多数已经转换到使用它——你可以把它设为默认版本，并允许客户"选择退出"，如果他们没有时间立即学习变更，可以在一段时间内继续使用旧版本。给这些人足够的通知，说明对旧版本的支持将在什么时候停止。对于一个拥有大量社区的重要客户端或服务，预计这个过程需要几个月。还要预料到你的工程和网站运营组织会有些不满，因为支持并行版本并不容易。

> Another gentle deployment approach is to deploy regionally—first trying out the new version in a limited area of the country or world, and then expanding. Or you can deploy the changes incrementally—introducing the changes in bite-size pieces over time.

另一种温和部署的方法是按地区部署——首先在一个有限的国家或世界地区试用新版本，然后扩展。或者你可以增量部署变更——随着时间的推移，以小而易消化的块引入变更。

> However you choose to go about it, the key is to be as sensitive as possible to the disruption that your changes will cause. Give people the opportunity to learn the differences when they have the time, and try to minimize the impact of any problems or issues your changes may cause.

无论你选择如何进行，关键是尽可能对你的变更将造成的破坏保持敏感。给人们机会在他们有时间的时候学习差异，并尽量减少你的变更可能引起的问题或问题的影响。

> If your users like your product or service, you've got a reserve of goodwill to draw upon. But save it for when you really need it—don't waste that goodwill through user abuse.

如果你的用户喜欢你的产品或服务，你就有一个可以利用的好感储备。但要把它留到真正需要的时候——不要通过用户滥用浪费这种好感。

第 25 章：快速响应

> Chapter 25: Rapid Response
>
> Finish What You Start And Save Entire Release Cycles
>
> I've discussed elsewhere in this book the pitfalls of confusing product launch with success, and how important it is to not lose focus after you ship your product or service. In this chapter, I take a closer look at what you should be doing during this critical phase of your project.

完成你开始的并节省整个发布周期

> In many organizations, the resources that have been marshaled to build and launch the product evaporate very quickly after launch so they can be applied to the next project coming along. This is

> especially unfortunate because this is the moment when your opportunity for learning and correcting is greatest. I consider this a project management and product development process failing that can be corrected simply by slightly extending the project to incorporate this critical phase. No phase of the process will provide a better ROI than this one, so the change is not a difficult pitch to management.

我在本书的其他地方讨论了将产品发布与成功混淆的陷阱，以及在你发布产品或服务后不失去焦点是多么重要。在本章中，我更仔细地看看你应该在这个项目的关键阶段做什么。

> I advocate that all project teams schedule a phase that begins at launch and lasts typically a few days to a week. I call this phase rapid response, to emphasize that it is all about responding quickly to what you learn once the product has been launched.

在许多组织中，用于构建和发布产品的资源在发布后很快就会消失，以便可以应用于下一个即将到来的项目。这尤其不幸，因为这是你学习和纠正的机会最大的时刻。我认为这是一个项目管理和产品开发流程的失败，可以通过稍微延长项目以纳入这个关键阶段来简单纠正。流程的任何阶段都不会比这个阶段的 ROI 更好，所以这种变更对管理层来说不是一个困难的推销。

> Note that while this approach was borne out of consumer Internet services where it is particularly critical, I believe it is important for platform, infrastructure, and enterprise products as well.

我主张所有项目团队安排一个从发布开始、通常持续几天到一周的阶段。我称这个阶段为快速响应，以强调它是关于快速响应你在产品发布后学到的东西。

> Even if you've done all of the early prototypes and validation testing prior to engineering that Iadvocate—and you have a great QA team so the product is reliable—you still need to expect that there will be issues that can only be discovered once you're live. The typical approach of waiting to see what the response is and if any issues exist—and then trying to schedule follow-on point releases following the same general cycle—takes far too long and costs far too much.

请注意，虽然这种方法源于消费者互联网服务，在那里它特别关键，但我认为它对平台、基础设施和企业产品也很重要。

> The question is not whether there will be issues but, rather, how quickly will you address them?

即使你在工程之前做了我倡导的所有早期原型和验证测试——而且你有一个很棒的 QA 团队，所以产品是可靠的——你仍然需要期望，一旦上线，就会发现只有通过上线才能发现的问题。典型的方法是等待看反应如何，是否存在任何问题——然后试图按照相同的一般周期安排后续点发布——这需要太长时间，成本太高。

> When measuring success, you need to have a clear, prioritized set of business metrics. Are you looking at page views? Registered users? Time on site? Conversion rates from visitor to member? Subscriptions? Advertising revenue? The right set of metrics will depend on your product and your business goals but, in any case, you need to know what you care about and what you'll be tracking.

> Further, you need to know what results you would consider to be a success and what results you would consider to be a problem.

问题不是是否会有问题，而是你会多快解决它们?

> For consumer Internet services, it has never been easier to understand how people are using your product or service. It is easy to instrument and track activity—there are many products in this space. Services such as Google Analytics (www.google.com/analytics) can quickly and

在衡量成功时，你需要有一套清晰、有优先级的业务指标。你在看页面浏览量吗？注册用户？网站停留时间？从访客到会员的转化率？订阅？广告收入？正确的指标集将取决于你的产品和业务目标，但无论如何，你需要知道你关心什么以及你将跟踪什么。此外，你需要知道什么结果你会认为是成功，什么结果你会认为是问题。

> easily tell you a great deal about how your users are using your service.

对于消费者互联网服务，了解人们如何使用你的产品或服务从未如此容易。埋点和跟踪活动很容易——这个领域有很多产品。Google Analytics（www.google.com/analytics）等服务可以快速、轻松地告诉你很多关于你的用户如何使用你的服务的信息。

> For enterprise software, I like to send members of the product team—product manager, engineers, designers—out to the customer site to be there with them when they install the software and work to get the software live and in use. It is amazing how much faster issues are identified and resolved when a team member understands she's going to be at the customer site until the customer is live and referenceable.

对于企业软件，我喜欢派产品团队的成员——产品经理、工程师、设计师——去客户现场，在他们安装软件时与他们在一起，并努力使软件上线并投入使用。当团队成员明白她将在客户现场直到客户上线并可作为参考时，问题被识别和解决的速度会快得多，这是令人惊叹的。

> Once the issues have been identified, the product team (product manager, interaction designer, lead engineers, customer service, marketing, etc.) needs to meet no less than daily, prioritize the issues, and discuss the best way to respond. The result might be a “hot fix” that is rushed to the site, or possibly additional content that explains problem areas. If the team is prepared for these changes—and understands how crucial it is to identify and respond quickly—then, in a very short amount of time, it can make substantial improvements to the effectiveness of the product or service.

一旦问题被识别，产品团队（产品经理、交互设计师、首席工程师、客户服务、营销等）需要每天至少开会，确定问题的优先级，并讨论最佳响应方式。结果可能是一个"热修复"，被紧急推送到网站，或者可能是解释问题领域的额外内容。如果团队为这些变更做好了准备——并理解快速识别和响应是多么关键——那么，在很短的时间内，它可以对产品或服务的有效性做出实质性的改进。

> Site analytics are not the only tool you should use to understand your users and how they feel about your site. Surveys, e-mail discussions, message boards, and field testing are other examples.

> But the data provides such near-real-time insights that I look at the Web site analytics for all of the sites that I am involved in nearly daily. I'm very often looking at where people are coming from, what their favorite pages and activities are, how long people are spending on the site, how many pages they are viewing, and how often they return. Customer service, sales, and partners are also good sources of input for rapid response fixes.

网站分析不是你用来了解你的用户以及他们对你的网站感受的唯一工具。调查、电子邮件讨论、留言板和现场测试是其他例子。但数据提供了如此近实时的洞察力，以至于我几乎每天查看我参与的所有网站的网站分析。我经常查看人们来自哪里，他们最喜欢的页面和活动是什么，人们在网站上花费多长时间，他们查看多少页面，以及他们多久回来一次。客户服务、销售和合作伙伴也是快速响应修复的良好输入来源。

第 26 章：成功运用敏捷方法

> Chapter 26: Succeeding With Agile Methods
>
> Top 10 List

十大清单

> Many software product teams are either currently experimenting with Agile methods, or have recently adopted some form of the methods. While the benefits of Agile methods—including Scrum and XP—are many, most product teams struggle initially as they work to understand how best to apply Agile methods which were originally developed for the custom software world to their product software environment.

许多软件产品团队目前正在尝试敏捷方法，或者最近采用了某种形式的方法。虽然敏捷方法——包括 Scrum 和 XP——的好处很多，但大多数产品团队在最初努力理解如何最好地应用敏捷方法时都会遇到困难，这些方法最初是为定制软件世界开发的，用于他们的产品软件环境。

在本章中，我强调了在产品软件环境中成功运用敏捷的关键。

如果你还不知道敏捷方法是什么，请访问 www.agilemanifesto.org。

请注意，这个清单是针对产品软件团队的。对于定制软件，有一些非常不同的考虑因素。

1. 产品经理是产品负责人，他代表客户。他需要与产品开发团队极度参与，帮助推动 backlog，特别是回答出现的问题。一些误入歧途的产品经理认为他们在敏捷环境中可以轻松——他们再错不过了。有些人还喜欢

让不同的人担任产品经理和产品负责人角色，但这通常只是更深层次问题的症状（参见"产品管理 vs. 产品营销"一章）。

. 使用敏捷不是缺乏产品规划的借口。作为产品经理/负责人，你仍然需要知道你要去哪里，你想实现什么，以及你将如何衡量成功。也就是说，在敏捷环境中，你的规划视野可以 somewhat 更短和滚动。你应该使用轻量级的机会评估，而不是繁重的 MRD（参见"机会评估"一章）。

1. 你和你的设计师应该总是比你的团队领先一两个冲刺。这允许你有足够的时间验证困难的功能以改进它们。坚持设计师（交互设计师和视觉设计师）在流程中处于最前沿，并确保他们不要试图在冲刺期间进行设计工作——而实施已经在进行中（参见"设计与实现"一章）。但是，要确保工程团队的某人在每一步审查你的想法和原型，以提供关于可行性、成本和对更好解决方案的洞察的反馈。

. 将设计工作分解成尽可能小和独立的块，但不要太小——确保你不要试图一次一个房间地设计一所房子。但记住强调想出尽可能小的产品。请注意，在敏捷环境中，设计师可能需要比他们舒适的更快地工作。你会发现某些设计师和某些设计方法——比如快速原型——比其他方法更适合敏捷环境的节奏。

1. 作为产品经理/负责人，你的主要职责是想出有价值和可用的原型和用户故事，你的团队可以从中构建。用原型和用户故事取代繁重的 PRD 和功能规格。出于三个原因做原型：（1）这样你可以用真实用户测试，（2）迫使自己思考问题；（3）这样你就有一个很好的方式向工程描述你在冲刺期间需要构建什么。确保用真实用户测试原型。尝试你的想法并迭代原型，直到你有值得构建的东西。你仍然需要确保你不要浪费冲刺周期。
2. 让工程以他们喜欢的任何粒度将冲刺分解。有时原型中的功能可以在单个冲刺中构建，其他时候可能需要几个冲刺。你会发现拥有好的原型将显著有助于估算构建所需的工作量和时间。记住，工程团队在质量、可扩展性和性能方面有考虑，所以让他们按他们认为合适的方式将功能分块到冲刺中。
3. 确保你作为产品经理/负责人和你的交互设计师参加每一个每日状态会议（又名站会或每日 scrum）。这些早晨会议是沟通过程的开始，而不是结束。关于产品会有持续的讨论流。设计师应该向开发人员和 QA 预览功能。开发人员应该向彼此、QA、设计师和产品经理展示已完成的代码。QA 和开发人员应该在原型制作期间识别潜在的陷阱，并帮助团队在功能、设计和实现权衡方面做出更好的决策。

> In this chapter I highlight the keys for succeeding with Agile in a product software environment.

1. 不要只是发布每个冲刺——在暂存区重新组装冲刺结果，直到你有足够的东西按照产品经理/负责人定义的那样进行发布。确保有足够的功能值得向用户发布是产品经理的工作。记住，在产品环境中，不断的变更可能会让你的客户感到不安（参见"温和部署"一章）。
2. 在每个冲刺结束时，确保你演示产品的当前状态，以及下一个冲刺的原型。让每个人看到你完成的东西验证了团队的辛勤工作，给整个公司提供了对产品的洞察力，并让布道继续进行。

3. 为你的整个团队提供敏捷培训。聘请顾问帮助你的产品团队转向敏捷，但要确保顾问有与产品软件团队合作的经验，并理解产品软件与 IT 或定制软件之间的区别。如果每个人都理解敏捷的机制，那么你就可以专注于执行。如果人们不理解，你会陷入语义和教条问题中。

> If you don't yet know what Agile methods are, take a look at www.agilemanifesto.org.

© cAN'T EARLY SPRINTS BE CONSIDERED A PROTOTYPE?

一些敏捷倡导者和实践者认为，团队应该只把早期的冲刺视为工作原型。事实上，对于定制软件工作，那里真的没有真正的产品管理，很少有用户体验设计，这基本上是你能做的最好的了。然而，对于产品软件组织，你可以而且必须比这做得更好，有三个原因：

> Note that this list is meant for product software teams. For custom software, there are some very different considerations.

首先，一个冲刺通常太长，无法等待尝试一个想法——这个想法很可能是错误的。用一次性原型在几天内尝试这个想法要快得多，而不是等待几个月的一个或多个冲刺周期。

其次，工程团队通常有太多关键的事情要做，不能把他们用于产品发现过程。通过占用他们的时间来做这种原型工作，他们无法做他们应该做的事情——构建生产软件。

第三，虽然敏捷方法做了很多鼓励团队学习和快速响应的工作，但一旦团队开始沿着一条路径走下去，并把长时间投入到特定的架构或方法中，团队就很难改变方向，而且耗时。

@ 敏捷可以用于产品软件吗?

像 Scrum 这样的敏捷方法确实解决了一些困扰软件团队数十年的关键问题。但许多产品经理和用户体验设计师——以及在较小程度上的 QA 人员——最初对敏捷感到困惑，不确定他们在这些方法中的角色。要明确的是，这些方法绝对需要这些角色，但我把困惑归因于敏捷方法的起源。我发现当我解释起源时，它有助于阐明敏捷旨在解决的问题，以及仍然存在哪些挑战。

> 1. The product manager is the product owner, and he represents the customer. He will need to be extremely involved with the product development team, helping to drive the backlog and especially answer questions as they arise. Some misguided product managers think they get off easy in an Agile environment—they couldn't be more wrong. Some also like to

许多人惊讶地得知，Scrum，最流行的敏捷方法，现在已经超过 20 岁了。它于 1986 年在日本创建。（这是新想法达到临界点可能需要多长时间的一个例子）。

但最重要的是，这些方法起源于定制软件世界。

定制软件世界——为特定客户构建特殊用途的软件——长期以来一直是一种极其困难的软件类型。这部分是因为客户 notoriously 不知道他们想要什么，但他们有需求，所以他们与定制软件供应商签订合同，或者与他们的内部 IT 人员坐下来，然后工作交付。当他们交付时，客户总是回应说，这并不是他们想要的，所以循环重复，挫折 mounts。但核心需求仍然存在，所以这为数不清的 IT 开发人员、定制软件商店和专业服务业务提供了工作保障。

此外，在招聘和留住顶级软件人才方面，定制软件长期以来一直处于不利地位。

这部分是因为许多顶级软件专业人士更喜欢为为数以千计（如果不是数以百万计）的客户创建软件的公司工作。部分原因是软件专业人士为产品软件公司工作获得更高的报酬，在这些公司中，产品团队负责想出能让许多人满意的软件产品，否则他们就无法赚钱。所以这些公司知道他们必须雇用必要的人才来创建成功的产品，并相应地支付报酬。但客观地说，只有相对较少百分比的软件人员实际从事商业产品软件——大多数人从事定制软件。

在定制软件模式中，由于客户相信他知道自己需要什么，你很少会发现产品经理的角色。同样，你几乎永远不会找到用户体验设计师。原因更复杂，涉及一定程度的无知（定制软件世界中相对较少的人意识到用户体验设计师做什么以及为什么需要他们），和成本敏感性（通过让开发人员设计来削减成本）。但公平地说——由于我们行业中用户体验设计师的短缺——少数可用的设计师立即被意识到他们有多关键的产品公司抢走，所以即使定制软件团队的领导意识到他们需要设计师，他们也很难找到。同样，QA 作为一门学科很少出现在定制软件项目中——同样，开发人员通常被期望做所需的测试。

理解定制软件世界的另一个关键因素是，绝大多数定制软件项目相对较小，是为了支持公司的内部运营而做的——诸如人力资源、计费和制造等应用程序——用户数量有限意味着可扩展性和性能等问题通常不那么关键。

从历史上看，定制软件世界使用瀑布流程，因为各种利益相关者需要一种方法来监控创建这些合同应用程序的长期过程中的进度。事实上，瀑布方法也起源于此。

在产品软件世界中，软件必须靠自身的优点销售，我们引入了产品经理的角色来代表广泛客户的需求，用户体验设计师来创建有效的用户体验，以及 QA 测试人员来确保软件在客户环境的范围内按宣传工作。

但在定制软件世界中，想出满足客户的东西的基本问题继续存在。

> have different people covering the product manager and the product owner role, but this is usually just a symptom of a deeper problem (see the chapter Product Management vs.

对于这些团队来说，敏捷方法代表着显著的改进。它们改善了客户和工程师之间的沟通。它们通过构建更小、更频繁的迭代显著降低了风险，这样客户就能更快地知道他是否真的喜欢某样东西——而不是等待漫长过程的结束。它们帮助引入了一些现代软件测试概念，它们帮助团队免于花费无数小时准备很少被阅读的文件——而且很快就会过时。

总的来说，这些对产品软件团队也是很好的好处，但我总是解释说需要一些调整。我之前写过关于这些主题的文章——比如如何将用户体验设计插入流程，以及如何管理发布和部署——但另一个苦苦挣扎的领域是架构设计。

敏捷方法鼓励工程师不要依恋他们的实现，相信事情可以相对快速和容易地重构或重新架构。对于绝大多数定制软件来说，这是正确的，但对于许多产品软件系统，如大规模消费者互联网服务——必须支持数十万（如果不是数百万）用户——这种方法可能是天真的。

所以许多产品软件团队在使用敏捷方法时遇到的主要问题源于其定制软件起源，这并不应该是一个大惊喜。许多敏捷书籍、文章和培训课程仍然没有提到产品经理——或任何形式的体验设计师（交互设计师和视觉设计师）——因为它们不是为产品软件团队准备的。

我对转向敏捷的团队的建议是，确保你雇用帮助你的组织转向敏捷的公司实际上理解产品软件所要求的差异。大多数不理解，但有些理解。

> Product Marketing)
>
> 1. Using Agile is not an excuse for a lack of product planning. As a product manager/owner, you still need to know where you're going, what you're trying to accomplish, and how you'll measure success. That said, in an Agile environment, your planning horizon can be somewhat shorter and rolling. You should use the lightweight opportunity assessment instead of a heavy MRD (see the chapter Opportunity Assessments).
> 2. You and your designers should always try and be one or two sprints ahead of your team. This allows you to validate difficult features with sufficient time to improve them. Insist that the designers (interaction designers and visual designers) are front and center in the process, and make sure they don't try to do their design work during the sprint—while the implementation is already underway (see the chapter Design vs. Implementation). Make sure, however, that someone from the engineering team is reviewing your ideas and prototypes every step of the way to provide feedback on feasibility, costs, and insights into better solutions.
> 3. Break the design work into as small and as independent chunks as possible, but not too small—make sure you don't try to design a house one room at a time. But remember the emphasis on coming up with the minimal product possible. Note that, in an Agile environment, the designers may need to work faster than they're comfortable with. You'll find that certain designers, and certain design methodologies—such as rapid prototyping—are more compatible with the pace of an Agile environment than others.
> 4. As a product manager/owner, your main responsibility is to come up with valuable and usable prototypes and user stories that your team can build from. Replace heavy PRDs
>
> and functional specs with prototypes and user stories. Do prototypes for three reasons: (1) so you can test with real users, (2) to force yourself to think through the issues; and (3) so you have a good way to describe to engineering what you need built during the sprint. Be sure to test prototypes with real users. Try out your ideas and iterate on the prototype until you've got something worth building. You still need to make sure that you don't waste sprint cycles.
>
> 1. Let engineering break up the sprints into whatever granularity they prefer. Sometimes the functionality in a prototype can be built in a single sprint, other times it may take several sprints. You will find that having good prototypes will help significantly in estimating the amount of work and time required to build. Remember that the engineering team has considerations in the areas of quality, scalability, and performance, so let them chunk the functionality into sprints as they see fit.
> 2. Make sure you as product manager/owner and your interaction designer are at every daily status meeting (aka standup or daily scrum). These morning meetings are the beginning

of the communication process, not the end. There will be a constant stream of discussion about the product. Designers should be previewing functionality to the developers and QA. Developers should be showing off completed code to each other, QA, and the designers and product manager. QA and developers should be identifying potential pitfalls during prototyping, and helping the team to make better functionality, design and implementation trade-offs.

3. Don't just launch every sprint—reassemble sprint results in a staging area until you have enough to make a release as defined by the product manager/owner. It's the product manager's job to ensure that there is sufficient functionality to warrant a release to the user. Remember that in a product environment, constant change can be upsetting to your

customers (see the chapter Gentle Deployment).

1. At the end of each sprint, make sure you demo the current state of the product, as well as the prototype for the next sprint. Having everyone see what you finished validates the team's hard work, gives the entire company insight into the product, and keeps the evangelism going.
2. Get Agile training for your entire team. Hire a consultant to help your product team move to Agile, but make sure the consultant has proven experience with product software teams and understands the difference between product software and IT or custom software. If everyone understands the mechanisms around Agile, then you can focus on the execution. If people don't understand, you'll get bogged down in the semantics and dogmatic issues.

@caAn'T EARLY SPRINTS BE CONSIDERED A PROTOTYPE?

Some Agile advocates and practitioners argue that the team should just consider the early sprints as the working prototype. And in fact, for custom software efforts where there really isn't true product management and rarely user experience design, this is the essentially the best you can do. However, for product software organizations, you can and must do better than this, for three reasons:

First, a sprint is typically far too long to wait to try out an idea—an idea which will most likely be wrong. It is much faster to try that idea out with a disposable prototype in days rather than wait months for one or more sprint cycles.

Second, there are typically too many critical things for the engineering team to do to use them for the product discovery process. By taking their time for this prototyping work they are not able to do what they should be doing—building production software.

Third, while Agile methods do much to encourage the team to learn and respond quickly, it is still difficult and time consuming for a team to change directions significantly once they have begun down a path, and put long hours into a particular architecture or approach.

CAN AGILE BE USED FOR PRODUCT SOFTWARE?

Agile methods like Scrum really do attack some key problems that have plagued software teams for decades. But many product managers and user experience designers—and to a lesser extent QA staff—are initially confused by Agile and unsure of their role in these methods. To be clear, these

methods absolutely require these roles, but I attribute the confusion to the origin of Agile methods. I've found that when I explain the origins, it helps to illuminate the problems that Agile was designed to solve, and what challenges remain. Many are surprised to learn that Scrum, the most popular of the Agile methods, is now over 20 years old. It was created in 1986 in Japan. (Yet another example of just how long it can take for a new idea to reach the tipping point).

But most importantly, these methods originated in the custom software world.

The custom software world—building special purpose software for specific customers—has long been a brutally difficult type of software. This is partly because customers notoriously don't know what they want, but they have a need so they write a contract with a custom software supplier, or sit down with their internal IT folks, who then work to deliver. When they do deliver, the customer invariably responds that it really wasn't what they had in mind, so the cycle repeats and frustration mounts. But the core need still exists, so this provides job security for countless IT developers, custom software shops, and professional services businesses.

Further, custom software has long been on the short end of the stick when it comes to recruiting and retaining top software talent.

This is partly the case because many top software professionals prefer to work for companies that are in the business of creating software for thousands, if not millions of customers. And partly it's because software professionals get paid more working for product software companies where the product team is responsible for coming up with software products that please many people, or they don't make money. So these companies know they must hire the talent necessary to create winning products, and they pay accordingly. But to put this in perspective, only a relatively small percentage of software people actually work on commercial product software—most work on custom software.

In the custom software model, since the customer believes he knows what he needs, you'll rarely find the role of the product manager. Likewise, you'll almost never find user experience designers. The reasons for this are more complex, and involve a degree of ignorance (relatively few in the custom software world realize what user experience designers do and why they're needed), and cost sensitivity (cut costs by letting the developers design). But to be fair—due to the shortage of user experience designers in our industry—the few available are immediately grabbed by the product companies that realize how critical they are, so custom software teams can rarely find designers even if their leaders realize they need them. Similarly, QA as a discipline is rarely found in custom software projects—again, the developers are typically expected to do the required testing.

Another crucial element in understanding the custom software world is that the vast

majority of custom software projects are relatively small and done to support the internal operations of a company—applications such as HR, billing, and manufacturing—where the limited number of users means that issues such as scalability and performance are usually less critical.

Historically the custom software world used the Waterfall process because the various stakeholders needed a way to monitor progress during the long process of creating these contract applications. In fact, the Waterfall methods originated here as well.

> In the product software world, where the software must sell on its own merits, we introduced the roles of product managers to represent the needs of a wide range of customers, user experience designers to create effective user experiences, and QA testers to ensure the software worked as advertised in the range of customer environments.
>
> But in the custom software world, the same fundamental issues of coming up with something that satisfied the customer continued.
>
> For these teams, especially, the Agile methods represent significant improvements. They improve communication between the customer and the engineers. They significantly reduce the risk by building smaller, more frequent iterations so that the customer can learn whether he really likes something or not much sooner—rather than waiting for the end of a long process. They help introduce some modern software testing concepts, and they help relieve the team from spending countless hours preparing documents that are rarely read—and quickly obsolete.
>
> In general, these are great benefits for product software teams as well, but I always explain that a few adjustments are required. I've written earlier about these topics—such as how to insert user experience design into the process, and how to manage releases and deployments—but another area that has struggled is architectural design.
>
> Agile methods encourage engineers to not get attached to their implementation, believing that things can be re-factored or rearchitected relatively quickly and easily. This is true for the vast majority of custom software, but for many product software systems, such as large-scale consumer Internet services—which must support hundreds of thousands if not millions of users—this approach can be naive.
>
> So it shouldn't be a big surprise that the main issues many product software teams encounter with Agile methods stem from their custom software origin. Many Agile books, articles, and training classes still don't mention product managers—or any form of user experience designers (interaction designers and visual designers)—because they aren't meant for product software teams.
>
> My suggestion to teams moving to Agile is to make sure the firm you hire to help your organization transition to Agile actually understands the differences that product software demands. Most don't, but some do.

第 27 章：成功运用瀑布流程

> Chapter 27: Succeeding With Waterfall Processes
>
> Proactively Addressing The Issues

主动解决问题

> In this chapter we look at the Waterfall process—the product development process that the majority of product teams still follow.

在本章中，我们来看看瀑布流程——大多数产品团队仍然遵循的产品开发流程。

> Even though the Waterfall development process is more than 30 years old—and even though it is often cursed by engineers and product managers alike, including me—it is still the most common process used to create software products.

尽管瀑布开发流程已经有 30 多年的历史——尽管它经常被工程师和产品经理（包括我）诅咒——但它仍然是创建软件产品最常用的流程。

> While it has long been unfashionable for a team to describe their product development process as waterfall, in most cases that is essentially what is still being followed, albeit by many different names including: Successive Refinement, SDLC, Phase-Gate, Stage Review, Staged Contracts, and Milestone-based.

虽然一个团队将他们的产品开发流程描述为瀑布已经不流行了，但在大多数情况下，这基本上仍然是正在遵循的，尽管有许多不同的名称，包括：连续细化、SDLC、阶段-门控、阶段评审、阶段合同和基于里程碑。

> In this chapter we'll explore the key weaknesses of this approach but, most importantly, we'll discuss what the product manager must do in order to maximize the chance of success with this process.

在本章中，我们将探讨这种方法的关键弱点，但最重要的是，我们将讨论产品经理必须做什么才能最大化使用此流程成功的机会。

> General Principles

一般原则

> The conventional waterfall process is extremely simple in concept:

传统的瀑布流程在概念上极其简单：

> 1. Phased development. Software progresses through a well defined series of phases, including: a written description of the requirements, user experience design, high-level architectural design, low-level detailed design, code, testing, and deployment.

1. 分阶段开发。软件通过一系列定义明确的阶段进行，包括：需求的书面描述、用户体验设计、高级架构设计、低级详细设计、代码、测试和部署。

> 1. Phase review. Each phase ends with a review of the deliverables from that phase, followed by sign-off and an explicit transition to the next phase.

1. 阶段评审。每个阶段以对该阶段交付物的评审结束，然后是签字和明确过渡到下一个阶段。

> The Waterfall method can be applied either informally or very formally, as in the US

瀑布方法可以非正式地或非常正式地应用，如美国国防部标准 2167A（后来的 498），它以令人痛苦的详细描述了流程的每一步以及许多文件交付物。

> Department of Defense Standard 2167A (and later 498), which describes in excruciating

同样，瀑布方法也是以下非常非正式和更常见场景的核心：营销部门的某个人收集一些市场需求并将其交付给工程部门，工程部门制定一个时间表并进行架构设计，然后对更复杂的区域进行一些详细设计。然后产品进入实现和测试，通常是测试版，最后是部署。

> detail every step of the process along with the many document deliverables.

虽然我们将很快讨论这种方法最严重的局限性，但承认使这一流程保持使用这么长时间的关键特征也是有用的：

> Similarly, the Waterfall method is also at the heart of the following very informal and much more common scenario: Someone from the marketing department gathers some market requirements and delivers them to engineering, which comes up with a schedule and works on an architectural design, and then some detailed designs for the more complicated areas.
> The product then moves into implementation and testing, often a beta, and finally deployment.
> While we will soon discuss the most serious limitations of this approach, it is also useful to acknowledge the key traits that have kept this process in use for so long:
>
> · Management appreciates the (perceived) predictability of the process. It is possible,
>
> although not common, to come up with fairly accurate schedules for even large and complex software projects. This assumes, however, that you completely and accurately understand the requirements and the technology, and that there will be no changes. With iterative approaches, you don't really know how many iterations will be required, and this can be disconcerting to management.
> *There are deliverables throughout the process. Many people (managers, customers/clients, and even some engineers) are reassured by seeing well thought-out and thorough documents and design diagrams. It helps these people to gauge progress towards the end, and it also helps them feel better about the level of thinking that has gone into the project (even though there is no way to test whether or not the confidence is justified because unlike software you can't execute paper documents). Many people make the mistake of feeling unjustifiably reassured by impressive specifications and documents.
> Product Management Concerns

«管理层欣赏流程的（感知到的）可预测性。有可能，虽然

> There are a number of well-known concerns with this process, especially from the product manager's perspective:

虽然不常见，但即使是大型和复杂的软件项目也能制定出相当准确的时间表。然而，这假设你完全和准确地理解需求和技术，并且不会有任何变化。使用迭代方法，你真的不知道需要多少次迭代，这可能会让管理层感到不安。

> Validation Occurs Too Late in the Process

«整个流程都有交付物。许多人（经理、客户/客户，甚至一些工程师）看到经过深思熟虑和彻底的文件和设计图会感到安心。这有助于这些人衡量向最终目标的进度，也有助于他们对项目中投入的思考水平感觉更好（尽管无法测试这种信心是否合理，因为与软件不同，你不能执行纸质文件）。许多人错误地对令人印象深刻的规格和文件感到不合理的安心。

> The most costly issue is that there is typically no actual working software until nearly the end of the process, so there is little if any visibility into whether the software will be useful until after the majority of the investment has been made.
>
> The product manager must ensure that, prior to moving into the expensive design and implementation phases, the product must be prototyped and tested on actual target users. This ensures the specification that is eventually provided to the product development organization describes a product that has been successfully validated with the target audience.
>
> Likewise, if there are major technical risks, these too should be explored and feasibility questions resolved (by the engineering organization) prior to beginning the actual architectural design and implementation. Before proceeding, the team needs to know that the product specification is something that can be successfully delivered.
>
> Changes Are Costly and Disruptive
>
> Any change to decisions from previous stages destabilizes the process and causes considerable pain and cost, as much work has to be reviewed and reworked. Moreover, the coding and testing process often uncovers issues in requirements and in architecture that can cause major delays and pain in this process.
>
> The product manager must constantly represent the voice of the customer and user and there will be times when changes are required. It is important to point out that the cost of postponing the change needs to include the cost of the follow-on release to make the correction. There will still be times when it makes the most sense to defer the change until the next release, but in many cases it will be less expensive to course correct sooner rather
>
> than later.
>
> Responding to the Market
>
> This approach has a relatively high overhead in terms of documentation and process for moving through the phases. One consequence of this is that it can take considerable time to make even relatively small changes to the software.
>
> This puts additional pressure on the product manager to ensure that they are providing a validated specification for a successful product in the first place, but it also means that the product manager will need to work with the product team to make course corrections as quickly as possible after release.

> Summary

产品管理关注点

> We have all seen the consequences of the Waterfall process in practice, and it's not hard to understand the motivation for alternatives such as the Agile methods Scrum and XP.

这个过程有许多众所周知的关注点，特别是从产品经理的角度来看：

> In many ways, the Waterfall process represents an idealistic but naive view of the software development process, where people are able to anticipate the key issues and fully understand the requirements. When this is the case—usually only for very small projects—this approach can provide a reasonable path to a quality implementation. Unfortunately, this is rarely the case with product software. In practice, the consequence is
>
> that the product ships later than planned due to changes, and then expensive, time-consuming follow-on releases are required to correct issues once real users have a chance to see and use the software.
>
> That said, the product development process is often deeply entrenched in the product development organization, and the best the product manager can do is to ensure that potential problems are avoided. The most important thing is to ensure that during the requirements and design phases, the emphasis must be on true product discovery—identifying a product that is valuable, usable and feasible—and that the product spec is validated (by building a prototype and testing it with real users) prior to moving to the implementation phase. If you do this, you'll not only stand a much better chance of defining an inspiring and successful product, but you'll also save your team significant time and cost.

验证发生在流程的太晚阶段

最昂贵的问题是，通常在流程快结束时才有实际工作的软件，所以在大部分投资完成之前，对软件是否有用几乎没有可见性。

产品经理必须确保，在进入昂贵的设计和实现阶段之前，产品必须在实际目标用户上进行原型化和测试。这确保最终提供给产品开发组织的规格描述了一个已成功与目标受众验证的产品。

同样，如果存在重大技术风险，这些也应该在实际架构设计和实现开始之前由工程组织探索和解决可行性问题。在继续之前，团队需要知道产品规格是可以成功交付的东西。

变更成本高昂且具有破坏性

对前一阶段决策的任何变更都会使流程不稳定并造成相当大的痛苦和成本，因为必须审查和返工大量工作。此外，编码和测试过程经常发现需求和架构中的问题，可能导致此流程中的重大延误和痛苦。

产品经理必须始终代表客户和用户的声音，有时需要变更。重要的是要指出，推迟变更的成本需要包括后续版本进行更正的成本。有时推迟变更到下一个版本仍然是最有意义的，但在许多情况下，尽早纠正比以后纠正成本更低。

响应市场

这种方法在文档和流程方面有相对较高的开销，用于经历各个阶段。其中一个后果是，即使是相对较小的软件变更也可能需要相当多的时间。

这给产品经理带来了额外的压力，确保他们首先为成功的产品提供经过验证的规格，但这也意味着产品经理需要与产品团队合作，在发布后尽快进行路线修正。

我们都看到过瀑布流程在实践中的后果，不难理解 Scrum 和 XP 等替代方法的动机。

在很多方面，瀑布流程代表了一种理想主义但天真的软件开发流程观点，人们能够预见关键问题并充分理解需求。当这种情况发生时——通常只对非常小的项目——这种方法可以提供通往质量实现的合理路径。

不幸的是，产品软件很少是这样。在实践中，结果是

由于变更，产品比计划晚些时候发货，然后一旦真实用户有机会看到和使用软件，就需要昂贵的、耗时的后续版本来纠正问题。

也就是说，产品开发流程通常深深扎根于产品开发组织，产品经理能做的最好的事情就是确保避免潜在的问题。最重要的是，确保在需求和设计阶段，重点必须放在真正的产品发现上——识别有价值、可用和可行的产品——并且产品规格在进入实现阶段之前经过验证（通过构建原型并用真实用户测试）。如果你这样做，你不仅会有更好的机会定义一个令人启发和成功的产品，而且还会为你的团队节省大量的时间和成本。

第 28 章：创业产品管理

> Chapter 28: Startup Product Management
> It's All About Product Discovery

一切都是关于产品发现

> I've been working with quite a few startups over the past few years—usually in an advisory capacity, but sometimes more directly involved. Startups are essentially all about new product creation, so they're a terrific place for product managers to do their thing, and it's why I love working with startups so much. Yet I believe that the prevalent model for how startups come up with their first product is terribly inefficient, and it's why so many otherwise good ideas never get funded or make it to market.

过去几年，我一直在与相当多的初创公司合作——通常是以顾问的身份，但有时更直接地参与。初创公司本质上都是关于新产品创建的，所以它们是产品经理施展拳脚的好地方，这也是我如此喜欢与初创公司合作的原因。然而，我相信初创公司提出其第一个产品的普遍模式效率极低，这就是为什么这么多原本好的想法从未获得资金或进入市场。

> Here's how the model typically works: Someone with an idea gets some seed funding, and the first thing he does is hire some engineers to start building something. The founder will have definite

> ideas on what she wants, and she'll typically act as product manager and often product designer, and the engineering team will then go from there. The company is typically operating in “stealth mode” so there's little customer interaction. It takes much longer than originally thought for the engineering team to build something, because the requirements and the design are being figured out on the fly.

这个模式通常是这样运作的：有想法的人获得一些种子资金，他做的第一件事就是雇用一些工程师开始构建东西。创始人会对她想要什么有明确的想法，她通常会充当产品经理，甚至产品设计师，然后工程团队从那里开始。公司通常在"隐形模式"下运营，所以很少有客户互动。工程团队构建东西需要比最初想象的更长的时间，因为需求和设计是在 fly 上 figuring out 的。

> After six months or so, the engineers have things in sort of an alpha or beta state, and that's when they first show the product around. This first viewing rarely goes well, and the team

大约六个月后，工程师们把东西做成 alpha 或 beta 状态，那是他们第一次展示产品的时候。这第一次展示很少进行得顺利，团队

> starts scrambling. The run rate is high because there's now an engineering team building this thing as fast as they can, so the money is running out and the product isn't yet there. Maybe the company gets additional funding and a chance to get the product right, but often it doesn't. Many startups try to get more time by outsourcing the engineering to a low-cost off-shore firm, but they're still left with the same process and the same problems. Here's a very different approach to new product creation, one that costs dramatically less and is much more likely to yield the results you want: The founder hires a product manager, an interaction designer, and a prototyper. Sometimes the designer can also serve as prototyper, and sometimes the founder can serve as the product manager, but one way or another, you have these three functions lined up—product management, interaction design, and prototyping—and the team starts a process of very rapid product discovery. I describe this process in detail in the chapter Reinventing the Product Spec, but there are two keys:

开始手忙脚乱。运营费用很高，因为现在有一个工程团队在尽可能快地构建这个东西，所以资金正在耗尽，而产品还没有准备好。也许公司会获得额外的资金并有机会把产品做好，但经常不会。许多初创公司试图通过将工程外包给低成本的离岸公司来争取更多时间，但他们仍然面临相同的流程和相同的问题。

这里有一种非常不同的新产品创建方法，成本显著更低，更有可能产生你想要的结果：创始人雇用产品经理、交互设计师和原型师。有时设计师也可以担任原型师，有时创始人可以担任产品经理，但无论如何，你有这三个功能排列起来——产品管理、交互设计和原型制作——团队开始一个非常快速的产品发现过程。

我在"重塑产品规格"一章中详细描述了这个过程，但有两个关键：

> 1. The idea is to create a high-fidelity prototype that mimics the eventual user experience

1. 想法是创建一个模仿最终用户体验的高保真原型

> 1. You need to validate this product design with real target users. In this model, it's normal to create literally dozens of versions of the prototype—it will evolve daily, sometimes with minor refinements and sometimes with very significant changes. But the point is that with each iteration you are getting closer to identifying a winning product. This process typically takes between a few weeks and a few months. At the end of the process, you have (a) identified a product that you have validated with the target market, (b) avery rich prototype that serves as a living spec for the engineering team to build from, and

1. 你需要用真实目标用户验证这个产品设计。

在这种模式下，创建几十个版本的原型是很正常的——它会每天演变，有时是小的改进，有时是很大的变化。但关键是，每次迭代你都更接近识别一个成功的产品。

> (©) a much greater understanding of what you're getting into, and what you'll need to do to succeed.

这个过程通常需要几周到几个月的时间。在流程结束时，你有（a）一个已经与目标市场验证过的产品，（b）一个作为工程团队构建的活规格的非常丰富的原型，以及（c）对你正在进入的领域以及你需要做什么才能成功有更深入的理解。

> Now, when you bring on an engineering team, they'll start off with a tremendous advantage—a clear understanding of the product they need to build and a stable spec. You'll find that the team can produce a quality implementation much faster than they would otherwise.

现在，当你引入工程团队时，他们会以一个巨大的优势开始——对需要构建的产品有清晰的理解和一个稳定的规格。你会发现团队可以比以其他方式更快地产生质量实现。

> So why don't all startup teams do this? Because we're such an engineering-driven industry that we just naturally start there. But any startup has to realize that everything starts with the right product, so the first order of business is to figure out what that is before burning through $500K or more in seed funding.

那么为什么不是所有的初创团队都这样做呢？因为我们是一个如此由工程驱动的行业，我们只是自然地从这里开始。但任何初创公司都必须意识到，一切都始于正确的产品，所以第一要务是在烧完 50 万美元或更多的种子资金之前弄清楚那是什么。

> This model applies beyond startups to much larger companies as well. The difference is that bigger companies are generally able to underwrite the several iterations it takes to get to a valuable

> product, while startups often can't. But there's no reason for the inefficiencies that larger companies regularly endure.

这种模式也适用于初创公司之外的大公司。不同之处在于，大公司通常能够承担几次迭代以获得有价值的产品，而初创公司经常无法承担。但没有理由让大公司经常忍受的低效继续下去。

> So on your next startup or new product development effort, give this approach a try.

所以在你的下一次创业或新产品开发工作中，试试这种方法。

第 29 章：在大公司中创新

> Chapter 29: Innovating In Large Companies
>
> Difficult But Worth The Effort

困难但值得努力

> There is a lot of cynicism out there about whether or not you can really innovate in a big company. Some believe that nearly all true innovation happens in the startup environment, and that the best a large company can do is either copy those innovations or acquire the successful startups. While I agree that it is certainly much easier to innovate in a startup, innovation most definitely can happen in larger companies as well.

关于你是否能真正在大公司中创新，有很多冷嘲热讽。有些人认为，几乎所有真正的创新都发生在初创公司环境中，而大公司能做的最好的事情就是复制这些创新或收购成功的初创公司。虽然我同意在初创公司中创新肯定更容易，但创新绝对也可以发生在更大的公司中。

> Unless you've worked at one of these large companies, you might be surprised to hear that innovation is actually a problem. After all, we hear so much about it—and see the success of large technology companies—we naturally assume that innovation is powering all this growth. But as we explained earlier, as organizations get larger, they invariably become more conservative, and less willing to take risks. Again, this is because large companies have much more to lose than smaller companies, so they get really good at protecting what they have. But there are also substantial advantages to shipping product from a large company and, despite their risk-averse nature, your company does need you to innovate.

除非你曾在这些大公司之一工作过，否则你可能会惊讶地听到创新实际上是一个问题。毕竟，我们听到关于它的很多——并看到大型科技公司的成功——我们自然地假设创新是推动所有这些增长的动力。但正如我们之前解释的，随着组织变大，它们不可避免地变得更加保守，更不愿意冒险。同样，这是因为大公司比小公司有更多损失，所以它们非常擅长保护它们所拥有的。但也有从大公司发货产品的实质性优势，而且，尽管它们厌恶风险，你的公司确实需要你创新。

> The two biggest factors influencing your ability to innovate in a large company are your corporate culture and your manager. In my experience, there is much that the typical large

影响你在大公司创新能力的两个最大因素是你的公司文化和你的经理。根据我的经验，典型的大公司可以做很多事情来提高员工创新的能力。

> company could do to improve the ability of their employees to innovate.

那么，如果你发现自己在创新和似乎困难的组织中，你能做什么？以下是几种为你的产品提出创新的技巧：

> So, what can you do if you find yourself in a team and an organization where innovation seems difficult? Here are several techniques for coming up with the innovations your product is looking for:
>
> Innovation via the 20% Rule
>
> Many of you have heard that at Google, engineers get to spend 20% of their time on the projects of their choice. More than 20 years ago, this was also the policy for our team at HP Labs, and we borrowed the idea from Xerox PARC. It worked then and it still works now. At HP Labs, our job was to come up with innovative technologies that the product divisions could then incorporate into commercial products. In the group I was in, we took five new products to market, and only one of them was for a technology that came from above (and that product was the one that actually failed badly in the market). The other four innovations were fruits of the 20% rule.
>
> As much as we might like to think that the great product ideas are the result of great strategic planning, or that they come down from the executive team, in many cases, the best ideas come from the bottom up. What's great about the 20% rule is that lots of ideas can be tried out. And when you inject the feeling of ownership that comes from thinking up the ideas yourselves, the ideas are pursued with more passion and creativity.
>
> If your company has the 20% rule, make sure it applies to product managers and interaction
>
> designers as well as to engineers. Unfortunately, most companies don't have the 20% rule. That's a shame, and I hope you bring this up and that management decides to give it a try. But if not, that's why skunk works was invented…
>
> Innovation via Skunk Works
>
> Skunk works is a very old industry term that originally referred to secret military projects, but today refers to chasing ideas under the radar, unhampered by bureaucracy, on your own time if necessary. Skunk works projects have saved countless large companies.
>
> In large organizations, it's hard to get permission to officially explore ideas. However, once you have proven an idea, it's remarkably easy to get the project funded. So long as you continue to carry out your official job responsibilities, management is usually supportive—many times they'll even pitch in and help.
>
> Just remember that your company likely owns the intellectual property of anything you come up with on the job, so I'm not suggesting that this is how you pursue your new startup idea. If you do decide to chase a skunk works idea, and the result looks good but your company doesn't want to

pursue for whatever reason, you might want to discuss an arrangement where you pursue the idea on your own. Those of you that know your Silicon Valley history may recognize this situation as the birth of Apple Computer when Steve Wozniak's employer HP wasn't ready to enter the personal computer market. Innovation via Observation

One of the easiest ways I know of to innovate is to just watch (and listen) as actual users attempt to use your current product or a competitor's product. Watch a few of these sessions and you'll start to see patterns of frustration and expectation. Watch some more and you'll start to get ideas of how to better meet the needs. Bring in an engineer who is familiar with the available technologies, and together you will start to come up with better ways of solving the problem.

The key is to get the product in front of actual target users, not early adopters, and not anyone from your company (including you). You don't need formal usability testing labs either. You can do this informally, and you can often take the software out to the users (ideally in their native habitat—their office, their home, or the mall).

And you don't just care about whether the software is usable or not. You care about whether or not the software is meeting their needs. Even if it's usable, do they care? What problem do they really need solved?

Remember: innovation is rarely about solving an entirely new problem. More often it is solving an existing problem in a new way. So watching people struggle with their existing solutions is a great way to highlight innovation opportunities.

Innovation via User Experience Design

Another good source of innovation is to step back, relax your technical constraints for a

moment, and consider what the ideal user experience would be for your product. Not just more efficient implementation of tasks, but eliminating tasks altogether. What is really essential, and what is there just because it's a side effect of the way the product is designed and built?

Every system has an implementation model that is the basis for how the product was built. But the user doesn't think this way—he has a conceptual model in mind for how he wants to think about this problem, and what he expects the system to present. Frustration happens when the user is presented not with something that reflects his conceptual model, but instead reflects the implementation model.

When you spot these incongruences, you have identified an opportunity for innovation (or at the very least, an opportunity for significant product improvements).

Innovation via Acquisition

Finally, we do need to talk about innovation via acquisition. Many product managers view an acquisition as a failure on their part. But in truth, acquisition can be an excellent technique for innovation—especially in areas where the risks are high. In essence, the company lets dozens of startups test the waters, try out their ideas, and either succeed or fail. The few remaining companies with products that work out may be good candidates for acquisition. Not only does this sort of acquisition bring in innovative new technologies, but they also bring in new blood with new ideas that you can leverage for your own purposes.

I encourage product managers at large companies to reach out and establish relationships with interesting, related startups. You can often help—and learn from—one another, and the nurturing

> you do might just save your company millions. There are many cases where the company that was acquired did not choose the highest bid, but instead the company that had the people they wanted to work with.
>
> It is important that acquisitions are handled well—as we all know most aren't. And realize that innovation via acquisition is a powerful tool for large companies to keep expanding their offerings and maintaining leadership in their markets.
>
> I hope you'l try out some of these techniques—your company truly does need you to innovate. Peter Drucker said “Every organization needs one core competence: innovation.” Innovation can absolutely happen in large companies. If you're still not convinced, just take a look at www.apple.com/iphone.

通过 20% 规则创新

你们中的许多人听说过，在谷歌，工程师可以花 20% 的时间在他们选择的项目上。20 多年前，这也是我们 HP 实验室团队的政策，我们从 Xerox PARC 借用了这个想法。那时有效，现在仍然有效。在 HP 实验室，我们的工作是想出产品部门可以整合到商业产品中的创新技术。在我所在的团队中，我们把五个新产品推向市场，其中只有一个来自上面的技术（而那个产品实际上在市场上惨败）。其他四项创新是 20% 规则的成果。

尽管我们可能愿意认为伟大的产品想法是伟大战略规划的结果，或者它们来自执行团队，但在许多情况下，最好的想法来自底层。20% 规则的好处是可以尝试很多想法。而且当你注入来自自己思考想法的归属感时，这些想法会以更多的热情和创造力被追求。

如果你的公司有 20% 规则，确保它也适用于产品经理和交互

设计师以及工程师。不幸的是，大多数公司没有 20% 规则。这很遗憾，我希望你能提出这个问题，管理层决定尝试一下。但如果没有，那就是 skunk works 被发明的原因……

通过 Skunk Works 创新

Skunk works 是一个非常古老的行业术语，最初指的是秘密军事项目，但今天指的是在雷达下追逐想法，不受官僚主义的阻碍，必要时在你自己的时间进行。Skunk works 项目拯救了无数大公司。

在大组织中，很难获得正式探索想法的许可。然而，一旦你证明了一个想法，获得项目资金就非常容易。只要你继续履行你的正式工作职责，管理层通常会支持——很多时候他们甚至会参与并帮助。

只要记住，你的公司可能拥有你在工作中想出的任何知识产权，所以我并不是建议这是你如何追求你的新初创公司想法的方式。如果你决定追逐一个 skunk works 想法，结果看起来不错，但你的公司由于某种原因不想追求，你可能想讨论一种安排，让你自己追求这个想法。那些了解硅谷历史的人可能会认出这种情况是苹果电脑的诞生，当时史蒂夫·沃兹尼亚克的雇主 HP 还没有准备好进入个人电脑市场。

通过观察创新

我所知道的最简单的创新方法之一就是观察（和倾听）实际用户尝试使用你当前的产品或竞争对手的产品。看几次这些会议，你就会开始看到沮丧和期望的模式。再看几次，你就会开始想出如何更好地满足需求的想法。带进来一位熟悉可用技术的工程师，你们一起会开始想出更好的解决问题的方法。

关键是把产品放在实际目标用户面前，而不是早期采用者，也不是你公司的任何人（包括你）。你也不需要正式的可用性测试实验室。你可以非正式地进行，而且你通常可以把软件带给用户（最好是在他们的原生环境中——他们的办公室、他们的家或商场）。

而且你不仅仅关心软件是否可用。你关心软件是否满足他们的需求。即使它是可用的，他们在乎吗？他们真正需要解决什么问题？

记住：创新很少是关于解决一个全新的问题。更常见的是以新的方式解决一个现有问题。所以看着人们与他们现有的解决方案挣扎是突出创新机会的好方法。

通过用户体验设计创新

另一个创新的好来源是退一步，放松你的技术约束

片刻，考虑你的产品理想用户体验会是什么。不仅仅是任务的更有效实现，而是完全消除任务。什么是真正必不可少的，什么只是因为产品设计和构建方式的副作用而存在？

每个系统都有一个实现模型，这是产品构建方式的基础。但用户不这样想——他有一个概念模型，用于他如何想要思考这个问题，以及他期望系统呈现什么。当用户呈现的不是反映他的概念模型，而是反映实现模型的东西时，沮丧就发生了。

当你发现这些不一致时，你就识别了一个创新的机会（或者至少是一个显著改进产品的机会）。

通过收购创新

最后，我们确实需要谈谈通过收购创新。许多产品经理将收购视为自己的失败。但事实上，收购可以是一种极好的创新技术——特别是在风险较高的领域。本质上，公司让几十个初创公司试水，尝试他们的想法，要么成功要么失败。少数剩下的产品成功的公司可能是收购的好候选者。这种收购不仅带来了创新的新技术，还带来了你可以用于自己目的的新血液和新想法。

我鼓励大公司的产品经理与有趣的、相关的初创公司接触并建立关系。你们经常可以互相帮助——并从彼此那里学习，你做的培养工作可能会为你的公司节省数百万美元。有很多案例，被收购的公司没有选择最高出价，而是选择了他们想要与之合作的人的公司。

重要的是要处理好收购——正如我们所知，大多数收购处理得不好。并意识到通过收购创新是大公司保持扩展其产品和维持市场领导地位的强大工具。

我希望你会尝试其中的一些技巧——你的公司确实需要你创新。彼得·德鲁克说："每个组织都需要一种核心能力：创新。"创新绝对可以发生在大公司中。如果你仍然不相信，只需看看 www.apple.com/iphone。

第 30 章：在大公司中取得成功

> Chapter 30: Succeeding In Large Companies
>
> Top 10 List

十大清单

> Many of the companies I work with are quite big, and countless product leaders in these companies ask: “How do I get things done in a large company?” I have worked in several large companies and, while it's not easy, I believe that those who figure out how to leverage the considerable resources of their company bring a substantial advantage to their product. For those of you not currently in a large company, as your company grows, you'll likely face these issues too. And if you partner with a big company, you are effectively in the same boat. You'll get more out of the relationship if you understand how these companies work.

我合作的许多公司都相当大，这些公司中无数的产品领导者问："我在大公司中如何把事情做成？"我曾在几家大公司工作过，虽然不容易，但我相信那些弄清楚如何利用公司相当资源的人会为他们的产品带来实质性优势。

> But before we get to the specific techniques for getting your product designed, built and launched—there are two important points to understand:

对于那些目前不在大公司的人来说，随着你的公司成长，你很可能也会面临这些问题。而且如果你与一家大公司合作，你实际上处于同一条船上。如果你理解这些公司如何运作，你会从关系中获得更多。

> First, it's critical to realize that an underlying dynamic in large organizations is that they are generally risk averse. This is not an accident—it's because large companies have much more to lose than smaller companies, and it's one of the biggest cultural changes that comes with success and growth. It's also why it's so much easier to innovate in a small company. So first and foremost, you need to realize that you will have to deal head-on with the many

但在我们进入设计、构建和发布产品的具体技巧之前——有两个重要的点需要理解：

> mechanisms that large companies put in place to protect what they have accumulated. Start by memorizing this paragraph.

首先，至关重要的是要意识到，大组织中的一个潜在动态是它们通常厌恶风险。这不是偶然的——这是因为大公司比小公司有更多损失，这是伴随成功和增长而来的最大文化变革之一。这也是为什么在小公司创新容易得多的原因。所以首先，你需要意识到你将不得不正面处理大公司建立的许多机制，以保护它们积累的东西。从背诵这段话开始。

> Second, many of these same organizations have at least some degree of matrix management and shared resources, where key members of the product team (most often design, engineering, QA,

> site operations, and marketing), are shared resources, and your project needs to secure the necessary people from the pool in order to staff up and create your product. It's not that this organizational design is particularly effective, it's just that this model has significant cost savings over project-oriented approaches (where much like a startup, you assemble a dedicated product team for the life of the project).

其次，这些相同的许多组织至少有某种程度的矩阵管理和共享资源，产品团队的关键成员（最常见的是设计、工程、QA、网站运营和营销）是共享资源，你的项目需要从池中获取必要的人员来组建和创建你的产品。并不是说这种组织设计特别有效，只是这种模式比面向项目的方法有显著的成本节约（很像初创公司，你为项目的生命周期组建一个专门的产品团队）。

> With these points in mind, here is a list of ten techniques for getting things done in a large company:

考虑到这些点，以下是在大公司中把事情做成的十种技巧：

> 1. Learn how decisions are really made in your organization.

1. 了解在你的组织中决策是如何真正做出的。

> Every organization is different. The key is to learn and accept how things get done in your organization. Don't try to change the culture. If you want to succeed in your company, you'll need to embrace it. Learn to love it. And be sure you look closely. Despite any formal decision processes that may exist, don't be surprised if your company requires one key person (or a few) to buy off on any significant decision. If this is the case, at least you know who you really need to convince, and then you can work on the best way to reach that person. And you'll need to learn how that person makes decisions, for example, does she base her decisions on a demo, or market data, or on customer commitments and testimonials?

每个组织都不同。关键是学习并接受在你的组织中事情是如何完成的。不要试图改变文化。如果你想在你的公司成功，你需要拥抱它。学会爱它。并确保你仔细观察。尽管可能存在任何正式的决策流程，但如果你的公司需要一个关键人物（或几个人）对任何重大决策进行认可，也不要感到惊讶。如果是这种情况，至少你知道你真正需要说服谁，然后你可以研究接触那个人的最佳方式。而且你需要了解那个人如何做决策，例如，她是基于演示、市场数据，还是基于客户承诺和推荐来做决策？

> 1. Build relationships before you need them.

1. 在你需要之前建立关系。

> If you prefer to go it alone, you might want to consider a startup, as large companies are all about people working with and depending on each other. You need to figure out all the people across the company that you might have to depend on to get your product designed, built, and launched. It'll probably be a long list. But well before you need the help of these people, you should introduce

> yourself, ask how you can best work with them, and start getting them excited about what you're working on. Try to figure out if there's anything you can do to help them in their job. Make friends.

如果你喜欢独自行动，你可能要考虑一家初创公司，因为大公司都是关于人们相互合作和依赖的。你需要弄清楚公司里所有你可能不得不依赖的人，以让你的产品设计、构建和发布。这可能是一个很长的清单。但在你需要这些人的帮助之前很久，你应该介绍自己，询问如何最好地与他们合作，并开始让他们对你正在做的事情感到兴奋。试着弄清楚你是否能做任何事情来帮助他们完成工作。交朋友。

> 3- Long live skunk works.

1. Skunk works 万岁。

> It really is easier to beg forgiveness than ask permission, especially in larger companies (see the point above about risk aversion). If you have a product idea, you can create a PowerPoint presentation and propose it through the proper channels, but it's all too likely that the idea won't go anywhere. However, if you take that idea—along with a few like-minded friends from across the company—and you flesh the idea out into a prototype, then if the idea is a good one, you'll be stunned at how quickly the resources of the company will line up to help. Countless great products were launched this way. More on this point in the chapter on innovation, but for now, know that your idea will have a much better chance of getting traction if you can actually show the idea works, rather than just talking about it.

乞求宽恕确实比请求许可更容易，尤其是在大公司（参见上面关于厌恶风险的观点）。如果你有一个产品想法，你可以创建一个 PowerPoint 演示文稿并通过适当的渠道提出，但这个想法很可能不会有任何进展。然而，如果你把这个想法——以及公司各地的几个志同道合的朋友——一起充实成一个原型，那么如果这个想法是好的，你会惊讶于公司的资源会以多快的速度排队帮助。无数伟大的产品都是以这种方式发布的。关于这一点的更多内容在关于创新的一章中，但现在要知道，如果你实际上能展示这个想法有效，而不是只是谈论它，你的想法将有更好的机会获得牵引力。

> 1. Just get it done.

1. 只管完成。

> One of the great ironies of large companies is that even though there may be thousands of

大公司的一个伟大讽刺是，即使可能有成千上万的员工，当你需要他们时，往往不可能找到人来帮忙。即使管理层非常愿意和支持，也可能没有合适的资源可用。在这种情况下，你可能需要发挥创造力。例如，你可能能够找到一些资金来聘请承包商，或者调用一些人情，或者你可能不得不自己介入。在具有正式流程和所需交付物的大公司中，介入并自己执行任务或创建交付物可能比与流程抗争更容易。我知道很多案例，产品经理需要帮助完成客户服务、销售培训、技术写作、QA、工程和营销的交付物。你必须愿意做任何需要做的事情。

> employees, it can often be impossible to get someone to help when you need them. Even when management is very willing and supportive, there may be no suitable resources available. In this

> case, you may need to get creative. You might, for example, be able to find some funding for a contractor, or call in some favors, or you might have to pitch in yourself. Ina large company with formal processes and deliverables required, it may be easier to step in and perform the task or create the deliverable yourself rather than fight the process. I know of many cases where the product manager needed to help out with deliverables for customer support, sales training, technical writing, QA, engineering, and marketing. You have to be willing to do whatever it takes.

1. 选择你的战斗。

> 1. Pick your battles.

在大组织中最有效的人的朋友远多于敌人。在大公司把事情做成并不容易，会有很多你有充分理由感到沮丧的情况，但你需要仔细选择你的战斗。确保你选择值得为之奋斗的东西，结果真的很重要，这样如果烧毁了任何桥梁，那也是值得的。当你战斗时，确保你是为你的产品而战，而不是针对另一个人。试着让公司与你同行，不要把你的对手逼到墙角。你不想赢了战斗却输了战争。

> The most effective people in a large organization have far more friends than enemies. Getting things done in a big company isn't easy, and there will be many situations where you'll have good reason to be frustrated, but you need to pick your battles carefully. Make sure you pick something worth fighting for, where the outcome truly matters, so that if any bridges are burned it's worth it. And when you do fight, make sure you're fighting for your product and not against another person. Try to bring the company along with you and not back your adversaries into a corner. You don't want to win the battle only to lose the war.

1. 在需要决策的重要会议之前建立共识。

> 1. Build consensus before important meetings where decisions are required. Always keep in mind than once someone opposes your position in a broad and public way, you have a major problem. It will not be easy for that person to publicly switch positions. In the long run, it takes much less time to build consensus beforehand when the outcome is important. In a large company, the main value of these decision meetings is for everyone to

永远记住，一旦有人以广泛和公开的方式反对你的立场，你就有大问题了。让那个人公开转换立场并不容易。从长远来看，当结果很重要时，事先建立共识花费的时间要少得多。在大公司中，这些决策会议的主要价值是让房间里的其他人都表明他们对你的产品或决策的支持。因此，确保在任何大型会议（或发送重要电子邮件）之前，你花时间与每个将要参加的人一对一交谈，以确保他们有机会向你私下表达任何顾虑。然后你可以直接解决他们的顾虑，并让他们上船并愿意公开表明这一点。

> see everyone else in the same room indicate their support for your product or decision. So make sure that before any big meeting (or before sending out an important e-mail), you take some time to talk one on one with each person who will attend to ensure they have a chance to privately voice

> any concerns to you. You can then address their concerns directly, and get them on board and willing to indicate this publicly.

1. 聪明地花费你的时间。

> 1. Be smart about how you spend your time.

在大公司中，很容易陷入一周不间断的会议。在周末，你会从一个会议赶到另一个会议，熬夜试图跟上你的电子邮件，但你实际上并没有做那些会对你的产品产生真正影响的事情。无情地分流你的会议。参加你必须参加的会议，但要习惯相信你的同事会做好他们的工作，并知道如果有什么真的需要你的关注，他们会让你知道。最重要的是，确保你在一周内有时间处理对产品成功至关重要的事情：你的产品战略、你的路线图、下一个发布的当前原型、你对竞争的理解——特别是——与实际用户和客户交谈。

> It is all too easy in a big company to get sucked into a week full of non-stop meetings. At the end of the week, you will have rushed from meeting to meeting and stayed up late trying to keep up with your e-mail, but you will not have actually done the things that will make a real difference to your product. Triage your meetings ruthlessly. Attend the meetings you must, but get used to trusting your colleagues to do their jobs and know that they'll let you know if something really needs your attention. Most importantly, make sure you have the time during the week to work on the items crucial to the success of your product: your product strategy, your roadmap, the current prototype of the next release, your understanding of the competition and—especially—talking to actual users and customers.

1. 分享信息。

> 1. Share information.

沟通在任何组织中都很困难。然而，在大公司中，沟通是一个严峻的挑战，信息成为一种货币。不幸的是，许多人确实把它当作货币，囤积而不是自由分享。不要采取信息就是力量的观点。你通过分享它获得更多，希望其他人也会回报并帮助你保持信息灵通。所以你能做的任何事情，通过在你得到它时尽快提供有用的信息来帮助你的同事，对你和你的公司都有好处。

> Communication is hard in any organization. In a large company, however, communication is a serious challenge, and information becomes a kind of currency. Unfortunately, many people actually do treat it like currency, and hoard it rather than sharing it freely. Don't take the view that information is power. You have more to gain by sharing it, and hopefully others will reciprocate and help keep you informed as well. So anything you can do to help your colleagues by providing useful information as soon as you get it is good for you—and

1. 让你的经理为你工作。

> good for your company.

在大公司中，你的经理可以为你的成功带来很大不同。假设你的经理受到相当好的评价，你应该利用他或她的关系，并使用你的经理来更好地了解公司和管理链。通过做你的功课并提供他或她需要的信息来向其他人证明你的情况，让你的经理轻松。确保你的经理知道他或她可以信任你与组织的各级人员交谈。

> 1. Put your manager to work.

1. 布道！

> Ina large company, your manager can make a big difference to your success. Assuming your manager is reasonably well regarded, you should leverage his or her relationships, and use your manager to get a better understanding of the company and your management chain. Make it easy for your manager by doing your homework and providing the information he or she needs to make your case to others. Make sure your manager knows he or she can trust you talking to all levels of the organization.

在大公司中，布道的需求从未停止。你需要不断传播消息，解释愿景和战略，演示原型，分享客户反馈。不要低估这种内部销售功能的重要性。确保每个甚至与你的产品稍有联系的人都理解为什么这个产品很重要，以及他们如何能帮助。

> 1. Evangelize!

虽然不可否认，克服内部障碍并让大公司的相当资源集中在你的产品上是困难的，但好处可能是巨大的。你将获得媒体、行业分析师、合作伙伴、客户和用户的高度关注，这是小公司无论如何都买不到的。所以学习如何充分利用公司的资产肯定对你有好处。

> Ina large company, the need to evangelize never stops. You need to continuously spread the word, explain the vision and strategy, demo the prototype, and share customer feedback. Don't underestimate the importance of this internal sales function. Make sure everyone even remotely connected with your product understands why the product is important, and how they can help.

关于这个话题，我的朋友大卫·韦登有一句很好的结束语，我认为很好地总结了大公司许多人的情况和机会："大多数人四处游荡，在黑暗中抱怨天黑了，而不是学习开关在哪里。"

> While it is undeniably hard to overcome the internal obstacles and get the considerable resources of a large company focused on your product, the benefits can be tremendous. You will get a level of attention from the press, industry analysts, partners, customers, and users that the small company can't buy at any price. So it definitely pays for you to learn how to use the assets of your company to the fullest.
>
> A great closing quote on this topic from my friend David Weiden, which I think sums up
>
> nicely the situation and opportunity for many people in large companies: “Most people wander around in the dark and bitch about it being dark, instead of learning where the light switches are.”

> I'M TRANSFERRING 4 THEN WE CAN
> YOu TO THE SALES ie MAKE USELESS

寻找令人启发的产品

> DEPARTMENT, 3] | prooucts ANO

究竟是什么让产品令人启发？在本节中，我们考虑令人启发和成功产品的特征，并讨论创建它们的关键。

> SCAPEGOAT. 5 BLAME YOU FOR
> OUR LOW SALES. ww, AA hal oS fe, || shed a5y Wh 2) lie AG re i TAN [xac\
>
> #] (WOULDN'T IT BE
>
>]] BETTER TO MAKE
> 3] (6000 pRooucts?
> j TIN A PERFECT
> 3 WORLD. ger Hain ae VEZ DILBERT: © Scott Adams/Dist. by United Feature Syndicate, Inc.
> In Search Of Inspiring Products
> What exactly makes a product inspiring? In this section we consider the characteristics of inspiring and successful products, and discuss the keys to creating them.

第 31 章：从苹果学到的教训

> Chapter 31: Lessons From Apple
> A Different Type Of Hardware Company

一种不同类型的硬件公司

> Ihave to admit to a strong bias up front: I love Apple. I think they're responsible for some of the best technology products our industry has produced in the past 25 years, and I have been a fan of the company ever since the Lisa (which I consider a prototype for the Mac) was introduced to the public in 1983. I view Steve Jobs as one of the best product managers of all time.

我必须承认有一个强烈的偏见：我爱苹果。我认为它们负责我们行业在过去 25 年中生产的一些最好的技术产品，自从 Lisa（我认为是 Mac 的原型）于 1983 年向公众推出以来，我一直是该公司的粉丝。我认为史蒂夫·乔布斯是有史以来最好的产品经理之一。

最近，苹果 iPhone 首次亮相，他们再次重新定义了行业。但当我和人们谈论那个产品，以及苹果一般时，我 struck 于对于是什么促成了他们的成功，有多少不同的意见。

我强烈反对那些将他们的成功归因于营销能力的人（尽管我认为他们在营销方面相当不错）。

为了解释，让我们看看 iPhone。不那么多关于它的具体细节，而是为什么我认为苹果能够持续重新定义主要消费市场——无论是个人电脑、数字音乐播放器还是手机。

> Recently, the Apple iPhone made its debut, and once again they have redefined the industry. But when I talk to people about that product, and Apple in general, I'm struck by how many different opinions there are as to what accounts for their success.

从苹果可以学到很多东西，但对我来说，有三个更高层次的教训：

1. 硬件服务于软件

与几乎所有其他硬件公司不同，苹果理解硬件的作用是服务于软件，而不是反过来。软件需要知道用户希望手机做什么，所以发明了多点触控显示器、加速计和接近传感器等硬件技术来实现这一点。每一项技术都有其目的。也就是说，虽然硬件和软件技术确实令人印象深刻，但苹果理解，一旦你超越了早期采用者，那不是人们关心的东西。这引出了下一点……

> I strongly disagree with those who attribute their success to marketing prowess (although I think they're quite good at marketing).

1. 软件服务于用户体验

如今，几乎每个消费品公司都口头上说用户体验，但苹果是认真的。可用性、交互设计、视觉设计、工业设计，都是公司优先事项的重中之重——而且这一点显而易见。可能需要两年半的时间来想出 iPhone，但团队知道，一切都是关于用户体验的，他们知道他们必须移山倒海才能创造出色的体验。此外，他们在公司的各个层面都有人才和毅力来实现这一点。这与微软努力为 Vista 做出即使是微不足道的——而且早就应该做的——用户体验改进的现在著名的例子形成对比。然而，尽管用户体验是基础，但苹果理解……

> To explain, let's take a look at the iPhone. Not so much about the specifics of it, but why I think Apple is able to consistently redefine major consumer markets—whether personal computers, digital music players, or cell phones.

1. 用户体验服务于情感

> There is a great deal to learn from Apple, but to me there are three higher-order lessons:

如果苹果作为一家技术公司有秘密武器，我相信就是这个：他们比任何人都更理解情感在让消费者渴望、购买和热爱产品方面所扮演的角色。他们知道如何创造与消费者这些情感对话的产品。人们渴望 iPhone。400 美元买一部手机？没问题，因为消费者不会将 iPhone 与 Razr 或 Treo 进行比较——它

处于完全不同的级别。看看机场休息室周围——人们对待他们的 PC 像租赁车，但他们呵护他们的 Mac 像它是他们的梦想车。而且，如果你足够勇敢，试着把一个青少年的 iPod 从他身边拿走。

> 1. The Hardware Serves the Software

有一百多种不同的手机可供选择，但很难找到真正热爱自己手机的人。他们对处理几十年没有改进的语音邮件系统、不兼容的通讯录、无法使用的网络浏览器和电子邮件 hack 感到沮丧。苹果带着一款直接与这些未满足需求对话的产品出现了。数字音乐播放器也发生了同样的事情。

> Unlike virtually every other hardware company, Apple understands that the role of the hardware is to serve the software, and not the other way around. The software needs to know what the user wants the phone to do, so hardware technologies like multi-touch displays, and accelerometer and proximity sensors are invented to enable this. Every technology is there for a purpose. That said, while the hardware and software technology are truly impressive, Apple understands that once you get beyond the early adopters, that's not what people care about. Which leads to the next point...

令我惊讶的是，有多少公司没有理解这些观点。即使是许多只是试图复制的公司，也只想到复制功能，而没有复制真正重要的东西。

> 1. The Software Serves the User Experience
>
> Almost every consumer company out there today gives lip service to the user experience, but Apple means it. Usability, interaction design, visual design, industrial design, are all front and center in the company's priorities—and it shows. It may have taken two-and-a-half years to come up with the iPhone, but the team knew that it was all about the user experience, and they knew they had to move mountains to make the experience great. In addition, they had the talent and persistence at all levels of the company to make this happen. Contrast this with the now famous example of Microsoft's effort to make even a very minor—and long overdue—user experience improvement in Vista. However, as fundamental as the user experience is, Apple understands that...
>
> 3- The User Experience Serves the Emotion
>
> If Apple has a secret sauce as a technology company, I believe it's this: They understand better than anyone else the role that emotion plays in getting consumers to crave, buy, and love a product. They know how to create products that speak to these emotions in consumers. People are craving the iPhone. $400 for a phone? No problem, because consumers aren't comparing the iPhone to a Razr or a Treo—it's in an entirely different league. Take a look around an airport lounge—people treat their PC like a rental car, but they coddle their Mac like it's their dream car. And, if you're brave enough, just try to take a teen's iPod away from him.
>
> There are well over a hundred different cell phones available, but it's hard to find people that actually love their phone. They get frustrated dealing with voice mail systems that haven't improved in decades, incompatible address books, unusable Web browsers, and e-mail hacks. Apple comes along with a product that speaks directly to these unmet needs. The same thing happened with digital music players.

> It's amazing to me how few companies get these points. Even the many companies that are just trying to copy, only think to copy the functionality, and don't copy what's really important.

第 32 章：警惕特殊需求

> Chapter 32: Beware Of Specials
>
> Don't Fall Down This Slippery Slope
>
> How many times have you seen the situation where a sales rep brings to the CEO a proposal from a prospect that says, “If you will just add these seven features to your product, then we'll buy your software.” Or, lest anyone thinks that this situation is unique to enterprise software companies, for consumer internet service companies, your ad salesperson comes over and says that “A big prospective partner will sign a seven-figure advertising and sponsorship deal with us if you'll just agree to these site integration and placement requirements.”

不要滑下这个 slippery slope

> Either way, these are what are known as specials. A special is when a company gets a big check from a prospective customer or partner with the condition that you build into your product exactly what they say.

你有多少次看到过这样的情况：销售代表向 CEO 提出一个来自潜在客户的提议，说"如果你能在你的产品上添加这七个功能，我们就会购买你的软件"。或者，以免有人认为这种情况对企业软件公司是独有的，对于消费者互联网服务公司，你的广告销售人员过来说"一个大的潜在合作伙伴将与我们签订七位数的广告和赞助协议，只要你同意这些网站整合和展示位置要求"。

> It is entirely understandable why large customers and partners may seek this arrangement. And if you're a small company that's strapped for cash, it's also very understandable that your CEO might be more than a little inclined to agree. After all, you want to be “marketdriven” and you're probably going to be adding these features at some point anyway, so why not let the customer underwrite them?”

无论哪种方式，这些就是所谓的特殊需求。特殊需求是指公司从潜在客户或合作伙伴那里获得一张大支票，条件是你按照他们说的确切内容构建到产品中。

> So what's wrong with doing a special? One of the surest ways to derail a product company is to confuse customer requirements with product requirements.

大型客户和合作伙伴寻求这种安排是完全可理解的。如果你是一家资金紧张的小公司，你的 CEO 可能非常倾向于同意，这也是完全可以理解的。毕竟，你想"以市场为导向"，而且你可能无论如何都会在某个时候添加这些功能，所以为什么不让客户为它们提供资金呢?

> I've talked in several chapters about the reasons why you can't count on customers to describe the product that they need, but to summarize: first, it's extremely difficult for the customer to know what he needs until he sees it; second, customers don't know what's possible; and third, customers don't often interact with each other in order to identify common themes.

那么做特殊需求有什么不好？让产品公司脱轨的最可靠方法之一就是将客户需求与产品需求混淆。

> But, more generally, even if the customer doesn't have these issues, it's not clear that these are the best things to focus on right now. By pursuing these special features now, what important work are you delaying? What is the business cost of that delay?

我在几章中谈到了为什么你不能指望客户描述他们需要的产品的原因，但总结一下：首先，客户在看到之前很难知道他需要什么；其次，客户不知道什么是可能的；第三，客户不经常相互交流以识别共同主题。

> Assuming these are not issues, specials are still dangerous. How come? Because your job is to meet the needs of a broad range of customers—that's what distinguishes product companies from customer software shops. If a year from now the market changes, you need to be able to quickly change and adapt. If you are contractually obligated to keep supporting a specific way of doing things, then your business will not be as nimble as it needs to be. Remember that every version of your product will have to be built, maintained, tested, released, documented, and supported. It doesn't take too many specials to weigh down a company to the point where it takes them months to do even the smallest release.

但，更一般地说，即使客户没有这些问题，也不清楚这些是现在关注的最佳事情。通过现在追求这些特殊功能，你在推迟什么重要工作？这种延迟的业务成本是什么？

> Don't get me wrong—there's nothing wrong with custom software shops. They provide an essential service for countless companies that need specialized solutions, and can often

假设这些不是问题，特殊需求仍然是危险的。为什么？因为你的工作是满足广泛客户的需求——这就是产品公司与客户软件店的区别。如果一年后市场变化，你需要能够快速变化和适应。如果你在合同上有义务继续支持特定的做事方式，那么你的业务就不会像它需要的那样灵活。记住，你的产品的每个版本都必须构建、维护、测试、发布、记录和支持。不需要太多特殊需求就能让一家公司沉重到连最小的发布都需要几个月的程度。

> deliver that specialized solution in a fraction of the time and at a fraction of the cost of inhouse developed solutions. But custom software is a very different business than commercial product software.

不要误会我的意思——定制软件店没有什么不对。它们为需要专业解决方案的无数公司提供必不可少的服务，而且通常可以在一小部分时间和成本内交付该专业解决方案

> So how do you avoid the pitfalls of specials? Undeniably, it takes corporate discipline to be able to recognize specials for what they are and be willing to walk away. This leadership comes from the CEO, but there is much you can do as product manager to help.

内部开发的解决方案。但定制软件与商业产品软件是非常不同的业务。

> First, it is natural for any customer to want to describe their problem in terms of the solution they can envision rather than the underlying problem itself. But as product manager it's your job to work with the customer to tease out the core issues and needs. You can help them recognize that there may be other approaches to this problem that provide a solution they would like even better. Most customers do not want to be running on a custom version of software. They want to be running on your mainstream product—the one that gets the most attention, support and improvement.

那么如何避免特殊需求的陷阱？不可否认，需要公司纪律来能够识别特殊需求的本质并愿意走开。这种领导力来自 CEO，但作为产品经理，你可以做很多事情来帮助。

> Second, consider looking at how you could keep your product general purpose but allow the product to be tailored/customized/ extended by the customer or by a solutions provider. And then have ready the names of a couple of system integrator/solution provider companies that can tailor your product to meet this specific need. You may need to partner with the solution provider so that your customer doesn't have to manage two relationships and have to worry about finger pointing if there are issues.

首先，任何客户自然都希望用他们能想到的解决方案来描述他们的问题，而不是根本问题本身。但作为产品经理，你的工作是与客户合作，梳理出核心问题和需求。你可以帮助他们认识到，可能有其他方法来解决这个问题，提供一个他们更喜欢的解决方案。大多数客户不想运行定制版本的软件。他们想运行你的主流产品——那个获得最多关注、支持和改进的产品。

> So far, my examples have mainly been in the enterprise software space. But the problem of specials is becoming increasingly severe in the consumer Internet services space where—for

其次，考虑如何保持你的产品通用，但允许产品由客户或解决方案提供商定制/定制/扩展。然后准备好几个系统集成商/解决方案提供商公司的名称，他们可以定制你的产品以满足这一特定需求。你可能需要与解决方案提供商合作，这样你的客户就不必管理两种关系，也不必担心如果有问题会互相指责。

> many companies—advertising that is not aligned with the site's objectives has significantly distracted or even damaged the user experience.

到目前为止，我的例子主要在企业软件领域。但特殊需求的问题在消费者互联网服务领域变得越来越严重，在那里——对于许多公司——与网站目标不一致的广告已经显著分散甚至损害了用户体验。

> For many advertisers, the main objective is very simply to move traffic from your site to their site. If this isn't your goal, you've gota strategic conflict. For some sites—such as directories or search

> engines—this is fine, but for others, you end up trading short-term traffic for your site's future. This really isn't in anyone's best interest.

对于许多广告商来说，主要目标非常简单，就是把流量从你的网站转移到他们的网站。如果这不是你的目标，你就有一个战略冲突。对于一些网站——比如目录或搜索引擎——这很好，但对于其他网站，你最终会用你网站的未来换取短期流量。这真的不符合任何人的最佳利益。

> Ihave found advertisers to be willing and interested in finding better, more synergistic ways to work together. When you have a strategy—and a clear role for them—they are more than willing to work with you. They know that old-style Internet advertising is of limited value, and they want something better as much as you do.

我发现广告商愿意并有兴趣找到更好、更协同的合作方式。当你有一个战略——以及为他们明确的角色时——他们非常愿意与你合作。他们知道老式的互联网广告价值有限，他们想要更好的东西，就像你一样。

> Whether it's enterprise software or consumer Internet services, it's the product manager's job to ensure that you're building the right product, and that the product will be applicable to and usable by a wide range of customers.

无论是企业软件还是消费者互联网服务，产品经理的工作是确保你正在构建正确的产品，并且该产品将适用于广泛范围的客户并为他们所用。

> @wuart aBout REQUIREMENTS MANAGEMENT TOOLS?

@wHatT 关于需求管理工具?

> There are several vendors that offer a category of tool called “requirements management software.” These tools are aimed at helping product managers and product marketing gather, track, prioritize and report customer requirements.

有几家供应商提供一类称为"需求管理软件"的工具。这些工具旨在帮助产品经理和产品营销收集、跟踪、优先排序和报告客户需求。

> While these tools have been offered for several years, I rarely find them in use, but I do sometimes get asked about these tools and if I recommend them in general.

虽然这些工具已经提供好几年了，但我很少发现它们被使用，但我确实有时会被问到这些工具以及我是否一般推荐它们。

> While I am all for tools that help people do their jobs better, the problem is that, while these tools are well intentioned, they make it extremely easy to fall into the trap of confusing customer requirements with product requirements.

虽然我支持帮助人们更好完成工作的工具，但问题是，虽然这些工具用心良苦，但它们使陷入将客户需求与产品需求混淆的陷阱变得极其容易。

> In the worst case, they institutionalize the misguided practices that are responsible for so many bad products.

在最坏的情况下，它们使导致如此多糟糕产品的误入歧途的做法制度化。

> In the best case, they can help automate what most product managers already do with Word, Excel or a project Wiki.

在最好的情况下，它们可以帮助自动化大多数产品经理已经用 Word、Excel 或项目 Wiki 做的事情。

> The thing is, in most cases the Wiki works just great, so there's little need for expensive software with its own learning curve that just ends up distracting you from the real thinking that must go on in order to come up with a winning product.

问题是，在大多数情况下，Wiki 工作得很好，所以几乎没有必要使用昂贵的软件，它有自己的学习曲线，最终只会分散你对想出成功产品所必须进行的真正思考的注意力。

第 33 章：新的旧事物

> Chapter 33: The New Old Thing
> What Is Possible Is Constantly Changing

什么是可能的在不断变化

> With apologies to one of my favorite authors Michael Lewis (see The New New Thing), in this chapter I talk about a common misconception among product managers and companies in general.

向我最喜欢的作者之一迈克尔·刘易斯道歉（参见《新新事物》），在本章中，我谈论产品经理和公司普遍存在的一个常见误解。

> Many companies believe they need to create an entirely new market in order to do something big. The media helps fuel this. Everyone wants to know: “What's going to be the next new thing?”

许多公司认为他们需要创造一个全新的市场才能做大。媒体助长了这一点。每个人都想知道："下一个新事物是什么？"

> While it's always fun to speculate on what the next new big thing is, much more often than not, the next big thing is not something altogether new, but rather a new incarnation of something old. The difference is that the new product does it so much better, faster, and/ or cheaper that they end up redefining their category.

虽然猜测下一个新大事总是很有趣，但更多时候，下一个大事不是全新的东西，而是旧事物的新化身。区别在于新产品做得更好、更快和/或更便宜，以至于它们最终重新定义了它们的类别。

让我们看一些例子：当谷歌进入搜索市场时，许多人嘲笑，因为他们认为市场已经成熟，已经有几十个搜索引擎（还记得 AltaVista、Infoseek 和 Snap 吗？）。区别在于谷歌实际上提供了有用的结果。始终如一。以至于他们很快开始定义该类别。

> Let's look at some examples: When Google entered the search market, many people scoffed because they considered the market mature, with dozens of search engines already out there (remember AltaVista, Infoseek, and Snap?). The difference was that Google actually provided useful results. Consistently. So much so that they soon came to define the

同样，当苹果推出 iPod 时，市场上已经有超过一百种 MP3 播放器，但产品如此出色，以至于他们迅速重新定义了该类别。

> category.

聪明的公司在成熟市场中创造成功产品有两种关键方法。

> Similarly, while there were literally over a hundred MP3 players on the market when Apple introduced the iPod, the product was so much better that they quickly redefined the category.

首先，他们理解他们的目标市场以及当前产品在哪些方面不足。产品可用性测试是我最喜欢的技术，你可以用竞争对手的产品以及你自己的产品来做这个。

> There are two key methods that smart companies use to create winning products in mature markets.

其次，伟大的产品领导者知道，现在什么是可能的总是在变化。新技术启用以前不可能或不可行的新解决方案。不断掌握相关技术并考虑如何应用它们来帮助你面临的问题并不容易，但它可以为你的产品带来所有不同。

> First, they understand their target market and where the current products fall short. Product usability testing is my favorite technique for doing this, and you can do this with your competitor's products in addition to your own.

记住：伟大的产品经理将令人向往的东西与刚刚可能的东西结合起来。苹果和谷歌理解这一点。产品机会无处不在，几乎在每个市场。但你必须识别需求，然后寻找应用技术解决问题的新方法。

> Second, great product leaders know that what is now possible is always changing. New technologies enable new solutions that may not have been possible or feasible until now. It is not easy to constantly stay on top of relevant technologies and consider how they might be applied to help solve the problems you face, but it can make all the difference for your product.

创新终结了吗?

> Remember: great product managers combine what is desirable with what is just now possible. Apple and Google understand this. Product opportunities exist everywhere, in virtually every

> market. But you must identify the need, and then search for new ways of applying technology to solve the problem.

最近，我正在接受一位媒体成员的采访，采访是关于硅谷的未来，我被问到这个问题："你认为还有什么好机会吗？"我花了一分钟才意识到他是认真的。整个概念对我来说似乎如此陌生，特别是因为我个人现在看到的机会比以往任何时候都多。

> the End Of Innovation?

但这个问题让我思考为什么我如此强烈地相信这一点。对于任何有任何怀疑的人，我会说三件事：

> Recently I was doing an interview with a member of the press having to do with the future of Silicon Valley, and I was asked this question: “Do you think there are any good opportunities left?” It took me a minute to realize that he meant this as a serious question. The whole concept seems so foreign to me, especially since I personally see more opportunity now than Ihave ever seen before.

首先，只要有产品让你发疯，就有人会做得更好的机会。一部不会掉线的手机怎么样？一台你父母实际上可以在没有你的帮助下管理的家庭电脑怎么样？

> But the question caused me to think about why I believe this so strongly. For anyone that has any doubts at all, I'd say three things:

其次，什么是可能的总是在变化。仅仅因为今天不可行并不意味着明天也不会。

> First, as long as there are products that drive you nuts, there are opportunities for someone to do it better. How about a cell phone that doesn't drop calls? How about a home computer that your parents can actually administer without your help?

第三，今天的应用是明天的基础。这就是我们行业的运作方式。最初，浏览器是一个应用程序，用于查看网站上的内容。今天互联网是一个基础，支持 eBay、Skype 和 PayPal 等应用。

> Second, what is possible is always changing. Just because something isn't feasible today doesn't mean it won't be tomorrow.

同样，直到最近，大多数人把 Facebook 视为社交网络应用，

> Third, today's applications are tomorrow's foundation. That's how things work in our business. Initially, the browser was an application to look at some content on a Web site. Today the Internet is a foundation enabling applications like eBay, Skype, and PayPal. Likewise, until recently, most people viewed Facebook as a social networking application,

但现在它是一波新互联网应用的基础。

> but nowit is a foundation for a new wave of Internet applications. There's no way that all the good opportunities are gone. In fact, there are more products I'd like to be working today than ever before.

所有好的机会都不可能消失。事实上，我今天想做的工作比以往任何时候都多。

第 34 章：恐惧、贪婪和欲望

> Chapter 34: Fear, Greed And Lust
>
> The Role Of Emotion In Products

情感在产品中的作用

> I find it ironic that so many of us in the product world come from science- and business-oriented backgrounds, yet such a large part of what we do every day is really all about emotion and human psychology. Most of us may not think of our job this way, but we should.

我发现具有讽刺意味的是，我们产品界的许多人来自科学和商业背景，但我们每天所做的很大一部分实际上是关于情感和人类心理学的。我们大多数人可能不会这样看待我们的工作，但我们应该这样看待。

> People buy and use products largely for emotional reasons. The best marketing people understand this, and the best product people ensure that their products speak to these emotions.

人们购买和使用产品主要是出于情感原因。最好的营销人员理解这一点，最好的产品人员确保他们的产品与这些情感对话。

> In the enterprise space, the dominant emotion is generally fear or greed. If I don't buy this product, my competitors will beat me to market, hackers will penetrate my firewalls, or my customers will desert me. Or, if I do buy this product, I will make more money, save more money, or stop spending so much money.

在企业领域，主导情感通常是恐惧或贪婪。如果我不买这个产品，我的竞争对手会击败我进入市场，黑客会穿透我的防火墙，或者我的客户会抛弃我。或者，如果我确实购买了这个产品，我会赚更多的钱，节省更多的钱，或者停止花这么多钱。

> In the consumer space, the dominant emotions get more personal. If I buy this product or use this Web site, I will make friends (loneliness), find a date (love or lust), win money (greed), or show off my pictures or my taste in music (pride).

在消费领域，主导情感变得更加个人化。如果我购买这个产品或使用这个网站，我会交朋友（孤独），找到约会对象（爱或欲望），赢得金钱（贪婪），或者展示我的照片或我的音乐品味（骄傲）。

> You may not have thought about your product or service in these terms before, but if you apply this emotional lens, you can start to view things much more in line with how your users and customers view your service—and potential competitors. Where else can they go to get these needs met? What

> could be done to the visual design to speak more directly to these emotions? What features can we provide that speak more directly to these emotions? What features get in the way of clearly speaking to these emotions?

你可能以前没有用这些术语思考过你的产品或服务，但如果你应用这种情感视角，你可以开始更符合你的用户和客户看待你的服务——以及潜在竞争对手的方式。他们还能去哪里满足这些需求？可以对视觉设计做些什么来更直接地与这些情感对话？我们可以提供什么功能来更直接地与这些情感对话？什么功能妨碍了清晰地向这些情感传达信息？

> Keep in mind also that different types of users may bring different emotional needs to the table. An eBay power seller is not the same as a buyer looking for a great bargain, or a buyer looking for the thrill of competing with others to “win” an item.

还要记住，不同类型的用户可能会带来不同的情感需求。eBay 的强力卖家与寻找便宜货的买家，或与他人竞争"赢得"物品的买家寻找刺激是不一样的。

> When you do prototype testing with your target users, after you determine whether or not the test subject can actually figure out how to use the product or service, you should take the opportunity to essentially do a one-on-one focus group to try to learn what emotion is driving this user, and how well your product meets that emotional need.

当你用目标用户进行原型测试时，在确定测试对象是否能真正弄清楚如何使用产品或服务之后，你应该利用这个机会基本上做一个一对一的焦点小组，试图了解是什么情感驱动着这个用户，以及你的产品满足该情感需求的程度。

> You can hopefully see why user experience design (interaction and visual design) and usability testing play such a key role in coming up with a winning product.

你希望能看到为什么用户体验设计（交互和视觉设计）和可用性测试在想出成功产品中发挥如此关键的作用。

> Once you have clearly identified and prioritized the dominant buying emotions your customers bring to your product, focus on that emotion and ask yourself where else they might be able to get that need met? That's your real competition. In many cases you'll find that the competition you should be worrying about is not the startup or big portal that's after the same thing you are, but rather the offline alternative.

一旦你清楚地识别并优先考虑了你的客户带给你的主导购买情感，就专注于那种情感，问问自己他们还能在哪里满足这种需求？那是你真正的竞争。在许多情况下，你会发现你应该担心的竞争不是追求与你相同目标的初创公司或大门户网站，而是线下替代方案。

第 35 章：情感采用曲线

> Chapter 35: The Emotional Adoption Curve
> An Interview With Jeff Bonforte

杰夫·邦福特访谈

> In his book, Crossing the Chasm, Geoffrey Moore introduces the powerful notion of a technology adoption curve, comprised of innovators and early adopters, followed by the early majority, the late majority and, finally, the laggards. He goes on to explain how few products get beyond the early adopters (they fall into the chasm).

在他的书《跨越鸿沟》中，杰弗里·摩尔介绍了技术采用曲线的有力概念，由创新者和早期采用者组成，然后是早期多数、晚期多数，最后是落后者。他接着解释了有多少产品无法超越早期采用者（它们掉入鸿沟）。

> Jeff Bonforte—as of this writing an exec at Yahoo! responsible for several industry-leading products used by millions—argues for adding a layer of analysis to the technology adoption curve, based on the driving emotions of the users in each group.

杰夫·邦福特——截至本文撰写时，他是雅虎！的高管，负责数百万用户使用的几个行业领先产品——主张为技术采用曲线增加一层分析，基于每组用户的驱动情感。

> In this interview, Jeff shares his views on the role of emotions in product development. Marty: Why do you like to focus on anger?

在这次采访中，杰夫分享了他对情感在产品开发中作用的看法。

> Jeff: Because angry people dictate the future of technology.

马蒂：你为什么喜欢关注愤怒?

> I like my product managers to focus on the most miserable thing people have to deal with everyday. If you can solve that problem, that actually changes behavior, and that can lead to

杰夫：因为愤怒的人决定技术的未来。

> the truly big product wins.

我喜欢我的产品经理关注人们每天必须处理的最痛苦的事情。如果你能解决这个问题，那实际上会改变行为，这可能导致真正的大产品胜利。

> Don't focus on the technology of your product, just think about the people that you're trying to help. What are the problems they're dealing with? What are the things that they're frustrated with?

不要关注你产品的技术，只要想想你试图帮助的人。他们正在处理什么问题？他们对什么事情感到沮丧？

> For example, every single one of us hates to travel nowadays—its just miserable. It's almost as if it's engineered for misery from start to finish. Or, we hate our telephone company—it's almost impossible not to. They send us a bill that's so complicated and so structured to screw you over. The rules are so elaborate that almost nobody understands them. The result is you feel like the entire billing process was engineered from the start to screw you over, and you're in a monthly battle to figure out how not to get screwed.

例如，我们每个人都讨厌现在的旅行——只是痛苦。几乎就像它是从头到尾为痛苦而设计的。或者，我们讨厌我们的电话公司——几乎不可能不讨厌。他们给我们寄来的账单如此复杂，结构如此复杂以至于要坑你。规则如此复杂，几乎没有人理解。结果是你觉得整个计费过程从一开始就是为坑你而设计的，而你正处于每月的战斗中，试图弄清楚如何不被坑。

> In my view, far too many product managers talk in terms of features and technology, and we don't really talk in terms of the user's core needs or emotions.

在我看来，太多产品经理用功能和技术来谈论，而我们并没有真正用用户的核心需求或情感来谈论。

> Marty: Let's go back to the technology adoption curve. What do you see as the underlying emotions and needs driving each of the groups of users?

马蒂：让我们回到技术采用曲线。你认为驱动每组用户的潜在情感和需求是什么？

> Jeff: In the Technology Adoption model, we're told that there's a technology adoption curve, and it's nice and clean. But there's also a chasm, and maybe there's a tornado in there too. But, what does that mean exactly? What are we supposed to do to design around these things?

杰夫：在技术采用模型中，我们被告知有一个技术采用曲线，它很好很干净。但也有一个鸿沟，也许里面还有一个龙卷风。但这到底意味着什么？我们应该做什么来围绕这些东西进行设计？

> Instead of thinking about these groups using the labels that we were given by Geoffrey

与其用杰弗里·摩尔给我们的标签来思考这些群体——顺便说一下，我发现这是反直觉的——相反，我为他们对技术的采用分配了一种情感状态。所以，我谈论的是爱好者、非理性者、高效者、嘲笑者和舒适者。

> Moore—which, by the way, I found to be counterintuitive—I instead assign one of the emotional states for their adoption of technology. So, I talk about the Lover, the Irrational, the Efficient, the Laugher, and the Comfortable.

马蒂：这些群体各代表什么？

> Marty: What does each of these groups represent?

杰夫：爱好者（创新者）是因为发现技术很酷而购买产品的技术人员。这些人对产品经理来说非常危险，因为他们的需求与更广大的人群非常不同。他们把解决艰难的技术问题视为乐趣。

> Jeff: The Lovers (Innovators) are the techies who buy the product because they find the technology cool. These people are very dangerous to product managers because they are driven by very different needs than the larger population. They look at solving tough technical problems as fun.

另一方面，非理性者（早期采用者）感受到与普通人群相同的情感，但更强烈。这些通常是愤怒、恐惧或孤独等负面情感，但无论如何，这些情感的强度可能导致购买行为在经济上是非理性的，例如，他们会花费比获得的价值更多的时间来学习某样东西，只是为了获得满足这些情感需求的满足感。

> On the other hand, the Irrationals (Early Adopters) feel the same emotions as the general population, but with more intensity. These are often negative emotions such as anger, fear, or loneliness, but in any case, the strength of these feelings can lead to buying behavior that is not economically rational, for example, they'll spend more time learning something than the value they get just so they can get the satisfaction of addressing these emotional needs. The good news is that as the product improves, ordinary people who feel the more subdued versions of the same emotions will also be motivated to buy.

好消息是，随着产品的改进，感受到相同情感的更温和版本的普通人也会有动力购买。

> The Efficients (Early Majority) will adopt when the technology becomes practical. Again, they feel the same emotion, but they're more pragmatic about the benefits versus the costs. The Laughers (Late Majority, and Yahoo's core constituency) feel the same emotion, but it's more muted and they don't want to deal with any grief in order to get the benefits.

高效者（早期多数）会在技术变得实用时采用。同样，他们感受到相同的情感，但他们对收益与成本更加务实。

> The Comfortable (Laggards) are the 15% that want the benefits but it just has to be drop dead simple and convenient for them to make the move.

嘲笑者（晚期多数，以及雅虎的核心选民）感受到相同的情感，但它更加微弱，他们不想为了获得好处而处理任何悲伤。

> In this view of adoption, there is tremendous power in the Irrationals.

舒适者（落后者）是那 15% 想要好处，但它必须非常简单和方便，他们才会做出改变。

> Lovers and Irrationals are often coming in the door at the same time, despite the traditional adoption curve that seems to imply there's first one and then the other. While Lovers and Irrationals may enter in at the same time, Lovers are the worst possible people in the world from a product manager's perspective.

在这种采用观点中，非理性者中有巨大的力量。

> Marty: Why is that?

爱好者和非理性者经常同时进门，尽管传统的采用曲线似乎暗示先有前者后有后者。虽然爱好者和非理性者可能同时进入，但从产品经理的角度来看，爱好者是世界上最糟糕的人。

> Jeff: Because they mislead you one hundred percent of the way. Lovers buy a Prius because they like the battery technology.

马蒂：为什么？

> On the other hand, Irrationals buy a Prius because they love the environment so much that they'll spend $22,000 over the benefit of the environment. They could just buy carbon credits and carbon neutralize themselves, or they could get a motorcycle, but they overspend on the solution because they're passionate about the problem they're trying to solve.

杰夫：因为他们百分之百地误导你。爱好者买普锐斯是因为喜欢电池技术。

> The bottom line is that Irrationals are really interesting, and Lovers are really not.

另一方面，非理性者买普锐斯是因为他们如此热爱环境，以至于他们会为环境的好处多花 22,000 美元。他们本可以买碳信用额和碳中和，或者买一辆摩托车，但他们在解决方案上超支，因为他们对试图解决的问题充满热情。

> People that obsess over your product because they like battery technology don't buy your product for the same reasons anyone else does, but the Irrationals do.

底线是非理性者真的很有趣，而爱好者真的不有趣。

> Intationals are essentially overreacting to the anger, but the emotional reaction they have is more of a multiplier times their logic. They exaggerate the value. But if you can tap into what they're thinking and what they feel, this can be very powerful.

那些因为喜欢电池技术而痴迷于你的产品的人，购买你的产品的原因与其他任何人都不同，但非理性者购买的原因相同。

> The Irrationals can teach you the value of your product all the way down the line.

非理性者本质上是对愤怒的过度反应，但他们的情感反应更像是乘以他们的逻辑。他们夸大了价值。但如果你能挖掘他们在想什么和感受什么，这可能是非常强大的。

> The latent frustration is highest amongst the Irrational and then it dissipates, but it's still always there. The Lovers are largely unconcerned with the core solution—they're more concerned with the technology involved.

非理性者可以一路教你产品的价值。

> One of the reasons startups in particular fall into the chasm is that they misread the situation—they confuse the Irrationals with the Lovers.

潜在的沮丧在非理性者中最高，然后消散，但它仍然存在。爱好者在很大程度上不关心核心解决方案——他们更关心所涉及的技术。

> Marty: So how do you address this at Yahoo!?

初创公司尤其掉入鸿沟的原因之一是他们误读了情况——他们把非理性者与爱好者混淆了。

> Jeff: One of the challenges Yahoo! and many large companies face is that we envy a lot of startups, and we think, “We want to do cool stuff like that.” But Yahoo has made its fame and fortune off serving the Laughers, so we will serve our user base much better by understanding Irrationals, and not by kowtowing to the Lovers.

马蒂：那你在雅虎！如何处理这个问题?

> Sometimes marketing folks confuse the emotional groups with demographics. They'll look at the Irrationals and say, “Oh, this group is comprised of males 18-30,” but it's not true. If you're in finance, you may have an Irrational group that's very different looking—it could be retired women. You have to look at each product individually, and look at the core emotions

杰夫：雅虎！和许多大公司面临的一个挑战是，我们羡慕很多初创公司，我们想，"我们也想做那样的酷东西。"但雅虎是靠服务嘲笑者而成名和致富的，所以我们会通过理解非理性者来更好地服务我们的用户群，而不是向爱好者卑躬屈膝。

> for this particular product.

有时营销人员会把情感群体与人口统计数据混淆。他们会看着非理性者说，"哦，这个群体由 18-30 岁的男性组成"，但事实并非如此。如果你在金融业，你可能有一个看起来非常不同的非理性群体——可能是退休女性。你必须单独看每个产品，看这个特定产品的核心情感。

> Marty: So what do teach your product managers to look for?

马蒂：那你教你的产品经理寻找什么?

> Jeff: Look for anger, exasperation, and frustration. If you just take a look at all those we love to hate—the telcos, banks, consumer credit firms, the tax man, government bureaucracy, airlines, health-care—these are all great opportunities for innovation because the consumer latent frustration is so high.

杰夫：寻找愤怒、恼怒和沮丧。如果你看看所有我们喜欢讨厌的——电信公司、银行、消费信贷公司、税务机关、政府官僚、航空公司、医疗保健——这些都是创新的好机会，因为消费者的潜在沮丧如此之高。

> Look at the music industry. We have to pay $15.98 for a CD with one good song on it. Is it any surprise that so many people feel good about stealing from these people?

看看音乐产业。我们必须花 15.98 美元买一张只有一首好歌的 CD。这么多人觉得从这些人那里偷东西感觉良好，这有什么奇怪的吗?

> We're all so impressed with Skype's growth, and yet, had we looked and seen the latent anger and frustration in the space, it would have been relatively predictable to say, it's not about standards or technology, or being open versus proprietary. The Skype guys rejected all that thinking and said we're just gonna make it work, and they then tapped into that latent frustration. It was like heroin, you couldn't stop it.

我们都对 Skype 的增长印象深刻，然而，如果我们看看并看到该领域潜在的愤怒和沮丧，相对可预测的是，这不是关于标准或技术，或开放与专有。Skype 的人拒绝了所有这些想法，说我们只是要让它工作，然后他们利用了那种潜在的沮丧。就像海洛因，你无法阻止。

> Contrast this with the webcam business. The problem here is there's no angry person in the webcam business—we don't hate our webcam providers, or our video conferencing guys—and webcam satisfies a need that's high up in Maslow's pyramid. It's “I wanna see my kid,” and while there's love involved there, not all of us have kids, and you can easily call your kids, so we're not all dying for a webcam.

与此形成对比的是网络摄像头业务。这里的问题是没有愤怒的人——我们不讨厌我们的网络摄像头提供商，或我们的视频会议人员——网络摄像头满足的需求在马斯洛金字塔上很高。它是"我想看到我的孩子"，虽然那里有爱，但不是我们所有人都有孩子，你可以很容易地给你的孩子打电话，所以我们并不都渴望网络摄像头。

> So, when you add video to Skype, not much happens. Their user base doesn't grow that

所以，当你给 Skype 添加视频时，并没有发生太多事情。他们的用户群增长不大，网络摄像头和即时通讯的使用量也没有下降——什么都没有改变。所以网络摄像头的鸿沟实际上很大，因为将其推向主流意味着你没有太多沮丧可以利用——你没有那种驱动的情感需求。

> much, the usage of webcam and messenger doesn't go down—nothing changed. And so the chasm for webcam is actually huge because taking it to the mainstream for webcam means you don't have much frustration to leverage—you don't have that driving emotional need. You really need the Irrationals to slingshot your business into the Efficients and the Laughers. Without that emotion from those irrational people you don't get the passion that carries the product over the chasm. So as with so many things in life we're brought back to Maslow's pyramid. If you look at the needs, the further down you go, the more you're tapping into core emotions, and the better off you are because these are the deepest emotions for humans.

你真的需要非理性者来把你的业务弹弓射入高效者和嘲笑者。没有那些非理性者的情感，你就不会获得将产品带过鸿沟的热情。所以，正如生活中的许多事情一样，我们回到了马斯洛金字塔。如果你看需求，你越往下走，你就越是在挖掘核心情感，你就越好，因为这些是人类最深的情感。

> Politicians like to tap into the fear emotion and explain that bad people are gonna destroy your cities, or come and bomb you, or you're not going to have food tomorrow. Well it's not that hard to motivate you to change your behavior when you energize these core emotions. It's much harder to get people to do something by appealing to their aspirations.

政治家喜欢利用恐惧情感，解释说坏人将要摧毁你的城市，或者来轰炸你，或者你明天就没有食物了。好吧，当你激发这些核心情感时，激励你改变行为并不难。通过诉诸他们的愿望来让人们做某事要困难得多。

> Google is the big winner in this way of thinking because the nature of a search engine is to help address countless critical human needs. If you've been diagnosed with a disease you go to Google to learn how to treat it. If you have to buy something, Google finds it for you. They're the good guys that are there to help you with whatever problem you have.

谷歌是这种思维方式的大赢家，因为搜索引擎的本质是帮助解决无数关键的人类需求。如果你被诊断出患有疾病，你去谷歌学习如何治疗它。如果你必须买东西，谷歌会为你找到。他们是好人，帮助你解决任何问题。

> So the question we ask is, “What are the emotions that are driving the behavior?” And then we look for ways to tap into those emotion with features and all the other things. We assess for each of our projects which core emotions they appeal to, and how acute is the user's pain.

所以我们问的问题是，"驱动行为的情感是什么？"然后我们寻找用功能和其他东西来利用这些情感的方法。我们评估我们的每个项目吸引哪些核心情感，以及用户的痛苦有多严重。

> Marty: Do you have a favorite approach for assessing these core emotions?

马蒂：你有一个最喜欢的方法来评估这些核心情感吗?

> Jeff: One technique I use is what I call the “freshman test.” Think back to the first day you walked into high school. You feel more pure emotions of human frailty in that one day than in any other day in your life. You're small, your hormones are all out of whack, anything you had acheived in your previous school completely gets washed away and you're a nobody—and you know you're a nobody.

杰夫：我使用的一种技术是我称之为"新生测试"。回想你走进高中的第一天。你在那一天感受到的人类脆弱性的纯粹情感比生活中任何其他日子都多。你很小，你的荷尔蒙都失调了，你在以前的学校取得的任何成就都被冲走了，你是一个无名小卒——而且你知道你是一个无名小卒。

> If you can tap into any one of those emotions that every human everyday feels—loneliness, insecurity, fear, frustration, anger—some bit of that freshman thing, then you're on the right track.

如果你能挖掘到每个人每天都感受到的那些情感中的任何一种——孤独、不安全感、恐惧、沮丧、愤怒——新生的那种感觉，那你就走对了路。

第 36 章：可用性与美学

> Chapter 36: Usability Vs. Aesthetics
>
> Both Are Important

两者都很重要

> I think most would agree that the general state of Web site design is still in its infancy, at least as practiced by most companies. While there are some notable exceptions, many sites—even from major players—are often either very difficult to use, downright ugly, or both. I have formed some theories as to why so many sites are bad, and what it will take to make this a better world as we all spend an increasing amount of our life interacting with the

我认为大多数人都会同意，网站设计的总体状态仍处于起步阶段，至少在大多数公司的实践中是如此。虽然有一些著名的例外，但许多网站——甚至来自主要参与者——往往要么非常难以使用，要么丑陋，或两者兼而有之。我对为什么这么多网站糟糕形成了一些理论，以及随着我们生活中越来越多的时间与网络互动，需要什么来使这个世界变得更好。

> I have long noted that too few companies invest the time and resources in user experience that they should. However, what I've had a harder time explaining is why companies that do invest the time and resources in user experience still often have such bad sites.

我早就注意到，太少公司在用户体验上投入时间和资源，而他们应该投入更多。然而，我更难解释的是，为什么确实在用户体验上投入时间和资源的公司仍然经常有如此糟糕的网站。

> Two edge cases in particular struck me as interesting. On the one hand, so many graphic/visual design firms have beautiful, artistic sites, that are difficult to read and poorly structured. On the other hand, many interaction design firms have very usable sites that are easy to navigate and find the info you need, yet are boring, primitive, and unappealing.

有两个极端案例特别让我觉得有趣。一方面，许多图形/视觉设计公司有美丽的、艺术性的网站，难以阅读且结构不佳。另一方面，许多交互设计公司有非常可用的网站，易于导航和找到你需要的信息，但却无聊、原始且没有吸引力。

> I think what these two cases illustrate is that the disciplines of interaction design and visual

我认为这两个案例说明了交互设计和视觉设计学科

> design are very different. To have a site that is both usable and appealing you need both skills on your design team. Some teams are very lucky and they have a designer talented at both types of

> design. But in many cases, I think they just expect that since they hired a “designer,” that person should be able to do both—and they can't.

设计是非常不同的。要拥有一个既可用又有吸引力的网站，你需要你的设计团队具备这两种技能。有些团队非常幸运，他们有一位在两种设计上都很有才华的设计师。但在许多情况下，我认为他们只是期望既然他们雇了一个"设计师"，那个人应该能够做两种设计——而他们做不到。

> Even worse—but most common of all—is when the company has neither type of designer, and they look to either the product manager or the UI engineer to design the site. When I talk to enterprise companies, this is unfortunately still the norm. When I talk to consumer startups, they usually have one or the other type of designer.

更糟——但最常见的是——当公司既没有这类设计师时，他们指望产品经理或 UI 工程师来设计网站。当我与企业公司交谈时，这不幸地仍然是常态。当我与消费者初创公司交谈时，他们通常有一种或另一种类型的设计师。

> Many teams feel that the visual design of a product or site is not really important. They argue that what matters is the functionality and the value proposition, and that things like nice colors, fonts, icons and layout are just unnecessary and superficial fluff. I strongly disagree with this view, and the more products I see, the stronger I believe in (a) the role that emotion plays in inspiring products, and (b) the direct role visual design plays in creating that emotion.

许多团队觉得产品或网站的视觉设计并不是真的很重要。他们认为重要的是功能和价值主张，而诸如漂亮的颜色、字体、图标和布局之类的东西只是不必要和肤浅的花哨。我强烈不同意这种观点，而且我看到的产品越多，我就越相信（a）情感在启发产品中的作用，以及（b）视觉设计在创造那种情感中的直接作用。

> You can show the exact same functionality to a user—one with wireframes and one with a good visual design—and the overall response can be dramatically different.

你可以向用户展示完全相同的功能——一个用线框图，一个用好的视觉设计——整体反应可能会有很大不同。

> Much as product management and product marketing are different functions requiring different training and skills, interaction design and visual design are different functions requiring different training and skills.

就像产品管理和产品营销是需要不同培训和技能的不同功能一样，交互设计和视觉设计是需要不同培训和技能的不同功能。

> I have oversimplified somewhat here—I haven't discussed the critical roles the product

我在这里有些过于简化了——我没有讨论产品经理或可用性工程师在开发一个既可用又令人愉快的网站中发挥的关键作用。

> manager or usability engineers play in developing a site that is both usable and enjoyable. And if the site performs like a dog, or is riddled with bugs, or is littered with advertising, then that will of course impact the experience in a big way too.

如果网站表现得像一只狗，或者充满错误，或者广告泛滥，那当然也会以很大的方式影响体验。

> But, fundamentally, I believe you need both interaction and visual design skill sets to deliver a good user experience, and that these people need to work closely with the product manager to define the product, which includes both the functionality and the user experience.

但是，从根本上说，我相信你需要交互和视觉设计技能来提供良好的用户体验，而且这些人需要与产品经理密切合作来定义产品，这包括功能和用户体验。

第 37 章：消费者互联网服务产品的关键

> Chapter 37: Keys To Consumer Internet Service PRODUCTS

十大清单

我喜欢构建非常大型的消费者服务。创建被世界各地数百万人使用和欣赏的产品是一件令人惊奇的事情。与大多数类型的产品不同，你和你的客户之间没有任何障碍——没有销售队伍或分销渠道需要打交道。你可以轻松即时地接触你的用户群，并近乎实时地尝试想法。你马上就能知道人们是喜欢还是讨厌你的新想法。

但这也非常难。

以下是一个对大规模消费者互联网服务公司特别重要的十种技巧的列表。这是一个通用列表，适用于多种类型的服务，包括电子商务、社交网络、搜索引擎和游戏，仅举几例。

1. 可用性。在我看来，大多数公司在每种类型的产品（尤其是企业软件！）上都对这一点关注太少，但是，对于消费者互联网服务，根本无法回避的是，它真的是关于用户体验的。如果你找不到一种方法让你的服务成为目标用户能够弄清楚如何使用的东西——并向他们提供为什么应该使用它的想法——那你就 nowhere fast。还要记住，性能是可用性的一个关键因素。如果一个页面加载时间太长，人们会认为它坏了，或者只是去别的地方，因为体验不好。
2. 用户画像。当你有数百万用户时，没有单一的"用户"，相反，你至少有几种重要的用户类型。将你的用户细分为最重要的用户画像并分别考虑每一个，这是至关重要的。当你创建一个功能时，你需要检查每个用户画像将如何评估和响应该功能。参见"产品管理的用户画像"一章。

3. 可扩展性。一旦数百万用户开始使用，产品就会发生奇怪的事情。数据库崩溃，性能瓶颈到处出现，用户界面变得不可用。你可以在 QA 期间进行一定程度的负载测试，但我发现这只找到容易的问题。我保证你会有惊喜——而且不是愉快的惊喜。管理可扩展性需求需要产品管理、设计、工程和运营的合作。它还需要一个积极主动的管理团队，持续分配大量工程和运营资源（我一般建议 20%）来扩展产品和基础设施。如果你到了一切都崩溃，工程团队告诉你"纸牌屋已经倒塌"的地步，你就大麻烦了。我知道避免这一点的唯一方法是从第一天开始持续地致力于扩展，不要让自己走到边缘。

4. 可用性。很快你会发现，真的没有一天、一周、一月或一年你的服务不需要运行。我不知道有谁达到了 100% 的可用性，但我知道中断对每个人来说都是痛苦的。消费者互联网服务提供商的生活不适合每个人。事情随时发生，工作日和周末，我认识的这个行业的每个人都有他们的故事。你需要像对待可扩展性一样，在你的所有思考中设计高可用性。

5. 客户支持。大多数消费者服务公司遇到的最大的（也是最不有趣的）惊喜之一是客户支持。传统的客户支持模式可以迅速使即使最好的服务公司破产。这是对一个复杂话题的过度简化，但你别无选择，只能设计和构建产品以绝对最小化客户支持成本——特别是如果你对你的服务收费。然而，永远记住，这不是关于控制客户支持成本——而是关于确保良好的客户体验。

6. 隐私和数据保护。消费者互联网服务公司可以很快积累比他们想象的更多的个人信息。你可能纯粹出于无辜的目的收集这些数据——比如提供个性化的用户体验。但今天，诸如电子邮件地址和信用卡号之类的客户数据是非常敏感的资产，你不希望数据落入坏人手中所带来的负面新闻、可能的罚款，特别是客户的愤怒。你需要尽早建立保护措施，绝对确保你正在保护客户委托给你的信息。你还需要保护实际的用户数据不被你自己的员工访问。

7. 病毒式营销。如果你的产品很好，你的用户会想要与朋友、家人和同事分享这种体验。这正是你想要的，但令人惊讶的是，有多少公司实际上没有做任何事情来促进这种最强大的营销形式。考虑你能用你的产品做什么来让用户邀请他们的朋友尝试它。此外，研究数字——大多数公司愿意为每个新用户支付一定的金额（客户获取成本，通常用于营销和广告）。也许你可以把那笔钱的一部分给你的用户？但找到让分享爱变得容易的方法——即使不使用财务激励。

8. 全球化。如果你有一个好服务，你会发现它会迅速蔓延到你国家的边界之外——首先是到其他讲相同语言的国家，然后是到所有其他主要的互联网连接国家。设计一个可以轻松本地化的产品，比试图为一个国家/货币/语言/文化设计的产品并重写为另一个要便宜得多——而且总体上，上市速度更快。你不必一开始为所有国家本地化——但随着你的业务扩展——你可以更快速、更经济地满足你的新用户的需求。

9. 温和部署。当数百万人使用你的服务时，任何变更都是一件大事，每个变更都需要仔细考虑。我们之前讨论过温和部署策略（参见"温和部署"一章），但在这里 suffice it to say 变更是需要智能部署的。这不仅仅意味着伟大的 QA——它还意味着逐步部署，并给你的用户时间在他们有时间学习变更时切换，而不是在你碰巧推出的时候。在许多情况下，这涉及将新版

本与旧版本一起部署，以便人们可以在一段时间内切换。此外，尽你所能消除不必要的变更。让你的社区吸收他们想要和需要的功能已经够难的了。

8.社区管理。每个公司都依赖于其客户，但对于消费者互联网服务公司，这个用户社区变得更加强大——他们可以成就你，也可以毁了你。他们从未如此容易地传达他们有多喜欢你的产品——或者你刚刚对他们有多糟糕。

如果你的用户重视你的产品，他们会想要忠诚，他们会想要成为你正在创造的伟大社区的一部分。但他们也想被欣赏和认可——不仅仅是口头上的。有很多方法可以接触你的社区——向他们学习并了解他们希望看到你的产品走向何方。也有很多方法可以向社区表达你的感激之情，并认可那些做出特别有价值贡献的人。这样做，让社区意识成为你公司每个人的首要任务——从 CEO 开始。

如果你能在创建消费者互联网服务时牢记这 10 个技巧，你会为自己省去很多 grief。但正如我上面所说，这些都是很好的产品工作，很容易对这类业务上瘾。

> Top 10 List
> I love building very large-scale consumer services. Creating products that are used and appreciated by millions of people around the world is an amazing thing. And unlike most types of products, there's nothing between you and your customer—no sales force or distribution channels to work through. You can reach your user base easily and instantly, and try ideas out in near real time. You know right away if people love or hate your new ideas.
> But it's also very hard.
> Below is a list of ten techniques that are especially important for large-scale consumer Internet service companies. It's a general list, meant to apply to many types of services including e-commerce, social networks, search engines, and games, as just a few examples.
>
> 1. Usability. In my opinion, most companies give far too little attention to this in every type of product (especially enterprise software!) but, with a consumer Internet service, there simply is no getting around that it's really all about the user experience. If you can't
>
> find a way to make your service something that the target user can figure out how to use—and provide them with an idea of why they should use it—then you're going nowhere fast. Also remember that performance is a key factor in usability. Ifa page takes too long to load, people will think it's broken or just go elsewhere because the experience is bad.
>
> 1. Personas. When you have millions of users, there is no single “user,” rather you have at least several important types of users. It's critical that you segment your users into the most important personas and consider each one separately. When you create a feature, you need to examine how each persona will value and respond to that feature. See the chapter Personas for Product Management.
> 2. Sealability. Weird things happen to products once millions of users start using them. Databases break, performance bottlenecks pop up all over, user interfaces become unusable. You can do some amount of load testing during QA, but I've found this only

finds the easy problems. I guarantee you that you'll have surprises—and not pleasant ones. Managing scalability needs takes a collaboration of product management, design, engineering, and operations. It also takes a proactive management team that allocates a significant amount of engineering and operations resources on an ongoing basis (I generally advise 20%) to scaling the product and infrastructure. If you get to the point where everything breaks and the engineering team is telling you that the "house of cards has collapsed," you're in big trouble. The only way I know to avoid that is to pay your taxes by working on scale continuously from day one, and don't let yourself get to the brink.

3. Availability. Very quickly you'll find that there really is no time of the day or week or month or year that your service doesn't need to be running. I don't know of anyone that

has achieved 100% availability, but I know outages are painful to everyone. The life of a consumer Internet service provider is not for everyone. Things happen at all hours, weekday and weekends, and everyone I know in the business has their stories. You need to design-in high-availability to all of your thinking just as you need to do with scalability.

1. Customer support. One of the biggest (and least fun) surprises that most consumer service companies run into is customer support. Traditional models of customer support can quickly bankrupt even the best services companies. This is oversimplifying a complicated topic, but you have no alternative other than designing and building the product to absolutely minimize customer support costs—especially if you charge a fee for your service. Yet, always remember that it is not about controlling customer support costs—it is about ensuring a great customer experience.
2. Privacy and data protection. Consumer Internet service companies can very quickly end up with far more personal information than they ever imagined. You may be collecting this data for purely innocent purposes—such as to provide a personalized user experience. But today customer data such as e-mail addresses and credit card numbers are a very sensitive asset, and you don't want the bad press, the possible penalties, and especially the customer anger that results if that data gets into the wrong hands. You need to put the protections in place early to absolutely ensure that you are protecting the information that your customers entrusted you with. You also need to protect the actual user data from your own employees.
3. Viral marketing. If your product is good, your users will want to share that experience with their friends, family, and co-workers. That's absolutely what you want, but it's surprising how few companies actually do anything to facilitate this most powerful form of marketing. Consider what you can do with your product to enable users to invite their

friends to try it. Also, work the numbers—most companies are willing to pay a certain amount per new user (customer acquisition cost, usually for marketing and advertising). Maybe you can funnel some of that money to your users instead? But find ways to make sharing the love easy—even without using financial incentives.

1. Globalization. If you have a good service, you'll find that it will quickly spread beyond your country's borders—first to other countries that speak the same language, then to the

> rest of the major Internet-connected countries. It is far less expensive—and overall, faster to market—to design a product that can easily be localized than it is to try and take a product for one country/currency/language/culture and rewrite it for another. You don't have to localize initially for all the countries but—as your business spreads—you can much more rapidly and economically meet your new users' needs.
>
> 7.Gentle deployment. When millions of people are using your service, any change is a big deal, and every change needs to be considered carefully. We talked earlier about gentle deployment strategies (see the chapter Gentle Deployment), but suffice it to say here that changes need to be deployed intelligently. This doesn't just mean great QA—it also means deploying gradually, and giving your users time to switch when they have the time to learn about the changes, not when you happen to roll it out. In many cases, this involves deploying the new version alongside the old so that people can switch over a period of time. Also, do everything you can to eliminate gratuitous changes. It's hard enough for your community to absorb features that they know they want and need.
>
> 8.Community management. Every company is dependent on its customers, but for consumer Internet service companies, this community of users takes on even more powerful importance—they can make you or break you. It has never been easier for them to communicate how much they love your product—or how badly you just treated them.
>
> If your users value your product, they will want to be loyal, and they will want to be a part of the great community you are creating. But they also want to be appreciated and recognized—and not just with lip service. There are many ways to reach out to your community—to learn from them and understand where they want to see your product go. There are also many ways to show your gratitude to the community and to recognize those that make especially valuable contributions. Do this, and make community awareness a top priority for everyone in your company—from the CEO on down.
>
> Ifyou can keep these 10 techniques in mind as you create your consumer Internet services, you'll save yourself a lot of grief. But as I said above, these are great products to work on, and it's easy to become hooked on these types of businesses.

第 38 章：企业产品的关键

> Chapter 38: Keys To Enterprise Products
>
> Top 10 List

十大清单

> I'm frustrated by the state of the enterprise software industry, and I have been for quite some time. While there are some notable exceptions, I find fewer examples of good products in this space than any other. Many people view the enterprise software market as mature—or worse—but I think customers are just frustrated and aren't anxious to spend yet more money on more disappointing

> products. And they're just not willing to dish out hundreds of thousands—or even millions—of dollars on professional services just to get them working. There are a number of reasons why the industry is in this state but, regardless, I believe that good product management can help improve the situation substantially.

我对企业软件行业的现状感到沮丧，而且已经有一段时间了。虽然有一些著名的例外，但我发现在这个领域的好产品例子比其他任何领域都少。许多人认为企业软件市场已经成熟——或者更糟——但我认为客户只是感到沮丧，并不急于在更多令人失望的产品上花更多钱。而且他们只是不愿意掏出数十万美元——甚至数百万美元——的专业服务费用，只是为了让他们工作。

> Before I get started, let me clarify what I mean by the very general term enterprise software. The two key points to clarify are who the software is sold to, and what type of software product we're talking about.

行业处于这种状态有很多原因，但无论如何，我相信好的产品管理可以帮助大大改善这种情况。

> Regarding who the software is sold to, the main point is that the products are sold to businesses rather than consumers. There is of course a full-spectrum of business size, and I'm intentionally not including the small office/home office/small business markets

在我开始之前，让我澄清一下我所说的非常通用的术语"企业软件"是什么意思。需要澄清的两个关键点是软件卖给谁，以及我们谈论的是什么类型的软件产品。

> because, in my view, they're really much more like the consumer market than the enterprise market (and the reason so many companies in the past tried and failed in the huge small biz market is because they didn't realize this fact and its implications for product requirements). I do, however, include medium-size businesses in this definition even though many people reserve the term enterprise for a larger group such as the Fortune 500. But I find many of the challenges start to show with mid-size businesses.

关于软件卖给谁，主要点是产品是卖给企业而不是消费者。当然，企业规模有一个完整的谱系，我故意不包括小型办公室/家庭办公室/小型企业市场，

> And then there's the type of software. Enterprise infrastructure products (e.g., security software, systems management, and communications software) have some significant differences from enterprise applications (e.g., SFA, CRM, ERP), but I'll speak to both here. I think the top-1o list is essentially the same, although I'd weight the importance of each item differently based on whether it's infrastructure or application.

因为，在我看来，它们真的更像消费者市场而不是企业市场（过去这么多公司在巨大的小企业市场尝试并失败的原因是，他们没有意识到这一事实及其对产品需求的影响）。然而，我确实在定义中包括中型企业，尽管许多人将企业一词保留给较大的群体，如财富 500 强。但我发现许多挑战开始在中型企业中显现。

> Here is a list of ten techniques that are especially important for enterprise software companies:

然后是软件的类型。企业基础设施产品（例如，安全软件、系统管理和通信软件）与企业应用（例如，SFA、CRM、ERP）有一些显著的差异，但我将在这里讨论两者。我认为前 10 名列表基本上是相同的，尽管我会根据它是基础设施还是应用对每个项目的重要性进行不同的加权。

> 1. Usability. Pity the poor souls who must sit at their desks all day and use the typical enterprise applications for purchasing, billing, customer service, and hundreds of other applications. If the people who actually had to use the systems were the same ones that made the actual buying decisions, I think we'd have a very different set of vendors. I'm very sorry to say it, but most of the applications are just awful. One of my favorite books is The Inmates are Running the Asylum by Alan Cooper. Nowhere is his argument truer than with enterprise software. I find too few enterprise companies doing any interaction design, visual design, or usability testing—and it certainly shows. Even when the application produces business results, there is still a sour taste in the mouth, and you rarely find true enthusiasm for these types of products. It really is time to raise the bar.

以下是对企业软件公司特别重要的十种技巧的列表：

> 1. Product actually needs to work. Even more egregious than poor product usability, too many enterprise products simply don't work—at least not without tremendous investments in time and money to “customize” and “integrate” or develop countless workarounds. This is certainly not the case with all products, but the many out there that ship with often hundreds of serious defects give a bad reputation to all of us. As product manager, it's essential to make sure your product does what you says it will.

1. 可用性。可怜那些必须整天坐在办公桌前使用典型的企业应用进行采购、计费、客户服务和数百个其他应用的可怜灵魂。如果实际必须使用系统的人与做出实际购买决定的人是同一个人，我想我们会有一组非常不同的供应商。我非常抱歉地说，但大多数应用都很糟糕。我最喜欢的书之一是 Alan Cooper 的《精神病院里的囚犯》。他的论点在企业软件中再真实不过了。我发现太少的企业公司进行任何交互设计、视觉设计或可用性测试——而且这一点显而易见。即使应用产生了业务结果，嘴里仍然有一种酸味，你很少发现对这些类型产品的真正热情。真的是时候提高标准了。

> 1. Specials. One of the biggest mistakes companies make is that they think they can get their product requirements from their customers. I've talked about this earlier (see Beware of Specials), but the most dangerous example of this is when the sales organization brings in a potential customer that has a big check all ready to sign if you'll just agree to put these seven features in your product. Soon you're becoming a custom software shop, and you don't have anything resembling a generally useful product. If that's not bad enough, there's a good chance the initial customer won't be happy with those seven features anyway (once they got it and tried it they realized they needed something different). Avoiding specials takes a lot of discipline—especially for a small company struggling to

> bring in some cash—but it is critical you create products that meet the needs of a wide range of customers.

1. 产品实际上需要工作。比糟糕的产品可用性更糟糕的是，太多企业产品根本不起作用——至少没有大量的时间和金钱投资来"定制"和"整合"或开发无数的变通方法。这当然并非所有产品都如此，但许多产品带有数百个严重缺陷，给我们所有人带来了坏名声。作为产品经理，确保你的产品做到你所说的至关重要。

> 1. Customers and Charter User Programs. The above is not to say that you shouldn't listen to your customers—you should absolutely meet with and listen to many customers. Just don't expect them to be able to dictate your product requirements. To determine the real product requirements, you'll need to meet with many customers. One valuable technique is to identify a half-dozen or so great potential customers (smart, enthusiastic, motivated, friendly) and invite them to participate on a charter customer program (see the chapter Charter User Programs). In exchange for the opportunity to work closely with

1. 特殊需求。公司犯的最大错误之一是，他们认为可以从客户那里获得产品需求。我之前谈过这一点（参见"警惕特殊需求"），但最危险的例子是，当销售组织带来一个潜在客户，他准备了一张大支票，只要你同意在你的产品中放入这七个功能。很快你就变成了一家定制软件店，你没有任何类似于普遍有用的产品的东西。如果这还不够糟糕，很有可能初始客户无论如何都不会对这七个功能满意（一旦他们得到并尝试了他们意识到他们需要不同的东西）。避免特殊需求需要很大的纪律——特别是对于一家努力带来一些现金的小公司——但你必须创建满足广泛客户需求的产品，这一点至关重要。

> the development team, they know that their needs will be understood and seriously considered, and they agree to be accessible to try out ideas, answer questions, and install software immediately—and, if it meets their needs, to be vocal reference customers. Your goal should always be to have at least half a dozen live, happy referenceable customers when you release your product.

1. 客户和特许用户计划。上面并不是说你不应该倾听你的客户——你绝对应该与许多客户会面并倾听他们的意见。只是不要期望他们能够规定你的产品需求。要确定真正的产品需求，你需要与许多客户会面。一种有价值的技术是识别大约六个优秀的潜在客户（聪明、热情、积极、友好），并邀请他们参加特许客户计划（参见"特许用户计划"一章）。作为密切合作的机会的交换，

> 1. Designing for the sales channel. It's critical to design your product around the needs of your sales and distribution channel. Different channels require different capabilities. The key is to provide value all along the distribution chain. If you're selling through systems integrators, you'll need to ensure that your product doesn't try to cut them out of the equation. If you're selling through VARs that supply a wide range of products, you'll need to ensure that your product doesn't overwhelm them with time-consuming technical

> requirements. Many otherwise good products struggle because they're not appropriate for their sales channel.

开发团队，他们知道他们的需求会被理解和认真考虑，他们同意可以尝试想法、回答问题并立即安装软件——而且，如果它满足他们的需求，就作为有声的参考客户。你的目标应该始终是，在发布产品时至少有六个活跃的、满意的、可参考的客户。

> 1. The customer versus the user. Many enterprise products are designed around the needs of the person who will actually buy the product—the customer. That's who the team. talks to when learning about needs, and that's who has to give the okay in order to sell the product. However, as we alluded to above, there are often several different users of the product who also bring needs and requirements. For example, the different types of end-users, the systems administrators, management, and often other business applications.

1. 为销售渠道设计。围绕销售和分销渠道的需求设计你的产品，这是至关重要的。不同的渠道需要不同的能力。关键是沿着分销链提供价值。如果你通过系统集成商销售，你需要确保你的产品不会试图把他们排除在外。如果你通过提供广泛产品的 VAR 销售，你需要确保你的产品不会用耗时的技术要求压倒他们。许多本来很好的产品因为不适合其销售渠道而苦苦挣扎。

> 1. Product installation. With consumer products, vendors know that they have to make the product absolutely bulletproof to install, and something that just takes minutes or even seconds. But with enterprise products, many vendors assume they'll be able to get dedicated systems administration staff that can craft custom installations that can take

1. 客户与用户。许多企业产品是根据实际购买产品的人——客户——的需求设计的。这就是团队在了解需求时交谈的对象，也是必须给予批准才能销售产品的人。然而，正如我们上面暗示的，通常有几个不同的产品用户，他们也带来需求和要求。例如，不同类型的最终用户、系统管理员、管理人员，通常还有其他业务应用。

> days or even weeks of work, and that these administrators will be able to watch over the applications daily. Even when this is true, when the person moves on—or is out due to vacation or illness, or just gets overloaded—things can quickly fall apart. Again, it's time to raise the bar.

1. 产品安装。对于消费产品，供应商知道他们必须使产品绝对防弹地安装，只需几分钟甚至几秒钟。但对于企业产品，许多供应商假设他们将能够获得专门的系统管理人员，可以制定需要几天甚至几周工作的定制安装，而且这些管理员将能够每天监控应用。即使这是真的，当这个人离开——或由于休假或疾病外出，或者只是超负荷——事情可能很快就会崩溃。再次，是时候提高标准了。

> 8.Product configuration, customization, and integration. An enormous professional services industry has emerged to fill the gap and try and get these enterprise software products to actually

> work, and further, to work with each other. I believe that the vast majority of the cost is simply due to poor product design and execution. However, even if you accept that the need for services is appropriate, there is still much that can be done by vendors to make their products easier to configure, customize and integrate. If you don't believe this is possible, look at how Salesforce.com has redefined the game in their market.

8.

> 1. Product update. Installing a new version of enterprise software is never fun. The vendor thinks it's the biggest event of the year, but the customer has a business to run, and installing updates of vendor software isn't something they typically want to be spending their precious time and resources on. Having problems upgrading or requiring complex data migration is extremely frustrating to the customer. Again, most consumer products realize this and make a much bigger investment in technology to upgrade, and in testing the upgrade process.

9.

> 1. The Sales Process. For many years, in the enterprise software market, far too much of the sale was driven by the talents/personality/charm of the respective sales staff. A product selection was too often driven by the relationship between the sales rep and the customer rather than by whether the product was the best fit or not. While the relationship aspects are still very important (more than they should be), today the

10.

> Internet has changed the product research and evaluation process, and vendors need to support this new sales process. Make your product easy to try and buy. If you can keep these 10 points front and center as you create your enterprise software products, you'll be far ahead of your competition—and your customers are sure to be grateful.

产品配置、定制和整合。一个巨大的专业服务行业已经出现，以填补空白，试图让这些企业软件产品实际工作，并进一步，彼此配合工作。我相信，绝大多数成本仅仅是由于糟糕的产品设计和执行。然而，即使你认为服务需求是适当的，供应商仍然可以做很多事情来使他们的产品更容易配置、定制和整合。如果你不相信这是可能的，看看 Salesforce.com 如何重新定义他们市场中的游戏规则。

> what About Solutions Products?

产品更新。安装企业软件的新版本从来都不是有趣的。供应商认为这是一年中最大的事件，但客户有业务要经营，安装供应商软件的更新并不是他们通常希望花费宝贵时间和资源的事情。升级出现问题或需要复杂的数据迁移，对客户来说非常令人沮丧。同样，大多数消费产品意识到这一点，并在升级技术和测试升级过程方面进行更大的投资。

> I realize that most of this book is focused on the development of Internet services, and

销售流程。多年来，在企业软件市场，太多销售是由各自销售人员的天赋/个性/魅力驱动的。产品选择往往是由销售代表与客户之间的关系驱动的，而不是产品是否最适合。虽然关系方面仍然非常重要（比应该的更重要），但今天互联网改变了产品研究和评估过程，供应商需要支持这个新的销售流程。让你的产品易于尝试和购买。

> mostly consumer Internet services at that. But many product managers out there are

如果你能在创建企业软件产品时始终牢记这 10 点，你将远远领先于你的竞争对手——你的客户肯定会感激。

> working hard on other types of software products, such as enterprise or infrastructure

O解决方案产品怎么样?

> software—both shipped software and hosted services.

我意识到这本书的大部分内容都集中在互联网服务的开发上，而且主要是消费者互联网服务。但那里有许多产品经理正在努力开发其他类型的软件产品，如企业或基础设施软件——既有出货软件，也有托管服务。

> One product area that seems to be a long-standing source of confusion for those in the

一个似乎长期以来一直困扰着行业内人士的困惑领域是所谓的解决方案产品以及相关解决方案营销。正如许多公司夸大其词地称他们的产品为平台一样，许多其他公司也喜欢声称他们的产品是解决方案——即使它不是。

> industry has to do with what are referred to as solutions products and the associated

平台和解决方案的概念都很重要和强大，那些不符合标准的产品只会稀释真正符合标准的产品的含义，并混淆市场。

> solutions marketing. Just as many companies stretch the truth to call their products a

在定义解决方案产品之前，让我们首先明确什么构成了一般意义上的产品。这听起来可能很基础，但很多软件实际上根本不是产品。

> platform, many other companies like to claim their product is a solution—even when it's not.

以下是我如何描述一个产品的： ¢ 人们会为此付费；它提供真实和独特的价值（通常有自己的SKU）。注意，有时人们通过容忍广告，或通过支付支持而非许可费来支付，但无论如何，他们都在补偿提供商。

> The concepts of platform and solution are both important and powerful, and those products

«它在多个客户安装中运行良好。这里的要点是，这不是一个特殊需求——这不是定制软件。

> that aren't really up to the standard just dilute the meaning for those that are and confuse

- 你的现场和/或渠道可以有效地销售它。你提供必要的销售工具和销售培训。

> the market.

«你的公司会支持它，提供支持并在必要时添加改进。

> Before defining a solutions product, let's first be clear on what constitutes a product in

« 你的客户和/或渠道和/或服务合作伙伴知道如何安装、配置和定制产品。

> general. This may sound basic, but much software is not actually a product at all.

你可能会认为我在这里定义的不仅仅是一个产品，而是某种质量的产品。我认为这是对的。我认为软件——即使是产品组织生产的软件——如果还没有被多个客户成功使用，也只能算是一个想要成为产品的产品。从某种意义上说，软件必须证明其被视为产品的权利，就像没有在其上运行任何应用的平台并不是真正的平台一样。

> Here is how I characterize a product:

A解决方案产品具有上述产品的所有特征，外加：« 软件解决业务层面的问题，通常针对特定的行业垂直领域。 « 产品可能基于一个或多个组件级产品的整合，这些产品可能来自你的公司或合作伙伴，它们是预整合的。 ¢ 如果合适，产品会按照必要与合作伙伴的产品一起认证（客户需要知道支持的配置）。

> - People will pay for it; it delivers real and distinct value (and typically has its own SKU).

我喜欢解决方案产品，因为它们直接针对业务层面的问题或需求。一般来说，客户对使用的底层技术不太关心（除了早期采用者），更关心你是否真的解决了他们的业务问题。你的业务问题可能是灾难恢复、客户关系管理或萨班斯-奥克斯利合规，但它不是你使用的操作系统是什么 flavor，或你选择的什么类型的路由器。

> Note that sometimes people pay by tolerating advertising, or by paying for support and not license fees, but one way or another they're compensating the provider.

请注意，有些解决方案产品是交钥匙的，其他则需要专业服务。解决方案产品可以适用于任何规模的客户——从单个消费者到最大的企业——软件可能是出货（安装）软件，或软件即服务。

> «It works well in multiple customer installations. The point here is that it's not a special—this is not custom software. + Your field and/or channel can effectively sell it. You provide the necessary sales tools and sales training. + Your company will stand behind it, providing support and adding improvements as necessary. + Your customers and/or channel and/or services partners know how to install, configure and customize the product. You might argue that what I'm defining here is not just a product, but a certain quality of product. And I think that's true. I consider software—even

> software produced by a product organization—to be just a wannabe product if it's not yet being successfully used by multiple customers. In a sense, the software has to prove its right to be considered a product, much like a platform that does not have any applications running on it isn't really much of a platform. A solutions product has all of the characteristics of a product above, plus: + The software solves a business-level problem, often for specific industry verticals. + The product may be based on an integration of one or more component-level products, which may come from your company or from partners, and they are pre-integrated. + If appropriate, the product is certified with partners' products as necessary (the customer needs to know the supported configurations). Ilike solutions products because they speak directly to a business-level problem or need. In general, customers care much less about the underlying technology you use (other than

但也要指出解决方案产品不是什么： - 关于如何以新方式使用现有产品的一套说明（客户不会为此付费）

> early adopters), and more about whether you really solve their business problem. Your business problem might be disaster recovery, customer relationship management, or Sarbanes-Oxley compliance, but it isn't what flavor of the operating system is used, or what type of router you select.
>
> Note that there are some solutions products that are turnkey, and others that require professional services. Solutions products can be for any size customer—from a single consumer to the largest enterprise—and the software may be shipped (installed) software, or software as a service.
>
> But it's also important to point out what a solutions product is not:
>
> - A set of instructions for how to use an existing product in a new way (customer's won't
>
> pay for that)
>
> - Aset of customizations/scripts from the field or other external source (not supportable) There are many field or marketing organizations that can clearly see the customer demand for true solutions, but their product organization only provides the underlying component products.
>
> The temptation is to get creative and to try to cobble together something that they hope they will have better luck selling. But while this might buy a little time, as soon as a competitor comes along with a true solutions product, the customer can easily tell the difference. Related—but the not the same as solutions products—is solutions marketing. I also like solutions marketing over other forms of product marketing because solutions marketing:
>
> - Speaks to the business-level problem, aligning products with business value + Speaks to vertical industry segments, aligning value with a particular industry's more specific—and sometimes regulated—needs. + Showcases live customer success stories in action, in order to prove the business value + Shows how to leverage products, professional services and business process knowledge or best practices to achieve business results To me the trend is very clear and has been underway for several years. Increasingly, customers are demanding products that directly address business-level needs, and they're less interested

> in reading and comparing a data sheet of technical specs. Good solutions products and solutions marketing speak directly to these business needs.

« 来自现场或其他外部来源的一套定制/脚本（无法支持）

有许多现场或营销组织可以清楚地看到对真正解决方案的客户需求，但他们的产品组织只提供底层组件产品。

诱惑是发挥创造力，试图拼凑一些他们希望会有更好的运气销售的东西。但虽然这可能会买一点时间，一旦竞争对手带着真正的解决方案产品出现，客户很容易就能看出区别。

相关——但不完全等同于解决方案产品——的是解决方案营销。我也比其他形式的产品营销更喜欢解决方案营销，因为解决方案营销：

« 针对业务层面的问题，将产品与业务价值对齐

- 针对垂直行业细分，将价值与特定行业的更具体——有时受监管的——需求对齐。

« 展示活生生的客户成功案例，以证明业务价值

- 展示如何利用产品、专业服务和业务流程知识或最佳实践来实现业务结果

对我来说，趋势非常清楚，而且已经进行了好几年。越来越多的客户需要直接解决业务层面需求的产品，他们对阅读和比较技术规格的数据表不太感兴趣。好的解决方案产品和解决方案营销直接针对这些业务需求。

第 39 章：平台产品的关键

> Chapter 39: Keys To Platform Products
> High Leverage But Not Easy

高杠杆但不容易

> One of the most difficult—but highest leverage—types of product management is to define successful platforms. By platforms, I am referring to foundation software that is used by application developers to create end-user solutions. Examples include operating systems (e.g., Windows, MacOS, Palm OS), operating environments (e.g., Java, Flash), Web services (e.g., Amazon's or eBay's integration APIs), game developer platforms (e.g., XNA), and application-level platform runtimes (e.g., Facebook and Salesforce.com). Before I go further, it's important to point out what is not a platform. There are too many so-called “platforms” out there that are really just unfinished products. The team didn't do the work required to provide a complete solution, so they market it as a platform and push the work off on the customer or a developer to finish. If you can't program the platform through some form of API, and if you don't have multiple commercial software products or services built upon your software, then you're not a platform in the sense I'm

> describing. But assuming you are, then you know how difficult platform product management can be. To begin with, there are three very different constituencies:

最困难——但杠杆率最高——的产品管理类型之一是定义成功的平台。通过平台，我指的是应用开发人员用来创建最终用户解决方案的基础软件。示例包括操作系统（例如 Windows、MacOS、Palm OS）、操作环境（例如 Java、Flash）、Web 服务（例如亚马逊或 eBay 的集成 API）、游戏开发者平台（例如 XNA）以及应用级平台运行时（例如 Facebook 和 Salesforce.com）。

在我进一步说明之前，重要的是指出什么不是平台。有太多所谓的"平台"实际上是未完成的产品。团队没有做提供完整解决方案所需的工作，所以他们将其作为平台销售，并将工作推给客户或开发人员来完成。如果你不能通过某种形式的 API 对平台进行编程，如果你没有多个商业软件产品或服务建立在你们的软件之上，那么你们就不是我描述意义上的平台。

但假设你是，那么你就知道平台产品管理有多么困难。首先，有三个非常不同的选民： ¢ 应用提供商——选择使用你们的平台构建其解决方案的企业。

> · The application providers—the businesses that choose to use your platform to build their

«开发人员——那些为应用提供商工作的人，他们使用你们的平台服务编写他们的软件。

> solution. +The developers—those who work for the application providers and who write their software using your platform services. + The end-users—the ones who run the application provider's products, and ultimately use

· 最终用户——那些运行应用提供商产品的人，最终使用你们的服务。

> your services. Each of these three constituencies brings to the table very different needs and requirements. You simply can't be a successful platform without meeting the key needs across all three. The application provider is going to be concerned with business viability—their viability if they use your services, and your viability in case you go out of business or discontinue support for the platform. The application provider cares about your pricing, licensing, quality, support, and global availability, among other factors. The developers are looking for services that make it easy for them to quickly create maintainable, reliable code in the languages they want to use, working with their favorite tools and infrastructure, on the devices they need to deliver on. The end-users care mainly about the end result. If the features and services they want aren't there or don't work in the way they need, they don't buy the application, and the application provider fails. You lose a customer, and eventually you fail too. One of the biggest mistakes platform product managers make is in the prioritization of the three constituencies. Developers are the most vocal group and the easiest for the company to relate to, so they usually get considered first. The application provider is the one writing the

这三个选民带来了非常不同的需求和要求。你根本不可能在不满足所有三个关键需求的情况下成为成功的平台。

应用提供商将关注业务可行性——如果他们使用你们的服务，他们的生存能力，以及你们的生存能力，以防你们倒闭或停止支持该平台。应用提供商关心你们的价格、许可、质量、支持和全球可用性等因素。

开发人员正在寻找服务，使他们能够轻松快速地使用他们想用的语言创建可维护、可靠的代码，与他们最喜欢的工具和基础设施一起工作，在他们需要交付的设备上工作。

最终用户主要关心最终结果。如果他们想要的功能和服务不存在，或者工作方式不是他们需要的，他们就不会购买应用，应用提供商就会失败。你失去一个客户，最终你也会失败。

> check, so they come close behind. But the end-user is often so far removed from the platform provider that they rarely interact directly. Unfortunately, this is precisely the reverse of what's needed. It is a big (but common) mistake to optimize for developers over the end-user.

平台产品经理犯的最大错误之一是对三个选民的优先级排序。开发人员是最直言不讳的群体，也是公司最容易理解的，所以他们通常被首先考虑。应用提供商是写支票的人，所以他们紧随其后。但最终用户往往与平台提供商相距如此之远，以至于他们很少直接互动。不幸的是，这恰恰与需要的情况相反。优化开发人员而不是最终用户是一个很大的（但常见的）错误。

> I know this may sound heretical, but it is okay for the developers to work a little harder if the application is something end-users like and use, versus making the developers happy but nobody wants the end result.

我知道这可能听起来是异端邪说，但如果最终用户喜欢和使用应用，开发人员工作稍微努力一点是可以的，而不是让开发人员满意但没人想要最终结果。

> Countless platform-wannabes made this mistake. The reasoning is very simple: I'm a developer—I know what other developers want, so I'll create something to help both my colleagues and myself. I would have to include client-side Java in this camp—great development environment, terrible user experience, terrific opportunity for Macromedia (now Adobe).

无数的平台想要者犯了这种错误。推理非常简单：我是一个开发人员——我知道其他开发人员想要什么，所以我会创造一些东西来帮助我的同事和我自己。我不得不将客户端 Java 包括在这个阵营中——伟大的开发环境，糟糕的用户体验，Macromedia（现在是 Adobe）的巨大机会。

> Many extremely successful platforms have been downright awful for the developers. But they succeeded because of compelling value to the end-users and—therefore—to the application providers. You don't have to look further than early Windows for an example of this.

许多极其成功的平台对开发人员来说是彻头彻尾的糟糕。但它们成功了，因为它们对最终用户有令人信服的价值——因此对应用提供商也是如此。你不必看比早期 Windows 更远的地方就能找到例子

> I'm not advocating platforms that make life miserable for developers, but product management is all about choices and priorities, and it's essential to understand that the delivered application is what matters the most.

我不是提倡让开发人员生活悲惨的平台，但产品管理就是关于选择和优先级的，必须理解交付的应用是最重要的。

> There are other important dimensions to platform product management that make this area

平台产品管理还有其他重要的方面，使这个领域

> challenging. For example, there are many different delivery models (e.g., embedded, private-label, co-branded, hosted) and many forms of customization that may be required (e.g., end-user, customer's IT, solution provider/SI, vendor, source code). Each of these is a topic in itself.

具有挑战性。例如，有许多不同的交付模式（例如，嵌入式、自有品牌、联合品牌、托管）和许多可能需要的定制形式（例如，最终用户、客户的 IT、解决方案提供商/SI、供应商、源代码）。每一个本身就是一个话题。

> Support is also very difficult for platform providers. The bar is high because you're a critical dependency for all of your customers. That said, the great thing about working on platforms is that they're very high leverage—if you do it well, you can create a thriving ecosystem where you and your application provider partners succeed together.

对平台提供商来说，支持也非常困难。门槛很高，因为你对所有客户来说都是关键的依赖。也就是说，从事平台工作的好处是，它们的杠杆率非常高——如果你做得好，你可以创造一个繁荣的生态系统，让你和你的应用提供商合作伙伴一起成功。

> OUR NEW SOFTWARE 3 WE'LL BUNDLE IT

在本节中，我们强调了创建令人启发的产品的最重要要点和最佳实践，并描述了如何了解更多关于本书讨论的主题。

> WILL GENTLY WARM iS WITH OUR SOFTWARE
> YOUR KEYBOARD SO |}z} |) THAT MAKES YOUR
> THE KEYS ARE EASTER H | LAPTOP LIGHTER. To PRESS.
> EI(IN A WORD, WE
> i] | HAVE BECOME
> 2] |S MARKET DRIVEN.”
> i (CREATE A DIVER")
> 3 iSTON. TLL RUN!
> 3 (FOR HELP. |
> HSS) _deh GiET DILBERT: © Scott Adams/Dist. by United Feature Syndicate, Inc. In this section we highlight the most important points and best practices for creating
> inspiring products, and describe how you can learn more about the topics discussed in this book.
> Chapter 4o:
> BEST PRACTICES SUMMARY

Top 10 List

In my more than 20 years of industry experience, I have observed many practices used to create successful and inspiring products—here are what I consider to be the 10 most important.

Each is described in detail elsewhere in the book, but this summary will hopefully give you a sense of what I consider the most important practices, and I hope you'll give them all a try.

1. The role of product management. Too many product leaders spend their time on other activities, especially product marketing and/or project management. These activities are not a substitute for true product management.

2.The role of user experience. For most software products, the user experience is all-important. Devising a good user experience requires that you collaborate closely with an interaction designer and an engineer to come up with something that is valuable, usable, and feasible.

1. Opportunity assessments. These lightweight, to-the-point assessments replace the old "MRD" (market requirements documents). Before you jump into the solution, this makes sure you know what problem you're trying to solve, who you're trying to solve it

for, and how you'll know if you are successful.

1. Charter user program. It is shocking to me how many product teams think they can come up with great products without talking to users and customers. If you only do this one thing, it will ensure that you do several other things right, starting with direct and intense user interaction.
2. Product principles. Product management is all about making choices, and many of the techniques here are about helping you make good choices. The product principles help you identify and prioritize what you believe is important.
3. Personas. Another key technique for making the difficult choices required for an inspiring product is to use personas as a way to focus your release and to understand the key behaviors and underlying emotions of the target users.
4. Focus on discovery. The primary responsibility of the product manager is to discover a product that is valuable, usable, and feasible. It makes no sense to proceed to building something until you have evidence that you have discovered this product.
5. The use of prototypes. One of the most important product discovery techniques is to create a high-fidelity prototype. We do this for several reasons: First, it forces you to think much deeper about the solution; second, it enables you test your ideas out with real users; and third, it is the most useful way to describe the product to be built to the rest of the product team.
6. Test prototype with target users. Once you have a prototype, you can find out which of your ideas works and which do not. The techniques for this prototype testing are something that all product managers and designers need to master. Knowing how to get feedback on product ideas is probably the single most important skill for product managers.
7. Measure to improve. The successful product manager uses data to make important decisions, especially when trying to improve an existing product. Improving a product is

> not about adding features that customers request; it is about analyzing the product's actual use, and then relentlessly driving the product to improve the key metrics.

第 40 章：最佳实践总结

> Chapter 40

十大清单n在我超过 20 年的行业经验中，我观察到许多用于创建成功和令人启发产品的实践——以下是我认为最重要的 10 个。

每一个在本书的其他地方都有详细描述，但这个总结希望能让你了解我认为最重要的实践，我希望你能尝试所有这些。

1. 产品管理的角色。太多产品领导者把时间花在其他活动上，特别是产品营销和/或项目管理。这些活动不能替代真正的产品管理。
2. 用户体验的角色。对于大多数软件产品，用户体验是至关重要的。设计一个好的用户体验要求你与交互设计师和工程师密切合作，想出一些有价值、可用和可行的东西。
3. 机会评估。这些轻量级、切中要点的评估取代了旧的"MRD"（市场需求文档）。在你跳入解决方案之前，这确保你知道你试图解决什么问题，你试图为谁解决它，

以及如何知道你是否成功。

1. 特许用户计划。令我震惊的是，有多少产品团队认为他们可以在不与用户和客户交谈的情况下想出伟大的产品。如果你只做这一件事，它将确保你做对其他几件事，从直接和深入的用户互动开始。
2. 产品原则。产品管理就是关于做选择，这里的许多技术都是关于帮助你做出好的选择。产品原则帮助你识别和优先考虑你认为重要的东西。
3. 用户画像。另一个用于做出令人启发产品所需的艰难选择的关键技术是，使用用户画像作为聚焦你的发布并理解目标用户的关键行为和潜在情感的方式。
4. 关注发现。产品经理的主要职责是发现有价值、可用和可行的产品。在你有证据表明你已经发现了这个产品之前，继续构建某样东西是没有意义的。
5. 使用原型。最重要的产品发现技术之一是创建高保真原型。我们这样做有几个原因：首先，它迫使你更深入地思考解决方案；其次，它使你能够与真实用户测试你的想法；第三，它是向产品团队的其他成员描述要构建的产品最有用的方式。
6. 用目标用户测试原型。一旦你有了原型，你可以找出哪些想法有效，哪些无效。这种原型测试的技术是所有产品经理和设计师都需要掌握的。知道如何获得产品想法的反馈可能是产品经理最重要的单一技能。

7. 测量以改进。成功的产品经理使用数据做出重要决策，特别是在试图改进现有产品时。改进产品不是关于添加客户要求的功能；它是关于分析产品的实际使用情况，然后不懈地推动产品改进关键指标。

第 41 章：产品经理担忧清单

> Chapter 41: Product Manager Worry List
> Top 10 List

十大清单

> The strong product manager is constantly obsessed with the current and future state of her

强大的产品经理不断痴迷于她产品的当前和未来状态。

以下是她不断问自己的问题：

> product. Here are the questions she is constantly asking herself:

1. 我的产品对我们的目标客户有吸引力吗？

> 1. Is my product compelling to our target customer?

1. 我们是否已使这个产品尽可能易于使用？

> 1. Have we made this product as easy to use as humanly possible?

1. 这个产品会在竞争中成功吗？不是今天的竞争，而是我们发布时市场上的竞争？

> 1. Will this product succeed against the competition? Not today's competition, but the competition that will be in the market when we ship?

1. 我认识真正会购买这个产品的客户吗？不是我希望我们要构建的产品，而是我们真正要构建的产品？

> 1. Do I know customers who will really buy this product? Not the product I wish we were going to build, but what we're really going to build?

5.我的产品真的有差异化吗？我能在两分钟内向公司高管解释差异化吗？向一个聪明的客户在一分钟内？向行业分析师在 30 秒内？

> 5.Is my product truly differentiated? Can I explain the differentiation to a company executive in two minutes? To a smart customer in one minute? To an industry analyst in 30 seconds?

1. 产品真的会工作吗?

> 1. Will the product actually work?

7.产品是完整的产品吗？客户实际上会如何思考和购买产品？它与我们计划销售它的方式一致吗?

> 7.Is the product a whole product? How will customers actually think about and buy the product? Is it consistent with how we plan to sell it?

1. 产品的优势是否与对客户重要的东西一致？我们是否尽可能积极地定位这些优势?

> 1. Are the product's strengths consistent with what's important to our customers? Are we positioning these strengths as aggressively as possible?

1. 产品值钱吗？多少钱？为什么？客户能在别的地方买到更便宜的吗？ 10. 我是否理解产品团队的其他成员认为产品有什么好的？它是否与我自己的观点一致?

> 1. Is the product worth money? How much money? Why? Can customers get it cheaper elsewhere? 10. Do I understand what the rest of the product team thinks is good about the product? Is it consistent with my own view? The reason that “thinking time” is so critical each day—and why the job of product manager is so all-consuming—is that these questions require serious and ongoing consideration.

"思考时间"每天如此关键的原因——以及为什么产品经理的工作如此耗费精力——是因为这些问题需要认真和持续的考虑。

> LEARNING MORE

了解更多

> To learn more about this topic, and to engage in an ongoing discussion with me about product discovery and creating products customers love, see the Silicon Valley Product Group site at www.svpg.com.

要了解更多关于这个主题，并与我进行关于产品发现和创造客户喜爱的产品的持续讨论，请参见 Silicon Valley Product Group 网站 www.svpg.com。

> You will find a blog and a free newsletter containing articles on these topics and more, a list of resources, and samples and examples of the many types of artifacts and deliverables discussed in this book.

你会找到一个博客和一个免费的新闻通讯，包含关于这些主题和更多的文章，一个资源列表，以及本书讨论的许多类型工件和交付物的样本和示例。

> For professional product managers, we offer executive coaching, product strategy assistance, and training workshops with hands-on learning of many of the practices described in this book, combined with current and relevant examples from industry.

对于专业产品经理，我们提供高管辅导、产品战略协助和培训研讨会，通过实践学习本书描述的许多实践，结合行业当前和相关的示例。

> Find out more at www.svpg.com.

在 www.svpg.com 了解更多。

> ACKNOWLEDGEMENTS
>
> The very nature of putting together this book on sharing best practices from the industry's best product companies means that I have learned from a great many exceptional people. I have been especially fortunate to have had the chance to work with and for some of our industry's best product minds and companies. I have learned from all of these people, but some of them have made such a deep impression on me that I must thank them here.
>
> First, my partners at the Silicon Valley Product Group. They are my colleagues now precisely because I have been so impressed with their talents and have learned so much from each of them over the years: Chuck Geiger, Martina Lauchengco, and Kyrie Robinson.
>
> At HP, in addition to learning the value of a strong corporate and product culture, I learned a great deal from Mike Bacco, Brian Beach, Ira Goldstein and Martin Griss especially.
>
> The genesis of this book was material that I developed at Netscape Communications along with two absolutely exceptional product leaders—Ben Horowitz and David Weiden. Netscape provided an unparalleled learning opportunity, and I gained much insight about product and leadership working for and with truly brilliant minds including Marc Andreesen, Jennifer Bailey, Jim Barksdale, Peter Currie, Eric Hahn, Basil Hashem, Mike Homer, Omid Kordistani, Keng Lim, Bob Lisbonne, Debby Meredith, Mike McCue, Danny Shader, Sharmila Shahani, Ram Shriram, Bill Turpin, and later Barry Appelman at AOL.
>
> At eBay, I have to especially credit Marty Abbott, Josh Kopelman, Shri Mahesh, Pierre Omidyar, Lynn Reedy and Maynard Webb.
>
> While researching this book, I also benefited from the insights of Jim Barton, Jeff Bonforte, Kevin Compton, Fred Cox, Audrey Crane, Pete Deemer, Mark Hurst, Guy Kawasaki, Amy Klement, Norm Meyrowitz, Andrew Sandler, and Bob Vallone.
>
> Each of these people have directly influenced me and informed specific topics in this book, either by their explicit help and coaching, or from simply their leadership and actions that I was fortunate enough to witness first hand.
>
> While my time working for these exceptional companies was an invaluable learning experience, I found that as I began working with client companies in my work as part of SVPG, I was able to benefit greatly by getting a chance to meet and work with the product leaders at many other

> leading companies in our industry. There are simply too many people to list, but they know who they are, and I am grateful to every one of them.
>
> This book is based on material produced for a blog and newsletter that I have published for several years, and each and every topic was improved thanks to feedback and comments from literally thousands of product leaders from every corner of the globe. I thank everyone who has read, shared, and commented on these articles.
>
> I also must thank Mark Coggins, Peter Economy, John Hornbaker, Benji Jasik, Cynthia Johanson, Jeff Lash, Bruce Williams and all of the people at Westminster Printing and
>
> Promotions for their very significant help in making this book as good as it can be.
>
> Finally, those people who know the culture of the companies I've worked at understand that many very long hours were involved, and I could not have contributed to these companies without the support of my wife and children.
>
> ABOUT THE AUTHOR
>
> During the course of the past 20 years, Marty Cagan has served as an executive responsible for defining and building products for some of the most successful companies in the world, including Hewlett-Packard, Netscape Communications, America Online, and eBay.
>
> Before founding the Silicon Valley Product Group to pursue his interests in helping others create inspiring and successful products through his writing, speaking, and workshops, Marty was most recently senior vice-president of product management and design for eBay, where he was responsible for defining products and services for the company's global e-commerce trading site.

致谢

> Acknowledgments

编写这本关于分享行业最佳产品公司最佳实践的书的本质意味着，我从许多杰出的人那里学到了东西。我特别幸运有机会与和为一些我们行业最好的产品头脑和公司工作。我从所有这些人那里学到了东西，但其中一些人给我留下了如此深刻的印象，我必须在这里感谢他们。

首先，我的硅谷产品集团合作伙伴。他们现在是我的同事，正是因为我对他们的才华印象深刻，多年来从每个人那里学到了很多东西：Chuck Geiger、Martina Lauchengco 和 Kyrie Robinson。

在 HP，除了学习强大公司和产品文化的价值之外，我特别从 Mike Bacco、Brian Beach、Ira Goldstein 和 Martin Griss 那里学到了很多东西。

这本书的 genesis 是我在网景通信公司开发的材料，与两位绝对杰出的产品领导者——Ben Horowitz 和 David Weiden 一起。网景提供了无与伦比的学习机会，我从与真正杰出的头脑一起工作和为他们工作中学到了很多关于产品和领导力的知识，包括 Marc Andreesen、Jennifer Bailey、Jim Barksdale、Peter Currie、Eric Hahn、Basil Hashem、Mike Homer、Omid Kordistani、

Keng Lim、Bob Lisbonne、Debby Meredith、Mike McCue、Danny Shader、Sharmila Shahani、Ram Shriram、Bill Turpin，以及后来在 AOL 的 Barry Appelman。

在 eBay，我特别要感谢 Marty Abbott、Josh Kopelman、Shri Mahesh、Pierre Omidyar、Lynn Reedy 和 Maynard Webb。

在研究本书的过程中，我还受益于 Jim Barton、Jeff Bonforte、Kevin Compton、Fred Cox、Audrey Crane、Pete Deemer、Mark Hurst、Guy Kawasaki、Amy Klement、Norm Meyrowitz、Andrew Sandler 和 Bob Vallone 的见解。

这些每个人都直接影响了我，并通过他们的明确帮助和指导，或者通过我有幸亲眼目睹的他们的领导和行动，为本书的具体主题提供了信息。

虽然我在这些杰出公司工作的时光是一次宝贵的学习经历，但我发现，当我开始作为 SVPG 的一部分与客户公司合作时，我有机会结识并与我们行业许多其他领先公司的产品负责人合作，从中受益匪浅。要列出的人实在太多，但他们知道自己是谁，我感谢他们每一个人。

本书基于我为博客和新闻通讯撰写的材料，这些材料我已出版多年，每一个主题都得益于来自全球各地数以千计的产品负责人的反馈和评论而得到改进。我感谢每一位阅读、分享和评论这些文章的人。

我还必须感谢 Mark Coggins、Peter Economy、John Hornbaker、Benji Jasik、Cynthia Johanson、Jeff Lash、Bruce Williams 以及 Westminster Printing 和

Promotions 的所有人，感谢他们为使这本书尽可能完善所提供的非常重要的帮助。

最后，那些了解我工作过的公司文化的人知道，这涉及很多很多长时间的工作，如果没有我的妻子和孩子的支持，我不可能为这些公司做出贡献。

关于作者

> About the Author
>
> INTRODUCTION
>
> In the mid 1980s, I was a young software developer working for HP on a high-profile product. It was when Artificial Intelligence was all the rage, and I was fortunate enough to be working at one of the industry's best companies, as part of a very strong software engineering team (several members of that team went on to substantial success in companies across the industry). Our assignment was a difficult one: to deliver software on a low-cost, general-purpose workstation that until then required a special-purpose hardware/software combination that cost over $100,000 per user—a price few could afford. We worked long and hard for well over a year, sacrificing countless nights and weekends. Along the way, we added several patents to HP's portfolio. We developed the software to meet HP's exacting quality standards. We internationalized the product and localized it for several languages. We trained the sales force. We previewed our technology with the press and received excellent reviews. We were ready. We released. We celebrated the release.

在过去的 20 年中，Marty Cagan 一直担任高管，负责为世界上一些最成功的公司定义和构建产品，包括惠普、网景通信、美国在线和 eBay。

> Just one problem: No one bought it.

在创立硅谷产品集团以通过他的写作、演讲和研讨会帮助他人创造令人启发和成功的产品之前，Marty 最近担任 eBay 的产品管理和设计高级副总裁，负责为公司的全球电子商务交易平台定义产品和服务。

> The product was a complete failure in the marketplace. Yes, it was technically impressive, and the reviewers loved it, but it wasn't something people wanted or needed.
>
> The team was of course frustrated with this outcome. But soon we began to ask some important questions: Who decides what products we should actually build? How do they
>
> decide? How do they know that what we build will be useful?
>
> Our young team learned something very profound—something I'm sure many teams have discovered the hard way: It doesn't matter how good your engineering team is if they are not given something worthwhile to build.
>
> More generally, we learned that it's not enough to do a good job engineering a product. At least as important is discovering a product that is valuable, usable, and feasible.
>
> When trying to track down the root cause of our failure, I learned that the decisions about what to build came from a "Product Manager"—someone who generally resided in the marketing organization and who was responsible for defining the products we built. But I also learned that Product Management wasn't something HP was particularly good at. I later learned that most companies weren't good at this, and in fact most still aren't.
>
> I promised myself that never again would I work so hard on a product unless I knew the product would be something that users and customers wanted.
>
> Over the next 20 years, I had the very good fortune to work on some of the most successful high-tech products of our time—first at HP during the rise of personal computers, then at Netscape Communications/AOL during the rise of the Internet where I served as vice-president of platform and tools, and finally at eBay during the rise of e-commerce where I served as the senior vice-president of product management and design.
>
> Not all of the product efforts have been as successful as others, but I am happy to say that
>
> none were failures, and several became loved and used by millions of people around the world.
>
> Soon after I left eBay, I started getting calls from product organizations wanting to improve how they produced products. As I began working with these companies, I discovered that there was a tremendous difference between how the best companies produced products, and how most companies produced them. I realized that the state of the art was very different from the state of the practice.
>
> Most companies were still using old and inefficient ways to define and create products. I also discovered that there was precious little help available, either from academia, including the best business school programs, or from industry organizations, which seemed hopelessly stuck in the failed models of the past—just like the one I worked in at HP.

Thave had some great rides, and I am especially thankful that I have had the chance to work for and with some of the best product minds in the industry. The best ideas in this book are from these people. You will find a list of many of them in the acknowledgements. I have learned from them all and I am grateful to each one of them.

I chose this career because I wanted to work on products that customers love; products that inspire and provide real value. I find that most product leaders also want to create inspiring and successful products. Yet most products are not inspiring, and life is too short for bad products.

My hope in writing this book is that it will help share the best practices of the most

successful product companies, and that the result will be more products that are truly inspiring—products that customers love.

Who This Book Is For

This book is specifically written for those members of software product teams—especially Internet software product teams—both consumer and business, who are responsible for defining the products to be built. Often these product leaders are called "product managers," but they may be company founders, executives, lead engineers, or designers.

In addition to product managers, this book is for user experience designers, software engineers and architects, project/program managers, product marketing managers, and the managers of the different parts of the product organization.

In my experience, the information described here is applicable to a wide variety of product development teams:

Your company may be a startup, or a very large business with many products, or something in between. You may be working on an all-new 1.0 product, or working on incremental improvements to an existing product. Your product development team may be using an Agile method such as Scrum, or conventional Waterfall-based methods.

Your product may be an Internet service, shipped software, a device, or a platform. It may be for consumers, small businesses, or enterprises. For instance, the products could be

e-commerce Web sites, fantasy sports or game sites, consumer electronics, hosted services for businesses, or platforms for enabling specific types of Internet applications and services such as social networking or video sharing.

One caveat I have to make is that the book is not intended for those working on non-software products, such as pharmaceuticals, and also for non-product software efforts, such as custom software projects. That's not to say that the methods and strategies I describe won't work in other environments, but I developed these concepts and practiced them in the product software world, so they may not be as effective outside of it.

How This Book Is Structured

When I first moved into senior management roles at Netscape, I found my day-to-day tasks fell into three distinct areas: People, Process, and Product:

Great Products by Design

I do not believe inspiring products happen by accident. In every case, behind every

successful, inspiring product, I find that there are certain truths. Here are ten such truths

that I try to keep in mind on every product effort:

1. The job of the product manager is to discover a product that is valuable, usable, and feasible.
2. Product discovery is a collaboration between the product manager, interaction designer, and software architect.
3. Engineering is important and difficult, but user experience design is even more important, and usually more difficult.
4. Engineers are typically very poor at user experience design—engineers think in terms of implementation models, but users think in terms of conceptual models.
5. User experience design means both interaction design and visual design (and for hardware-based devices, industrial design).
6. Functionality (product requirements) and user experience design are inherently intertwined.
7. Product ideas must be tested—early and often—on actual target users in order to come up with a product that is valuable and usable.
8. We need a high-fidelity prototype so we can quickly, easily, and frequently test our ideas on real users using a realistic user experience.
9. The job of the product manager is to identify the minimal possible product that meets the objectives—valuable, usable and feasible—minimizing time to market and user

complexity. 2.Once this minimal successful product has been discovered and validated, it is not something that can be piecemealed and expect the same results. I continue to talk to far too many product teams stuck in old, failed ways of creating products. Coming to terms with these truths is what this book is all about. People refers to the product organization, and the roles and responsibilities of the members of the team as they define and develop the product. Process refers to the processes, activities and best practices used to repeatedly discover and build inspiring and successful products. Product refers to the defining characteristics of these inspiring products. All three of these areas are essential to discovering and creating inspiring products. Everything starts with the people, but the process is what enables these people to consistently produce inspiring and successful products. I have organized this book into these same three parts. Each part is broken up into several independent topics. The order within each of these parts is generally not important—you can jump from one topic to another. Each topic is meant to be self-contained. At the end of the book, I sum everything up by describing what I think are the 10 most important practices and techniques.

Many of the topics discussed in this book reveal the best practices applied by some of the best companies in the world. Others are based on interviews with some of the best product minds in our industry. And yet others are based on my own experiences in the companies I have worked for and with.

Remember: This book is only useful if it helps you deliver better products, so the intention is for each and every topic to be thought provoking, relevant, and actionable.

It is my hope that this information will help you create more successful products, and I would love to hear about your experiences. Please visit me online at the Silicon Valley Product Group site (www.svpg.com) and let me know what you think.

Here's to your success, and to the discovery of inspiring products that customers love.

° Examples

Iam a big believer in the power of examples, both good and bad. But, because our industry changes so quickly, one of the challenges in a book like this is providing timely and relevant examples. Another problem is that including my favorite examples would add more than one hundred extra pages to this book.

I therefore put many of the examples on the Silicon Valley Product Group web site (www.svpg.com/examples). This way I can continue to update and add to the examples without needing to update the content of the book. As with all of the material on the site, there is no fee or registration required.

The online examples include opportunity assessments, product principles, product strategies, product roadmaps, product specs, prototypes, personas, and prototype testing questions and tasks.

I apologize that this means that you will need Internet access to see many of the examples, and I realize that this breaks the flow of reading a book, but I hope you find the benefits of this approach outweigh the inconvenience.

You will find references in the chapters that follow to the examples on the site.

OKAY, LET'S START 8] NO, LET'S START BY

BY DOCUMENTING 2] YOU TELLING ME ALL

YOUR MARKET 3] THE THINGS YOU CAN

REQUIREMENTS 3 DESIGN. THEN T'LL

=] TELLYOU WHICH ONE

) § 1 LIKE

fee all dh i

Sf 3 Sha

(| Sianf | atin ees =] work CAN WHAT'S

2] BEVERY = THaT

3] REWARDING. pooutcKey

3] YOU SHOULD yoy HAVE

p| UNA THERE? DILBERT: © Scott Adams/Dist. by United Feature Syndicate, Inc.

The Product Organization

Every product begins with the people on the product team. How you define the roles, and who you select to staff the team, will very likely prove to be a determining factor in its success or failure.

In this section we will describe the key roles and responsibilities of modern software and Internet product teams.

> This is an area where many product teams fall short, stuck in old models of the past. For many organizations the roles and responsibilities discussed here represent significant differences from what they're used to.